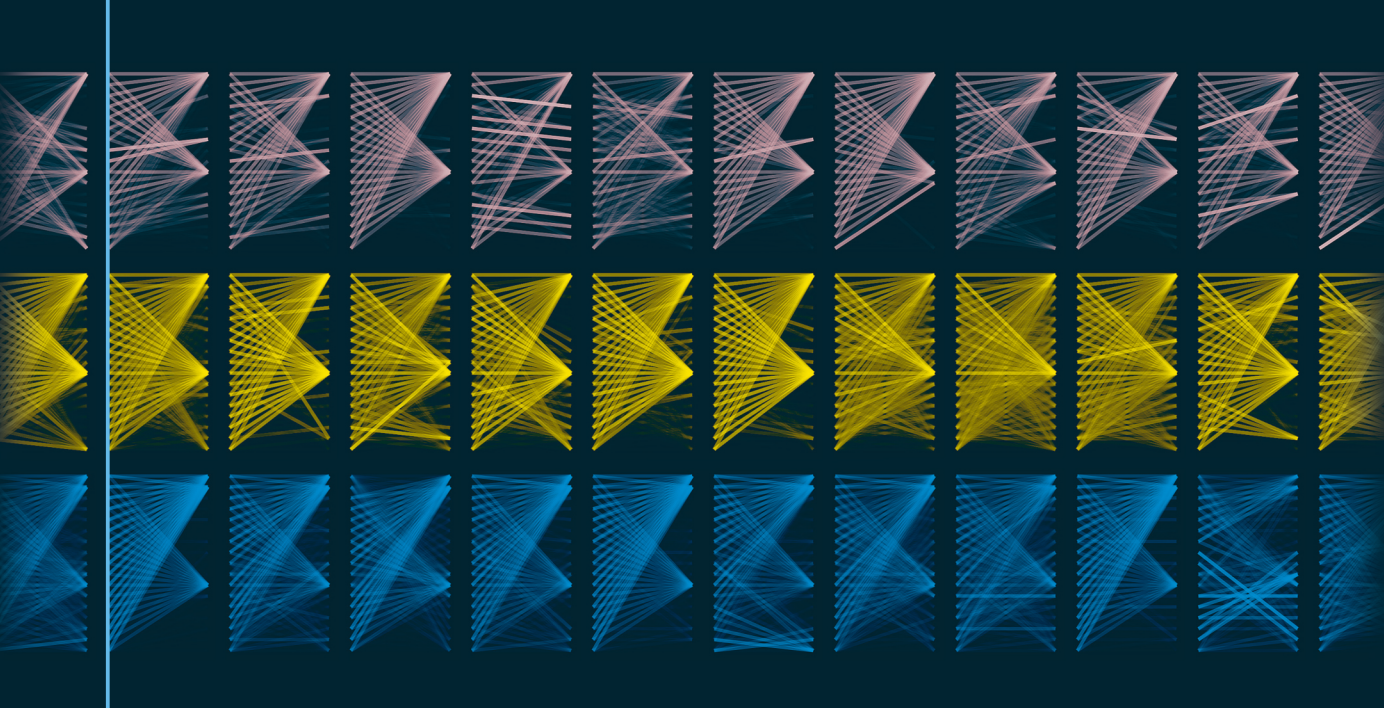




Uday Kamath · Kevin Keenan
Garrett Somers · Sarah Sorenson

Large Language Models: A Deep Dive

Bridging Theory and Practice

 Springer

Large Language Models: A Deep Dive

Uday Kamath • Kevin Keenan
Garrett Somers • Sarah Sorenson

Large Language Models: A Deep Dive

Bridging Theory and Practice

 Springer

Uday Kamath
Smarsh
Ashburn, VA, USA

Kevin Keenan
Smarsh
Castlewellan, Down, UK

Garrett Somers
Smarsh
Nashville, TN, USA

Sarah Sorenson
Smarsh
Reston, VA, USA

ISBN 978-3-031-65646-0 ISBN 978-3-031-65647-7 (eBook)
<https://doi.org/10.1007/978-3-031-65647-7>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

To my parents, Krishna and Bharathi, my wife, Pratibha, my children, Aaroh and Brandy, and my dear family and friends (Anu, Jyoti, Maithilee, Munnu, Nana, Priyal, Shilpa, Sunila, Swati, and Swetha) — for your unwavering support, love, and care, especially during my illness.

—Uday Kamath

To my parents, Eugene and Evelyn, my relentlessly patient and loving wife, Kaitlin, and my beautiful children, Éirinn and Méabh, all of whom have gracefully supported me in many things in life, few more ambitious as this book.

—Kevin Keenan

To my wonderful daughters, Octavia and Lyra.

—Garrett Somers

To my husband, Matthew, who endured my frequent lack of availability in the course of writing this book (and complained only occasionally).

—Sarah Sorenson

Foreword

Large Language Models have revolutionized the field of artificial intelligence, transforming how we interact with technology and reshaping various industries. As a course director at the University of Oxford for various AI courses and an entrepreneur involved in multiple ventures across the globe, I have seen firsthand how these models can solve complex problems and streamline everyday tasks. This book arrives at an opportune moment, providing a comprehensive guide to understanding and utilizing LLMs. The authors have done an excellent job of breaking down the complex architecture and algorithms behind these models, making them accessible to a broad audience.

I have known the first author, Dr. Uday Kamath, for some time and have followed his previous work with great interest. His expertise and insights into AI are well-regarded, and this book is a testament to his deep understanding and innovative thinking. The book covers everything in detail, from pre-training and prompt-based learning basics to more advanced topics like fine-tuning techniques and Retrieval-Augmented Generation (RAG). One of the most empowering features of this book is its practical focus. Each chapter is designed to equip the reader with the skills and knowledge to apply LLMs in real-world scenarios. With hands-on tutorials and real-world examples, one will not only understand the theory but also gain the confidence to implement these models effectively in their work.

A dedicated chapter on LLMOps and productionizing is particularly valuable. It provides detailed guidance on operationalizing and deploying these models in practical settings, ensuring one can take the theoretical understanding and turn it into tangible results. Additionally, the book includes an extensive compilation of datasets, benchmarks, and evaluation metrics, providing a solid foundation for anyone looking to explore LLM applications. The chapter on multimodal LLMs, which goes beyond text to include audio, images, video, and robotic inputs/outputs, is particularly exciting and points to the future of AI interaction. The book also addresses the ethical challenges associated with LLMs, such as bias, fairness, and privacy. It's crucial that as we leverage the power of these models, we do so responsibly. The authors provide valuable strategies for addressing these issues, helping ensure that LLMs are used in a way that aligns with human values.

In conclusion, this book is an essential resource for anyone interested in Large Language Models. It offers a thorough understanding of the technology, practical insights, and ethical considerations, making it a valuable guide for navigating the future of AI. I commend the authors for their detailed research and clear presentation, and this book will be a key reference in the field for years to come.

**- Ajit Jaakar, Course Director and Visiting Fellow Artificial Intelligence,
University of Oxford**

Reviews

“As a seasoned tech executive and industry thought leader I realized how critical LLMs were becoming in all phases of product development - from developer efficiency to product and marketing deployment. Looking to strengthen my foundation, I found this book on Large Language Models to be an invaluable guide and has now become my go to resource, as my team and I look to harness the power of LLMs within our product.

It demystifies the complexities of LLMs, from their intricate architectures to the ethical considerations of their deployment, and highlights the future of multimodal LLMs, which extend their capabilities beyond text to audio, images, and video. With its comprehensive coverage and practical insights, this book is a must-read for anyone looking to understand and leverage the transformative power of LLMs in today’s AI-driven world.”

-Shalini Govil Pai, VP and GM, Google

“The writing is precise and highly technical, catering to readers with a solid background in machine learning and AI. The explanations are dense with information, and the book assumes familiarity with advanced mathematical concepts and programming skills. Despite its technical depth, the book is well-structured, with clear explanations and logical progression.”

-Dr. Sherry Marcus, Director of Applied Science GenAI AWS, Amazon

“Generative AI is a hot topic today, but is it a game-changer for society and business, or just another buzzword? This book is a must-read to understand the vast potential, risks, and challenges of Generative AI. It offers a thorough journey through the lifecycle of Generative AI, making it an ideal choice for those seeking a comprehensive guide. The book starts by setting the stage with the history of language models (LLMs), then dives into transformer architecture, prompt engineering, fine-tuning, retrieval-augmented generation (RAG), and concludes with practical steps for deploying these solutions. I found the book incredibly versatile and engaging, suitable for both developers and AI enthusiasts. The final chapter, which looks ahead at the future of Generative AI, is particularly insightful. I highly recommend it.”

-Eduardo Ordax, GenAI Lead and AWS Evangelist, Amazon

“This book is impressively comprehensive and up-to-date. The authors have meticulously sifted through recent developments in LLMs, organizing and explaining various techniques in a practical context. This makes it an indispensable resource for professionals and researchers, particularly for enhancing user experience and interactivity in information systems.”

– Prithvi Prabhu, Chief Technologist, H2O.ai

“Through this book, Kamath and co-authors provide a comprehensive resource for researchers and students interested in obtaining a deep understanding of large language models. The material elegantly bridges theory and practice and integrates recent, cutting-edge advancements, such as multi-modal models, thus appealing to and becoming a must-have for both academic researchers and industry-based scientists.”

- **Dr. Amarda Shehu, Professor of Computer Science at GMU and Associate Dean for AI Innovation in the College of Engineering and Computing**

“Comprehensive overview of what it takes to build reliable LLM-powered software - understand how to select, adjust, and evaluate models for your product and business goals and how to complement your LLM system with RAG, even if your data is quite complex. Must read for all AI Engineers!”

- **Daniel Svonava, CEO and Founder, Superlinked**

Preface

Why This Book

In the panorama of technological evolution, Large Language Models (LLMs) have emerged as a cornerstone, transforming our interaction with information, reshaping industries, and redefining the boundaries of artificial intelligence. As we stand on the cusp of this transformation, the impact of LLMs extends beyond mere computational advancements, influencing everything from day-to-day tasks to complex problem-solving mechanisms. This seismic shift has not only intrigued technologists and researchers but has also captivated the imagination of a broader audience keen on understanding and harnessing the power of LLMs.

The inception of LLMs marks a significant departure from traditional computing paradigms, offering an unprecedented ability to understand, generate, and interact with human language in a manner that is both intuitive and insightful. This evolution of models that learn from vast datasets of human language has opened new avenues for innovation, creativity, and efficiency. The ability of LLMs to process and produce language has led to transformative applications across various domains, including but not limited to automated content creation, sophisticated chatbots, enhanced search engines, and groundbreaking research tools.

However, with great power comes great complexity. The workings of LLMs, while fascinating, are not immediately accessible to all. The intricate architecture, the underlying algorithms, and the ethical considerations accompanying the deployment of LLMs are subjects of vital importance that require thorough exploration. Here, the need for a comprehensive book on LLMs becomes evident. A pressing demand exists for a resource that not only demystifies the technical workings of these models but also contextualizes their impact, explores their applications, and addresses the ethical dilemmas they pose. This book aims to be that resource.

The book provides an in-depth exploration into the reality of large language models. It begins with an overview of pre-trained models, categorizing them based on different criteria and delving into architectures like Transformers. This foundation paves the way for a deeper understanding of prompt-based learning. It highlights a

variety of prompt-based learning mechanisms, the significance of extracting knowledge from LLMs, and different techniques to accomplish it. The book elaborates on the methods for fine-tuning LLMs, discussing different strategies and trade-offs. Next, the book comprehensively examines integrating reinforcement learning into LLM training to align with human values, a core component of LLMs. The book further discusses the convergence of LLMs in fields such as computer vision, robotics, and speech processing. The book emphasizes practical applications, detailing real-world use cases such as Retrieval-Augmented Generation (RAG). These examples are carefully chosen to illustrate the diverse and impactful ways in which LLMs can be applied in various industries and scenarios.

Additionally, the book provides valuable insights into operationalizing and deploying LLMs. It guides readers through the implementation of these models using contemporary tools and libraries, ensuring they know how to use LLMs practically. The book explores the challenges associated with LLMs, from inherent biases and unpredictability to the broader ethical implications of their emergent behaviors. Finally, the book examines the cutting-edge realm of Multimodal Large Language Models, extending their reach beyond text to encompass audio, images, video, and robotic inputs and heralding a new era of comprehensive AI interaction. Furthermore, each chapter includes hands-on tutorials that showcase the functionality of LLMs in Natural Language Processing (NLP) tasks for LLM-specific topics.

This book is structured around several key features designed to offer readers an in-depth and accessible journey through the landscape of LLMs. Among these salient features are:

- Delve into over **100+ techniques and state-of-the-art methods**, including pre-training, prompt-based tuning, instruction tuning, parameter-efficient and compute-efficient fine-tuning, end-user prompt engineering, and building and optimizing Retrieval-Augmented Generation systems, along with strategies for aligning LLMs with human values using reinforcement learning.
- Utilize over **200 datasets** compiled in one place, covering everything from pre-training to multimodal tuning, providing a robust foundation for diverse LLM applications.
- Explore **50+ strategies** to address key ethical issues such as hallucination, toxicity, bias, fairness, and privacy. Discover methods for measuring, evaluating, and mitigating these challenges to ensure responsible LLM deployment.
- Bridge the gap from promise to practice by learning how to select the right LLM, optimize training and inference costs, improve latency and performance, and leverage essential tools.
- Access over **200+ benchmarks** covering LLM performance across various tasks, ethical considerations, multimodal applications, and **50+ evaluation metrics** for the LLM lifecycle.
- Engage with **9 detailed tutorials** that guide readers through pre-training, fine-tuning, alignment tuning, bias mitigation, multimodal training, and deploying large language models using tools and libraries compatible with Google Colab, ensuring practical application of theoretical concepts.

- Benefit from **100+ practical tips** for data scientists and practitioners, offering implementation details, tricks, and tools to successfully navigate the LLM life-cycle and accomplish tasks efficiently.

Who This Book Is For

This book has been meticulously crafted to serve a diverse audience, aiming to be a comprehensive one-stop resource for anyone looking to grasp the essence and intricacies of LLMs. Whether you're an undergraduate or graduate student in computer science, data science, or artificial intelligence, keen on unraveling the complexities of AI, a researcher in AI or NLP diving deep into the theoretical advancements and practical applications of language models, or a data scientist in the industry looking to leverage the cutting-edge capabilities of LLMs in solving real-world problems, this book is designed for you.

The content is structured to cater to a broad spectrum of readers, from those taking their first steps in AI to seasoned professionals and academics who wish to deepen their understanding and expand their knowledge base. This book can be a foundational text for students covering the fundamental concepts, methodologies, and tools necessary to understand and work with LLMs. It bridges the gap between academic learning and the skills required to navigate the challenges and opportunities presented by AI in a practical context.

For researchers and academics, this book provides comprehensive coverage of cutting-edge research in every aspect of LLMs, including prompt engineering techniques, learning strategies, Reinforcement Learning from Human Feedback (RLHF), multimodal LLMs, and an in-depth analysis of challenges and mitigation strategies.

Data scientists and industry professionals will consider this book an essential toolkit for mastering efficient techniques to fine-tune LLMs for domain-specific applications. It goes beyond fine-tuning to explore applications such as Retrieval-Augmented Generation (RAG) and learning strategies, equipping readers with the skills to successfully deploy LLMs in production systems. Moreover, the book delves into critical methods to evaluate and mitigate challenges such as hallucination, bias, fairness, and privacy issues, ensuring readers are prepared to address these concerns in practical settings.

Before diving into this book, readers are expected to have a certain level of prerequisite knowledge, including:

- Basic understanding of Linear Algebra, Calculus, Statistics, and Probability.
- Understanding Machine Learning and AI concepts at an intermediate level.
- Understanding Natural Language Processing concepts and deep learning techniques at a basic level.
- Intermediate-level Python programming and familiarity with associated libraries such as Pandas, Matplotlib, PyTorch, etc.

What This Book Covers

To set the stage for what will be covered, we provide a comprehensive overview of each chapter, unpacking the content and themes to give readers a nuanced understanding of the material covered.

Chapter 1: Large Language Models: An Introduction begins with a discussion of the historical context and progression of natural language processing.. Tracing back to the origins of human linguistic capabilities, the chapter explains the gradual transition to computational language modeling, emphasizing the importance of the intricate interplay between biology and technology. The evolution of language models in computational domains is presented in a coherent timeline, showcasing how rudimentary models transformed into the sophisticated LLMs we are familiar with today. Various critical factors influencing this transformative journey, including algorithmic advancements, computational power, and data availability, are discussed. LLMs are defined and delineated, ensuring readers grasp their significance in contemporary AI paradigms.

Chapter 2: Language Models Pre-training delves deeply into the realm of pre-trained models, offering a foundational understanding of their core mechanisms and structures. It starts with thoroughly examining the attention mechanism, showcasing how it has reshaped NLP by enabling models to focus on relevant information. The groundbreaking nature of the Transformer architecture is then presented, highlighting its significance in modern NLP endeavors. The chapter transitions to categorizing LLMs, explaining the specifics of encoder-decoder, autoregressive, and masked language models. Pioneering architectures like BERT, T5, GPT (1-3), and Mixtral8x7B are discussed, focusing on their unique training techniques and primary applications. A section on key datasets offers insights into the foundational data powering these state-of-the-art models. The chapter concludes with a practical guide to essential models, tools, and hubs, preparing readers for the advanced topics in the subsequent chapters.

Chapter 3: Prompt-based Learning offers an insightful exploration into prompt-based learning, a technique central to current advances in NLP. This chapter methodically introduces the reader to the principles of this approach, illustrating how diverse NLP tasks can be effectively mapped to specific prompts. It delves into the nuances of prompt engineering, answer engineering, and multi-prompting, shedding light on the art and science of crafting effective and efficient prompts that can guide models to desired outputs. This chapter provides a comparative analysis between the traditional pre-trained/fine-tuning methodologies and the prompt-based approach.

Chapter 4: LLM Adaptation and Utilization delves into the intricate dynamics surrounding the impressive capabilities of LLMs and the practical challenges they present, especially when fine-tuning becomes essential. It provides the reader with an in-depth exploration of various strategies geared toward parameter-efficient learning. Notable methods like serial and parallel adapters, LoRA, and VeRA, among others, are elucidated, all viewed through the lens of “delta-tuning”—a concept that

aids in discerning the efficiency of these methods relative to desired outcomes. The chapter addresses the scenarios of limited data availability, elaborating on zero-, few-, and multi-shot learning approaches within the LLM framework. The nuances of prompt design and context length, pivotal for enhancing in-context learning, are highlighted. Furthermore, the significance of chain-of-thought reasoning, especially in data-scarce settings, is emphasized. Finally, the chapter broaches the topical subject of making full-parameter tuning in LLMs more financially and computationally viable, spotlighting innovations like post-training quantization and quantization-aware fine-tuning, thereby ensuring that LLM capabilities are not just the preserve of heavily-funded enterprises.

Chapter 5: Tuning for LLM Alignment introduces the concept of alignment with human preferences defined as 3H—Helpful, Harmless, and Honest—and discusses the challenges of encoding complex human values into LLMs. The chapter explores how reinforcement learning, particularly Reinforcement Learning from Human Feedback (RLHF), is utilized to align LLMs with human values through feedback mechanisms. It addresses the challenges associated with RLHF, such as the high resource demands and scalability issues, and presents breakthroughs like Constitutional AI and Direct Preference Optimization as innovative solutions to enhance the ethical and responsible application of LLMs.

Chapter 6: LLM Challenges and Solutions explores the inherent challenges and ethical quandaries surrounding LLMs. Beginning with an overview of the limitations and challenges, the chapter dives into epistemological issues arising from the vast and varied data on which these models are trained. The narrative transitions to an intricate examination of the embedded moral norms within pre-trained models, raising questions about their inherent biases and the sociocultural values they may inadvertently propagate. A subsequent section delves into the task of discerning the moral direction of LLMs and the intricacies involved in ensuring their ethical alignment. The chapter further addresses the pertinent issue of neural toxic degeneration, discussing strategies to mitigate and counteract such tendencies within LLMs. As the narrative progresses, emphasis is laid on ethical concerns, specifically the vulnerabilities associated with privacy attacks on language models. A comprehensive discourse on privacy-enhancing technologies tailored for LLMs highlights cutting-edge solutions to safeguard user data and interactions.

Chapter 7: Retrieval-Augmented Generation delves into the foundational elements of Retrieval-Augmented Generation (RAG) and outlines the critical considerations in designing RAG systems. We explore a variety of modular enhancements that can be integrated into a RAG workflow aimed at broadening functionalities and fortifying against potential vulnerabilities. Additionally, we examine key test metrics employed to assess RAG performance, focusing on the accuracy of dense retrieval processes and the effectiveness of chatbots in responding to queries.

Chapter 8: LLMs in Production focuses on the operational and engineering dimensions of LLMs, particularly in the context of prompt-based approaches that are increasingly becoming integral to various functional applications. This chapter pro-

vides a comprehensive guide to deploying LLMs effectively in production settings. It discusses crucial considerations such as choosing the appropriate LLM, understanding evaluation metrics, benchmarking, and optimizing for various factors, including latency, cost, quality, adaptability, and maintenance. The chapter provides essential tools and techniques, guiding readers through the intricacies of LLM application development.

Chapter 9: Multimodal LLMs This chapter delves into the rapidly evolving domain of multimodal large language models (MMLLMs), representing a significant advancement in language modeling. We present a general MMLLM framework, discussing its various components both theoretically and practically and mapping each to state-of-the-art implementations. The chapter explores the adaptation of techniques like instruction tuning, in-context learning, chain-of-thought prompting, and alignment tuning from traditional LLMs to multimodal contexts, showcasing how these adaptations enhance adaptability and reasoning across different modalities. Various benchmarks, datasets, and distinctions between the architectures are elaborately described. We highlight three leading MMLLMs—Flamingo, Video-LLaMA, and NExT-GPT—offering a comprehensive overview and mapping them to the general MMLLM framework.

Chapter 10: LLMs: Evolution and New Frontiers This concluding chapter provides an overview of the evolution of LLMs, emphasizing significant trends and developments. It explores the shift toward using synthetic data to sustain model scaling and the expansion of context windows to enhance interpretative capabilities. The chapter also discusses the progression of training techniques aimed at improving efficiency and depth of knowledge transfer, along with the transition from traditional Transformer architectures to alternative approaches like state space models, which offer improved scalability and efficiency. Furthermore, it highlights trends toward smaller, more efficient models, the democratization of technology, and the rise of domain-specific models. These trends illustrate a movement toward more customized, accessible, and industry-specific AI solutions. Additionally, the chapter delves into the frontiers of LLM technologies and their use in agent-based applications and search engines, which are increasingly replacing traditional technologies.

How to Navigate This Book

This book is designed to be versatile, offering various paths through its content to suit readers from different backgrounds and with specific interests. For example, an industry-based data scientist focused on fine-tuning large language models through custom datasets, understanding associated challenges and mitigations, and deploying these models in production might find the most value in exploring Chapters 1, 3, 4, 7, 8, and 9.

We have endeavored to organize the chapters in such a manner that complex topics are progressively layered on top of more fundamental concepts. With that said,

readers should take heed that the material is not always presented in a strictly sequential nature. For instance, in Chapter 2, we touch on foundational LLMs which have achieved their success through training techniques that are not fully explained until Chapter 6. In such cases, we frequently provide references to the sections of the book where the relevant information is covered in more depth; jumping forward or backward as needed to focus in on a particular topic of interest is encouraged. Similarly, readers should not feel that they need complete mastery of all previous chapters before continuing on to the next one.

Throughout this book, important points are highlighted in gray boxes in every chapter to ensure that readers can easily recognize and reference key concepts and critical information.

This is an important concept.

Each chapter includes “Practical Tips”, highlighted in attention boxes, which provide practical advice and strategies.

! Practical Tips

These boxes highlight essential strategies for deployment, tuning, customization, tools, parameters, and more, offering actionable guidance for real-world application of the concepts discussed.

In this book, prompts are consistently formatted and presented in a standard list style.

```
passage: "Look What You Made Me Do" is a song recorded by
        American singer-songwriter Taylor Swift, released on August
        24, 2017 by Big Machine Records as the lead single from her
        sixth studio album Reputation (2017). Swift wrote the song
        with her producer Jack Antonoff. "Look What You Made Me Do"
        is an electroclash and pop song, with lyrics about various
        issues that built Swift's reputation. Right Said Fred band
        members Fred Fairbrass, Richard Fairbrass, and Rob Manzoli
        are also credited as songwriters, as it interpolates the
        melody of their song "I'm Too Sexy" (1991).

question: "did taylor swift write look what you made me do"

label: 1
```

Listing 1: *GLUE BoolQ* example

Python code and listings throughout the book are presented in a clear, standardized format to facilitate understanding and practical application of programming concepts related to the topics discussed.

```
from transformers import BertTokenizer, BertForMaskedLM

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForMaskedLM.from_pretrained("bert-base-uncased")
```

Listing 2: Python code for initializing BERT tokenizer and model

All tutorials from this book are hosted on a dedicated GitHub repository, accessible via <https://github.com/springer-llms-deep-dive>. The repository is organized into chapter-wise folders containing Jupyter notebooks and associated code, which readers can run on Google Colab using GPU settings for optimal efficiency. The authors recommend subscribing to Colab Pro, which comes at a small cost that we consider quite reasonable for the amount of added capability it provides. We have intentionally designed the tutorials to be widely accessible to all interested practitioners, regardless of their compute spending threshold; however, some of the provided notebooks will likely encounter memory issues without a minor investment in Colab Pro or comparably performant GPU resources.

Acknowledgments

The construction of this book would not have been possible without the tremendous efforts of many people. Firstly, we want to thank Springer, especially our editor, **Paul Drougas** and coordinator **Jacob Shmulewitz**, for working very closely with us and seeing this to fruition. We extend our gratitude to **Smarsh** for providing us the opportunity to tackle real-world multimodal, multilingual challenges and for fostering a culture of research and innovation that has significantly influenced our work here.

We want to extend our heartfelt thanks to (in alphabetical order) **Felipe Blanco, Shekar Gothoskar, Gaurav Harode, Dr. Sarang Kayande, Ankit Mittal, Sasi Mudigonda, Raj Pai, Gokul Patel, Sachin Phadnis, Dr. Ross Turner, Sameer Vajre, and Vedant Vajre** for their content feedback, suggestions and contributions, which have been instrumental in bringing this book together. Finally, we would like to express our sincere appreciation to the industry experts and researchers who have read, reviewed, and contributed to the foreword and reviews of this book. Your insights and expertise have been invaluable. Special thanks to **Ajit Jaokar, Shalini Govil Pai, Dr. Sherry Marcus, Prithvi Prabhu, Dr. Amarda Shehu, and Daniel Svonava**.

Declarations

Competing Interests The authors have no conflicts of interest to declare that are relevant to the content of this book.

Image and Table Reproduction All images reproduced or adapted from research papers in this book are created by the authors. While these images may not explicitly mention the original sources within the images themselves, the corresponding sections in the text provide appropriate citations to the original work. The same applies to all tables included in this book.

Ethics Approval This book does not include primary studies with human or animal participants; therefore, no ethics approval was required.

Notation

Calculus

$\approx$	Approximately equal to
$ \mathbf{A} $	L_1 norm of matrix $\mathbf{A}$
$\ \mathbf{A}\ $	L_2 norm of matrix $\mathbf{A}$
$\frac{da}{db}$	Derivative of a with respect to b
$\frac{\partial a}{\partial b}$	Partial derivative of a with respect to b
$\nabla_x Y$	Gradient of Y with respect to x
$\nabla_{\mathbf{X}} Y$	Matrix of derivatives of Y with respect to $\mathbf{X}$

Datasets

$\mathcal{D}$	Dataset, a set of examples and corresponding targets, $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$
$\mathcal{X}$	Space of all possible inputs
$\mathcal{Y}$	Space of all possible outputs
y_i	Target label for example i
$\hat{y}_i$	Predicted label for example i
$\mathcal{L}$	Log-likelihood loss
Ω	Learned parameters

Functions

$f : \mathbb{A} \rightarrow \mathbb{B}$	A function f that maps a value in the set $\mathbb{A}$ to set $\mathbb{B}$
$f(\mathbf{x}; \theta)$	A function of $\mathbf{x}$ parameterized by θ . This is frequently reduced to $f(\mathbf{x})$ for notational clarity.
$\log x$	Natural log of x
$\sigma(a)$	Logistic sigmoid, $\frac{1}{1+\exp -a}$
$\llbracket a \neq b \rrbracket$	A function that yields a 1 if the condition contained is true, otherwise it yields 0
${}_x f(x)$	Set of arguments that minimize $f(x)$, ${}_x f(x) = \{x \mid f(x) = \min_{x'} f(x')\}$
${}_x f(x)$	Set of arguments that maximize $f(x)$, ${}_x f(x) = \{x \mid f(x) = \max_{x'} f(x')\}$

Linear Algebra

a	Scalar value (integer or real)
-----	--------------------------------

$\begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix}$	Vector containing elements a_1 to a_n
$\begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{m,1} & \cdots & a_{m,n} \end{bmatrix}$	A matrix with m rows and n columns
$A_{i,j}$	Value of matrix $\mathbf{A}$ at row i and column j
$\mathbf{a}$	Vector (dimensions implied by context)
$\mathbf{A}$	Matrix (dimensions implied by context)
$\mathbf{A}^\top$	Transpose of matrix $\mathbf{A}$
$\mathbf{A}^{-1}$	Inverse of matrix $\mathbf{A}$
$\mathbf{I}$	Identity matrix (dimensionality implied by context)
$\mathbf{A} \cdot \mathbf{B}$	Dot product of matrices $\mathbf{A}$ and $\mathbf{B}$
$\mathbf{A} \times \mathbf{B}$	Cross product of matrices $\mathbf{A}$ and $\mathbf{B}$
$\mathbf{A} \circ \mathbf{B}$	Element-wise (Hadamard) product
$\mathbf{A} \otimes \mathbf{B}$	Kronecker product of matrices $\mathbf{A}$ and $\mathbf{B}$
$\mathbf{a}; \mathbf{b}$	Concatenation of vectors $\mathbf{a}$ and $\mathbf{b}$
Probability	
$\mathbb{E}$	Expected value
$P(A)$	Probability of event A
$X \sim \mathcal{N}(\mu, \sigma^2)$	Random variable X sampled from a Gaussian (Normal) distribution with μ mean and σ^2 variance.
Sets	
$\mathbb{A}$	A set
$\mathbb{R}$	Set of real numbers
$\mathbb{C}$	Set of complex numbers
$\emptyset$	Empty set
$\{a, b\}$	Set containing the elements a and b .
$\{1, 2, \dots, n\}$	Set containing all integers from 1 to n
$\{a_1, a_2, \dots, a_n\}$	Set containing n elements
$a \in \mathbb{A}$	Value a is a member of the set $\mathbb{A}$
$[a, b]$	Set of real values from a to b , including a and b
(a, b)	Set of real values from a to b , including a but excluding b
$a_{1:m}$	Set of elements $\{a_1, a_2, \dots, a_m\}$ (used for notational convenience)

Most of the chapters, unless otherwise specified, assume the notation given above.

Contents

- Notation xxiii
- 1 Large Language Models: An Introduction 1
 - 1.1 Introduction 1
 - 1.2 Natural Language 3
 - 1.3 NLP and Language Models Evolution 5
 - 1.3.1 Syntactic and Grammar-based methods: 1960s-1980s 5
 - 1.3.2 Expert Systems and Statistical Models: 1980s-2000s 6
 - 1.3.3 Neural Models and Dense Representations: 2000s-2010s ... 8
 - 1.3.4 The Deep Learning Revolution: 2010s-2020s 9
 - 1.4 The Era of Large Language Models 10
 - 1.4.1 A Brief History of LLM Evolution 10
 - 1.4.2 LLM Scale 12
 - 1.4.3 Emergent Abilities in LLMs 12
 - 1.5 Large Language Models in Practice 13
 - 1.5.1 Large Language Model Development 15
 - 1.5.2 Large Language Model Adaptation 17
 - 1.5.3 Large Language Model Utilization 20
 - References 24
- 2 Language Models Pre-training 29
 - 2.1 Encoder-Decoder Architecture 29
 - 2.1.1 Encoder 30
 - 2.1.2 Decoder 30
 - 2.1.3 Training and Optimization 31
 - 2.1.4 Issues with Encoder-Decoder Architectures 31
 - 2.2 Attention Mechanism 32
 - 2.2.1 Self-Attention 33
 - 2.3 Transformers 33
 - 2.3.1 Encoder 33
 - 2.3.2 Decoder 34

2.3.3	Tokenization and Representation	35
2.3.4	Positional Encodings	35
2.3.5	Multi-Head Attention	36
2.3.6	Position-Wise Feed-Forward Neural Networks	36
2.3.7	Layer Normalization	37
2.3.8	Masked Multi-Head Attention	38
2.3.9	Encoder-Decoder Attention	38
2.3.10	Transformer Variants	38
2.4	Data	43
2.4.1	Language Model Pre-Training Datasets	43
2.4.2	Data Pre-Processing	45
2.4.3	Effects of Data on LLMs	47
2.4.4	Task-Specific Datasets	48
2.5	Pre-trained LLM Design Choices	48
2.5.1	Pre-Training Methods	48
2.5.2	Pre-training Tasks	50
2.5.3	Architectures	55
2.5.4	LLM Pre-training Tips and Strategies	58
2.6	Commonly Used Pre-trained LLMs	61
2.6.1	BERT (Encoder)	61
2.6.2	T5 (Encoder-Decoder)	64
2.6.3	GPT (Decoder)	68
2.6.4	Mixtral 8x7B (Mixture of Experts)	73
2.7	Tutorial: Understanding LLMs and Pre-training	76
2.7.1	Overview	76
2.7.2	Experimental Design	76
2.7.3	Results and Analysis	77
2.7.4	Conclusion	78
	References	79
3	Prompt-based Learning	83
3.1	Introduction	83
3.1.1	Fully Supervised Learning	85
3.1.2	Pre-train and Fine-tune Learning	86
3.1.3	Prompt-based Learning	88
3.2	Basics of Prompt-based Learning	90
3.2.1	Prompt-based Learning: Formal Description	90
3.2.2	Prompt-based Learning Process	92
3.2.3	Prompt-based Knowledge Extraction	94
3.2.4	Prompt-based Learning Across NLP Tasks	96
3.3	Prompt Engineering	97
3.3.1	Prompt Shape	97
3.3.2	Manual Template Design	101
3.3.3	Automated Template Design: Discrete Search	103
3.3.4	Automated Template Design: Continuous Search	108

3.3.5	Prompt-based Fine-tuning	112
3.4	Answer engineering	113
3.4.1	Answer Shape	113
3.4.2	Defining the Answer Space	114
3.4.3	Manual Answer Mapping	115
3.4.4	Automated Answer Mapping: Discrete Search	115
3.4.5	Automated Answer Mapping: Continuous Search	115
3.5	Multi-Prompt Inference	116
3.5.1	Ensembling	116
3.5.2	In-context Learning	118
3.5.3	Prompt Decomposition	119
3.6	First Tutorial: Prompt vs. Pre-train and Fine-tune Methods in Text Classification and NER	120
3.6.1	Overview	120
3.6.2	Experimental Design	122
3.6.3	Results and Analysis	122
3.6.4	Conclusion	124
3.7	Second Tutorial: Approaches to Prompt Engineering	125
3.7.1	Overview	125
3.7.2	Experimental Design	126
3.7.3	Results and Analysis	126
3.7.4	Conclusion	129
	References	130
4	LLM Adaptation and Utilization	135
4.1	Introduction	135
4.2	Instruction Tuning	136
4.2.1	Instruction Tuning Procedure	137
4.2.2	Instruction Tuning Data	139
4.2.3	Instruction Tuning for Domain Adaptation	142
4.3	Parameter-Efficient Fine-Tuning	144
4.3.1	Adapters	147
4.3.2	Reparameterization	150
4.4	Compute-Efficient Fine-Tuning	157
4.4.1	LLM Quantization	158
4.5	End-User Prompting	162
4.5.1	Zero-Shot Prompting	163
4.5.2	Few-Shot Prompting	164
4.5.3	Prompt Chaining	164
4.5.4	Chain-of-Thought	166
4.5.5	Self-Consistency	167
4.5.6	Tree-of-Thoughts	167
4.6	Tutorial: Fine-Tuning LLMs in a Resource-Constrained Setting	168
4.6.1	Overview	168
4.6.2	Experimental Design	168

4.6.3	Results and Analysis	169
4.6.4	Conclusion	172
	References	172
5	Tuning for LLM Alignment	177
5.1	Alignment Tuning	178
5.1.1	Helpfulness	179
5.1.2	Honesty	181
5.1.3	Harmlessness	182
5.2	Foundation: The Reinforcement Learning Framework	184
5.3	Mapping the RL Framework to LLMs with Human Feedback	188
5.4	Evolution of RLHF	189
5.4.1	Safety, Quality, and Groundedness in LLMs	189
5.4.2	Deep Reinforcement Learning from Human Preferences	193
5.4.3	Learning Summarization from Human Feedback	196
5.4.4	Aligning LLMs to be Helpful, Honest, and Harmless with Human Feedback	200
5.5	Overcoming RLHF Challenges	206
5.5.1	Instilling Harmlessness with AI Feedback	206
5.5.2	Direct Preference Optimization	210
5.6	Tutorial: Making a Language Model More Helpful with RLHF	213
5.6.1	Overview	213
5.6.2	Experimental Design	214
5.6.3	Results and Analysis	215
5.6.4	Conclusion	217
	References	217
6	LLM Challenges and Solutions	219
6.1	Hallucination	219
6.1.1	Causes	220
6.1.2	Evaluation Metrics	221
6.1.3	Benchmarks	223
6.1.4	Mitigation Strategies	225
6.2	Bias and Fairness	229
6.2.1	Representational Harms	230
6.2.2	Allocational Harms	230
6.2.3	Causes	231
6.2.4	Evaluation Metrics	233
6.2.5	Benchmarks	237
6.2.6	Mitigation Strategies	238
6.3	Toxicity	247
6.3.1	Causes	247
6.3.2	Evaluation Metrics	248
6.3.3	Benchmarks	250
6.3.4	Mitigation Strategies	251

6.4	Privacy	256
6.4.1	Causes	256
6.4.2	Evaluation Metrics	257
6.4.3	Benchmarks	259
6.4.4	Mitigation Strategies	259
6.5	Tutorial: Measuring and Mitigating Bias in LLMs	262
6.5.1	Overview	262
6.5.2	Experimental Design	263
6.5.3	Results and Analysis	263
6.5.4	Conclusion	265
	References	265
7	Retrieval-Augmented Generation	275
7.1	Introduction	275
7.2	Basics of RAG	276
7.3	Optimizing RAG	279
7.4	Enhancing RAG	281
7.4.1	Data Sources and Embeddings	281
7.4.2	Querying	283
7.4.3	Retrieval and Generation	287
7.4.4	Summary	293
7.5	Evaluating RAG Applications	293
7.5.1	RAG Quality Metrics	294
7.5.2	Evaluation of RAG System Capabilities	301
7.5.3	Summarizing RAG Evaluation	306
7.6	Tutorial: Building Your Own Retrieval-Augmented Generation System	307
7.6.1	Overview	307
7.6.2	Experimental Design	308
7.6.3	Results and Analysis	308
7.6.4	Conclusion	311
	References	311
8	LLMs in Production	315
8.1	Introduction	315
8.2	LLM Applications	316
8.2.1	Conversational AI, chatbots and AI assistants	317
8.2.2	Content Creation	317
8.2.3	Search, Information Retrieval, and Recommendation Systems	318
8.2.4	Coding	318
8.2.5	Categories of LLMs	320
8.3	LLM Evaluation Metrics	322
8.3.1	Perplexity	323
8.3.2	BLEU	323
8.3.3	ROUGE	325

8.3.4	BERTScore	325
8.3.5	MoverScore	326
8.3.6	G-Eval	328
8.3.7	Pass@k	329
8.4	LLM Benchmark Datasets	330
8.5	LLM Selection	333
8.5.1	Open Source vs. Closed Source	335
8.5.2	Analytic Quality	335
8.5.3	Inference Latency	337
8.5.4	Costs	339
8.5.5	Adaptability and Maintenance	341
8.5.6	Data Security and Licensing	341
8.6	Tooling for Application Development	343
8.6.1	LLM Application Frameworks	343
8.6.2	LLM Customization	345
8.6.3	Vector Databases	346
8.6.4	Prompt Engineering	347
8.6.5	Evaluation and Testing	350
8.7	Inference	350
8.7.1	Model Hosting	350
8.7.2	Optimizing Performance	352
8.7.3	Optimizing Cost	354
8.8	LLMOps	354
8.8.1	LLMOps Tools and Methods	355
8.8.2	Accelerating the Iteration Cycle	358
8.8.3	Risk Management	359
8.9	Tutorial: Preparing Experimental Models for Production Deployment	362
8.9.1	Overview	362
8.9.2	Experimental Design	362
8.9.3	Results and Analysis	363
8.9.4	Conclusion	365
	References	365
9	Multimodal LLMs	375
9.1	Introduction	375
9.2	Brief History	376
9.3	Multimodal LLM Framework	377
9.3.1	Modality Encoder	378
9.3.2	Input Projector	379
9.3.3	Pre-training: Core LLMs, Datasets and Task-Specific Objectives	383
9.3.4	MMLLM Tuning and Enhancements	390
9.3.5	Multimodal RLHF	395
9.3.6	Output Projector	397
9.3.7	Modality Generator	398

9.4	Benchmarks	398
9.5	State-of-the-Art MLLMs	400
9.5.1	Flamingo (Image-Video-Text)	401
9.5.2	Video-LLaMA (Image-Video-Audio-Text)	405
9.5.3	NExT-GPT (Any-to-Any)	408
9.6	Tutorial: Fine-Tuning Multimodal Image-to-Text LLMs	411
9.6.1	Overview	411
9.6.2	Experimental Design	412
9.6.3	Results and Analysis	412
9.6.4	Conclusion	415
	References	415
10	LLMs: Evolution and New Frontiers	423
10.1	Introduction	423
10.2	LLM Evolution	424
10.2.1	Synthetic Data	424
10.2.2	Larger Context Windows	425
10.2.3	Training Speedups	425
10.2.4	Multi-Token Generation	426
10.2.5	Knowledge Distillation	427
10.2.6	Post-Attention Architectures	428
10.3	LLM Trends	428
10.3.1	Small Language Models	428
10.3.2	Democratization	430
10.3.3	Domain-Specific Language Models	430
10.4	New Frontiers	431
10.4.1	LLM Agents	431
10.4.2	LLM-Enhanced Search	433
10.5	Closing Remarks	435
	References	436
A	Deep Learning Basics	439
A.1	Basic Structure of Neural Networks	439
A.2	Perceptron	439
A.3	Multilayer Perceptron	440
A.3.1	Structure and Function of MLPs	441
A.3.2	Training MLPs	442
A.4	Deep Learning	442
A.4.1	Key Components of Deep Neural Networks	443
A.4.2	Activation Functions	443
A.4.3	Loss Functions	445
A.4.4	Optimization Techniques	446
A.4.5	Model Training	449
A.4.6	Regularization Techniques	449

B	Reinforcement Learning Basics	451
B.1	Markov Decision Process	451
B.1.1	Tasks	452
B.1.2	Rewards and Return	453
B.1.3	Policies and Value Functions	454
B.1.4	Optimality	456
B.2	Exploration/Exploitation Trade-off	457
B.3	Reinforcement Learning Algorithms	457
B.3.1	Q-Learning	459
B.3.2	Deep Q-Network (DQN)	460
B.3.3	Policy Gradient-based Methods	461
	Index	465

Selected Acronyms

AI Artificial Intelligence
BLEU Bilingual Evaluation Understudy
CBOW Continuous Bag-of-Words
CBS Categorical Bias Score
CLIP Contrastive Language-Image Pre-training
CNN Convolutional Neural Network
CoT Chain-of-Thought
CPT Continual Pre-Training
CUDA Compute Unified Device Architecture
DMN Dynamic Memory Network
DPO Direct Preference Optimization
EOS End-of-Sentence (token)
GeLU Gaussian Error Linear Unit
GLU Gated Linear Unit
GNN Graph Neural Network
GPT Generative Pre-trained Transformer
GPU Graphical Processing Unit
HHH/3H Helpful, Honest, and Harmless
ICL In-Context Learning
IT Instruction Tuning
ITG Image-Text Generation
ITM Image-Text Matching
KD Knowledge Distillation
KL Kullback-Leibler
LLM Large Language Model
LLMOps Large Language Model Operations
LM Language Model
LoRA Low-Rank Adaptation
LPBS Log-Probability Bias Score
LSTM Long Short-Term Memory
MDP Markov Decision Process

ME Modality Encoder
MM-IT Multimodal Instruction Tuning
MLM Masked Language Modeling
MLOps Machine Learning Operations
MM-COT Multimodal Chain-of-Thought
MM-ICL Multimodal In-Context Learning
MMLLM Multimodal Large Language Model
MoE Mixture of Experts
MRR Mean Reciprocal Rank
nDCG Normalized Discounted Cumulative Gain
NER Named-Entity Recognition
NLG Natural Language Generation
NLI Natural Language Inference
NLP Natural Language Processing
OCR Optical Character Recognition
PEFT Parameter-Efficient Fine-Tuning
PII Personally identifiable information
PLM Pre-trained Language Model
PPO Proximal Policy Optimization
PTFT Pre-Train and Fine-Tune
PTQ Post-training Quantization
PTS Pre-training From Scratch
QA Question Answering
QLoRA Quantized Low-Rank Adaptation
RAG Retrieval-Augmented Generation
RL Reinforcement Learning
RLHF Reinforcement Learning with Human Feedback
RNN Recurrent Neural Network
ROUGE Recall-Oriented Understudy for Gisting Evaluation
RRF Reciprocal Rank Fusion
SFT Supervised Fine-Tuning
SMoE Sparse Mixture of Experts Model
SOTA State of the Art
SQL Structured Query Language
SVD Singular Value Decomposition
TCO Total Cost of Ownership
TF-IDF Term Frequency/Inverse Document Frequency
TI Task Instructions
ToT Tree-of-Thoughts
TPU Tensor Processing Unit
TRPO Trust Region Policy Optimization
VeRA Vector-Based Random Matrix Adaptation
VLM Visual Linguistic Matching
VQA Visual Question Answering
VSM Video-Subtitle Matching



Chapter 1

Large Language Models: An Introduction

Abstract This chapter begins with a discussion of the historical context and progression of natural language processing. Beginning with the origins of human linguistic capabilities, this chapter explains the gradual transition to computational language modeling, emphasizing the importance of the intricate interplay between biology and technology. The evolution of language models in computational domains is presented in a coherent timeline, showcasing how rudimentary models transformed into the sophisticated LLMs that we are familiar with today. The critical factors influencing this transformative journey, including algorithmic advancements, computational power, and data availability, are discussed. LLMs are defined and delineated, ensuring that readers grasp their significance in contemporary AI paradigms. The chapter concludes with an overview of the subsequent chapters, enabling readers to anticipate the breadth and depth of topics covered throughout the book.

1.1 Introduction

In November 2022, the San Francisco-based tech company OpenAI announced the public release of ChatGPT, a web-based chatbot trained to respond to user queries. The subsequent publicity and viral attention around ChatGPT caused a global sensation, with the platform attracting more than 100 million monthly users by January 2023, making it the fastest-growing consumer app in history. Discussion of ChatGPT centered on the human-like quality of its text, the depth of its responses to technical questions on many subjects, its exceptional performance on standardized tests such as the GRE and LSAT, and its safety guardrails, which suppress responses to questions on controversial topics. Several competing chatbots appeared on the market within the next several months, including Microsoft's *Bing Chat* (February 2023), Google's *Bard* (March 2023), Anthropic's *Claude* (March 2023), and Baidu's *Ernie* (August 2023).

At around the same time, researchers achieved rapid advances in another type of generative model: text-to-image. These models take as input a description written by a human user and produces a digital image that conforms to the description. Starting in 2022, state-of-the-art (SOTA) text-to-image models reached photorealistic quality outputs of a seemingly endless variety of prompt subjects, with notable viral instances leaving many fooled ([Di Placido, 2023](#)). These programs further impressed by creating images of arbitrary topics that effectively copycat the styles of famous artists and art styles throughout history. Players in this space include OpenAI (*DALL-E 2*; April 2022), *Midjourney* (July 2022), and StabilityAI (*Stable Diffusion*; August 2022). Related AI applications creating buzz in 2022 and 2023 include the creation of synthetic speaking and singing voices imitating celebrities and doctoring live videos to alter the appearance or speech of individuals (*deep fakes*).

While differing in detail, each of these programs shares a common underlying technological basis – *Large Language Models* (LLMs). The explosion of this technology into the public consciousness has catalyzed a burst of investment in generative AI companies. The valuation of OpenAI skyrocketed to \$80 billion by February 2024, more than five-times its value in 2021. NVIDIA, a key manufacturer of the GPUs central to AI technology, saw its market capitalization increase by more than a factor of ten in the year and a half following October 2022, eventually surpassing Microsoft in June 2024 to become (at least temporarily) the largest company in the world by market cap. The global generative AI market, valued at \$8.2 billion in 2021, shot up to \$29 billion by the end of 2022 and is projected to top \$667 billion by 2030 ([Fortune, 2023](#); [Jin and Kruppa, 2023](#); [Valuates, 2023](#)). Financial markets and private investors anticipate monumental growth in this space over the next decade.

Along with these investments in AI has come public scrutiny. Discussion of the social, political, and existential risks, economic implications, ethics, and long-term consequences of LLM-based AI has become commonplace. Artists worry about AI-generated art and text intruding on their domains. Educators ponder whether college admission essays can be trusted as authentic in a world with ChatGPT. Governments worldwide have considered regulations on AI research, and the legality of training LLMs on scraped internet data is being adjudicated in numerous lawsuits. In short, the public discussion of AI has been revolutionized in every dimension in just two years. Although specialists in AI who were up to speed on recent developments may not have been surprised at the achievements of these platforms, they were the wider public's first taste of the revolution occurring in machine learning over the last decade. Indeed, 2022 can be said to mark the beginning of the global era of large language models.

What are these large language models? How have they developed such astonishing capabilities? What underlies their ability to acutely absorb, process, and deploy natural language? And how have the past decades of machine learning research primed LLMs for their big debut on the world stage? To answer these questions, it is helpful to first to step back and consider the fundamental system at the root of their power – human language.

1.2 Natural Language

Natural language is unique among modes of communication in animals. No other form of communication – the process of exchanging thoughts, feelings, ideas, or information among individuals – is more expressive, efficient, and abstract than human natural language. At its core, language encodes meaning through systems of symbols, gestures, and sounds combined in complex ways to encode that meaning precisely. Indeed, humanity's ability to precisely communicate meaning about the world is considered one of the most influential factors in our ecological dominance on Earth. With language, meaning is no longer limited to the individual – it can be transferred from one individual to many others, allowing highly useful information to spread quickly to the benefit of its lucky recipients.

It is easy to imagine how natural language communication would have been a game changer for early human populations. Humans originated from social ancestors, and one thing about social species is that communication is necessarily at the heart of their evolutionary success. Try to imagine existing within a community of other people where linguistic communication was absent. How efficiently could disputes be resolved, the location of mutually beneficial resources be shared, or the direction from which threats to the community are coming be communicated in the absence of natural language? Not impossible, as every other species on the planet uses modes of communication less expressive than natural language, but our adaptive potential within our environment would be severely limited, resulting in humanity being a very different beast, for better or worse, than we are today.

Effectively, language facilitates new dimensions of behavioral adaptability to the environment around us. As it turns out, this is a game changer concerning whether your species is locked into a rigid life history (i.e., an ecological specialist) or a life history that is much more flexible (i.e., an ecological generalist). For example, consider Salmonids, a family of fish made up of many species of salmon and trout. This family is characterized by its distinctive life history, whereby juveniles of the species typically develop within natal rivers and streams, and once they reach a threshold age and size, they migrate staggering distances to feed in environments richer in resources to allow them to grow into adults. These adults then migrate back to their natal environment when they have reached reproductive age to spawn, thus setting the cycle in motion for the next generation of the species. This life history is virtually invariant for individuals in this family because doing anything other than what your parents did as a salmon or a trout is a precarious business.

On the other hand, humans have many life histories, with individual survivability remaining more or less unaffected. So, how does natural language enable this remarkable ability within our species? The answer is surprisingly simple. Communication of meaning in precise ways allows us to learn information about the world, not simply through just-in-time, first-hand experience of events and facts that the information encodes, but indirectly and independently of our spatio-temporal proximity to those events! A profound capability to have evolved.

The conceptualization of the significance of the things we experience during our lives is foundational to interacting beneficially (in the evolutionary sense) with the

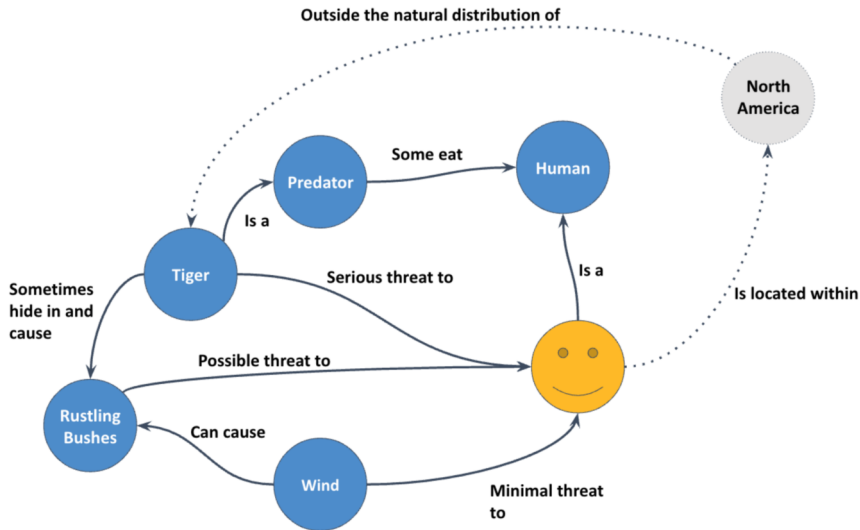


Fig. 1.1: A knowledge graph representation of information with potential significance or meaning to a human (self), concerning the threat posed by a rustling bush observed by the “self”. Solid edges and solid borders on entity nodes represent the self’s current knowledge. In contrast, dotted edges and borders on entity nodes represent potential new knowledge/meaning that the self could learn. Learned information can allow the self to make extremely useful inferences about their personal risk posture in the environment. A language must be capable of precisely encoding this complexity to maximize real-world utility while minimizing potentially lethal misunderstandings.

world. To use the typical example of the significance of a rustling bush to a human, where the cause of the rustling is unknown, Fig. 1.1 illustrates how a subset of the knowledge relevant to this phenomenon might be encoded within the brain of a human individual. Multiple meanings can be derived from the observation that a bush in close proximity to the “self” is rustling. One might dismiss the observation as being innocuous concerning survival if their chain of reasoning concludes that something other than a survival threat is the cause. On the other hand, based only on the entities and their relationships encoded in the knowledge graph in Fig. 1.1, if there is insufficient wind to cause the magnitude of rustling observed, then it might be safer to conclude that a tiger is the cause.

Obviously, having the ability to reason about the world in this way is not necessarily contingent on natural language of the human variety, since many other animals appear to exhibit similar high-functioning cognitive abilities. But what language enables is the scaling of functional units of meaning to any other individual capable of decoding the information encoded within it. So rather than meaning that improves survival being limited by the need for first-hand experience or low-capacity and im-

precise communication modes, human language allows us to scale out knowledge and meaning about the world in unprecedented ways in the animal kingdom. This ability is fundamental to our species' ecological success.

To accomplish this difficult task, human language systems have developed great complexity. Not only do these languages have to encode rigid facts about the world, but they also have to be able to embed these facts into different conceptual contexts that often alter their meaning or significance to the powerful brain within which they are being cognitively manipulated. A language that does this insufficiently will do a poor job of enabling communication of information with the kind of precision needed to allow the recipient to operationalize it in valuable ways. Until very recently, only humans possessed the sophisticated hardware (or wetware if you like) required to both create and utilize human language. But within the domain of artificial intelligence, the subdomains of machine learning and natural language processing have contributed to the emerging revolution in human language understanding and generation, which have culminated in the form of LLMs. Next we will overview this historical process, before examining closer the capabilities of LLM.

1.3 NLP and Language Models Evolution

In the realm of computer science, natural language processing (NLP) stands as a pivotal discipline focused on facilitating interactions between machines and human language. The field of NLP aims to formulate algorithms and techniques that empower computers to comprehend and interpret human language through natural language understanding (NLU) and generate human-like text via natural language generation (NLG). As highlighted previously, the profound intricacy of human language is undeniable; however, the escalating demand for algorithms proficient in linguistic understanding reinforces the significance of NLP. Historically, NLP strategies have adopted a linguistic-centric paradigm, rooting their analyses in foundational semantic and syntactic constituents, such as parts of speech. However, contemporary deep learning methodologies might obviate the necessity for such intermediary components, potentially crafting their distinct representations for broad tasks. This section will offer a concise synopsis of seminal studies that have shaped the trajectory of NLP, particularly concerning language models, as shown in Fig. 1.2.

1.3.1 Syntactic and Grammar-based methods: 1960s-1980s

While the 1940s witnessed preliminary explorations in the domain, the 1954 IBM-Georgetown experiment, which demonstrated the machine translation of approximately 60 sentences from Russian to English, stands out as a significant landmark in the field (Hutchins et al., 1955). In the late 1950s, seminal contributions transformed the landscape of language understanding. A pivotal moment in linguistic research oc-

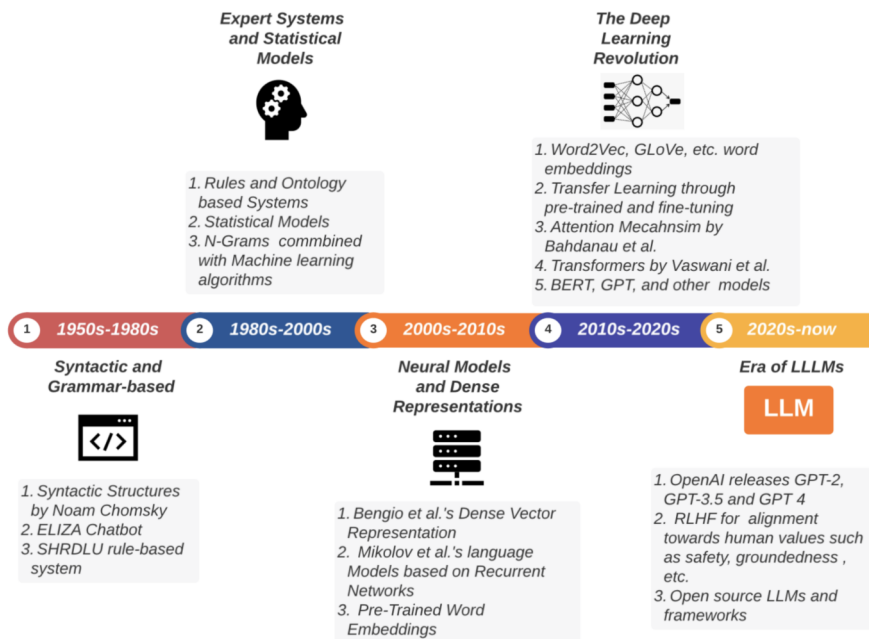


Fig. 1.2: Timeline illustrating the progression of NLP and LLM from the 1950s to the present, highlighting major events and breakthroughs.

curred in 1957 when Noam Chomsky introduced his work, *Syntactic Structures*. This publication underscored the crucial role of sentence syntax in the comprehension of language (Chomsky, 1957). Concurrently, the emergence of the phase-structure grammar further influenced linguistic and computational perspectives during this period. Furthermore, advancements in artificial intelligence were marked by endeavors such as the creation of LISP by John McCarthy in 1958 and the development of ELIZA, recognized as the inaugural chatbot. These achievements have shaped the evolution of NLP and left an indelible mark on the broader realm of artificial intelligence. SHRDLU emerged as a rudimentary system proficient in discerning basic queries and responses by integrating syntax, semantics, and reasoning. Systems of this era, exemplified by ELIZA (1966) and SHRDLU (1970), predominantly hinged on predetermined lexicons and rulesets for language generation and comprehension.

1.3.2 Expert Systems and Statistical Models: 1980s-2000s

During the early 1980s, NLP predominantly employed symbolic methodologies, often called expert systems. These systems were characterized by manually established rules and ontologies, which essentially served as structured knowledge repositories

detailing facts, concepts, and their interconnections within a particular domain. By the late 1980s, the limitations of symbolic AI and its inability to scale and handle ambiguities in natural language became apparent. This realization led to a gradual transition toward statistical methods. The idea was simple: use data to learn patterns rather than trying to hardcode every possible rule. Essential linguistic repositories, encompassing annotated compilations like the Penn TreeBank, British National Corpus, Prague Dependency Treebank, and WordNet, have proven invaluable for both academic research and commercial ventures (Hajicová et al., 1999; Marcus et al., 1994; Miller, 1995). Hidden Markov models (HMMs), introduced in the 1980s, represented one of the first successful applications of statistical methods to language. They treated language as a series of states and transitions and found extensive use in early NLP, especially in speech recognition systems.

N-gram models became the foundation of statistical language modeling, representing a fundamental approach to capturing the sequential nature of language. N-gram models operate on the principle of conditional probabilities. The core idea is to estimate the likelihood of a word based on the history of $n - 1$ preceding words. This is represented as:

$$P(w_n|w_{n-1}, w_{n-2}, \dots, w_1)$$

However, directly estimating this probability for large values of n can be computationally intensive and prone to data sparsity issues. Thus, n-gram models often make the Markov assumption, which simplifies the history to just the last $n - 1$ words:

$$P(w_n|w_{n-1}, w_{n-2}, \dots, w_1) \approx P(w_n|w_{n-1}, w_{n-2}, \dots, w_{n-(n-1)})$$

For a *unigram model* ($n=1$), the probability of a word is estimated independently of any preceding words. This is represented as:

$$P(w_n)$$

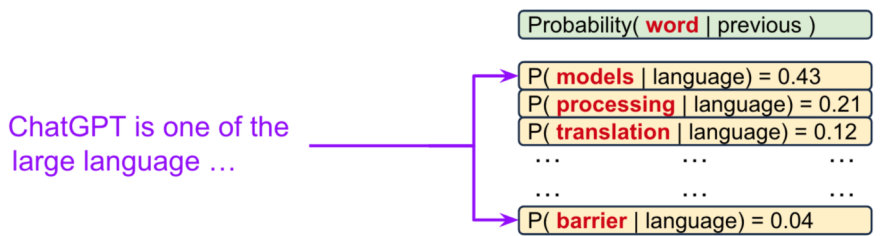


Fig. 1.3: This figure illustrates the process of sentence completion using a hypothetical bigram model with the partial sentence “ChatGPT is one of the large language”. Each yellow cell displays the conditional probability of a word that may follow “language” based on the bigram probabilities calculated from the training corpus.

In this case, each word is assumed to be generated independently, and the probability associated with the word is its frequency in the corpus divided by the total number of words. For a *bigram model* ($n=2$), the probability of a word is conditioned on the immediately preceding word:

$$P(w_n|w_{n-1})$$

For a *trigram model* ($n=3$), the probability of a word is conditioned on the two immediately preceding words:

$$P(w_n|w_{n-1}, w_{n-2})$$

Fig. 1.3 shows an illustration of this probabilistic inference process at work. Given a sequence of words, the model predicts the likelihood of possible subsequent words based on its parameters – which themselves have been learned from training data consisting of natural language sentences – and produces a rank-ordering by probability of the likeliest continuations. **Models that operate in this probabilistic framework are what we refer to as language models (LMs) or pre-trained language models (PLMs).**

A majority of NLP undertakings traditionally leveraged methodologies such as n -grams integrated with machine learning techniques, including multinomial logistic regression, support vector machines, Bayesian networks, and the expectation-maximization algorithm, to address a large number of NLP tasks in both supervised and unsupervised settings (Brown et al., 1992; Manning and Schütze, 1999).

1.3.3 Neural Models and Dense Representations: 2000s-2010s

In the early 21st century, seminal research by Bengio et al. (2000) led to the first-ever neural language model. This model employs a lookup table to map n preceding words and feeds them through a feed-forward network with hidden layers, the output of which is smoothed into a softmax layer to predict the subsequent word. Significantly, this research marked a departure from traditional n -grams or bag-of-words models, instead introducing “dense vector representation” into the annals of NLP. Subsequent language models, harnessing recurrent neural networks and long short-term memory architectures, have emerged as leading-edge solutions in the field (Graves, 2013; Mikolov et al., 2010). Collobert and Weston (2008) produced research of paramount significance, providing an early glimpse at concepts like pre-trained word embeddings and the adoption of convolutional neural networks for textual analysis. Additionally, their contribution emphasized the utility of the lookup table, now known as the embedding matrix, in multitask learning.

1.3.4 The Deep Learning Revolution: 2010s-2020s

The advancements in word embedding technology were notably propelled by the contributions of Mikolov et al. (2013a,b), who refined the training efficiency of embeddings originally conceived by Bengio et al. (2000). By eliminating the hidden layer and adopting an approximate learning objective, they introduced “word2vec”, a large-scale, efficient implementation of word embeddings. This framework comes in two variants: the continuous bag-of-words (CBOW), which forecasts a central word based on surrounding words, and the skip-gram, which conversely predicts adjacent words. The efficiency gains realized from extensive corpus training allowed these dense embeddings to encapsulate diverse semantic relationships. Utilizing such word embeddings as initial representations, followed by their pre-training on expansive datasets, has become a cornerstone methodology in neural-based architectures.

Fig. 1.4 visually represents the word2vec process, highlighting the transformation of words into high-dimensional embeddings. Starting with individual words, they are first mapped to their respective embeddings in a high-dimensional space and visually represented in the lower 2D plane.

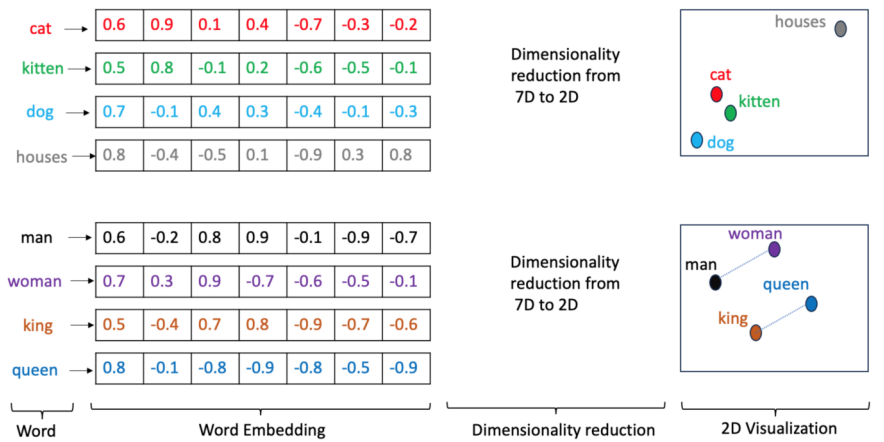


Fig. 1.4: This illustrative example highlights the relationship between the representation of words and their similarity in semantic space. The vector from “man” to “woman” closely mirrors the vector from “king” to “queen”. This arrangement signifies that the relationship (or difference) between “man” and “woman” is similar to that between “king” and “queen” in this embedded space. Additionally, the word “dog” appears closely positioned to “cat” and “kitten.” Conversely, the word “houses” is distinctly separated from these animal words.

Another pivotal development in neural architectures for NLP and speech processing was the attention mechanism, introduced by Bahdanau et al. (2014). Subsequently, Vaswani et al. (2017) introduced the Transformer architecture in 2017,

constituting a paradigm shift in language models. Employing attention mechanisms, the Transformer architecture permitted models to selectively concentrate on various segments of input data based on their relevance to the tasks, thereby significantly enhancing the performance across multiple NLP tasks. This foundational architecture paved the way for subsequent models such as BERT and GPT-1. Introduced by Google in 2018, BERT represented a landmark achievement in transfer learning within the NLP sphere. This LM underwent initial training on extensive text corpora and was fine-tuned for specific tasks. This approach of initially generalizing the model, followed by task-specific fine-tuning, became a prevalent practice in NLP.

1.4 The Era of Large Language Models

Following the deep learning revolution the impressive achievements of small language models developed with the Transformer architecture, the pieces were in place for the emergence of LLMs. The promise of this new class of language models is so clearly evident that they have driven a paradigm shift in how machine learning practitioners aim to solve common NLP problems. From text classification to named entity recognition (NER), long-standing language tasks are being reformulated as text generation tasks by development and research teams around the world to take advantage of the unprecedented language understanding and text generation capabilities of LLMs (Zhao et al., 2023).

! Practical Tips

From an model architecture perspective, LLMs are primarily distinguished from smaller Transformer-based LMs or PLMs by their number of parameters. There is no canonical quantity that distinguishes LLMs from smaller language models, but typically LLMs have hundreds of millions to trillions of parameters. The earliest Transformer-based models such as GPT-1 and BERT can be considered the first generation of LLMs, while models designed to be smaller (typically for use in low-compute situations) can be considered small PLMs.

1.4.1 A Brief History of LLM Evolution

The release of ChatGPT in November of 2022 is undoubtedly a pivotal moment in LLM research since it represents the first time that an LLM's capabilities so ubiquitously captured the public's imagination. However this event is far from the full story of how we ended up where we are today. In this section, we will take a look at other pivotal events, both before and since the release of ChatGPT, that have contributed to LLM development.

Following the invention of the *attention* mechanism and the *Transformer* architecture, the first major innovation that started to show glimmers of the promise of contemporary LLMs occurred with the release of T5 (Raffel et al., 2020). T5 is a text-to-text language model built entirely without any recurrence or convolution network structures, instead leveraging only Transformer attention networks. T5, thanks to its scale (11 billion parameters at the time), and the scale of the pre-training corpus (1 trillion tokens) demonstrated SOTA performance in multiple text-to-text translation tasks. Additionally, T5 was published with various evaluations across a diverse range of NLP tasks reformulated as text-to-text problems, showing impressive performance. Shortly after Google released the T5 model, OpenAI released an updated version of their own language models, GPT-1 and GPT-2, in the form of GPT-3 (Brown et al., 2020). GPT-3 was shown to have impressive generalization capabilities, including remarkable *in-context learning* abilities (see Sect. 1.5.3.1), all from unsupervised pre-training without the need for additional task-specific fine-tuning.

Several important events occurred between the early summer of 2020, when GPT-3 was released, and November 2022, when ChatGPT was released. One key milestone was the release of Anthropic’s 52 billion parameter model, which for the first time developed approaches to better align LLM responses to human values (Askell et al., 2021). Before this work, it was clear to users of models like GPT-3 that they had a fundamental limitation: their responses were often misaligned with human-level values. This work demonstrated the promise of alignment tuning for making LLMs more human-like in their conversation behavior. Shortly thereafter, OpenAI released another innovative model, InstructGPT (Ouyang et al., 2022). InstructGPT is a fine-tuned variant of the GPT-3 family of models, that is tuned using human feedback during an alignment phase. After this alignment, InstructGPT showed impressive abilities in responding in ways that human evaluators preferred, even when comparing the 1.3 billion parameter InstructGPT variant to the 175 billion parameter GPT-3 models. This demonstrated that although scaling model parameters was necessary for LLMs to exhibit some of the most useful emergent abilities, fine-tuning the language model is another path to achieve superior performance in multiple tasks. The groundwork had been laid for the creation of ChatGPT.

Since ChatGPT was released in November 2022, OpenAI has released GPT-4 (OpenAI, 2023), which demonstrates even more impressive capabilities than the original version of ChatGPT in both standard NLP and NLU benchmarks. It also shows markedly improved results on human tests such as the bar exam (Martínez, 2023)), as well as new capabilities in understanding relationships between images and text. In addition to the impressive contributions of OpenAI, other notable LLMs available today include Llama-2 from Meta (Touvron et al., 2023), PaLM from the Google Research team¹, and Claude-2 from Anthropic².

Readers should fully expect the innovation in LLM research and the resulting LLM applications to continue to evolve. These technologies provide unprecedented human-machine interaction opportunities and represent one of the single most

¹ <https://blog.google/technology/ai/google-palm-2-ai-large-language-model>

² <https://www.anthropic.com/index/claude-2>

promising avenues through which human intentions and goals can be scaled through the use of computation. Be it in more efficiently and comprehensively helping to solve traditional NLP problems or opening up avenues for unprecedented applications, we, the authors, are excited to be on this journey with the reader as we delve into this fascinating space together.

1.4.2 LLM Scale

Three core scale factors contribute to any large language model;

1. **Pre-training corpus scale**, which defines the breadth and depth of knowledge trained into the model
2. **Number of learned parameters**, which determines the complexity of the learned states.
3. **Computational Scale**, which marks the tractability of training and running inference with a given architecture.

Much work has been done to understand how the scaling of these three factors contributes to what has become known as *emergent ability*, which is effectively the emergence of competencies that the LLM was not explicitly trained on during pre-training or any subsequent fine-tuning (Hoffmann et al., 2022; Wei et al., 2022).

1.4.3 Emergent Abilities in LLMs

Before diving into what *emergent abilities* are, it is helpful to clarify what they are not. Specifically, early attempts that resulted in larger and larger-scale models did indeed bear fruit in many NLP problems. For example, consider BERT’s performance on the entity-relation classification task (Soares et al., 2019). In applications of this sort, smaller pre-trained language models (PLMs) achieve remarkable performance. However, such performance gains typically occur due to the fine-tuning process, wherein a carefully crafted objective is engineered, significant effort is invested in curating a dataset that encapsulates this objective (e.g., labeled examples or question-answer pairs), and additional tuning of the PLM’s parameter space is carried out. In this context, the valuable aspects of the model’s performance are explicitly taught.

On the other hand, emergent abilities occur without the need for these additional fine-tuning steps or even having to explicitly teach them to the model. That is to say that LLMs with emergent abilities can “learn” to solve such problems without modifying the pre-trained model’s weights at all (Wei et al., 2022). Instead sufficiently large LLMs, trained on sufficiently comprehensive corpora with sufficiently large computational budgets, begin to exhibit high competency, both in specific NLP

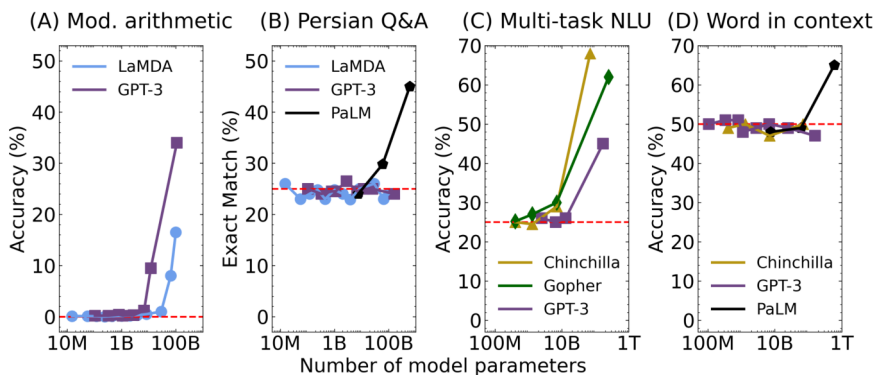


Fig. 1.5: Emergent abilities arising in large language models of various scales. Model scale, as measured in a number of model parameters, is represented by the x -axis, while task accuracy is represented by the y -axis. Five LLMs, LaMDA (Thoppilan et al., 2022), GPT-3 (Brown et al., 2020), Gopher (Rae et al., 2022), Chinchilla (Hoffmann et al., 2022) and PaLM (Chowdhery et al., 2022) are evaluated for their performance on 4 wide-ranging NLP tasks in a few-shot setting; A and B are benchmarks from the BIG-Bench suite (Srivastava et al., 2023), namely, A) tests 3-digit addition/subtraction and 2-digit multiplication ability; B) tests question-answering in the Persian language. C) is the combine performance across 57 wide-ranging tasks, and D) tests for semantic understanding. A clear trend of emergence in these abilities is seen for at least one LLM in each task.

tasks and higher-level abilities such as language understanding, arithmetic, and multistep reasoning (Radford et al., 2019; Wei et al., 2022). Fig. 1.5 shows the effects on accuracy for four different NLP tasks due to LLM scaling. In each instance, smaller language models do no better than random at the task, but at least one begins to greatly exceed random above a given parameter scale. The emergence of these abilities in LLMs could not have been anticipated *a priori* based on the performance of LLMs with fewer parameters, as indicated by the often sharp increase in accuracy in Fig. 1.5, partially reproduced from Wei et al. (2022). How or why emergent abilities arise in LLMs is an active area of research.

1.5 Large Language Models in Practice

As with any new technology, there is a strong overlap between the research and application phases of LLM evolution toward maturity and, eventually, ubiquity. As a result, navigating the most valuable or useful research literature or adopting the most suitable methodology for a given application can be daunting. To assist the reader in this task, the following sections aim to introduce structure to LLMs in practice.

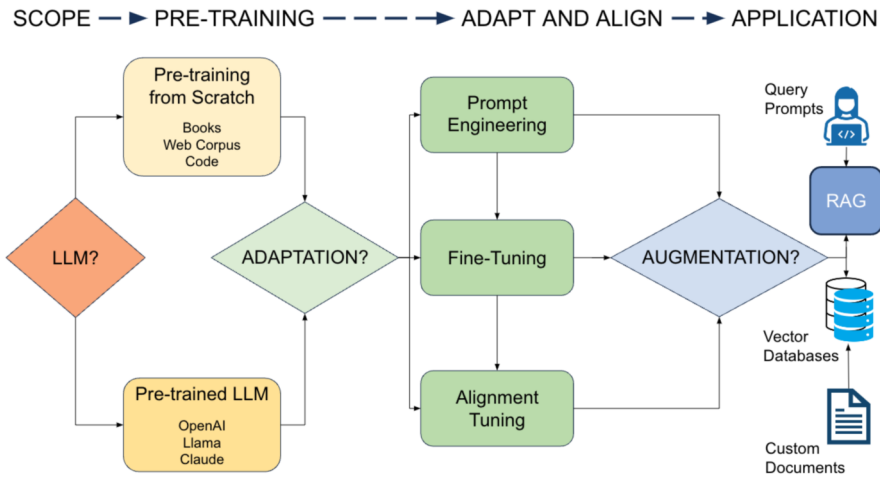


Fig. 1.6: A high-level view of the various paths to consider when planning to develop, adapt, or utilize large language models in an application context.

Generally speaking, there are three core areas of practical concern in the context of LLMs:

- **Development** focuses on how to build an LLM from scratch. This area encompasses pre-training from the perspectives of datasets, learning objectives, and architectures used to develop LLMs.
- **Adaptation** focuses on how pre-trained LLMs can be modified to exhibit more desirable outcomes. Often, these outcomes are measured in the context of the LLM application, which also influences the approaches used to adapt them. Topics such as instruction tuning, alignment, and fine-tuning are important here.
- **Utilization** focuses on how to interact with LLMs, in order to leverage their valuable capabilities. Here, the topics of end-user prompting and application development are key.

In this section, we begin with an overview of LLM development, which is subsequently covered in deeper detail in Chapter 2. Next, we explore LLM adaptation by giving overviews of the most commonly used approaches, including alignment tuning and parameter-efficient fine-tuning (PEFT) – we will expand on these subjects in Chapter 4 and Chapter 5. Lastly, we look at end-user prompting and applications in the context of LLM utilization. These topics are also covered in deeper detail later, with the most relevant coverage in Chapter 6, Chapter 7 and Chapter 8. Let us explore the core concepts in each.

1.5.1 Large Language Model Development

Although not expected to be a particularly common development pathway, owing to the complexity and upfront costs associated with doing so, pre-training one's own LLM from scratch or adaptively pre-training an existing model is possible. Motivations for taking this path, as opposed to leveraging one of the many open-source or closed-source pre-trained LLMs available, might include the need for the LLM to have specialized domain knowledge, having more explicit control over the biases, capabilities or limitations of the LLM, or lowering the total cost of ownership on a long-term basis. This section introduces the key factors associated with these approaches.

1.5.1.1 Large Language Model Pre-training

Much of the impressive capabilities of LLMs emerge as a result of the pre-training process, which enables them to learn fundamental language skills and knowledge from large-scale datasets (Zhao et al., 2023). Here, we review three key aspects of pre-training: commonly used *datasets*, commonly used *network architectures*, and commonly used *learning objectives*.

- **Datasets:** Unlike the learning objectives and network architectures used to develop LLMs, the datasets used do not vary as much from model to model. Typically, general purpose LLMs are trained on some combination of *web-scraping* (e.g., Common Crawl), *Wikipedia*, *Book corpora* and *code*, the latter of which has been shown to significantly improve model reasoning capabilities (Ma et al., 2023). These diverse content sources are further curated to standardize quality and minimize the negative impacts of attributes such as duplication and harmful or hateful content. Once datasets are deemed sufficient in coverage and quality, they are normalized and tokenized according to the preference of the development team. Once these steps are complete, the data are ready to begin the pre-training process. All of this information is covered in greater detail in Sect. 2.4.
- **Network Architectures:** The most common architecture used for LLMs is the Transformer, which was introduced in the seminal paper “Attention is All You Need” (Vaswani et al., 2017). This original architecture consists of an encoder and a decoder, both of which are built using self-attention mechanisms and feed-forward neural networks. While Sect. 2.5.3 provides detailed break-downs of the various ways the original Transformer architecture has been innovated upon, here we highlight some of the most influential models and their architectural innovations to give the reader a sense of the architectural-capability association:
 - The **encoder-only architecture** is leveraged to enable contextual representation of the input sequence. These contextual representations are valuable in many NLP tasks, such as classification and named entity recognition. A

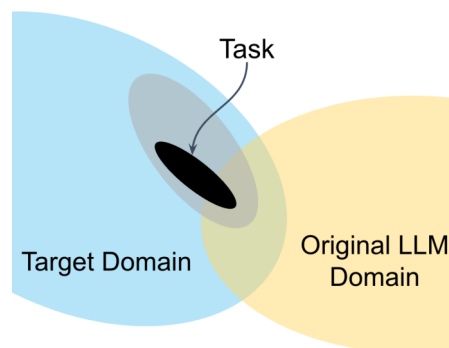
popular model that leverages this architectural design is BERT (Devlin et al., 2019).

- The **encoder-decoder architecture** is leveraged for sequence-to-sequence tasks such as machine translation or question-answering. A popular model built using this architecture is T5 (Raffel et al., 2020).
 - The **causal decoder architecture** is used when the learning objective is autoregressive sequence generation. Sequence generation is achieved by unidirectionally constraining the attention mechanism. Models built using this architecture are adept at text generation tasks, with the GPT series of models being the most familiar (e.g. Brown et al., 2020).
 - The **prefix-decoder architecture** is also known as the non-causal decoder architecture and is a variant of the causal decoder discussed above, with the key difference being the bidirectional attention mechanism applied to the input sequence (i.e., the prefix). Attention is still unidirectional on the generated sequence, and generation is still autoregressive. A popular model leveraging this architecture is Google’s PaLM, which is particularly adept at tasks where bidirectional encoding is beneficial, such as machine translation (Chowdhery et al., 2022).
- **Learning objectives:** The learning objectives used in pre-training strongly influences the resulting LLM’s emergent capabilities. Generally, the objectives used in pre-training aim to maximize natural language understanding and coherent generative capabilities. The most common approach to achieve these capabilities is *full language modeling*, which involves autoregressively predicting the next token in a sequence given preceding tokens (Zhao et al., 2023). Other important learning objectives used include *denoising autoencoding*, which leverages a strategy of corrupting input sequences of text and training the network to recover the corrupted spans (Raffel et al., 2020), and *mixture-of-denoisers*, which aims to leverage three core pre-training tasks through a mixture of denoisers specializing in a) standard language modeling, b) short-span, low noise recovery and c) long-span and/or high noise recovery (Chowdhery et al., 2022). The former was leveraged for the popular T5 sequence-to-sequence LLM, while the latter was leveraged for Google’s PaLM LLM.

1.5.1.2 Adaptive Pre-training of LLMs

While pre-training of LLMs is typically done using general purpose datasets, such as those discussed in the previous section, domain-adaptive pre-training leverages more domain-specific datasets to further train the LLM (Gururangan et al., 2020). The objective of adaptive pre-training is to better align the LLM’s capabilities to domains where there is specialized vocabulary or language usage. For example, there is much technical language in the biological domain, including the use of Latin nomenclature for species names or anatomical descriptions. An LLM trained only on general purpose datasets may not have sufficient knowledge of this biological terminology,

Fig. 1.7 An illustration of the relationships between the data distributions associated with general purpose LLM pre-training (Original LLM Domain), domain-adaptive pre-training (Target Domain), and task fine-tuning (Task). The light-gray area within which the task distribution exists highlights that the task is typically an observable subset of a larger distribution.



and so adapting the base model with a dataset such as *bioRxiv*, a specialized preprint server for biological scientific literature, can overcome these limitations (Zhang et al., 2024a).

Adaptive pre-training has also been shown to significantly improve the effectiveness of task fine-tuning (Gururangan et al., 2020), such as domain-specific topic classification, as illustrated in Fig. 1.7. This effect has been observed across multiple domains, with BloombergGPT being a well-known example in the financial services industry. This LLM was adaptively pre-trained on a mixture of financial domain data such as news articles and financial reports, as well as Bloomberg’s proprietary data. Despite making up only 0.7% of the total pre-training data used, significant performance improvements were seen across multiple tasks in the financial domain, relative to other general purpose LLMs with three times as many model parameters as BloombergGPT (Wu et al., 2023).

Both general purpose pre-training and domain-adaptive pre-training have their benefits when suitable datasets, sufficient computing power, and a substantial budget are available. Building LLMs from scratch like this can have significant advantages with respect to control over outcomes, performance, or privacy, but only if the upfront costs of doing so can be absorbed. This is unlikely to be the case for the majority of development teams. As such, the next section introduces the reader to adaptation concepts that are more aligned with fine-tuning instead of pre-training, and typically involve both datasets and compute costs that are a fraction of those required for the pre-training approaches discussed.

1.5.2 Large Language Model Adaptation

Of course, the emergent abilities of LLMs are remarkable in and of themselves, but it is not guaranteed that the out-of-box performance exhibited by an LLM on a given task will be sufficient for every use case, even after adaptive pre-training. As such, research and innovation around tools, techniques, and procedures for further tuning of LLMs in the direction of a given outcome has rapidly produced many options for de-

velopers. Some of these approaches focus on traditional NLP task fine-tuning, such as classification or NER. In contrast, others focus on stylistically aligning the generated text to the value-based aesthetic preferences of the developers. In either case, the key distinction from the pre-training approaches is the use of labeled training datasets. This section explores four key areas of supervised adaptation: Instruction tuning, alignment tuning, full-parameter fine-tuning, and parameter-efficient fine-tuning.

1.5.2.1 Instruction Tuning

Instruction tuning (IT) is a fine-tuning technique for enhancing the capabilities and controllability of LLMs. The core objective of IT involves fine-tuning a general purpose LLM to more accurately follow the specific instructions provided by users (Zhang et al., 2024b). This is accomplished by training the LLM on labeled datasets formatted as (instruction, output) pairs in a supervised fashion. Tasks such as code generation, summarization, question-answering, and task planning/execution can be formulated as IT data, enabling developers to improve instruction following in the context of those tasks.

Typically, IT is achieved by either full-parameter or parameter-efficient fine-tuning, wherein the learning process enables the LLM to better associate instructions provided to the desired outputs as specified in the dataset pairs, resulting in responses that are better aligned with human instructions. As one would expect, full-parameter instruction tuning can be costly, especially when large IT datasets are used along with very large LLMs. As such, parameter-efficient fine-tuning approaches like LoRA (Hu et al., 2021), or the IT specific approach HINT (Iverson et al., 2023), have emerged as viable methods that enable better trade-offs between the scale of the IT dataset used and the cost of fine-tuning.

1.5.2.2 Alignment Tuning

In contrast with instruction tuning, which aims to fine-tune LLMs to follow specific human instructions, alignment tuning aims to more globally “align” the LLM’s outputs to human preferences and values (Lin et al., 2023). Similar to instruction tuning, alignment tuning is a supervised fine-tuning technique that depends heavily on human annotators who are tasked with ranking LLM responses according to their alignment to a pre-defined set of preferences or values. This technique involves inherently subjective determinations on behalf of the annotators, especially if they originate from a diverse set of cultural or social backgrounds. This can be a significant challenge in the context of providing consistent alignment feedback for the fine-tuning process, and thus it is important to adopt a clear definition of the human values we wish to uphold. The three core principles typically used in alignment tuning are defined as follows:

- **Helpfulness** refers to the ability of the model to adhere closely to the prompt instructions and help the user accomplish their task.

- **Honesty** refers to the ability of the model to provide accurate information to the user.
- **Harmlessness** refers to the model’s ability not to generate text that is harmful to, or otherwise contrary to, the values and morals of the user. Examples of issues that degrade an LLM’s harmlessness include hateful content generation or biased behaviors.

While there are many innovative approaches to alignment tuning, which are given a fuller treatment in Chapter 5, two of the most well-known are reinforcement learning from human feedback (RLHF; see Kaufmann et al., 2024) and direct preference optimization (DPO; see Rafailov et al., 2023). RLHF involves the use of human judgment-based feedback to fit a reward model that reflects these human preferences. This reward model is then used to fine-tune the LLM to maximize this reward. DPO was proposed in response to some of the complexities in fitting the reward models and achieving stability in LLM alignment. This alignment approach leverages direct preference pairs to fine-tune the LLM according to a simple classification objective, such as maximum likelihood. DPO has been proposed as a simpler approach to achieve alignment tuning in LLMs.

1.5.2.3 Full Parameter Fine-tuning

Early efforts to fine-tune LLMs focused on the instruction tuning approach, where labeled datasets are reformulated into natural language instructions and passed through the LLM to update their parameters (Sect. 1.5.2.1). However the enormous computational cost of updating billions of parameters with thousands of instruction samples is prohibitive for all but a few enterprises with budgets to meet the costs necessary. Therefore, much attention has been given to more memory/computation-efficient full-parameter tuning. The most common approach to achieving better computational efficiency is quantization, which compresses the memory footprint required for a model either during pre-training or after pre-training (Gholami et al., 2021). In addition to quantization approaches for more efficient fine-tuning of LLMs, recently lower memory optimization has also been demonstrated as a practical approach, both in facilitating task outcomes, but also in reducing overall fine-tuning cost (Lv et al., 2023). We discuss these approaches further in Sect. 4.4.

1.5.2.4 Parameter-Efficient Fine-Tuning

Another set of approaches for fine-tuning LLMs seeks to minimize the number of parameters to be tuned while achieving improved performance on a given task (Zhao et al., 2023). Below are two of the most notable parameter-efficient fine-tuning approaches (PEFT).

Low-Rank Adaptation (LoRA) is an approach presented by Hu et al. (2021) that reduces the number of parameters to be tuned by proposing that trainable rank decomposition matrices be injected into each Transformer layer of an LLM, the pre-

trained model weights for which have been frozen. These injected matrices improve fine-tuned task performance and do so without significantly impacting inference latency, as we will see is not the case for other parameter-efficient fine-tuning methods.

Adapters are another approach to fine-tuning in this category. Rather than leveraging rank decomposition matrices, adapters are small neural network modules injected into each Transformer layer and placed between input and output components. The adapter parameters are then optimized while keeping the much larger Transformer components fixed. Adapters reduce the total number of tuned parameters significantly and thus cut down considerably on training time. However, adding extra components into the pipeline leads to longer inference times. A more comprehensive treatment of these and other interesting PEFT methods will be provided in Chapter 4.

1.5.3 Large Language Model Utilization

LLM utilization at the lowest level essentially refers to *end-user prompting* as this is the core method for interacting with LLMs. However, in this section, we also address LLM utilization in the context of applications, which takes a higher-level perspective on leveraging LLMs and is discussed in more detail within Chapter 8. Here, we first introduce the reader to the concept of in-context learning, an extremely useful emergent ability of LLMs (Wei et al., 2022), and then provide details on an advanced prompting technique known as chain-of-thought prompting to provide a sense of the key ideas associated with end-user prompting. While there are many innovations within the prompt engineering space, we do not exhaustively cover them here; instead, we provide details throughout the rest of the book chapters. From the perspective of LLM application, we provide a high-level view of the core categories of applications and some insights into conceptual and framework innovations that enable them. Many of these topics are treated in more detail in later chapters of the book, such as *conversational LLMs and retrieval-augmented generation* in Chapter 7, *LLM challenges and evaluation* in Chapter 6, and *LLM application development and operations* in Chapter 8.

1.5.3.1 In-Context Learning

Often, the elicitation of emergent task performance in LLMs is done using an emergent ability in and of itself, namely *in-context learning* (ICL). First demonstrated in early OpenAI GPT models (Brown et al., 2020; Radford et al., 2019), this ability of LLMs allows them to learn from natural language inputs during inference alone (i.e., no model parameters are updated). Typically, these natural language inputs are referred to as *prompts* and can be categorized as zero-shot, few-shot, or multi-shot prompts, depending on the number of demonstrations of the task included in the input prompt as context.

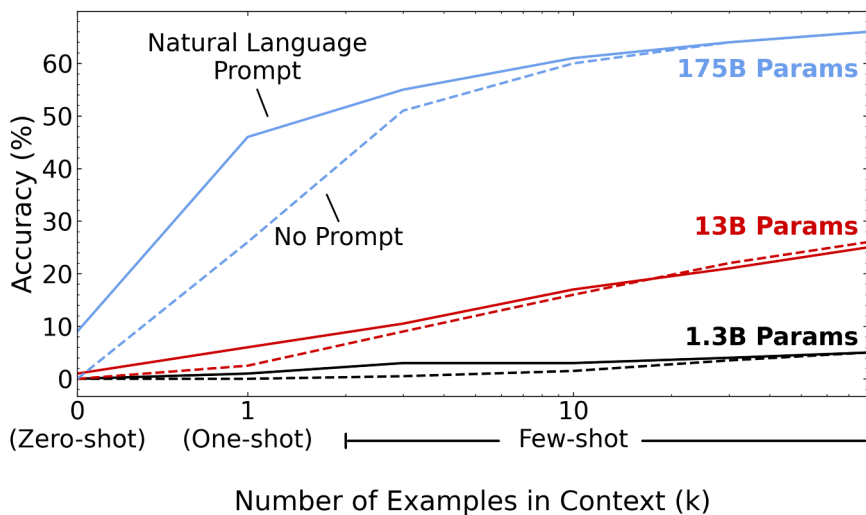


Fig. 1.8: Demonstration of *in-context learning*. Three GPT-3 models with varying numbers of parameters (1.3 billion, 13 billion, and 175 billion) are evaluated for their performance in removing random symbols that have been intentionally injected into words. Each model was tested for zero-, one- and few-shot settings. Dashed lines show model performance when no natural language task description is provided in the prompt, while solid lines show LLM performance when a natural language task description is provided as context in the prompt.

Fig. 1.8, adapted from Fig. 1.2 of [Brown et al. \(2020\)](#), illustrates the effect of ICL on LLM performance in solving a simple task that involved removing random symbols injected into otherwise valid words. Seen most remarkably for the largest GPT-3 LLM tested (175B parameters), simply providing the LLM with a prompt containing a natural language task description can achieve significantly better performance than not providing the task description. Furthermore, by giving this natural language description and only one demonstration of the task, GPT-3 performance jumped even more significantly relative to no description. This property of the most sophisticated LLMs (e.g., GPT-3.5, 4, PaLM, Llama-2) is a core driver of ongoing innovation leveraging LLMs. Strategies for designing prompts to optimize language model performance on a given task are discussed in detail in Chapter 3 and, most relevant to end-user prompting, in Chapter 7.

1.5.3.2 Chain-of-Thought Reasoning

A particular case of ICL worth noting is chain-of-thought prompting (CoT), which aims to structure the context, instructions, and queries within a prompt in a way that

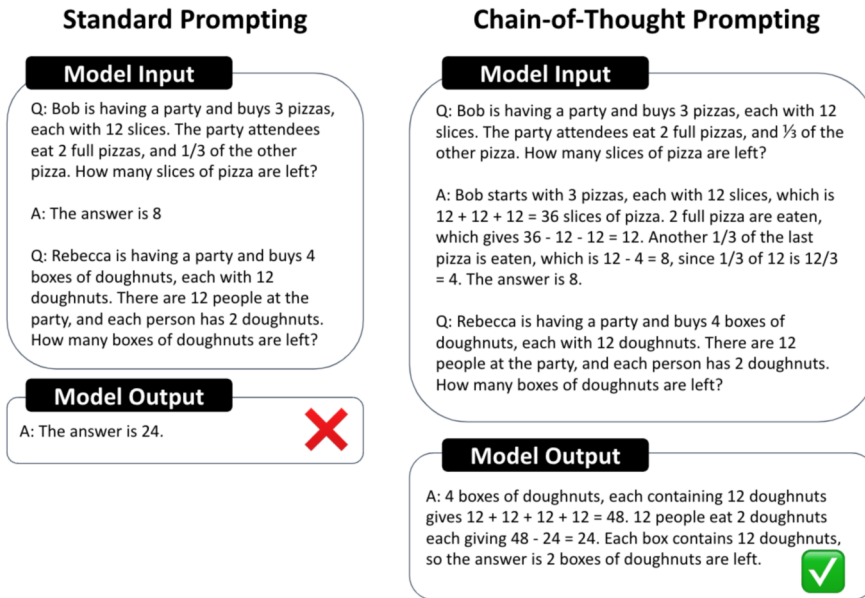


Fig. 1.9: Comparison of a hypothetical standard prompt and a chain-of-thought prompt, showing how additional reasoning instruction can be provided to the LLM to improve both the accuracy of answers, but also the explainability of how the LLM arrived at the answer.

induces the LLM to follow a logical sequence of constituent steps when solving a larger task (Wei et al., 2023). Fig. 1.9 demonstrates the *chain-of-thought* concept, illustrating how, by including sequential reasoning steps in the prompt demonstration, the LLM can better answer the sample question relative to a standard prompt. ICL and its special case, CoT, are examples of inference-time, natural language prompting for eliciting knowledge, language understanding, and reasoning from LLMs.

1.5.3.3 Applications

As hinted in the Chapter introduction, the most prominent and public-facing applications are LLMs trained to produce and assist with writing. These are typically autoregressive models, which begin with a string of text as input, predict the subsequence token, append the token, and then repeat the process with the newly enhanced string of text until the generation process is complete. This approach also underlies the chatbots mentioned in Sect. 1.1 and has been leveraged to build writing assistance software to aid in drafting emails, legal documents, technical manuals, data reports, and almost any other writing task imaginable. Achieving high-quality output that conforms to a given use case usually requires fine-tuning a base LLM on

hand-vetted data, for example, question/answer pairs, which helps guide the application to provide the desired answers (see Chapter 5).

Writing assistance applications are also notable in the world of computer programming. The concept is the same as with natural language, but instead of tuning a model to approximate natural language, the LLMs are tuned with vast blocks of computer code in various coding languages. Variants of coding assistants include suggesting auto-completions in real-time, generating functions based on pseudo-code with a compatible notation to an existing code base, and populating dashboards with data from a database based on natural language instructions. Applications such as Github's Copilot have already proliferated widely among coders, helping to streamline more tedious aspects of software development and augment their creative and general problem-solving abilities.

Other applications in the conversational or search/retrieval domains attempt to reduce the negative impacts of LLM fail states, such as their tendency to *hallucinate*, meaning that they return factually inaccurate responses to queries. One method to overcome this issue is retrieval-augmented generation (RAG). In RAG, an LLM is paired with a knowledge base for a specific subject from which it can draw context, such as a car user manual or a set of documents on a certain legal case. A RAG process takes a query from a user, assesses whether the query is related to its specific subject, and then searches its paired knowledge base to extract information related to the user question. Any relevant context in the knowledge base is then passed to the LLM along with the original query, and an answer is produced. Thus, RAG leverages the ability of LLMs to accurately answer questions about the context in which it is provided at inference time without requiring the model to contain that information beforehand. We discuss RAG systems in great detail in Chapter 7.

RAG is especially prominent in industries where privacy and data protection are significant concerns. In these fields, domain-relevant knowledge is primarily out of distribution for generic LLMs, meaning that out-of-the-box conversational applications can be of limited value. Simply domain-adapting a given LLM to protected information – for example, patient medical histories – is not an option since any information an LLM is trained on is liable to be deeply embedded into the model's weights themselves, running the risk that they will become part of any given future response to a user query. RAG allows protected information to remain outside of the training data of an LLM but within the scope of knowledge that it can draw from.

LLMs provide SOTA entity recognition and reasoning capabilities in fields such as law or NLP research. Historically, teams of data scientists would spend months developing high-quality entity recognition models, using time and resource-intensive fully-supervised approaches coupled with complex business logic systems necessary to accurately reason around and act on identified entities of interest. Today, LLM-enabled applications are now capable of both identifying these entities and reasoning around them with something approaching the legal competency of human lawyers in the top 10% of Bar exam scores (although it is still unclear how robustly evaluations of these types measure LLM competency in line with human performance) (Martínez, 2023).

This section offers just a taste of the ever-expanding litany of applications. We will discuss more applications throughout this book, especially in Chapter 8. As practitioners within various domains continue to explore the benefits and limitations of applying LLMs to their areas of endeavor, there is no doubt that the rate with which innovations emerge around these remarkable technologies will continue to grow. Indeed, the impressive performance of LLMs on a plethora of learning, evaluation, and generation benchmarks has naturally produced an interest in guiding these capabilities toward solving business and consumer problems. By adapting LLMs to various domains using techniques such as fine-tuning or ICL, researchers have produced a flurry of new applications that take advantage of their novel capabilities. We hope this book is a valuable introduction and reference to the core concepts around LLMs and their use.

References

- Amanda Askell et al. A general language assistant as a laboratory for alignment, 2021.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. A neural probabilistic language model. In *Proceedings of the 13th International Conference on Neural Information Processing Systems*, pages 893–899. MIT Press, 2000.
- Peter F. Brown, Peter V. deSouza, Robert L. Mercer, Vincent J. Della Pietra, and Jenifer C. Lai. Class-based n-gram models of natural language. *Comput. Linguist.*, 18(4):467–479, December 1992.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Noam Chomsky. *Syntactic Structures*. Mouton and Co., 1957.
- Aakanksha Chowdhery et al. Palm: Scaling language modeling with pathways, 2022.
- Ronan Collobert and Jason Weston. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th International Conference on Machine Learning*, pages 160–167. ACM, 2008.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- Dani Di Placido. Why did “balenciaga pope” go viral?, Mar 2023. URL <https://www.forbes.com/sites/danidiplacido/2023/03/27/why-did-balenciaga-pope-go-viral/>.
- Fortune. Generative ai market size, share covid-19 impact analysis, by model, by industry vs application, and regional forecast, 2023-2030, Aug 2023. URL <https://www.fortunebusinessinsights.com/generative-ai-market-107837>.

- Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference, 2021.
- Alex Graves. Generating sequences with recurrent neural networks. *CoRR*, abs/1308.0850, 2013.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks, 2020.
- Eva Hajicová, Ivana Kruijff-Korbayová, and Petr Sgall. Prague dependency treebank: Restoration of deletions. In *Proceedings of the Second International Workshop on Text, Speech and Dialogue*, pages 44–49. Springer-Verlag, 1999.
- Jordan Hoffmann et al. Training compute-optimal large language models, 2022.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- W. John Hutchins, Leon Dostert, and Paul Garvin. The georgetown-i.b.m. experiment. In *In*, pages 124–135. John Wiley And Sons, 1955.
- Hamish Ivison, Akshita Bhagia, Yizhong Wang, Hannaneh Hajishirzi, and Matthew Peters. Hint: Hypernetwork instruction tuning for efficient zero- few-shot generalisation, 2023.
- Berber Jin and Miles Kruppa. Wsj news exclusive | chatgpt creator is talking to investors about selling shares at \$29 billion valuation, Feb 2023. URL <https://www.wsj.com/articles/chatgpt-creator-openai-is-in-talks-for-tender-offer-that-would-value-it-at-29-billion-11672949279>.
- Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. A survey of reinforcement learning from human feedback, 2024.
- Bill Yuchen Lin, Abhilasha Ravichander, Ximing Lu, Nouha Dziri, Melanie Sclar, Khyathi Chandu, Chandra Bhagavatula, and Yejin Choi. The unlocking spell on base llms: Rethinking alignment via in-context learning, 2023.
- Kai Lv, Yuqing Yang, Tengxiao Liu, Qinghui Gao, Qipeng Guo, and Xipeng Qiu. Full parameter fine-tuning for large language models with limited resources, 2023.
- Yingwei Ma, Yue Liu, Yue Yu, Yuanliang Zhang, Yu Jiang, Changjian Wang, and Shanshan Li. At which training stage does code data help llms reasoning?, 2023.
- Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.
- Mitchell Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. The penn treebank: Annotating predicate argument structure. In *Proceedings of the Workshop on Human Language Technology*, pages 114–119. Association for Computational Linguistics, 1994.
- Eric Martínez. Re-evaluating gpt-4’s bar exam performance. 2023. URL <http://dx.doi.org/10.2139/ssrn.4441311>.
- Tomas Mikolov, Martin Karafiát, Lukás Burget, Jan Cernocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In Takao Kobayashi,

- Keikichi Hirose, and Satoshi Nakamura, editors, *INTERSPEECH*, pages 1045–1048. ISCA, 2010.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *CoRR*, abs/1301.3781, 2013a.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc., 2013b.
- George A. Miller. Wordnet: A lexical database for english. *Commun. ACM*, 38(11): 39–41, November 1995.
- OpenAI. Gpt-4 technical report, 2023.
- Long Ouyang et al. Training language models to follow instructions with human feedback, 2022.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9, 2019.
- Jack W. Rae et al. Scaling language models: Methods, analysis insights from training gopher, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2020.
- Livio Baldini Soares, Nicholas FitzGerald, Jeffrey Ling, and Tom Kwiatkowski. Matching the blanks: Distributional similarity for relation learning, 2019.
- Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models, 2023.
- Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.
- Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Valuates. Generative ai market size to grow usd 126.5 billion by 2031 at a cagr of 32%, Jun 2023. URL <https://www.prnewswire.com/news-releases/generative-ai-market-size-to-grow-usd-126-5-billion-by-2031-at-a-cagr-of-32--valuates-reports-301846316.html>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- Jason Wei et al. Emergent abilities of large language models, 2022.
- Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance, 2023.
- Qiang Zhang et al. Scientific large language models: A survey on biological chemical domains, 2024a.
- Shengyu Zhang et al. Instruction tuning for large language models: A survey, 2024b.
- Wayne Xin Zhao et al. A survey of large language models, 2023.



Chapter 2

Language Models Pre-training

Abstract Pre-training forms the foundation for LLMs' capabilities. LLMs gain vital language comprehension and generative language skills by using large-scale datasets. The size and quality of these datasets are essential for maximizing LLMs' potential. It is also crucial to have suitable model structures, speed-up methods, and optimization approaches for effective pre-training. We start the chapter by introducing the encoder-decoder architectures, their applicability in a wide range of NLP tasks, and their shortcomings. We then introduce the readers to the attention mechanism and help them understand the Transformers' architecture, which is the central part of most LLMs. We will then cover data collection and processing, followed by key design aspects such as model architectures, pre-training objectives, and optimization tactics, all of which are vital for LLM pre-training. We then examine primary LLMs such as BERT, T5, GPT (1-3), and Mixtral8x7B, which have inspired numerous variations to highlight their architectures and training differences. Finally, at the end of the chapter, we provide a tutorial that delves into LLM architectures, highlighting the differences between masked and causal models, examining the mechanisms behind pre-trained models' outputs, and providing a succinct overview of the training procedure.

2.1 Encoder-Decoder Architecture

The encoder-decoder architecture, as illustrated in Fig. 2.1, represents a pivotal advancement in natural language processing (NLP), particularly in sequence-to-sequence tasks such as machine translation, abstractive summarization, and question answering (Sutskever et al., 2014). This framework is built upon two primary components: an encoder and a decoder.

2.1.1 Encoder

The input text is tokenized into units (words or sub-words), which are then embedded into feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_T$. A unidirectional encoder updates its hidden state $\mathbf{h}_t$ at each time t using $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ as given by:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t) \quad (2.1)$$

The final state $\mathbf{h}_T$ of the encoder is known as the *context variable* or the *context vector*, and it encodes the information of the entire input sequence and is given by :

$$\mathbf{c} = m(\mathbf{h}_1, \dots, \mathbf{h}_T) \quad (2.2)$$

where m is the mapping function and, in the simplest case, maps the context variable to the last hidden state

$$\mathbf{c} = m(\mathbf{h}_1, \dots, \mathbf{h}_T) = \mathbf{h}_T \quad (2.3)$$

Adding more complexity to the architecture, the encoders can be bidirectional; thus, the hidden state would not only depend on the previous hidden state $\mathbf{h}_{t-1}$ and input $\mathbf{x}_t$, but also on the following state $\mathbf{h}_{t+1}$.

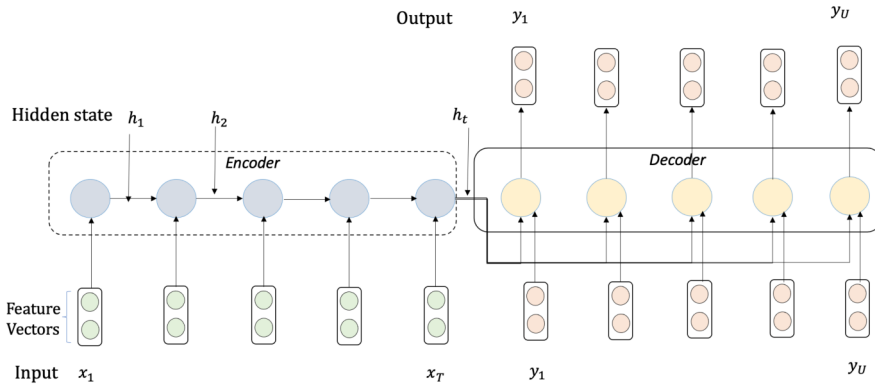


Fig. 2.1: A unidirectional encoder-decoder for sequence-to-sequence processing.

2.1.2 Decoder

Upon obtaining the context vector $\mathbf{c}$ from the encoder, the decoder starts to generate the output sequence $y = (y_1, y_2, \dots, y_U)$, where U may differ from T . Similar to the

encoder, the decoder's hidden state at any time t is given by

$$\mathbf{s}_{t'} = g(\mathbf{s}_{t-1}, \mathbf{y}_{t'-1}, \mathbf{c}) \quad (2.4)$$

The hidden state of the decoder flows to an output layer and the conditional distribution of the next token at t' is given by

$$P(\mathbf{y}_{t'} | \mathbf{y}_{t'-1}, \dots, \mathbf{y}_1, \mathbf{c}) = \text{softmax}(\mathbf{s}_{t-1}, \mathbf{y}_{t'-1}, \mathbf{c}) \quad (2.5)$$

2.1.3 Training and Optimization

The encoder-decoder model is trained end-to-end through supervised learning. The standard loss function employed is the categorical cross-entropy between the predicted output sequence and the actual output. This can be represented as:

$$\mathcal{L} = - \sum_{t=1}^U \log p(y_t | y_{t-1}, \dots, y_1, \mathbf{c}) \quad (2.6)$$

Optimization of the model parameters typically employs gradient descent variants, such as the Adam or RMSprop algorithms.

2.1.4 Issues with Encoder-Decoder Architectures

As outlined in the preceding section, the encoder component condenses the information from the source sentence into a singular context variable $\mathbf{c}$ for subsequent utilization by the decoder. Such a reductionist approach inherently suffers from information loss, particularly as the input length increases. Moreover, natural language's syntactic and semantic intricacies often entail long-range dependencies between tokens, which are challenging to encapsulate effectively within a singular context vector. However, it should be noted that the hidden states at each time step in the encoder contain valuable information that remains available for the decoder's operations. These hidden states can exert variable influence on each decoding time step, thereby partially alleviating the limitations of a singular context variable. Nevertheless, Recurrent Neural Networks (RNNs), the foundational architecture for many encoder-decoder models, have shortcomings, such as susceptibility to vanishing and exploding gradients (Hochreiter, 1998). Additionally, the sequential dependency intrinsic to RNNs complicates parallelization, thereby imposing computational constraints.

2.2 Attention Mechanism

The attention mechanism helps address problems found in the RNN-based encoder-decoder setup. As illustrated in Fig. 2.2, an attention mechanism is like a memory bank. When queried, it produces an output based on stored keys and values (Bahdanau et al., 2014).

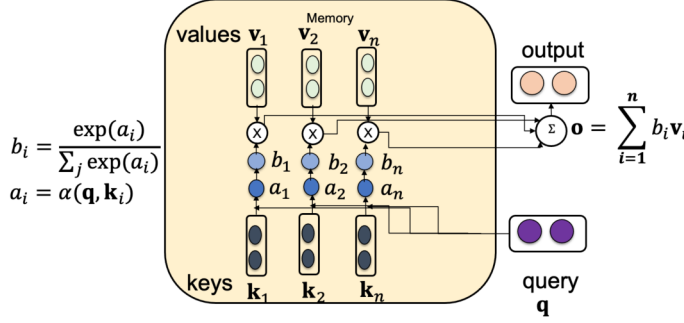


Fig. 2.2: The attention mechanism and its interplay among queries, keys, values, and the resultant output vectors.

Let us consider the memory unit consisting of n key-value pairs $(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_n, \mathbf{v}_n)$ with $\mathbf{k}_i \in \mathbb{R}^{d_k}$ and $\mathbf{v}_i \in \mathbb{R}^{d_v}$. The attention layer receives an input as query $\mathbf{q} \in \mathbb{R}^{d_q}$ and returns an output $\mathbf{o} \in \mathbb{R}^{d_v}$ with the same shape as the value $\mathbf{v}$.

The attention layer measures the similarity between the query and the key using a score function α , which returns scores $a_1, \dots, a_n$ for keys $\mathbf{k}_1, \dots, \mathbf{k}_n$ given by

$$a_i = \alpha(\mathbf{q}, \mathbf{k}_i) \quad (2.7)$$

Attention weights are computed as a softmax function on the scores

$$\mathbf{b} = \text{softmax}(\mathbf{a}) \quad (2.8)$$

Each element of $\mathbf{b}$ is

$$b_i = \frac{\exp(a_i)}{\sum_j \exp(a_j)} \quad (2.9)$$

The output is the weighted sum of the attention weights and the values.

$$\mathbf{o} = \sum_{i=1}^n b_i \mathbf{v}_i \quad (2.10)$$

The score function $\alpha(\mathbf{q}, \mathbf{k})$ exists in various forms, leading to multiple types of attention mechanisms. The dot product-based scoring function is the simplest, re-

quiring no tunable parameters. A variation, the scaled dot product, normalizes this by $\sqrt{d_k}$ to mitigate the impact of increasing dimensions (Luong et al., 2015; Vaswani et al., 2017).

$$\alpha(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}} \quad (2.11)$$

2.2.1 Self-Attention

In self-attention, each input vector $\mathbf{x}_i$ is projected onto three distinct vectors: query $\mathbf{q}_i$, key $\mathbf{k}_i$, and value $\mathbf{v}_i$. These projections are performed via learnable weight matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$, resulting in $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q$, $\mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K$, and $\mathbf{v}_i = \mathbf{x}_i \mathbf{W}_V$, respectively. These weight matrices are initialized randomly and optimized during training. The simplified matrix representation with each of the query, key, and value matrices as a single computation is given by:

$$\text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \quad (2.12)$$

2.3 Transformers

The Transformer model, which was introduced by Vaswani et al. (2017), is a cornerstone in sequence-to-sequence tasks. The Transformer architecture, shown in Fig. 2.3, employs an encoder-decoder setup, each consisting of multiple identical layers with the specifics of its essential components discussed in the following section.

2.3.1 Encoder

The encoder is responsible for processing the input sequence and compressing the information into a context or *memory* for the decoder. Each encoder layer comprises three main elements:

- **Multi-Head Attention:** This component allows the model to focus on different parts of the input for each attention head, thereby capturing various aspects of the data.
- **Feed-Forward Neural Network:** A simple yet effective neural network that operates on the attention vectors, applying nonlinear transformation and making it available for the next encoder layer (and the decoder layer).
- **Add & Norm:** The Add & Norm layer aids in stabilizing the activations by combining residual connections and layer normalization, ensuring smoother training and mitigating the vanishing gradient problem in the encoder (and the decoder).

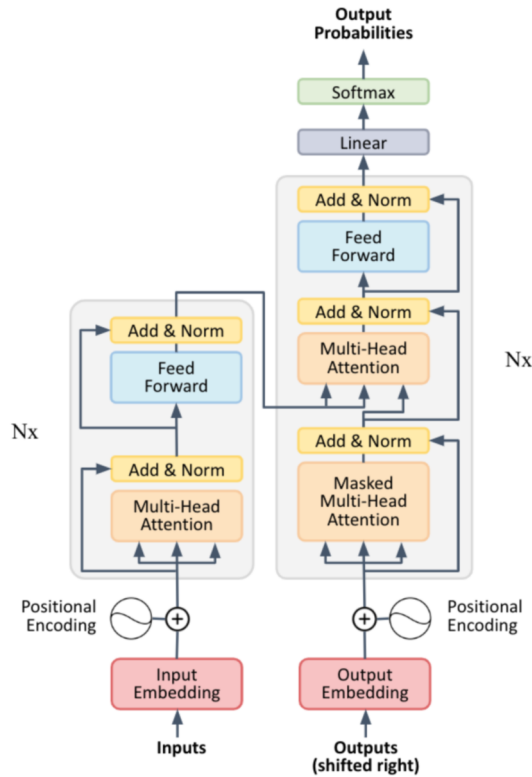


Fig. 2.3: The Transformer's architecture uses encoder and decoder components, both of which employ multi-head attention.

2.3.2 Decoder

The decoder takes the context from the encoder and generates the output sequence. It is also composed of multiple layers and has many commonalities with the encoder, but with minor changes:

- **Masked Multi-Head Attention:** Similar to multi-head attention but with a masking mechanism to ensure that the prediction for a given word doesn't depend on future words in the sequence.
- **Encoder-Decoder Attention:** This layer allows the decoder to focus on relevant parts of the input sequence, leveraging the context provided by the encoder.
- **Feed-Forward Neural Network:** Identical in architecture to the one in the encoder, this layer further refines the attention vectors in preparation for generating the output sequence.

Next, we describe various components and sub-components of the Transformer architecture.

2.3.3 Tokenization and Representation

In Transformer models, tokenization typically converts sentences into a machine-readable format. This can be done at the level of words or subwords, depending on the granularity required for the specific application. Each word in the sentence is treated as a distinct token in word-level tokenization. These tokens are then mapped to their corresponding vector representations, such as word embeddings, which serve as the input to the Transformer model. This approach may face limitations when dealing with out-of-vocabulary words. Subword-level approaches such as byte-pair encoding (BPE) or WordPiece often address the limitations of word-level tokenization. In these methods, words are broken down into smaller pieces or subwords, providing a way to represent out-of-vocabulary terms and capture morphological nuances. These subwords are then mapped to embeddings and fed into the Transformer.

For instance, the word “unhappiness” could be split into subwords such as “un” and “happiness”. These subwords are then individually mapped to their embeddings. This method increases the model’s ability to generalize and handle a broader range of vocabulary, including words not seen during training.

A hybrid approach combining word and subword-level tokenization can also leverage both. Such a strategy balances the comprehensiveness of subword-level representations with the interpretability of word-level tokens.

2.3.4 Positional Encodings

Since the Transformer model processes all tokens in the input sequence in parallel, it does not have a built-in mechanism to account for the token positions or order. Positional encoding is introduced to provide the model with information about the relative positions of the tokens in the sequence. The positional encoding is usually added to the input embeddings before they are fed into the Transformer model.

If the length of the sentence is given by l and the embedding dimension/depth is given by d , positional encoding $\mathbf{P}$ is a 2-d matrix of the same dimension, i.e., $\mathbf{P} \in \mathbb{R}^{l \times d}$. Every position can be represented with the equation in terms of i , which is along the l , and j , which is along the d dimension as

$$\mathbf{P}_{i,2j} = \sin(i/1000^{2j/d}) \quad (2.13)$$

$$\mathbf{P}_{i,2j+1} = \cos(i/1000^{2j/d}) \quad (2.14)$$

for $i = 0, \dots, l - 1, j = 0, \dots, \lfloor (d - 1)/2 \rfloor$. The function definition above indicates that the frequencies decrease along the vector dimension and form a geometric progression from 2π to $10000 \cdot 2\pi$ on the wavelengths. For $d = 512$ dimensions for a maximum positional length of $l = 100$, the positional encoding visualization is shown in Fig. 2.4.

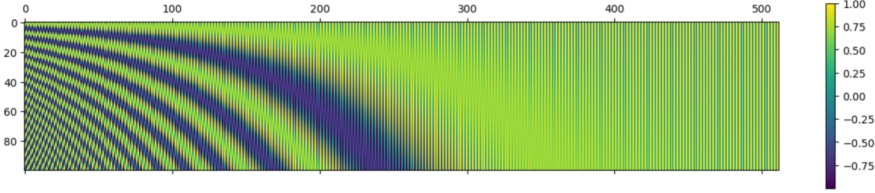


Fig. 2.4: Positional encoding for 100 positions with a dimensionality of 512.

2.3.5 Multi-Head Attention

Rather than a single self-attention head, multi-head attention employs h parallel self-attention heads, enhancing the model's representational capacity. In the original Transformer model, $h = 8$ heads were used to allow the model to capture various aspects and dependencies within the input data, such as grammar and tense in machine translation tasks.

Each head operates with its own set of learnable query, key, and value weight matrices in multi-head attention. This results in distinct query, key, and value matrices and unique output matrices for each head. These output matrices are concatenated and subsequently linearly transformed using an additional weight matrix. The parallel input-to-output transformations for all the heads are depicted in Fig. 2.5.

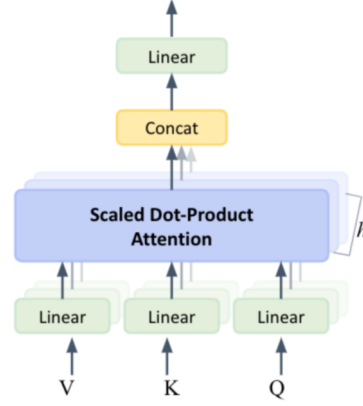
$$head_i = \text{attention}(\mathbf{W}_Q^i \mathbf{Q}, \mathbf{W}_K^i \mathbf{K}, \mathbf{W}_V^i \mathbf{V}) \quad (2.15)$$

$$\text{multihead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{W}_O \text{concat}(head_1, \dots, head_h) \quad (2.16)$$

2.3.6 Position-Wise Feed-Forward Neural Networks

Following the attention mechanism, the next component in the architecture of the Transformer model is the feed-forward neural network. This network transforms the attention vectors further, rendering them compatible with the input to the subsequent encoder or decoder layer. The feed-forward neural network often comprises two lay-

Fig. 2.5 The multi-head attention mechanism learns multiple query/key/value matrices and subsequently combines them.



ers with a rectified linear unit (ReLU) activation function applied after the first layer to allow nonlinearity. Mathematically, if $\mathbf{z}$ is the input attention vector, the transformation $F(\mathbf{z})$ performed by the feed-forward neural network can be represented as:

$$F(\mathbf{z}) = \text{ReLU}(\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \quad (2.17)$$

where $\mathbf{W}_1$ and $\mathbf{W}_2$ are the weight matrices, and $\mathbf{b}_1$ and $\mathbf{b}_2$ are the bias vectors for the first and second layers, respectively. Each of the N layers in the Transformer encoder (and decoder) perform identical linear transformations on every word in the input sequence. However, they utilize distinct weight ($\mathbf{W}_1, \mathbf{W}_2$) and bias ($\mathbf{b}_1, \mathbf{b}_2$) parameters for these transformations.

2.3.7 Layer Normalization

In a manner akin to ResNets, the Transformer model employs a residual connection where the input $\mathbf{X}$ is added to the output $\mathbf{Z}$ (He et al., 2016). This normalization procedure ensures that each layer's activations have a zero mean and a unit variance.

For each hidden unit h_i , the layer normalization is formulated as:

$$h_i = \frac{g}{\sigma}(h_i - \mu) \quad (2.18)$$

where g is the gain variable (often set to 1), μ is the mean calculated as $\frac{1}{H} \sum_{i=1}^H h_i$, and σ is the standard deviation computed as $\sqrt{\frac{1}{H} \sum_{i=1}^H (h_i - \mu)^2}$.

The layer normalization technique minimizes *covariate shift*, i.e., the gradient dependencies between layers, thus accelerating convergence by reducing the required iterations (Ba et al., 2016).

2.3.8 Masked Multi-Head Attention

In the Transformer model, the decoder aims to predict the next token (word or character) in the sequence by considering both the encoder's output and the tokens already seen in the target sequence. The first layer of the decoder adopts a particular strategy: it only has access to the tokens that come before the token it is currently trying to predict. This mechanism is known as masked multi-head attention.

The masking is implemented using a particular weight matrix $\mathbf{M}$. In this matrix, entries corresponding to future tokens in the sequence are set to $-\infty$, and those for previous tokens are set to 0.

This masking is applied after calculating the dot product of the Query ($\mathbf{Q}$) and Key ($\mathbf{K}^T$) matrices but before applying the softmax function. As a result, the softmax output for future tokens becomes zero, effectively masking them from consideration. This ensures that the decoder cannot peek into future tokens in the sequence, thereby preserving the sequential integrity required for tasks such as language translation.

$$\text{maskedAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{M}}{\sqrt{d_k}}\right)\mathbf{V} \quad (2.19)$$

2.3.9 Encoder-Decoder Attention

The encoder-decoder attention mechanism serves as the bridge that connects the encoder and the decoder, facilitating the transfer of contextual information from the source sequence to the target sequence. Conceptually, the encoder-decoder attention layer works similarly to standard multi-head attention but with a critical difference: the Queries ($\mathbf{Q}$) come from the current state of the decoder, while the Keys ($\mathbf{K}$) and Values ($\mathbf{V}$) are sourced from the output of the encoder. This mechanism allows the model to focus on relevant portions of the source sequence while generating each token in the target sequence, thus capturing intricate relationships between the source and target.

2.3.10 Transformer Variants

Numerous Transformer models have emerged, each featuring modifications to the original Transformer discussed in the previous Sect. (Lin et al., 2022). These alterations can be categorized into three types: architectural changes, pre-training methods, and applications, as illustrated in Fig. 2.6. We detail in the following sections key variables between different Transformer variants. A selection are summarized at the end in Table 2.1.

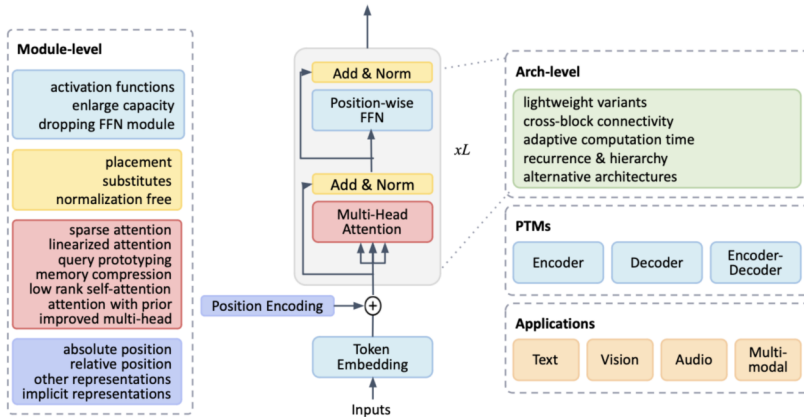


Fig. 2.6: The Transformer has spun off numerous variants that can be taxonomized based on architecture modifications, training objectives, and input types.

2.3.10.1 Normalization Methods

Training instability is challenging in the pre-training phase of LLMs. Normalization methods are employed to stabilize training. Initially, BatchNorm was commonly used but proved inefficient with variable-length sequence and small-batch data. Consequently, LayerNorm (LN) was introduced to perform layer-wise normalization, recalculating the mean and variance for each layer's activations. RMSNorm was later proposed to enhance the training speed of LayerNorm by rescaling activations using the root mean square of summed activations, demonstrating improved training speed and performance in Transformer models.

! Practical Tips

Models such as Gopher and Chinchilla have adopted RMSNorm. DeepNorm, proposed by Microsoft, aids in stabilizing the training of deep Transformers, allowing them to scale up to 1000 layers. This method has been adopted for models requiring stability and performance during training, such as GLM-130B.

2.3.10.2 Normalization Position

There are three primary normalization positions: post-LN, pre-LN, and sandwich-LN. Post-LN, utilized in the original Transformer, is positioned between residual blocks. However, due to large gradients near the output layer, post-LN Transformers often exhibit instability during training. As a result, post-LN is infrequently used in

LLMs unless combined with other strategies, such as integration with pre-LN in the GLM-130B model. Pre-LN is applied before each sub-layer, with an additional layer normalization (LN) before the final prediction.

! Practical Tips

Transformers using pre-LN demonstrate greater training stability than post-LN, albeit with reduced performance. Despite this trade-off, pre-LN is commonly adopted for its training stability, with exceptions noted in models such as GLM with over 100 billion parameters, where pre-LN exhibits instability. Sandwich-LN, an extension of pre-LN, incorporates extra LN before residual connections to mitigate value explosion in Transformer layer outputs. However, this approach does not consistently stabilize LLM training and may result in training collapse.

2.3.10.3 Activation Functions

The proper setting of activation functions is essential for optimal performance in feed-forward networks. GeLU activations are commonly used in existing LLMs.

! Practical Tips

In recent LLMs, such as PaLM and LaMDA, variants of GLU activation, including SwiGLU and GeGLU, are utilized, often resulting in improved performance. However, these variants require approximately 50% more parameters in the feed-forward networks than GeLU.

2.3.10.4 Positional Embeddings

Absolute position embeddings, used in the original Transformer, are added to the input embeddings at the bottom of the encoder and decoder. Two variants exist: sinusoidal and learned position embeddings, with the latter prevalent in pre-trained language models.

! Practical Tips

Relative position embeddings, generated based on offsets between keys and queries, were introduced in Transformer-XL and modified in T5, simplifying the embeddings by adding learnable scalars to attention scores based on distances between query and key positions. Transformers using relative embeddings can handle sequences longer than those seen during training. Rotary position embedding (RoPE) utilizes rotatory matrices based on token positions, allowing for the calculation of scores with relative position information. Due to its performance and long-term decay property, RoPE is

used in recent LLMs such as PaLM and Llama. xPos, built on RoPE, enhances the translation invariance and length extrapolation of Transformers by adding a special exponential decay to each dimension of the rotation degree vector, stabilizing training over increased distances. ALiBi improves Transformer extrapolation by biasing attention scores with a distance-based penalty between keys and queries without trainable parameters. It has demonstrated superior extrapolation performance and training stability compared to other position embedding methods, including sinusoidal PE, RoPE, and T5 bias.

2.3.10.5 Attention Mechanism

The original Transformer utilizes full attention, conducting attention pairwise and considering all token pairs in a sequence. It employs scaled dot-product attention and multi-head attention, where queries, keys, and values are projected differently in each head, with the concatenated output of each head forming the final output. Sparse attention addresses the quadratic computational complexity challenge of full attention, especially with long sequences.

! Practical Tips

Efficient Transformer variants, like locally banded sparse attention (e.g., Factorized Attention in GPT-3), allow each query to attend to a subset of tokens based on positions, reducing complexity. Multi-query attention, where different heads share the same linear transformation matrices on keys and values, offers computational savings with minimal impact on model quality. Models such as PaLM and StarCoder utilize multi-query attention. FlashAttention optimizes the speed and memory consumption of attention modules on GPUs without compromising model quality. It organizes input into blocks and introduces recomputation to utilize fast memory (SRAM) on GPUs efficiently. Integrated into platforms such as PyTorch, DeepSpeed, and Megatron-LM, FlashAttention optimizes attention modules from an IO-aware perspective. For optimal generalization and training stability, pre-RMSNorm is recommended for layer normalization, with SwiGLU or GeGLU as the activation function. It is advised not to use layer normalization immediately after embedding layers to avoid performance degradation. Some methods, such as Realformer and Predictive Attention Transformer, reuse attention distributions from previous blocks to guide the current block, creating more direct paths through the network. Transparent Attention eases optimization using a weighted sum of encoder representations from all layers in cross-attention modules. Adaptive Computation Time (ACT) has been introduced to tailor computation time based on input difficulty, leading to strategies such as Universal Transformer and Conditional Computation Transformer, which either refine representations iteratively or utilize gating mechanisms to optimize computational resources.

Table 2.1: In our network configurations, *Sublayer* refers to either a feed-forward neural network (FFN) or a self-attention module within a Transformer layer. The symbol d represents the size of the hidden states in the network. The position embedding at a specific position i is denoted by p_i . In the attention mechanism, A_{ij} signifies the attention score computed between a given query and its corresponding key. The difference in positions between the query and the key is represented by r_{i-j} , a learnable scalar value. Finally, the term $R_{\theta,t}$ refers to a rotary matrix, which rotates by an angle determined by multiplying t by θ .

Configuration	Method	Equation
Normalization position	Post Norm [1]	$\text{Norm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$
	Pre Norm [2]	$\mathbf{x} + \text{Sublayer}(\text{Norm}(\mathbf{x}))$
	Sandwich Norm [3]	$\mathbf{x} + \text{Norm}(\text{Sublayer}(\text{Norm}(\mathbf{x})))$
Normalization method	LayerNorm [4]	$\frac{\mathbf{x} - \mu}{\sqrt{\sigma}} \cdot \gamma + \beta, \mu = \frac{1}{d} \sum_{i=1}^d \mathbf{x}_i, \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (\mathbf{x}_i - \mu)^2}$
	RMSNorm [5]	$\frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma, \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d \mathbf{x}_i^2}$
	DeepNorm [6]	$\text{LayerNorm}(\alpha \cdot \mathbf{x} + \text{Sublayer}(\mathbf{x}))$
Activation	ReLU [7]	$\text{ReLU}(\mathbf{x}) = \max(0, \mathbf{x})$
function	GeLU [8]	$\text{GeLU}(\mathbf{x}) = 0.5\mathbf{x} \otimes \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}} (\mathbf{x} + 0.044715\mathbf{x}^3)\right)\right)$
	Swish [9]	$f(x) = x \cdot \frac{1}{1 + e^{-x}}$
	SwiGLU [10]	$f(x) = x \odot \sigma(Wx + b)$
	GeGLU [10]	Similar to SwiGLU with GeLU
Positional embeddings	Absolute [1]	$\mathbf{x}_i = \mathbf{x}_i + p_i$
	Relative [11]	$A_{ij} = W_q \mathbf{x}_i \mathbf{x}_j^T W_k + r_{i-j}$
	RoPE [12]	$A_{ij} = W_q \mathbf{x}_i R_{\theta, i-j} \mathbf{x}_j^T W_k$
	Alibi [13]	$A_{ij} = W_q \mathbf{x}_i \mathbf{x}_j^T W_k - m(i - j)$

Key: [1] (Vaswani et al., 2017), [2] (Radford et al., 2019), [3] (Ding et al., 2021), [4] (Ba et al., 2016), [5] (Zhang and Sennrich, 2019), [6] (Wang et al., 2022), [7] (Nair and Hinton, 2010), [8] (Wang et al., 2019), [9] (Ramachandran et al., 2017), [10] (Shazeer, 2020), [11] (Raffel et al., 2020), [12] (Su et al., 2021), [13] (Press et al., 2021)

2.3.10.6 Structural Modifications

To address the computational demands of the Transformer, various high-level modifications have been proposed. The *Lite Transformer* introduces a two-branch structure, combining attention for long-range contexts and convolution for local dependencies, making it suitable for mobile devices. Meanwhile, *Funnel Transformer* and *DeLight* introduce funnel-like encoder architectures and replace standard Transformer blocks with specialized modules, respectively, aiming to achieve efficiency in terms of FLOPs, memory, and model size. Transformers face challenges in handling long sequences due to their quadratic complexity. Divide-and-conquer strategies, such as recurrent and hierarchical Transformers, have emerged to address this issue. *Recurrent Transformers* utilize cache memory to store historical information, with techniques such as *Transformer-XL* extending context lengths. At the same time,

hierarchical Transformers break down inputs into smaller pieces, first processing low-level features and then aggregating them for higher-level processing, aiding in handling long inputs and generating richer representations.

2.4 Data

Thus far, in this chapter, we have primarily discussed the technical concepts behind LLMs. In addition to the architectural elements of the models themselves, the data used to train them are equally essential to understanding how they work. This section will provide a view of the types of training data commonly utilized and their effects on the capabilities of LLMs.

2.4.1 Language Model Pre-Training Datasets

Transfer learning has dominated all areas of NLP since 2018. In that year, three significant language models were released: ULMFiT, followed by GPT and BERT. Each of these models varied substantially in their architectures, but they all shared a common theme: using only a self-supervised language modeling objective for pre-training and then fine-tuning on task-specific labeled training data. This approach can leverage massive bodies of text for general language understanding without requiring the data to be labeled, which is highly beneficial since labeled data are often difficult to obtain. This section describes the most commonly used data sources for language model pre-training.

The objective during pre-training is to condition the LLM with general language understanding and world knowledge. As such, the selected training data should cover a broad range of topics and use an extensive vocabulary while also capturing a representative distribution of the patterns found in written language. In addition, of course, it also needs to be available in vast quantities. Effective sources include the following:

- **Web-scraping:** Web pages are collected in an automated fashion by following the links within a given page, then following the links in all of those pages, etc. This type of data offers an extensive range of language, but its quality can be suspect. The internet contains slang, typos, and other non-standard language that can increase the robustness of a model. However, by the same token, much of the text may be indecipherable or counterfactual, leading to detrimental effects if not cleaned adequately. The Common Crawl data is the most notable publicly available web scrape.
- **Wikipedia:** Training on Wikipedia data has several benefits. First, it provides a wealth of factual information. It is generally well edited and consistently formatted, making it less prone to the data quality issues of the wider web. As a

bonus, Wikipedia has articles in many languages, allowing for expansion beyond English.

- **Books:** Novels are an excellent narrative source about how humans think and interact with each other and their environments. This type of language is not found in a knowledge base such as Wikipedia, which contains only third-person accounts of events. Most books are also great at modeling long-term dependencies. The obvious downside is that much of the information in story books is fictional.
- **Code:** As generative models have become increasingly powerful, code generation has become a popular application. Data from GitHub and StackExchange are frequently used to train models capable of producing code. Interestingly, training on code may also enhance LLM capabilities on other logical reasoning tasks (Fu and Khot, 2022).

Early Transformer models were trained on a scale at which it was typical to choose one or two of the data sources described above. At the scale of modern LLMs, it is now more common to combine all of these (and more) to realize the unique benefits that each can provide. The Pile (Gao et al., 2020) introduced a corpus spanning 22 sources, such as legal and medical texts, academic research papers, and code from GitHub. They demonstrated that these sources improved downstream performance over models trained on less diverse corpora such as Common Crawl. Taking this idea further, the ROOTS corpus (Laurençon et al., 2023) incorporates 46 natural and 13 programming languages from hundreds of sources.

Table 2.2: Descriptions of various corpora widely adopted for pre-training LLMs.

Corpus	Source
BookCorpus	Books
Wikitext103	Wikipedia
Common Crawl	Internet
OpenWebText	Internet
The Pile	Internet, Academic Research, Books, Dialog, Code
ROOTS	High and Low Resource Languages, Internet, Code

2.4.1.1 Multilingual and Parallel Corpora

Many LLMs are trained exclusively or primarily in a single language, but models that can interpret and translate between many different languages require data spanning all of the desired languages. These data fall broadly into two categories:

- In a parallel corpus, each text example has a corresponding translation in a second language. These language pairs are then used with a training objective

wherein one language is the input and the other is the target. The model predictions are then scored based on how closely they match the target.

- A multilingual corpus contains data in multiple languages without any explicit translation between languages. These corpora are useful for language modeling objectives, not the machine translation objective used with parallel corpora.

In recent years, modern LLMs have reached a scale that allows them to perform well on translation tasks in a few-shot setting without specific training on parallel data (Workshop et al., 2023). Translation capabilities emerge from the model’s joint conditioning on multiple languages rather than learning from explicit language pairs.

2.4.2 Data Pre-Processing

Since the corpora used for pre-training are far too large to be manually reviewed, various methods exist to filter out data that might hinder the model’s performance or cause unintended effects. Any text that falls too far outside the language distribution, as well as text that is offensive or contains sensitive personal information, should be removed.

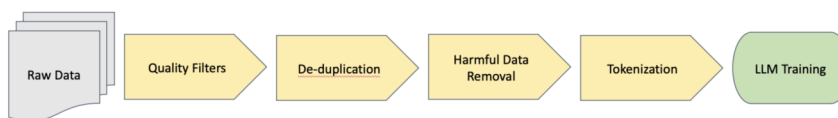


Fig. 2.7: A general sequence of steps to prepare a large corpus for use in LLM pre-training.

2.4.2.1 Low-Quality Data

As shown in Fig. 2.7, the first pre-processing stage is focused on overall data quality. Since the raw corpora tend to be substantially large, one can usually afford to remove sizable portions of data that show any signs of being unsuitable for training. As such, this stage of pre-processing can be somewhat coarse-grained.

One typical quality issue that may arise in large corpora is languages that fall outside the model’s intended use. If the model is being trained specifically for Spanish applications, for instance, then the presence of any languages other than Spanish will decrease training efficiency. These data can be filtered out with a language classification model or a more rule-based approach.

Another helpful pre-processing step is statistical filtering based on unusual text patterns. Some examples include a high frequency of strings much longer than a typical word, a high density of punctuation characters, and a prevalence of very long or short sentences. Any of these patterns indicate that the document or set of documents will be less generalizable and, therefore, less valuable for the model during training.

2.4.2.2 Duplicate Data

There has been considerable discussion about the effects of duplicate training data. [Hernandez et al. \(2022\)](#) observed several potential negative consequences from training on repeated data. As a counterpoint, analysis by [Biderman et al. \(2023\)](#) indicated that training on duplicated data neither benefits nor hurts the model. At any rate, training on duplicated data appears to be a suboptimal use of compute cycles, even in the best-case scenario. It is, therefore, a standard practice to remove repeated text wherever possible during the pre-processing stage.

2.4.2.3 Harmful Data

The above issues are primarily about optimizing training cycles using only the most applicable data. A further concern is that certain information may be undesirable for the model to capture. For example, it could be problematic if real people's names and email addresses appear in LLM-generated outputs after being scraped from the web. Toxicity and bias present in the training data are also significant areas of concern. Combating these elements is a more complex matter that will be discussed in later chapters, but removing offensive language in the pre-processing stage is worthwhile wherever possible.

2.4.2.4 Text Normalization

Some data may suffer from less severe issues that need to be cleaned up but don't warrant complete removal of the text. For example, data scraped from the web will naturally contain remnants of HTML tags that should be stripped out. Another common step is Unicode normalization, which addresses the fact that equivalent strings can be represented with multiple possible encodings. Rather than forcing the model to try to learn these equivalencies, it is usually preferable to standardize the representation as much as possible using one of several methods. Similarly, if desired, one can optionally choose to lowercase all text so that the model will not treat capital letters as distinct characters.

2.4.2.5 Tokenization

Upon completion of pre-processing, the data are then used to train a tokenizer such as those described in Sect. 2.3.3. Naturally, this must be done before the actual LLM can be trained since the tokenized output is the input to the model. A frequent practice is to use an existing tokenizer rather than training one from scratch, but this is only an option if similar data sources are used. First and foremost, the tokenization must reflect the languages (or programming languages) included in the training data. Additionally, conversational data might gravitate toward shorthand tokens such as “thx” or “omg”, while the academic literature might have a rather different distribution of tokens representing technical terminology.

The data are fed through the tokenizer in chunks of text, each of which is mapped to a sequence of tokens. For efficiency, the tokens are represented as vectors of integers with length l given by the number of subwords. The first layer of the model, also called the embedding layer, has dimensions $n \times m$, where n corresponds to the total number of tokens learned by the tokenizer and m is a predetermined embedding size. Thus, the tokenized output is a list of index lookups to retrieve vectors of size m for every token identified in the original input. The text has now been converted into a $l \times m$ matrix of floating point values that can be passed through the model to initiate the learning process.

2.4.3 Effects of Data on LLMs

As discussed previously, many data sources are available for training LLMs. The results produced by Gopher [Rae et al. \(2022\)](#) demonstrated that varying the percentages of data from each source had notable effects on the overall performance of the LLM for an assortment of downstream tasks. In general, data diversity consistently results in better performance across many tasks; however, it is also essential to consider the intended applications of the model. In building a chatbot, one would likely want a substantial portion of the training data to be conversational. Conversely, unless the chatbot dispenses legal advice, including many legal documents would not be sensible.

The amount of data seen by the model during pre-training has a substantial effect. This became abundantly clear with the release of Chinchilla [Hoffmann et al. \(2022\)](#), which demonstrated that previous LLMs had been undertrained. In pursuing the powerful capabilities that emerge with increasing model size, the effects of data size have been miscalculated. Through empirical trials, the Chinchilla researchers sought to establish a formula for determining the optimal number of parameters *and* training tokens for a given compute budget. They found that model size and data size should increase roughly in proportion, a stark contrast to previous work that emphasized the increase in parameters. This was a significant result, showing that highly capable LLMs could be

smaller than previously thought. Following these guidelines, the pre-training budget is used more efficiently, and fine-tuning and inference are less expensive.

2.4.4 Task-Specific Datasets

For research purposes, NLP “tasks” are often used as a general measure to approximate how well a given model will perform in various real-world settings. Most task-specific datasets are carefully curated and labeled for supervised training and evaluation. As a result, they tend to be much smaller than the very large unlabeled datasets used for LLM pre-training.

Task-specific datasets are generally pre-split into train and test sets to ensure that all researchers train and test on the same examples. Evaluating the performance on these standardized datasets allows direct comparisons between different architectures and training strategies. Importantly, LLMs can often achieve favorable evaluation metrics on a test set without seeing examples from the corresponding training data; this is called zero-shot learning.

2.5 Pre-trained LLM Design Choices

This section explores the multifaceted design elements that set apart various LLMs (Zhao et al., 2023). Specifically, we will discuss the nuances of pre-training tasks, delve into different pre-training objectives, examine the intricacies of Transformer architectural choices, and shed light on various decoding strategies.

2.5.1 Pre-Training Methods

Understanding the diverse methodologies for pre-training is critical for effectively deploying language models in various domains. Each method has benefits and challenges and suits particular tasks and data types. This section will explore five main pre-training methods, providing a clear overview of how each works, where it is used, and its pros and cons (Kalyan et al., 2021).

2.5.1.1 Pre-training from Scratch

Pre-training from scratch (PTS) involves training Transformer models from the ground on extensive volumes of unlabeled text. This foundational method is cru-

cial for initializing Transformer-based pre-trained language models, which typically comprise an embedding layer followed by multiple Transformer layers. PTS is beneficial because it does not rely on prior knowledge, making it a versatile starting point for various applications. However, this approach requires substantial computational resources and time, especially when dealing with large models and datasets. Models like BERT, RoBERTa, ELECTRA, and T5 are pre-trained from scratch on large volumes of unlabeled text.

2.5.1.2 Continual Pre-training

Continual pre-training (CPT) is a subsequent step following PTS, where the model undergoes further training on a domain-specific corpus. This method is helpful for tasks requiring specialized knowledge, enhancing the model's performance in specific domains. For instance, BioBERT is a variant of BERT that has undergone CPT on biomedical texts, making it adept at tasks related to the biomedical and clinical domains. The drawback of CPT is that it might lead the model to overfit the domain-specific corpus, potentially losing its generalizability.

2.5.1.3 Simultaneous Pre-training

Simultaneous pre-training (SPT) is a method in which models are simultaneously pre-trained on a combination of domain-specific and general-domain corpora. This approach aims to strike a balance, allowing the model to acquire general and domain-specific knowledge concurrently. An example of SPT is ClinicalBERT, which is pre-trained on a mixed corpus of clinical notes and general-domain text. While SPT offers a balanced knowledge base, the challenge lies in effectively selecting and combining corpora to avoid bias toward either domain.

2.5.1.4 Task Adaptive Pre-training

Task adaptive pre-training (TAPT) is a technique for pre-training on a small, task-related corpus. This method is less resource intensive than other methods and is particularly useful when the available data for a specific task are limited. TAPT can complement other pre-training approaches, as it can further refine models that have undergone PTS or CPT, enhancing their performance on specific tasks. However, the effectiveness of TAPT relies heavily on the relevance and quality of the task-related corpus used for pre-training.

2.5.1.5 Knowledge Inherited Pre-training

Knowledge inherited pre-training (KIPT) is a novel method that utilizes self-supervised learning and inherits knowledge from existing pre-trained models. This approach is inspired by the human learning process, which involves learning from knowledgeable individuals in addition to self-learning. KIPT is efficient because it reduces the time and resources required for pre-training from scratch. However, the success of KIPT depends on the quality and relevance of the knowledge inherited from existing models, and it might not always be straightforward to combine or transfer this knowledge effectively.

2.5.2 Pre-training Tasks

Supervised learning has been pivotal in AI advancement, necessitating extensive human-annotated data for practical model training. While proficient in specific tasks, these models often require substantial amounts of labeled data, making the process costly and time intensive, especially in specialized fields like medicine and law, where such data is scarce. Furthermore, supervised learning models lack generalization capabilities, often learning only from provided data, leading to generalization errors and unintended correlations. Recognizing these limitations, researchers are exploring alternative paradigms such as self-supervised learning (SSL). SSL is a learning paradigm in which labels are automatically generated based on data attributes and the definition of pre-training tasks. It helps models learn universal knowledge through pseudo-supervision provided by pre-training tasks. The primary objectives of SSL are to learn universal language representations and improve generalization ability by utilizing a large amount of freely available unlabeled data.

The loss function for SSL is given by:

$$\mathcal{L}_{SSL} = \lambda_1 \mathcal{L}_{PT1} + \lambda_2 \mathcal{L}_{PT2} + \dots + \lambda_m \mathcal{L}_{PTm} \quad (2.20)$$

where:

- $\mathcal{L}_{SSL}$ is the total loss function for SSL.
- $\mathcal{L}_{PT1}, \mathcal{L}_{PT2}, \dots, \mathcal{L}_{PTm}$ are the loss functions associated with each pre-training task.
- $\lambda_1, \lambda_2, \dots, \lambda_m$ are the weights assigned to each pre-training task's loss, controlling their contribution to the total loss.

Numerous self-supervised pre-training tasks have been established to train various LLMs (Kalyan et al., 2021). The following section will explore some of the prevalent pre-training tasks employed in LLMs.

2.5.2.1 Causal Language Model

Causal language modeling (CLM) is utilized for predicting the next word in a sequence based on the context, which can be either left-to-right or right-to-left. For a given sequence $x = \{x_1, x_2, x_3, \dots, x_{|x|}\}$, where $|x|$ represents the number of tokens in the sequence, the loss function for CLM is defined as:

$$\mathcal{L}_{\text{CLM}}^{(x)} = -\frac{1}{|x|} \sum_{i=1}^{|x|} \log P(x_i | x_{<i}) \quad (2.21)$$

where $x_{<i}$ represents the tokens preceding x_i in the sequence.

2.5.2.2 Masked Language Model

Masked language modeling (MLM) is used in the pre-training phase, where selected tokens are masked in the input sequence, and the model is trained to predict these masked tokens. Let $x_{\setminus M_x}$ represent the masked version of x , and M_x represent the set of masked token positions in x . The loss function for MLM is defined as:

$$\mathcal{L}_{\text{MLM}}^{(x)} = -\frac{1}{|M_x|} \sum_{i \in M_x} \log P(x_i / x_{\setminus M_x}) \quad (2.22)$$

The model aims to minimize this loss by learning to predict the masked tokens accurately, thereby gaining a deeper understanding of the language structure. BERT, a prominent model in natural language processing, employs MLM as a pre-training task, selecting tokens to be masked with a probability of 0.15.

2.5.2.3 Replaced Token Detection

Replaced token detection (RTD) mitigates the drawbacks of MLM by enhancing the training signals and minimizing the discrepancy between the pre-training and fine-tuning phases. Unlike MLM, which uses special mask tokens for corruption, RTD corrupts sentences with tokens generated by a model pre-trained with the MLM objective. This approach transforms the task into a binary classification at the token level, where each token is classified as either replaced or not. The procedure involves two steps: first, training a generator model with the MLM objective, and second, training a discriminator model (initialized from the generator) with the RTD objective. The loss function for RTD is expressed as:

$$\mathcal{L}_{\text{RTD}}^{(x)} = -\frac{1}{|\hat{x}|} \sum_{i=1}^{|\hat{x}|} \log P(d / \hat{x}_i) \quad (2.23)$$

where $d \in \{0, 1\}$ denotes whether a token is replaced (1) or not (0), $\hat{x}$ is the corrupted sentence, and $P(d / \hat{x}_i)$ represents the probability of a token being replaced or not.

2.5.2.4 Shuffled Token Detection

Shuffled token detection (STD) is designed to improve the model's understanding of coherent sentence structures, ultimately enhancing its performance across various tasks. In this task, tokens within a sequence are shuffled with a probability of 0.15. The loss function associated with STD is given by:

$$\mathcal{L}_{\text{STD}}^{(\mathbf{x})} = -\frac{1}{|\hat{\mathbf{x}}|} \sum_{i=1}^{|\hat{\mathbf{x}}|} \log P(d/\hat{x}_i) \quad (2.24)$$

In this equation, $d \in \{0, 1\}$ denotes whether a token is replaced (1) or not (0), and $\hat{\mathbf{x}}$ is the corrupted sentence. The model aims to minimize this loss by learning to identify and comprehend the shuffled tokens within the sequence context effectively.

2.5.2.5 Random Token Substitution

Random token substitution (RTS) is a method introduced by [Liello et al. \(2021\)](#) for identifying tokens that have been randomly substituted in a sequence. In this technique, 15% of the tokens in a given sequence are randomly replaced with other tokens from the vocabulary. This approach is efficient because it does not require a separate generator model to corrupt the input sequence. The loss function for RTS is articulated as:

$$\mathcal{L}_{\text{RTS}}^{(\mathbf{x})} = -\frac{1}{|\hat{\mathbf{x}}|} \sum_{i=1}^{|\hat{\mathbf{x}}|} \log P(d/\hat{x}_i) \quad (2.25)$$

where $d \in \{0, 1\}$ signifies whether a token has been randomly substituted (1) or not (0), and $\hat{\mathbf{x}}$ is the sequence obtained by randomly substituting 15% of the tokens in the original sequence $\mathbf{x}$.

2.5.2.6 Swapped Language Modeling

Swapped language modeling (SLM) addresses the discrepancy in the MLM pre-training task caused by using a special mask token. This discrepancy occurs between the pre-training and fine-tuning stages. SLM mitigates this by corrupting the input sequence with random tokens selected from the vocabulary with a probability of 0.15. Although SLM is akin to MLM in predicting the corrupted tokens, it differs by replacing tokens with random ones instead of mask tokens. Although SLM and RTS both employ random tokens for corruption, SLM is not as sample-efficient as RTS. This inefficiency arises because SLM involves only 15% of input tokens, whereas RTS engages every token in the input sequence. The loss function for SLM is defined as:

$$\mathcal{L}_{\text{SLM}}^{(x)} = -\frac{1}{|R_x|} \sum_{i \in R_x} \log P(x_i / x_{\setminus R_x}) \quad (2.26)$$

where R_x represents the set of positions of randomly substituted tokens, and $x_{\setminus R_x}$ represents the corrupted version of x .

2.5.2.7 Translation Language Modeling

Translation language modeling (TLM) is designed for pre-training multilingual models. Given a pair of sentences in different languages, TLM masks some tokens in both sentences and trains the model to predict the masked tokens. The loss function for TLM is defined as:

$$\mathcal{L}_{\text{TLM}}^{(x)} = -\frac{1}{|M_x|} \sum_{i \in M_x} \log P(x_i / x_{\setminus M_x}, y_{\setminus M_y}) - \frac{1}{|M_y|} \sum_{i \in M_y} \log P(y_i / x_{\setminus M_x}, y_{\setminus M_y}) \quad (2.27)$$

In this context, M_x and M_y denote the sets of masked positions within sentences x and y , while $x_{\setminus M_x}$ and $y_{\setminus M_y}$ signify the masked versions of x and y respectively.

2.5.2.8 Alternate Language Modeling

Alternate language modeling (ALM) is used for cross-lingual model pre-training. It involves alternating the language of each sentence in the input sequence. Given a pair of parallel sentences (x, y) , a code-switched sentence is created by randomly replacing some phrases in x with their translations from y . ALM follows the same masking procedure as the standard MLM for selecting tokens to be masked. By pre-training the model on these code-switched sentences, the model can learn relationships between languages more effectively.

$$\mathcal{L}_{\text{ALM}}^{(z(x,y))} = -\frac{1}{|M|} \sum_{i \in M} \log P(z_i / z_{\setminus M}) \quad (2.28)$$

In this context, z represents the code-switched sentence generated from x and y , $z_{\setminus M}$ denotes the masked version of z , and M is the set of masked token positions within $z_{\setminus M}$.

2.5.2.9 Sentence Boundary Objective

Sentence boundary objective (SBO) involves predicting masked tokens based on span boundary tokens and position embeddings. The loss function for SBO is defined as:

$$\mathcal{L}_{\text{SBO}}^{(x)} = -\frac{1}{|S|} \sum_{i \in S} \log P(x_i / f(x_{s-1}, x_{e+1}, p_{s-e+1})) \quad (2.29)$$

where $f()$ is a two-layered feed-forward neural network, S represents the positions of tokens in the contiguous span, s and e represent the start and end positions of the span, respectively, and p represents the position embedding.

2.5.2.10 Next Sentence Prediction

Next sentence prediction (NSP) is a binary sentence pair classification task. The loss function for NSP is defined as:

$$\mathcal{L}_{\text{NSP}}^{(x,y)} = -\log P(d/x, y) \quad (2.30)$$

where d is a binary variable representing whether the sentences (x, y) are consecutive (1) or not (0).

2.5.2.11 Sentence Order Prediction

Sentence order prediction (SOP) focuses on sentence coherence, unlike NSP, which also includes topic prediction. SOP, introduced by ALBERT, involves determining whether sentences are in the correct order or swapped. The training instances are balanced with 50% swapped. The SOP loss is defined as:

$$\mathcal{L}_{\text{SOP}}^{(x,y)} = -\log P(d/x, y) \quad (2.31)$$

where $d \in \{1, 0\}$ indicates whether the sentences are swapped.

2.5.2.12 Sequence-to-Sequence Language Modeling

Sequence-to-Sequence Language Modeling (Seq2Seq) is an extension of MLM used for pre-training encoder-decoder-based models. The loss function for Seq2Seq is defined as:

$$\mathcal{L}_{\text{Seq2Seq}}^{(x)} = -\frac{1}{l_s} \sum_{s=i}^j \log P(x_s / \hat{x}, x_{i:s-1}) \quad (2.32)$$

where $\hat{x}$ is the masked version of x and l_s represents the length of the masked n-gram span.

2.5.2.13 Denoising Autoencoder

The denoising autoencoder (DAE) involves reconstructing the original text from the corrupted text. The loss function for DAE is defined as:

$$\mathcal{L}_{\text{DAE}} = -\frac{1}{|\mathbf{x}|} \sum_{i=1}^{|\mathbf{x}|} \log P(x_i/\hat{x}, x_{<i}) \quad (2.33)$$

where $\hat{x}$ is the corrupted version of x .

2.5.3 Architectures

Initially proposed by [Vaswani et al. \(2017\)](#), Transformers are composed of stacks of encoder and decoder layers. A Transformer-based language model can be pre-trained using a stack of encoders, decoders, or both, thus resulting in various architectures, as shown in [Fig. 2.8](#).

2.5.3.1 Encoder-Decoder

The encoder-decoder architecture is a two-part structure in which the encoder processes the input sequence, and the decoder generates the output. The encoder transforms the input into a continuous representation that holds all the learned information of the input. The decoder then uses this representation to generate the output sequence. This architecture is beneficial for sequence-to-sequence tasks such as machine translation and text summarization. For instance, in a machine translation task, the encoder processes the input sentence in the source language, and the decoder generates the translation in the target language. The attention mechanism in this architecture allows the model to focus on different parts of the input sequence while generating the output, providing a dynamic computation of context.

2.5.3.2 Causal Decoder

The causal decoder architecture is designed for autoregressive tasks where the model generates the output token by token. This architecture employs a unidirectional attention mechanism, meaning that each token can only attend to previous tokens and itself during the generation process. This is particularly useful for text generation tasks where the model needs to generate coherent and contextually appropriate text. For example, in text completion tasks, the model predicts the next token based on the previous ones, ensuring that the generated text is coherent and contextually relevant.

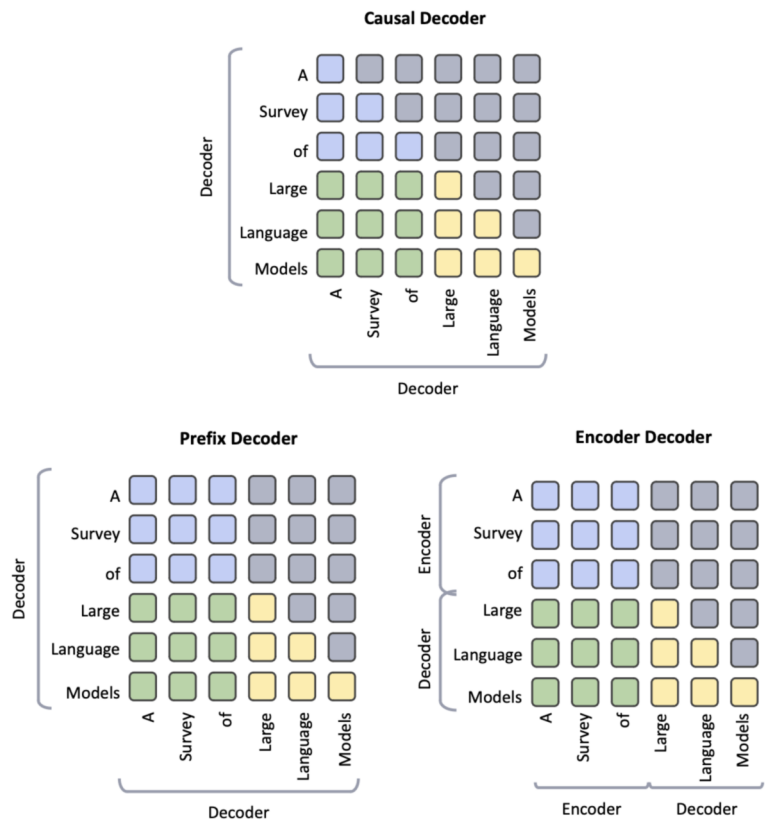


Fig. 2.8: Analysis of attention patterns across three primary architectures. In this context, the blue, green, yellow, and gray rounded shapes represent attention within prefix tokens, attention between prefix and target tokens, attention among target tokens, and masked attention, respectively.

2.5.3.3 Prefix Decoder

The prefix decoder architecture is a variation of the causal decoder where the model can attend bi-directionally to a prefix of tokens while maintaining unidirectional attention for the rest. This hybrid attention mechanism allows the model to have a broader context while generating each token, making it effective for tasks that require understanding both previous and subsequent tokens in a sequence. For instance, the model can attend to the dialog history and the partially generated response in a dialog system while generating the next token.

2.5.3.4 Encoder

The encoder is designed to efficiently process and understand the contextual information embedded within input sequences, making it a preferred choice for certain NLP tasks. Each encoder layer within the architecture generates a robust contextual representation of the input sequence. The final output from the last encoder layer is utilized as the contextual representation, serving as a valuable input for diverse downstream tasks. The encoder architecture is particularly advantageous for tasks requiring a deep understanding of token context without requiring sequence generation, such as classification tasks.

2.5.3.5 Mixture-of-Experts

The Mixture-of-Experts (MoE) architecture is a variant of Transformer models that incorporates MoE layers, replacing the standard feed-forward blocks as shown in Fig. 2.9. These layers contain multiple parallel units called “experts”, each with unique parameters. A router directs input tokens to specific experts based on their capabilities. Experts, which are feed-forward layers following the attention block, process tokens independently. Unlike traditional models where capacity increases lead to higher computational costs, the MoE architecture simultaneously activates only a few experts. This sparse activation allows the architecture to support larger model sizes without a proportional increase in computational demand, maintaining efficient performance.

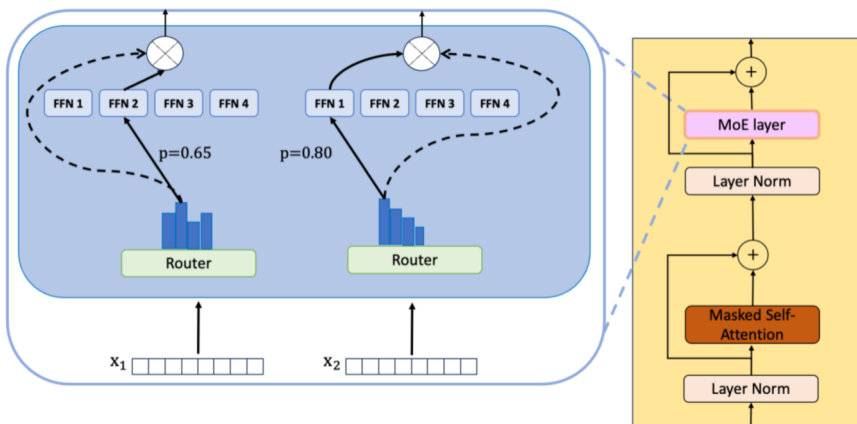


Fig. 2.9: Mixture-of-experts variant of the Transformer architecture.

2.5.4 LLM Pre-training Tips and Strategies

This section will explore the key configurations, methods, and strategies for training LLMs.

2.5.4.1 Training Methods

- **Learning Rate** Most LLMs follow a similar learning rate schedule with warm-up and decay phases during pre-training. Initially, the learning rate is gradually increased for approximately 0.1% to 0.5% of the training steps, typically ranging from 5×10^{-5} to 1×10^{-4} . After this phase, the learning rate is progressively reduced using a cosine decay strategy.
- **Batch Size** During language model pre-training, it is common to use large batch sizes, often with 2,048 examples or 4M tokens, to enhance stability and efficiency. Models such as GPT-3 and PaLM employ a dynamic approach, adjusting the batch size throughout training, with GPT-3's batch size, for instance, expanding from 32K to 3.2M tokens. This adaptive batch sizing has been shown to stabilize LLM training effectively.
- **Optimizers** For training LLMs such as GPT-3, the Adam and AdamW optimizers are commonly used. These optimizers adapt based on gradient estimations with typical hyper-parameters: $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 10^{-8}$. Additionally, the Adafactor optimizer, a memory-efficient variant of Adam, is employed for models such as PaLM and T5. Its hyper-parameters are $\beta_1 = 0.9$ and β_2 adjusted based on the number of training steps.

2.5.4.2 Decoding Strategies

Greedy Search

This autoregressive decoding mechanism is one of the techniques utilizing decoder-only architectures. A most common decoding method herein is the *greedy search*. This method predicts the most probable token at each generation step, conditioned on the previously generated tokens. The mathematical formulation of this process is as follows:

$$x_i = \arg \max_x P(x|x_{<i}),$$

where x_i denotes the token predicted at the i -th step, which is the most probable token given the context $x_{<i}$. Consider a partial sentence, “The sky is so”, for illustration. The greedy search method might predict “blue” as the next token, given its high likelihood of completing the sentence appropriately. This approach is efficient in text generation tasks such as machine translation and text summarization, where there is a strong dependency between the input and the expected output.

The greedy search offers reliable results by leveraging probability and context in scenarios where the output must align closely with the input. This decoding strategy is not limited to decoder-only architectures and can be applied to encoder-decoder and prefix-decoder models. Many improvements to greedy search have been proposed, and we will discuss some of them here. Beam search is a notable strategy, holding onto the top- n probable sentences during each decoding step and ultimately choosing the one with the highest probability.

! Practical Tips

Typically, a beam size between 3 to 6 is adequate, though increasing it may reduce performance. Length penalty, or length normalization, is another improvement that compensates for beam search's tendency to prefer shorter sentences. This method modifies sentence probability about its length, applying an exponential power as a divisor. Penalties for generating previously used tokens have been introduced to mitigate the issue of generating repetitive tokens or n -grams. Additionally, diverse beam search offers a valuable improvement, yielding a variety of outputs from a single input.

Random Search

Sampling-based methods offer an alternative decoding strategy, introducing a probabilistic approach to token selection to foster diversity and randomness in text generation. This strategy is beneficial when the goal is to generate both varied and engaging text. For instance, given the context sentence, "I am thirsty. I would like a cup of", the probability distribution of the next token might favor words such as "tea," "coffee," or "water." However, sampling-based methods still allow the selection of words with lower probabilities, albeit at a reduced likelihood. While "tea" has the highest probability, words such as "coffee," "water," and "juice" still have a chance of being selected, introducing diversity to the responses. This approach applies to various architectures, including decoder-only, encoder-decoder, and prefix decoder models, offering flexibility for different language generation tasks.

Improvements to random sampling have been developed to enhance the quality of generated text by mitigating the selection of words with extremely low probabilities. One such improvement is temperature sampling, which adjusts the softmax function's temperature coefficient when calculating each token's probability over the vocabulary. This is given by:

$$P(x_j|x_{<i}) = \frac{\exp(l'_j/t)}{\sum_{j'} \exp(l'_{j'}/t)}$$

where l'_j denotes the logits of each word and t is the temperature coefficient. By reducing the temperature, words with higher probabilities are more likely to be selected, while those with lower probabilities are less likely. For instance, with a temperature of 1, the method defaults to random sampling. As the temperature ap-

proaches 0, it becomes akin to a greedy search, and as it increases indefinitely, it transitions to uniform sampling.

! Practical Tips

Another improvement is Top- k sampling. This approach involves truncating tokens with lower probabilities and only sampling from those with the top k highest probabilities. Top- p sampling, or nucleus sampling, is another strategy. It samples from the smallest set of tokens whose cumulative probability is greater than or equal to a specified value p . This set is constructed by progressively adding tokens (sorted by descending generative probability) until the cumulative probability surpasses p . For example, if the tokens are sorted and added until their cumulative probability exceeds 0.8, only those tokens are considered for sampling.

2.5.4.3 3D Parallelism

3D parallelism integrates three key parallel training techniques—data, pipeline, and tensor parallelism—for efficiently training LLMs.

Data Parallelism

This method enhances training speed by distributing model parameters and the training dataset across multiple GPUs. Each GPU processes its data and calculates gradients, and then these gradients are combined and used to update the model on each GPU.

! Practical Tips

The ZeRO technique, introduced by the DeepSpeed library, addresses memory redundancy in data parallelism. Typically, data parallelism forces every GPU to store an identical copy of an LLM, encompassing model parameters, gradients, and optimizer parameters (Rajbhandari et al., 2020). However, this redundancy leads to extra memory usage. ZeRO's solution is to keep only a portion of the data on each GPU, fetching the rest from other GPUs as needed. Three strategies based on data storage are proposed: optimizer state partitioning, gradient partitioning, and parameter partitioning. Tests show that the first two do not add to communication costs, while the third increases communication by approximately 50% but conserves memory based on the GPU count. PyTorch has also introduced a technique akin to ZeRO, named FSDP.

Pipeline Parallelism

Here, different layers of an LLM are spread across several GPUs. Sequential layers are assigned to the same GPU to minimize the data transfer costs. While basic imple-

mentations might under-utilize GPUs, advanced methods like GPipe and PipeDream enhance efficiency by processing multiple data batches simultaneously and updating gradients asynchronously (Harlap et al., 2018; Huang et al., 2019).

Tensor Parallelism

This technique divides LLMs' tensors or parameter matrices for distribution across multiple GPUs. For instance, the parameter matrix can be split column-wise and processed on different GPUs during matrix multiplication. The results from each GPU are then merged. Libraries such as Megatron-LM support tensor parallelism, which can be applied to more complex tensors (Shoeybi et al., 2019).

2.6 Commonly Used Pre-trained LLMs

This section delves into three prominent LLM architectures, examining them from the perspectives of the datasets employed, their alignment with the Transformer architecture, essential insights, and their diverse variants.

2.6.1 BERT (Encoder)

The *Bidirectional Encoder Representation from Transformer* (BERT) is a pre-trained model that employs an attention mechanism to better comprehend linguistic context (Devlin et al., 2019). BERT consists of multiple encoder segments, each contributing to its robustness. Upon its introduction, BERT set new benchmarks for a range of NLP tasks, such as question answering on the SQuAD v1.1 dataset and natural language inference on the MNLI dataset. Unlike traditional language models that process text sequences in a unidirectional manner, BERT's bidirectional training approach offers a more comprehensive understanding of linguistic context and sequence flow.

2.6.1.1 Dataset

BERT's training data primarily comprise Wikipedia, accounting for approximately 2.5 billion words, and the BooksCorpus, which contains approximately 800 million words.

2.6.1.2 Architecture

BERT is an encoder-only Transformer and offers various pre-trained models differentiated by their architectural scale. Two examples include:

- **BERT-BASE** consists of 12 layers, 768 hidden nodes, 12 attention heads, and 110 million parameters.
- **BERT-LARGE** is a more extensive version with 24 layers, 1024 hidden nodes, 16 attention heads, and 340 million parameters.

The training of BERT-BASE utilized four cloud TPUs over four days, while BERT-LARGE required 16 TPUs for the same duration.

2.6.1.3 Training

BERT operates in two phases—pre-training and fine-tuning—as shown in Fig. 2.10. The model learns from unlabeled data across various tasks in the initial pre-training phase. During the fine-tuning phase, the model starts with the parameters acquired from the pre-training and then optimizes these parameters using labeled data specific to the target tasks.

BERT's training methodology combines two objectives: the *masked language model* (MLM) and *next sentence prediction* (NSP). The combined loss function of these techniques is minimized during training. For BERT, each training instance is a pair of sentences that may or may not be sequential in the original document. The special tokens [CLS] and [SEP] denote the beginning of the sequence and the separation between sentences, respectively. A subset of tokens in the training instance is either masked with a [MASK] token or substituted with a random token. Before being input into the BERT model, tokens are transformed into embedding vectors. These vectors are then enhanced with positional encodings, and in BERT's unique approach, segment embeddings are added to indicate whether a token belongs to the first or second sentence.

Once pre-trained, BERT can be adapted for various downstream tasks, whether for individual texts or pairs of texts. General linguistic representations, derived from BERT's 350 million parameters trained on 250 billion tokens, have significantly advanced the state of the art in numerous NLP tasks. During the fine-tuning process, additional layers can be incorporated into BERT. These layers and the pre-trained BERT parameters are updated to align with the training data of specific downstream tasks. The Transformer encoder, essentially a pre-trained BERT, accepts a sequence of text and uses the [CLS] representation for predictions. For example, [CLS] is replaced with actual classification labels in sentiment analysis or classification tasks. During this fine-tuning phase, the cross-entropy loss between the predictions and actual labels is minimized via gradient-based methods. The additional layers are trained from scratch, and the pre-trained BERT parameters undergo updates.

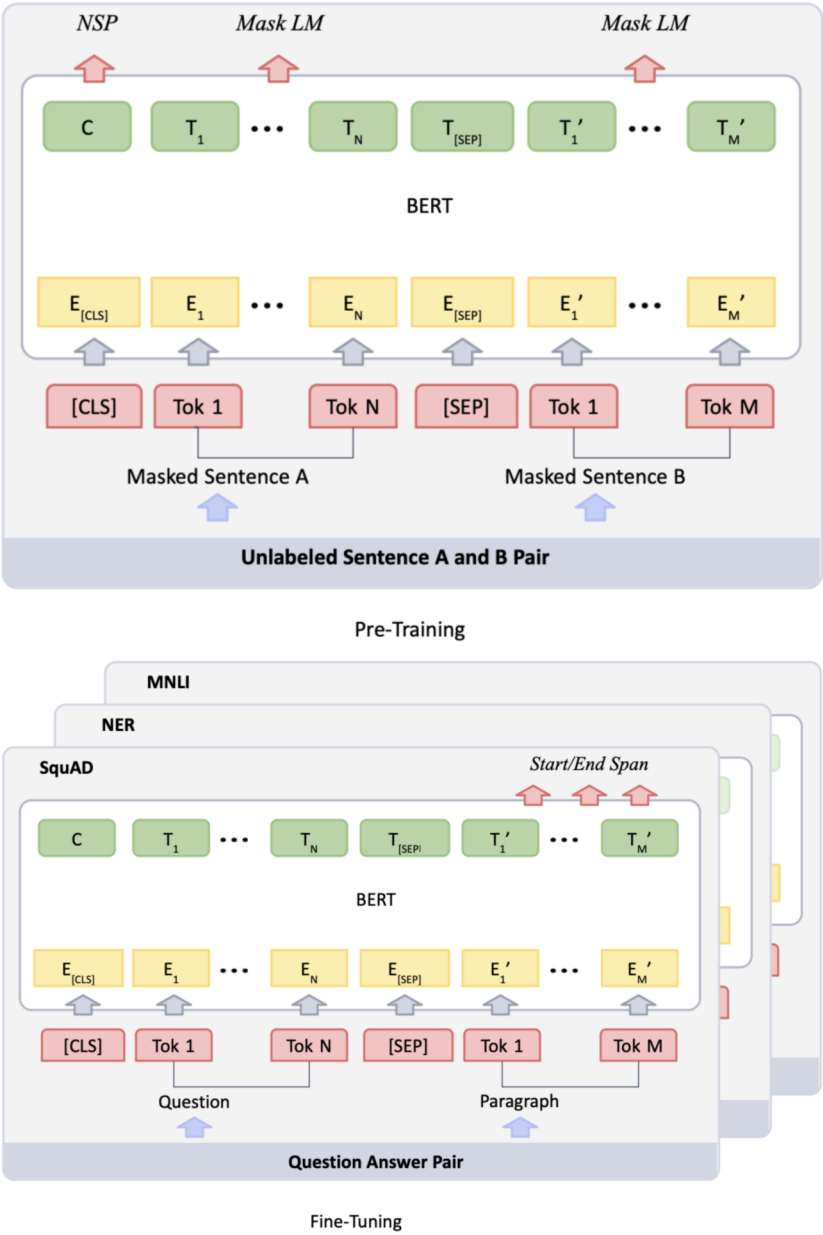


Fig. 2.10: BERT can adapt its pre-training objective to fine-tune on task-specific input data.

2.6.1.4 Key Takeaways

1. The scale of the model is crucial. BERT-LARGE, encompassing 345 million parameters, is the most extensive model in its category. Despite having the same structure, it outperforms BERT-BASE, which contains “merely” 110 million parameters.
2. Given sufficient training data, increasing training steps correlates with enhanced accuracy. For example, in the MNLI task, BERT-BASE’s accuracy sees a 1.0% boost when trained for 1 million steps (with a batch size of 128,000 words) instead of 500K steps with an identical batch size.
3. While BERT’s bidirectional method (MLM) may converge at a slower rate than unidirectional (left-to-right) methods (given that only 15% of words are predicted in each batch), it surpasses the latter in performance after a limited number of pre-training iterations.

2.6.1.5 Variants

Subsequent developments and variations of BERT have been introduced to enhance model architectures or pre-training objectives ([Kamath et al., 2022](#)). Notably:

- **RoBERTa**: A BERT variant of the same size, pre-trained on 200 billion tokens. The loss function used in BERT was found to be less impactful in this context.
- **ALBERT**: Improves efficiency by enforcing parameter sharing.
- **SpanBERT**: Focuses on representing and predicting text spans.
- **DistilBERT**: A lightweight version achieved through knowledge distillation.
- **ELECTRA**: Emphasizes replaced token detection.

2.6.2 T5 (Encoder-Decoder)

The *Text-to-Text Transfer Transformer* (T5) model introduces a comprehensive framework that consolidates various NLP transfer learning process elements ([Raffel et al., 2020](#)). This includes diverse unlabeled datasets, pre-training goals, benchmarks, and methods for fine-tuning. The framework identifies optimal practices to achieve superior performance by integrating and comparing these components via ablation experiments.

2.6.2.1 Dataset

T5 sources its data from text extracted from the Common Crawl web archive. The researchers implemented basic heuristic filtering and pre-processing on these data. Post extraction, they eliminated inappropriate language, placeholder text (such as Lorem Ipsum), code brackets such as “{”, duplicate content, and sentences lacking terminal punctuation. Given that the primary tasks target English text, they employed *langdetect*7 to exclude pages not identified as English with a confidence level of 99% or higher.

2.6.2.2 Architecture

The primary architecture employed for T5 is the encoder-decoder structure, which, with minor alterations, closely resembles the original Transformer design. A distinctive feature of T5 is its use of relative position embeddings, which generate learned embeddings based on the offset between the “key” and “query” in the self-attention process rather than fixed position embeddings. The research introduced five model variants:

- **Base:** A baseline model mirroring BERT_base, comprising 222 million parameters.
- **Small:** A reduced version of the Base, containing 60 million parameters and six layers for both encoders and decoders.
- **Large:** An enhanced version of the Base, equipped with 770 million parameters.
- **3B:** An expansion of the Base, boasting 3 billion parameters.
- **11B:** The largest variant, scaling the Base to 11 billion parameters.

2.6.2.3 Training

T5 employs a multi-task learning approach, combining various tasks during its pre-training phase. These tasks are categorized into two primary groups based on their training methodology:

1. Unsupervised Training:

- Involves training on the C4 dataset using traditional language model training tasks with a maximum likelihood objective.
- For unsupervised tasks like MLM, T5 utilizes 100 unique tokens, ranging from <extra_id_0> to <extra_id_99>, to format both input and output text. For instance, to mask “name is” in the sentence “My name is John Smith”, the input becomes “My <extra_id_0> John Smith” and the expected output is “<extra_id_0> name is <extra_id_1>”.

2. Supervised Training:

- Incorporates various NLP tasks like question-answering, summarization, and classification. The model is trained using curated data in a supervised manner. However, all tasks are adapted to fit the text-in-text-out format, which is suitable for encoder-decoder models, as shown in Fig. 2.11.
- The research employs a prompting technique, requesting the language model to produce answers textually. Every NLP task tackled by T5 is transformed into a text-to-text format. For instance, an input might appear as “translate English to German: The house is wonderful.</s>” and the corresponding output as “<pad> Das Haus ist wunderbar.</s>”.
- A series of ablation experiments were conducted to identify optimal component strategies, as shown in Fig. 2.12. Initially, three primary approaches were considered: (1) Language modeling, (2) BERT-style masking, and (3) Deshuffling. The BERT-style approach yielded the best results and was chosen for further analysis.
- Subsequent experiments explored corruption strategies, such as only masking tokens without swapping, masking tokens and replacing them with a sentinel token, and removing tokens.

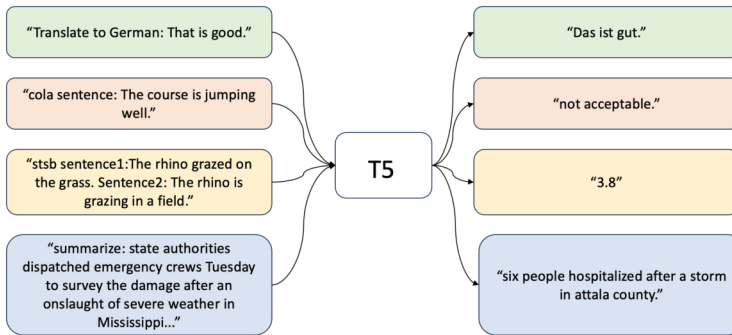


Fig. 2.11: T5 architecture unifying different NLP tasks as sequence-to-sequence and generating appropriate responses based on prompts.

2.6.2.4 Key Takeaways

- T5, especially the 11B variant, achieved state-of-the-art results in most NLP tasks, marking its dominance in 18 out of 24 tasks.
- The experiments underscored the value of providing the model with bi-directional context, enhancing its predictive capabilities.

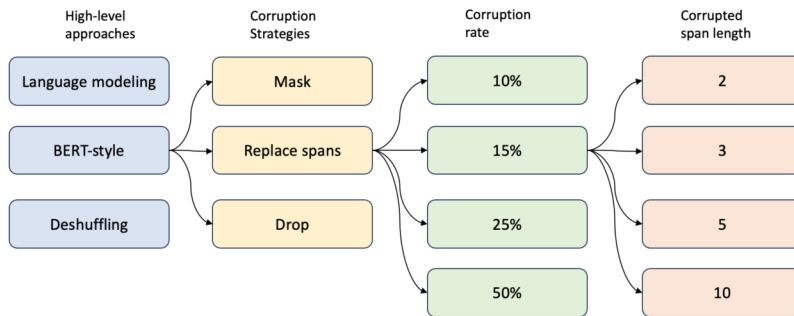


Fig. 2.12: Ablation experiment setup for choosing the winning strategy for T5.

- Word corruption objectives were the most effective, especially those resulting in shorter target sequences. This is attributed to the reduced computational cost of pre-training on shorter sequences.
- Filtering the training data, especially removing non-English content, proved beneficial. Moreover, domain-specific pre-training, such as on news articles, significantly improved performance on related downstream tasks.
- While the idea of training a single model on multiple tasks simultaneously seems appealing, it led to a decline in performance, especially on tasks like GLUE, SQuAD, and SuperGLUE.
- Making the model deeper and wider and extending the training duration led to marked improvements. Additionally, training multiple models and using ensemble methods further boosted performance.

2.6.2.5 Variants

1. **T5v1.1**: An enhanced version of T5 with architectural modifications, pre-trained solely on C4 without incorporating supervised tasks.
2. **mT5**: A multilingual T5 variant trained on the mC4 corpus encompassing 101 languages.
3. **byT5**: A T5 variant trained on byte sequences instead of SentencePiece subword token sequences.
4. **UL2**: A model similar to T5 that is pre-trained using diverse denoising objectives.
5. **Flan-T5**: T5 models trained using the Flan pre-training method, which is prompt-based. The datasets include taskmaster2, djaym7/wiki_dialog, deepmind/code_contests, and others.
6. **FLan-UL2**: The UL2 model fine-tuned with the “Flan” prompt tuning and dataset collection.

7. **UMT5:** A multilingual T5 model trained on the refreshed mC4 multilingual corpus with 29 trillion characters across 107 languages using the UniMax sampling method.

2.6.3 GPT (Decoder)

The *Generative Pre-trained Transformer* (GPT) models of OpenAI have revolutionized the NLP landscape with advanced language modeling capabilities (Brown et al., 2020; Radford et al., 2018, 2019). Remarkably, they can execute various NLP tasks without supervised training, from question answering and textual entailment to text summarization. This section delves into the three pivotal GPT iterations, GPT-1, GPT-2, and GPT-3, tracing their evolution. Subsequent models such as GPT 3.5, founded on InstructGPT (a method that utilizes Reinforcement Learning from Human feedback), will be thoroughly discussed in Chapter 5.

2.6.3.1 Dataset

1. Initially, GPT-1 language model pre-training was performed using the BooksCorpus dataset. Following this, it was fine-tuned on various specific language understanding tasks. For Natural Language Inference, datasets such as SNLI, MultiNLI, Question NLI, RTE, and SciTail were utilized. The model uses the RACE and Story Cloze datasets to address question-answering. Datasets such as the MSR Paraphrase Corpus, Quora Question Pairs, and STS Benchmark were selected to gauge the LM's performance in terms of sentence similarity. For tasks centered around classification, the Stanford Sentiment Treebank-2 and CoLA datasets served as the benchmarks.
2. For GPT-2 training, the authors curated the WebText dataset by extracting data from highly upvoted Reddit articles' outbound links. This 40GB dataset, comprising over 8 million documents, was more significant than the Book Corpus used for GPT-1. To ensure test set integrity, Wikipedia articles were excluded from WebText. Notably, GPT-2 was trained without task-specific fine-tuning, achieving results through zero-shot inference.
3. GPT-3 training utilized a combination of five distinct corpora, each assigned a specific weight for sampling. Datasets of higher quality were frequently sampled, with the model undergoing multiple training epochs. The datasets included were the Common Crawl, WebText2, Books1, Books2, and Wikipedia datasets.

2.6.3.2 Architecture

Table 2.3 illustrates the variations in the decoder-only architectures adopted by all the GPT models.

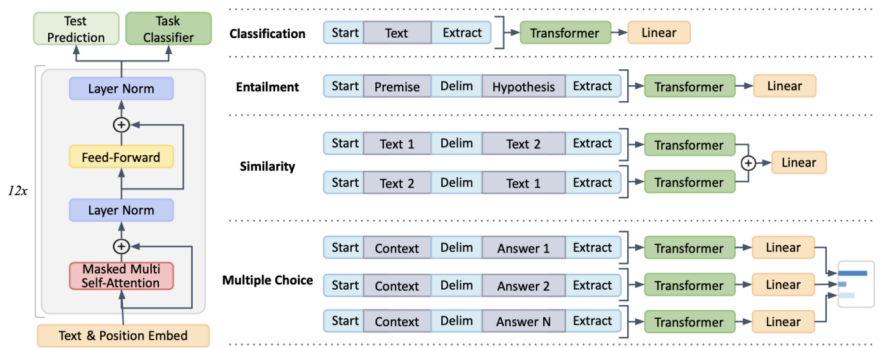


Fig. 2.13: The GPT-1 architecture and designated training objectives employed for training. Structured inputs are converted into sequences of tokens for fine-tuning different tasks, which the pre-trained model processes, followed by implementing a linear layer with a softmax layer.

Table 2.3: Comparisons of the GPT-1, GPT-2, and GPT-3 models.

Characteristic	GPT-1	GPT-2	GPT-3
Parameters	117 Million	1.5 Billion	175 Billion
Decoder Layers	12	48	96
Context Token Size	512	1024	2048
Hidden Layer Size	768	1600	12288
Batch Size	64	512	3.2M

1. GPT-1 employs a 12-layer Transformer structure that is solely decoder-based, aligning with the original Transformer decoder, except for utilizing learnable positional embeddings instead of the fixed positional embeddings in the original Transformer as shown in Fig. 2.13.
2. GPT-2 retains the architectural design of GPT-1 but is significantly larger, with 1.5 billion parameters, which is a tenfold increase from GPT-1’s 117 million parameters.
3. GPT-3 maintains the architectural foundation set by GPT-1 and GPT-2. However, it distinguishes itself from GPT-2 in aspects such as context token size and the number of layers.

2.6.3.3 Training

1. GPT-1 follows a two-stage training procedure, starting with unsupervised pre-training and then supervised fine-tuning. The initial stage involves training a high-capacity language model on a large text corpus. Given an unsupervised

corpus of tokens $U = \{u_1, \dots, u_n\}$, the standard language modeling objective is used:

$$\mathcal{L}_1(U) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta) \quad (2.34)$$

where k is the size of the context window, and the conditional probability P is modeled using a neural network with parameters Θ . These parameters are trained using stochastic gradient descent.

After unsupervised pre-training, the model parameters are adapted to a supervised target task. Given a labeled dataset C , where each instance consists of a sequence of input tokens $x_1, \dots, x_m$ and a label y , the inputs are processed through the pre-trained model to obtain the final Transformer block's activation hm_j :

$$P(y | x_1, \dots, x_m) = \text{softmax}(hm_j W_y) \quad (2.35)$$

This results in the following objective:

$$\mathcal{L}_2(C) = \sum_{(x,y)} \log P(y | x_1, \dots, x_m) \quad (2.36)$$

Additionally, using language modeling as an auxiliary objective during fine-tuning improves generalization and convergence. The combined objective is:

$$\mathcal{L}_3 = \mathcal{L}_2(C) + \lambda \times \mathcal{L}_1(U) \quad (2.37)$$

Here, $\mathcal{L}_1(U)$ is the unsupervised pre-training objective, and $\mathcal{L}_2(C)$ is the supervised fine-tuning objective. The combined objective $\mathcal{L}_3$ leverages both stages. Task-specific transformations ensure that the pre-trained model can handle structured inputs for various tasks without significant architectural changes.

2. The primary methodology for training GPT-2 is rooted in language modeling, which is conceptualized as unsupervised distribution estimation from a collection of examples $(x_1, x_2, \dots, x_n)$, where each x_i is a sequence of symbols. The model is conditioned on the input and the specific task to achieve generalization across diverse tasks. Formally, the model aims to estimate:

$$p(\text{output} | \text{input}, \text{task}) \quad (2.38)$$

For instance, a translation task can be represented in the model as ("translate to French", "English text", "French text"). Traditionally, language models have been trained on singular domains. However, the approach here emphasizes the importance of a diverse dataset encompassing various domains and contexts to capture a broad spectrum of natural language patterns described in the dataset discussion.

3. GPT-3 was trained using autoregressive next-word prediction on an expansive corpus, as detailed in the datasets section. Instead of the traditional approach of fine-tuning models on specific tasks with dedicated training data, GPT-3 intro-

duces a paradigm shift by harnessing in-context learning. This means that GPT-3 can dynamically adapt to new tasks it has not been explicitly trained on simply by interpreting the context or examples in the prompt. Its various learning modes further exemplify the versatility of in-context learning in GPT-3. Few-shot learning involves guiding the model using multiple examples within the prompt. For instance, one might offer several English-French sentence pairs before presenting a new English sentence for translation to facilitate English-to-French translation. On the other hand, one-shot learning provides the model with only a single guiding example. In contrast, zero-shot learning does not rely on explicit examples; instead, GPT-3 is tasked based on a descriptive prompt, showcasing its ability to understand and execute tasks based purely on pre-training. We will cover this topic in-depth in the next several chapters.

2.6.3.4 Key Takeaways

1. GPT-1 demonstrated the efficacy of using language models for pre-training, enabling strong generalizability. Its architecture supported transfer learning, allowing it to handle diverse NLP tasks with minimal fine-tuning. This model highlighted the promise of generative pre-training, paving the way for subsequent models to harness these capabilities using larger datasets and increased parameters.
2. GPT-2 demonstrated that larger datasets and increased parameters enhanced a language model's proficiency, often outperforming state-of-the-art results in zero-shot scenarios. The research indicated a log-linear rise in performance with model capacity. Interestingly, the model's perplexity consistently decreased with added parameters without showing signs of saturation. GPT-2 underfitted the WebText dataset, suggesting that further training could improve the results. This finding suggested the potential benefits of even larger models for advancing natural language understanding.
3. In zero-shot settings, GPT-2 surpassed the prevailing benchmarks in 7 of 8 language modeling datasets. On the Children's Book dataset, it enhanced the state-of-the-art accuracy by approximately 7% for common nouns and named entity recognition. For the LAMBADA dataset, GPT-2 notably decreased the perplexity from 99.8 to 8.6, indicating a significant improvement in accuracy. In reading comprehension tasks, it outdid three of the four baseline models. However, GPT-2's performance in text summarization was comparable to or even lower than that of traditional summarization-trained models.
4. GPT-3 was assessed across various language modeling and NLP datasets. It excelled on datasets such as LAMBADA and Penn Tree Bank, often outperforming or matching state-of-the-art models, especially in few or

zero-shot settings. While it did not always surpass the top benchmarks, it consistently improved the zero-shot performance. GPT-3 showcased proficiency in diverse NLP tasks, including closed-book question answering and translation, often rivaling or exceeding fine-tuned models. It generally fared better in few-shot scenarios than in one-shot or zero-shot scenarios. Additionally, GPT-3's capabilities were tested on unconventional tasks such as arithmetic, word unscrambling, and novel word usage. Here, its performance scaled with parameter size and was notably better in few-shot settings.

5. It was shown that GPT-3 can generate high-quality text but sometimes lacks coherence in longer sentences and tends to repeat text. It struggles with tasks such as natural language inference, fill-in-the-blanks, and specific reading comprehension tasks, possibly due to its unidirectional nature. The research suggests that bidirectional models might address this issue in the future. GPT-3's objective treats all tokens equally, lacking task-specific predictions. Solutions were discussed, including objective augmentation, reinforcement learning, or the addition of other modalities. It was also highlighted that GPT-3's large architecture makes inference complex and costly, making its outputs difficult to interpret. Additionally, it emphasized the risk of GPT-3's human-like text generation, including its misuse for phishing or spreading misinformation.

2.6.3.5 Variants

1. **Gopher**: Gopher is a 280B parameter model trained on 300 billion tokens with a 2048-token context window using the MassiveText dataset, which includes web pages, books, news articles, and code. Gopher outperformed then state-of-the-art models such as GPT-3 (175B parameters) on 81% of 100 tasks (Rae et al., 2021).
2. **Chinchilla**: Chinchilla is a compute-optimal 70B model trained on 1.4 trillion tokens. It outperforms the larger Gopher model and has a reduced model size, significantly lowering inference costs (Hoffmann et al., 2022).
3. **Llama**: Meta's GPT variant of Llama, currently at version 3, is an open-source LLM with 8B and 70B parameter sizes and is optimized for dialog with pre-trained and instruction-tuned models, utilizing supervised fine-tuning and reinforcement learning with human feedback (Touvron et al., 2023).
4. **Claude**: The Claude 3 model family by Anthropic includes Claude 3 Opus (20B), Sonnet (70B), and Haiku (2T), each designed for different performance needs (Anthropic, 2023). These models offer multilingual capabilities, vision processing, and improved steerability. Opus provides top-tier performance for complex tasks, Sonnet balances performance and cost, and Haiku is the fastest and most affordable, processing 21K tokens per second for prompts under 32K tokens with a 1:5 input-to-output token ratio.

5. **Command R:** Command R (35B), developed by Cohere, is a generative model optimized for long-context tasks such as Retrieval-Augmented Generation (RAG) and the use of external APIs and tools. It is designed for scalable implementation with strong accuracy in RAG and tool use, low latency, high throughput, and a long 128k context (Cohere, 2024). Command R also supports strong capabilities across 10 key languages.
6. **Gemma:** The Gemma model family, developed by Google, includes 2B and 7B parameter versions trained on 6 trillion tokens (Team et al., 2024). These models demonstrate strong language understanding, reasoning, and safety performance, outperforming similarly sized models on 11 of 18 tasks.

2.6.4 Mixtral 8x7B (Mixture of Experts)

Mixture of Experts (MoE) models have significantly evolved since their inception by Jacobs et al. (1991). Initially designed to tackle complex problems by dividing them into manageable sub-problems, MoE models combine outputs from multiple “expert” networks, each specializing in different facets of the overall task. This approach leverages a gating network to weigh each expert’s contribution dynamically.

A key advancement came with the introduction of top-k routing in 2017 by Shazeer et al.. This method, which only computes outputs from the top k experts, enabled the creation of large-scale models with billions of parameters while maintaining manageable computational costs and showcasing remarkable improvements in tasks such as language modeling.

The evolution continued with the Switch Transformer, which took top- k routing further by using “hard routing”, where $k = 1$, selecting only the most relevant expert for each input token (Lepikhin et al., 2020). This model replaced traditional feed-forward network layers in the T5 Transformer with 128 hard-routed experts, incorporating various optimization techniques to enhance training efficiency and performance on tasks such as the GLUE benchmark.

Mixtral 8x7B is a high-quality sparse mixture of experts model (SMoE) that is openly available under the Apache 2.0 license (Jiang et al., 2024). It outperforms Llama-2 70B on most benchmarks and offers 6x faster inference speeds, matching or surpassing GPT3.5 on most standard benchmarks.

2.6.4.1 Dataset

Details on pre-training are not specified, but it is reported that the model was trained using a multilingual dataset sourced from an open web corpus. It can process multiple languages, including English, French, Italian, German, and Spanish.

2.6.4.2 Architecture

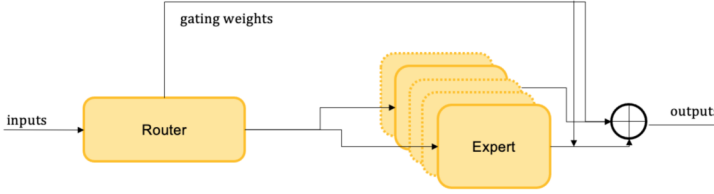


Fig. 2.14: The mixture-of-experts layer in Mixtral, where each input vector is routed to 2 out of 8 experts, and the output of the layer is a weighted sum from the outputs of the selected experts, utilizing standard feed-forward blocks found in traditional Transformer architectures.

In the Mixtral architecture, the traditional Transformer setup is modified by replacing the standard feed-forward network sub-blocks with MoE layers, where each token is processed independently, as shown in Fig. 2.14. The output of the MoE layer for an input vector $\mathbf{x}$ is calculated by a weighted sum of the outputs from several expert networks. These weights are determined by the outputs from a gating network. Considering n expert networks denoted as $\{E_0, E_1, \dots, E_{n-1}\}$; the output of the MoE layer is expressed as:

$$y = \sum_{i=0}^{n-1} G(\mathbf{x})_i \cdot E_i(\mathbf{x}) \quad (2.39)$$

where $G(\mathbf{x})_i$ represents the output of the gating network for the i -th expert and $E_i(\mathbf{x})$ is the output of the i -th expert network. The gating vector, if sparse, enables the omission of computations for experts corresponding to zero-valued gates.

Multiple implementations of $G(\mathbf{x})$ exist (Clark et al., 2022; Hazimeh et al., 2021). A simple yet efficient approach involves computing the softmax function over the top k logits from a linear layer (Shazeer et al., 2017). The gating function is defined as:

$$G(\mathbf{x}) := \text{Softmax}(\text{TopK}(\mathbf{x} \cdot \mathbf{W}_g)) \quad (2.40)$$

where $\text{TopK}(\ell)_i = \ell_i$ if ℓ_i is among the top k coordinates in the logits vector $\ell \in \mathbb{R}^n$, and $\text{TopK}(\ell)_i = -\infty$ otherwise. The choice of k , which denotes the number of experts utilized per token, is a hyper-parameter that controls the computation intensity per token.

In the Mixtral implementation, the SwiGLU architecture is used as the expert function $E_i(\mathbf{x})$, with $k = 2$. Therefore, each token is routed to two SwiGLU sub-blocks with distinct weight sets. This setup computes the output y for a given input

token $\mathbf{x}$ as follows:

$$y = \sum_{i=0}^{n-1} \text{Softmax}(\text{Top2}(\mathbf{x} \cdot \mathbf{W}_g))_i \cdot \text{SwiGLU}_i(\mathbf{x}) \quad (2.41)$$

2.6.4.3 Training

The researchers did not provide information regarding the pre-processing, training methodologies, or hardware used in training Mixtral 8x7B.

2.6.4.4 Key Takeaways

1. A key finding from the research is that Mixtral excels on multilingual benchmarks while maintaining strong performance in English by significantly increasing the proportion of multilingual data during pre-training. Compared to Llama-2 70B, Mixtral demonstrates notable French, German, Spanish, and Italian superiority. The results show that Mixtral outperforms Llama-2 70B across these languages on benchmarks such as the ARC Challenge, Hellaswag, and MMLU.
2. Efficient execution of MoE layers on single GPUs is feasible using specialized high-performance kernels, such as those provided by Megablocks (Gale et al., 2023), which treat the feed-forward network operations of the MoE layer as large sparse matrix multiplications. Mixture of Experts (MoE) layers can be effectively distributed across several GPUs, leveraging both standard parallelism and a targeted partitioning method termed Expert Parallelism (EP) (Shazeer et al., 2017). This approach ensures that during execution, each token assigned to a specific expert is processed by the corresponding GPU, and the resulting output is precisely routed back to its original position in the token sequence.
3. One issue with this setup is the substantial VRAM requirement, as all experts must be loaded into memory, even though only one or two may be actively used at any given time.

2.6.4.5 Variants

1. **Mixtral 8x22B** is a larger sparse MoE variant that leverages up to 141B parameters while utilizing approximately 39B during inference. Thus, it improves inference throughput with a higher VRAM requirement. This model can handle up to 64,000 tokens.

2. For the chat-oriented version of the model, **Mixtral 8x7B-Instruct**, supervised fine-tuning and direct preference optimization were performed utilizing a paired feedback dataset (the specific dataset was not disclosed). The human evaluation results reported by LMSys indicated that this model version achieved a higher Elo rating than did the GPT 3.5 Turbo and Claude 2.1.
3. **MegaBlocks** enhances the efficiency of MoE training on GPUs through block-sparse operations and custom GPU kernels, optimizing token utilization and hardware mapping. This system achieves training speeds up to 40% faster than those of the Tutel library and 2.4 times the speed of dense neural networks trained with the **Megatron-LM** framework without compromising model quality.

2.7 Tutorial: Understanding LLMs and Pre-training

2.7.1 Overview

In this tutorial, we will explore the mechanics of LLM architectures, emphasizing the differences between masked models and causal models. In the first section, we will examine existing pre-trained models to understand how they produce their outputs. Once we have demonstrated how LLMs can do what they do, we will run an abbreviated training loop to provide a glimpse into the training process.

Goals:

- Inspect the inputs and outputs of an LLM, including the tokenizer.
- Step through code to demonstrate the token prediction mechanisms of both masked LLM's and causal LLMs.
- Illustrate on a small scale how to train a LLM from scratch.
- Validate that a training loop is working as intended.

Please note that this is a condensed version of the tutorial. The full version is available at <https://github.com/springer-llms-deep-dive/llms-deep-dive-tutorials>.

2.7.2 Experimental Design

The eventual result of this tutorial is to see the pre-training process at work, but we begin by analyzing the elements of LLM architectures. We first look at the forward pass, which introduces the various components and how they operate together to fulfill the language modeling objective. This code is repeated for both the BERT

and GPT-2 models to highlight the similarities and differences between the masked (encoder only) and autoregressive (decoder only) models.

Once we have dissected the steps involved in token prediction, it becomes natural to understand the LLM training cycle as a typical backpropagation of gradients through the model layers. We assume basic familiarity with deep learning and do not spend time exploring the impact of specific hyperparameters or other details of the training loop. Readers who need a brief refresher may refer to the appendix.

By the end of the exercise, the code will yield a toy model that has memorized a small chunk of Wikipedia data. The notebook we provide only includes a training loop for GPT-2 and not for a masked model, but the reader could easily extend this experiment to other LLMs if desired.

2.7.3 Results and Analysis

In our LLM pre-training experiment, the training loss decreased quickly, while the validation loss remained high. This behavior is depicted in Fig. 2.15, and we expect it when the model overfits the training data. It would take far more documents and training steps for the model to capture enough information to generalize well to the validation data, which is unsurprising since the number of viable token sequences in English is enormous.

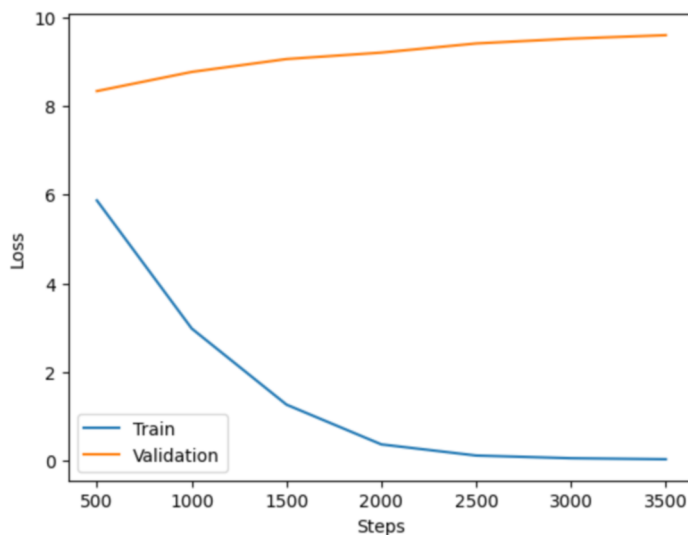


Fig. 2.15: The loss curve obtained as GPT-2 learns the contents of a minimal set of Wikipedia documents.

Although the model has not been adequately trained to perform well on the validation data, we can still see that it has learned much from the training data. To verify, we can test on a training example.

```
print(raw_datasets["train"][0]["text"])
# Output:
# William Edward Whitehouse (20 May 1859 - 12 January 1935) was
  an English cellist.

# Career
# He studied for one year with Alfredo Piatti, for whom he
  deputised (taking his place in concerts when called upon),
  and was his favourite pupil. He went on to teach at the Royal
  Academy of Music, Royal College of Music and King's College,
  Cambridge...
```

Listing 2.1: Accessing Dataset Text Example

Given the first few tokens, we then confirm that our model can complete this text for us.

```
text = "William Edward Whitehouse (20 May 1859 - 12 January 1935)
  was an English cellist.\n\nCareer\nHe studied for one year
  with"

model_inputs = tokenizer(text, return_tensors='pt')
output_generate = model.generate(**model_inputs, max_new_tokens
  =5)
sequence = tokenizer.decode(output_generate[0])
print(sequence)
# Output:
# William Edward Whitehouse (20 May 1859 - 12 January 1935) was
  an English cellist.
#
# Career
# He studied for one year with Alfredo Piatti,
```

Listing 2.2: Generating Text with Model

In this case, the model correctly identified Alfredo Piatti, showing that it has memorized this information from repeated exposure to a specific Wikipedia article. This gives us confidence that our tokenizer and model are able to learn language patterns from Wikipedia. Of course, this does not immediately guarantee that the same training approach will directly translate to a full-sized dataset. Specific parameters, such as the learning rate, may need to be adjusted.

2.7.4 Conclusion

We have shown how masked and causal language models can predict tokens. We then demonstrated that these models can internalize information by repeatedly attempting to predict these tokens and applying subsequent weight updates to decrease the loss.

References

- Anthropic. The claude 3 model family, 2023. URL <https://www.anthropic.com/news/claude-3-family>.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- Stella Biderman et al. Pythia: A suite for analyzing large language models across training and scaling, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Aidan Clark, Diego de Las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, et al. Unified scaling laws for routed language models. In *International conference on machine learning*, pages 4057–4086. PMLR, 2022.
- Cohere. Command r: A generative model for long context tasks, 2024. URL <https://cohere.com/command>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, et al. Cogview: Mastering text-to-image generation via transformers. *Advances in Neural Information Processing Systems*, 34:19822–19835, 2021.
- Hao Fu, Yao; Peng and Tushar Khot. How does gpt obtain its ability? tracing emergent abilities of language models to their sources. *Yao Fu’s Notion*, Dec 2022. URL <https://yaofu.notion.site/How-does-GPT-Obtain-its-Ability-Tracing-Emergent-Abilities-of-Language-Models-to-their-Sources-b9a57ac0fcf74f30a1ab9e3e36fa1dc1>.
- Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems*, 5, 2023.
- Leo Gao et al. The pile: An 800gb dataset of diverse text for language modeling, 2020.
- Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training. *arXiv preprint arXiv:1806.03377*, 2018.
- Hussein Hazimeh, Zhe Zhao, Aakanksha Chowdhery, Maheswaran Sathiamoorthy, Yihua Chen, Rahul Mazumder, Lichan Hong, and Ed Chi. Dselect-k: Differentiable selection in the mixture of experts with applications to multi-task learning. *Advances in Neural Information Processing Systems*, 34:29335–29347, 2021.

- Ji He, Jianshu Chen, Xiaodong He, Jianfeng Gao, Lihong Li, Li Deng, and Mari Ostendorf. Deep reinforcement learning with a natural language action space, 2016.
- Danny Hernandez et al. Scaling laws and interpretability of learning from repeated data, 2022.
- Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2):107–116, 1998. URL <http://dblp.uni-trier.de/db/journals/ijufks/ijufks6.html#Hochreiter98>.
- Jordan Hoffmann et al. Training compute-optimal large language models, 2022.
- Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Katikapalli Subramanyam Kalyan, Ajit Rajasekharan, and Sivanesan Sangeetha. Ammus: A survey of transformer-based pretrained models in natural language processing. *arXiv preprint arXiv:2108.05542*, 2021.
- Uday Kamath, Kenneth L Graham, and Wael Emara. *Transformers for Machine Learning: A Deep Dive*. CRC Press, 2022.
- Hugo Laurençon et al. The bigscience roots corpus: A 1.6tb composite multilingual dataset, 2023.
- Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Luca Di Liello, Matteo Gabburo, and Alessandro Moschitti. Efficient pre-training objectives for transformers, 2021.
- Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers. *AI Open*, 2022.
- Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation, 2015.
- Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.
- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9, 2019.
- Jack W Rae et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.
- Jack W. Rae et al. Scaling language models: Methods, analysis insights from training gopher, 2022.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2020.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
- Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.
- Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks, 2014.
- Gemma Team, Mesnard, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.
- Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022.

BigScience Workshop et al. Bloom: A 176b-parameter open-access multilingual language model, 2023.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

Wayne Xin Zhao et al. A survey of large language models, 2023.



Chapter 3

Prompt-based Learning

Abstract This chapter explores prompt-based learning, a technique central to current advances in LLMs. We introduce prompt-based learning by contrasting it with two older techniques: fully supervised learning and fine-tuning pre-trained models. We then zoom in and discuss the steps necessary for prompt-based inference, exploring its utility as an LLM knowledge extraction tool and overviewing its applications across the field of natural language processing. Next, we explore the nuances of prompt engineering, shedding light on the art and science of crafting effective and efficient prompts that can guide models to desired outputs. This leads to a discussion of answer engineering, where we overview techniques to optimize models for more accurate and contextually relevant responses. Multi-prompting techniques that are useful for more complex queries are discussed. The chapter concludes with two tutorials that further illustrate the practical advantages of prompt-based learning. This first tutorial demonstrates how prompt-based learning can achieve better results with fewer training examples than traditional head-based fine-tuning, and the second tutorial explores different approaches to prompt engineering.

3.1 Introduction

The primary function of language models is to predict the likelihood of individual tokens appearing within a sequence of other tokens based on the semantic representations learned during the pre-training process (Chapter 2). This capability can be leveraged for language generation, in the case of autoregressive tasks, or for fill-in-the-blank inference in masked language model tasks. In both of these approaches, a beginning sequence of tokens must be passed to a model for inference: we refer to these beginning sequences of tokens as *prompts*.

In their seminal research, Brown et al. (2020) illustrated that large language models can effectively address many NLP tasks with a prompt. The authors assembled

long strings of text, consisting of a series of question and answer pairs, and ending with a final question without an answer. Fig. 3.1 illustrates one such task of translating from English to French, employing prompts and varying numbers of preceding examples. Every English phrase is followed by “=>”, and then the French translation, except in the final case. They then used several GPT-3 variants to predict the most likely following token or tokens in the slot where the answer should appear. Remarkably, their language models accurately translated the sentences in many instances with no fine-tuning. These results demonstrate that instead of training language models to learn tasks separately, prompting enables us to use the semantic knowledge embedded in LLMs to complete tasks without additional tuning.

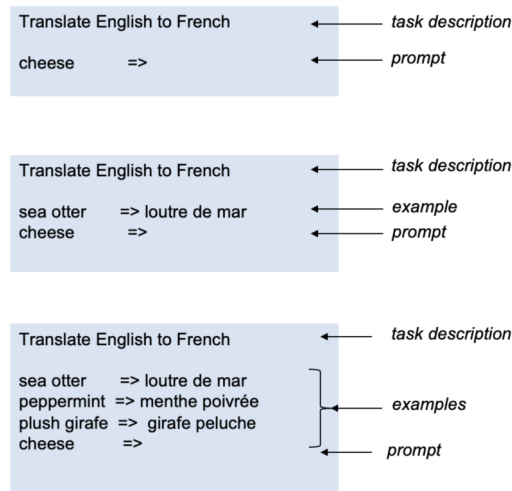


Fig. 3.1: Three different examples of prompt-based inference for English-to-French language translation. In each case, the examples and prompts are passed to an LLM, and the model is allowed to predict the most likely term to come next, in this case “fromage”, thus accomplishing the prompt task. The three examples from top to bottom illustrate zero-shot, one-shot, and few-shot inference.

The notion of prompting can be attributed to the work by [Kumar et al. \(2016\)](#), which introduced the dynamic memory network (DMN). DMN comprises a neural network architecture designed to process input sequences and questions, establish episodic memories, and generate pertinent answers ([Xiong et al., 2016](#)). Tasks corresponding to questions (prompts) initiate an iterative attention mechanism, allowing the model to concentrate on the inputs and outcomes of previous iterations. [Radford et al. \(2019\)](#) revealed the potential of this approach for achieving expertise in various natural language processing tasks without requiring explicit supervision, provided that the models are trained on adequately extensive datasets.

Since these discoveries, a wealth of literature has developed, examining many different approaches and improvements to prompt-based inference and learning. This chapter will introduce and systematically examine the critical aspects of prompt-based inference, including the basic procedure, details of prompt shape, prompt optimization, answer space engineering, and practical applications to various NLP tasks.¹ But first, to place prompting in its proper historical context, we will describe two prominent approaches that have shaped the field in the last few years – supervised learning and pre-trained model fine-tuning – and distinguish them from prompt-based learning.

3.1.1 Fully Supervised Learning

In this traditional approach, NLP models are trained on labeled data, which consists of input-output pairs that serve as examples for the desired task (Kotsiantis et al., 2007). The model learns to map inputs to the corresponding outputs, generalizing from the training examples to make predictions on unseen data. Fig. 3.2 shows an example using a logistic regression classifier, which learns the relationships between the sentences and the labels.

Models trained by supervised learning have a well-defined learning process, resulting in reliable performance on tasks with sufficient labeled data. As such, they have been used across a diverse range of NLP tasks, from sentiment analysis to machine translation. However, this method has several drawbacks.

First, the success of supervised learning depends strongly on the availability and quality of labeled data, which can be scarce, expensive, or time consuming to create. Second, supervised learning models traditionally rely on expert-driven feature engineering to define their predictive features. This engineering process requires significant manual effort and substantial expertise while also being inefficient due to incomplete knowledge of how features are naturally distributed within a dataset (Och et al., 2004; Zhang and Nivre, 2011). Finally, supervised learning creates models that struggle to generalize beyond the scope of the provided training data, particularly when faced with examples that differ significantly from the training set.

Pros:

- Predictability
- Wide applicability

Cons:

- Heavy data dependency
- Feature engineering requirements

¹ The terminology and procedural formulations employed in this chapter are largely informed by the comprehensive survey paper authored by Liu et al. (2023), titled “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing”.

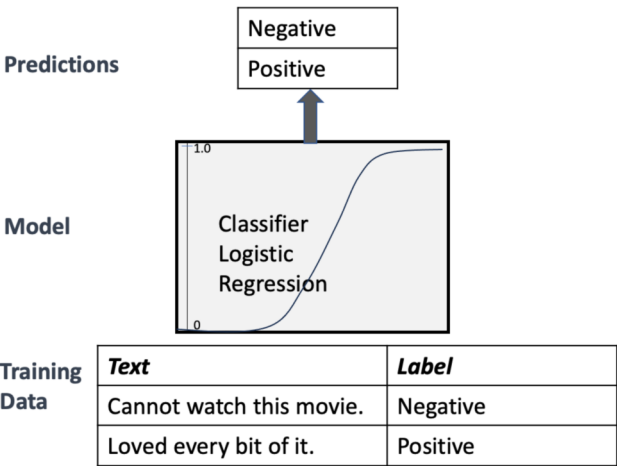


Fig. 3.2: Logistic regression classifier model for sentiment classification from training data. The weighting of features in the training samples is learned by maximizing the likelihood of the labels. Learned feature weights are then summed, and this value is passed through the logistic function (also known as the sigmoid function) to generate a probability between 0 and 1. Class label mapping is then achieved by identifying the point along the probability distribution above which a particular input is considered positive or negative; 0.5 is common for balanced classification.

- Limited generalization

3.1.2 Pre-train and Fine-tune Learning

In this approach, LLMs trained on large corpora in an unsupervised manner are subsequently fine-tuned in a supervised manner using smaller datasets labeled according to the desired task. Thus, the model is honed for a specific task but retains semantic knowledge gained from pre-training (Peters et al., 2019; Radford et al., 2018). Fig. 3.3 shows an example of pre-training and fine-tuning (PTFT) using a BERT model (Sect. 2.6.1).

This approach has led to state-of-the-art results across numerous NLP benchmarks. These impressive results are due to several key advantages of the PTFT paradigm. First, the pre-training process allows for substantial transfer learning from the pre-training phase, enhancing performance across different NLP tasks (Kamath et al., 2019). Second, because of this transfer learning, there is a reduced reliance

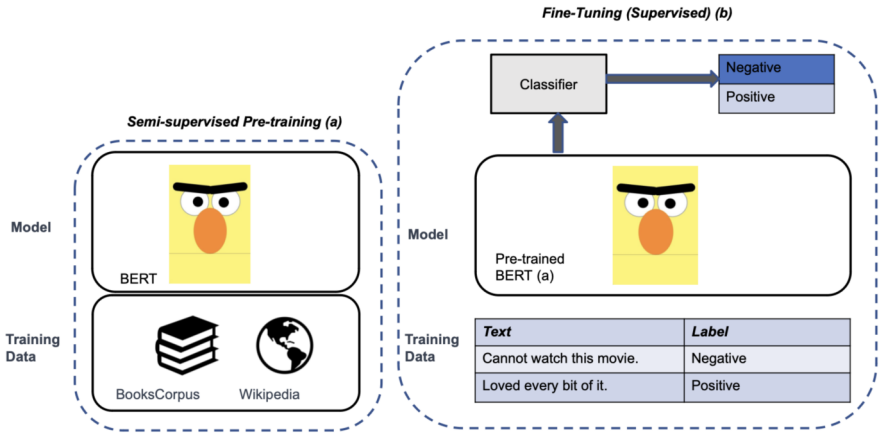


Fig. 3.3: Illustration of the pre-train and fine-tune approach, using BERT. The model has been pre-trained in a semi-supervised manner with data encompassing a wide range of language and subject matter to learn a rich semantic representation of language. It is then fine-tuned with data specifically curated for the sentiment classification task. By pre-training and then fine-tuning, the semantic language learned by BERT can be transferred to the fine-tuned task, improving performance.

on labeled data in the fine-tuning phase compared to fully supervised learning. This makes PTFT suitable for low-resource settings. Finally, in the realm of natural language processing the procedure for fine-tuning pre-trained models has become increasingly standardized and industry-accepted, owing to the development of various platforms and frameworks, such as HuggingFace².

These improvements come at the cost of additional downsides. First, training and fine-tuning large-scale pre-trained models require significant computational resources, which may not be accessible to all researchers or developers. Second, the architectures of models suitable to pre-training, such as deep neural networks, can be challenging to interpret and explain, hindering understandability and potentially raising ethical concerns. Finally, the objectives of pre-training and fine-tuning are generally distinct, with the former being about learning general semantic relations and the latter being about assigning labels to text. If the pre-trained model remains static while a new task-specific head is fine-tuned, this can create some deterioration in the outcomes.

Pros:

- Benefits from transfer learning
- Improved performance compared to fully supervised learning

² <https://huggingface.co/>

- Reduced reliance on labeled data

Cons:

- Heavy computational requirements
- Imprecise mapping between semantic space and tasks
- Model complexity and poor explainability

This approach represented SOTA until very recently when prompt-based learning emerged as a new pathway toward LLM tuning.

3.1.3 Prompt-based Learning

Prompt-based learning, also known as *prompt-based inference*, represent an innovative approach to harnessing the power of language models, as they can generate task-specific responses without the necessity of fine-tuning. A prompt is often thought of informally as a line of communication from a human to a model. When a person writes a question to a chatbot, this is one example of a prompt. However, the use of prompts can also be applied as a strategy for solving natural language processing tasks.

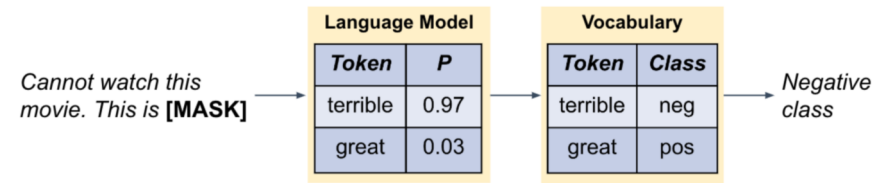


Fig. 3.4: Demonstration of “pre-train, prompt, and predict” as a classification process. We pass a prompt with a space left blank for prediction ([MASK]) to a pre-trained language model. We then look at the probabilities predicted by the model of certain words filling in the empty space. We select the most probable token, in this case “terrible”, and return the label class mapped to that token, in this case “negative”.

In this application, a prompt is a string of natural language text with one or more words left blank for an LLM to fill in based on its semantic model. We show an example in Fig. 3.4. Instead of fine-tuning a model to predict a positive or negative label, we can pass the following sentence to an LLM: “Cannot watch this movie. This is [MASK].”, and determine whether the model calculates “great” or “terrible” as the more likely term for the masked token. In this case, the LLM will predict “terrible” as the more probable continuation, as it creates a much more

semantically coherent sentence than a positive term. These predictions can then be mapped to a label class, in this case “negative”. Prompt-based prediction can also be open-ended generative, such as a translation prompt phrased as: “English: Cannot watch this movie. German: ”, and the model made to predict additional tokens at the end, which will lead the LLM to produce a German translation of the input sentence.

This method requires formulating prompts that guide the language model in producing desired outputs corresponding to a particular NLP task. This technique leverages the pre-trained language models’ ability to generate coherent text, reducing the need for labeled data while enabling zero-shot or few-shot learning. As a result, prompt-based learning has become an area of active research and has demonstrated strong performance on various NLP tasks.

! Practical Tips

This book uses the terms zero-shot, one-shot, and few-shot to describe different training dataset sizes for prompt-based learning. In the *zero-shot* setting, no additional training samples are needed for the pre-trained model to perform the desired task. In the *one-shot* and *few-shot* settings, we use one task-specific training examples (one-shot), or a small number of such samples (few-shot; $\lesssim 100$) to guide the model. The fact that prompt-based models perform well with limited training data is a significant advantage over other techniques that may require a large number of samples.

Prompt-based learning has several advantages. First, they can be adapted to a wide range of NLP tasks as long as they can be formulated as fill-in-the-blank problems. There is also greater cohesiveness between the prediction task and the semantic knowledge learned by the pre-training process compared to PTFT. Since the model’s weights are not updated, the model will not “forget” old information when fitting to the new information. By extension, we also gain the ability to apply these original model weights across multiple use cases, which greatly reduces training costs while also simplifying application deployment. Finally, by leveraging the text generation capabilities of these models, prompt-based learning reduces the need for labeled data, thus enabling zero-shot or few-shot learning.

On the other hand, the performance of prompt-based learning models can be heavily influenced by the choice of prompts, making prompt engineering critical and potentially challenging. Moreover, as with pre-trained model fine-tuning, the underlying mechanisms driving prompt-based learning models can be challenging to understand and explain. Finally, prompt-based learning models may produce inconsistent outputs, particularly in cases where the model has not been sufficiently exposed to the target task or when the prompt is ambiguous.

Pros:

- Applicable to many NLP tasks
- Cohesive semantics in training and inference
- Reduced labeled data requirement

Cons:

- High sensitivity to prompt design
- Limited interpretability
- Inconsistency in results

The following sections will introduce the conceptual basics of prompt-based learning, describe strategies for creating optimal prompts, and discuss different prompt-based approaches to LLM fine-tuning.

3.2 Basics of Prompt-based Learning

3.2.1 Prompt-based Learning: Formal Description

Prompting, as it pertains to language models, refers to providing an initial input or a series of textual cues to the model, which subsequently generates a contextually relevant and coherent response based on the given input. Consider the movie review sentence “Cannot watch this movie” in the context of sentiment analysis. This section will use this sentence as an illustrative example to compare and delineate the distinctions between supervised learning, pre-trained combined with fine-tuning, and prompt-based learning.

As depicted in Fig. 3.5, supervised learning involves training the model with parameters θ to learn from the data represented by example pairs in the training set $(\mathbf{x}, \mathbf{y})$. In the example, the training pair corresponding to $(\mathbf{x}, \mathbf{y})$ is (Cannot watch this movie, negative).

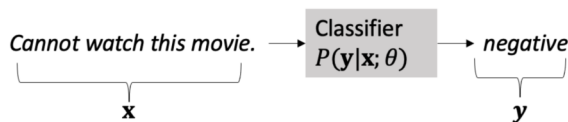


Fig. 3.5: Supervised learning, represented as the probability of output $\mathbf{y}$ given input $\mathbf{x}$

In PTFT, the input is modified to include a head token [CLS] corresponding to the class label, such as “positive” or “negative” (for a positive or negative movie

review), that the model aims to predict. The fine-tuning process, also referred to as head-based tuning, involves learning the model parameters θ for this classifier using both the label and input as shown in Fig. 3.6.

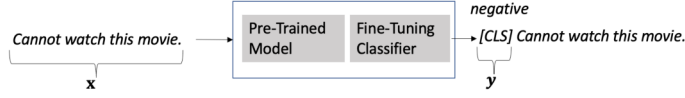


Fig. 3.6: Pre-trained model with fine-tuning, where the classifier head acts on the pre-trained model's embedding of input x to produce output y

In the context of prompt-based learning, each input is placed with a prompt template that incorporates the input and a slot for predicting the output in a manner relevant to the inference task. For instance, in our example in Fig. 3.7, "It is $[z]$ " is appended to the input, and the word z is inferred as the highest probability token for the slot according to the language model.

Thus, prompt-based inference encompasses two primary components: (1) a template that transforms the downstream task into a language modeling problem and (2) a collection of label words that facilitate the conversion of the language model's textual output into classification labels. We chose "great" and "terrible" as our two outputs, which complete the prompt more naturally than "positive" and "negative" and are thus more likely to be predicted by the model. Fig. 3.7 illustrates the template transformation and choice of label words. This approach eliminates the need to introduce new parameters.

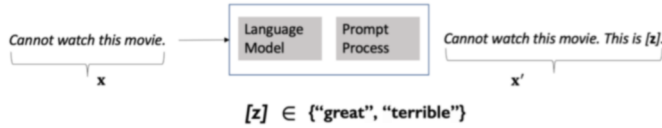


Fig. 3.7: Prompt-based learning, where the task is formulated with a prompt function from x to x' such that the output of the language model can be mapped to y

Formally, if the label word mapping $\mathcal{M}$ maps labels y to individual words, given the original input x and the modified input after prompt addition x' , then the prompt-based fine-tuning process can be written as:

$$p(y|x) = p([z = \mathcal{M}(y)|x']) \quad (3.1)$$

$$p(y|x) = \frac{\exp(\mathbf{w}_{\mathcal{M}(y)} \cdot \mathbf{h}_z)}{\sum_{y' \in y} \exp(\mathbf{w}_{\mathcal{M}(y')} \cdot \mathbf{h}_z)} \quad (3.2)$$

where $\mathbf{h}_z$ is the hidden vector of answer $\mathbf{z}$, and $\mathbf{w}$ is the pre-softmax vector associated with the subset of words mapping to y .

! Practical Tips

This prompt-based method often performs better than head-based fine-tuning (discussed in Chapter 2), especially in low-data scenarios. This can be attributed to the fact that a pre-trained model such as BERT incorporates new, randomly initialized parameters, which prove challenging to optimize effectively when provided with a limited number of examples. We show a concrete example in the tutorial in Sect. 3.6.

3.2.2 Prompt-based Learning Process

Let us consider the example of sentiment classification based on movie reviews to elucidate the various steps involved in data flow and output class prediction. As depicted in Fig. 3.8, three high-level steps are involved during the prompt-based learning process.

1. A prompting function $f_{prompt}(\cdot)$ modifies the input text $\mathbf{x}$ into a prompt $\mathbf{x}'(\mathbf{z}) = f_{prompt}(\mathbf{x})(\mathbf{z})$.
2. A function $f_{fill}(\mathbf{x}'; \mathbf{z})$ determines the most probable token to fill the slot $\mathbf{z}$
3. A mapping function is used to associate the highest-scoring answer $\mathbf{z}^*$ with the highest-scoring output $\mathbf{y}$.

In our example, the input is the sentence “Cannot watch this movie,” represented by $\mathbf{x}$. The template is “[$\mathbf{x}$] It was a [$\mathbf{z}$] movie”. And the output is the sentiment prediction “negative,” represented by $\mathbf{y}$. In the following sections, we will discuss these individual steps in more detail.

3.2.2.1 Prompt Addition

The process of *prompt addition* can be expressed mathematically as a function $f_{prompt}(\cdot)$, responsible for taking an input text $\mathbf{x}$ and a designated template with an answer slot $\mathbf{z}$, subsequently generating a prompt $\mathbf{x}' = f_{prompt}(\mathbf{x})$. The prompting function entails a two-phase procedure:

1. Implement a predetermined template, characterized by a textual string containing two slots: an input slot [$\mathbf{x}$] designated for the input and an answer slot [$\mathbf{z}$] intended for an intermediate generated answer text that will subsequently be mapped to the output $\mathbf{y}$.
2. Replace the input slot [$\mathbf{x}$] with the provided input.

Continuing with the movie sentiment classification illustration, the input sentence “Cannot watch this movie.” undergoes a transformation utilizing a template “[$\mathbf{x}$] It

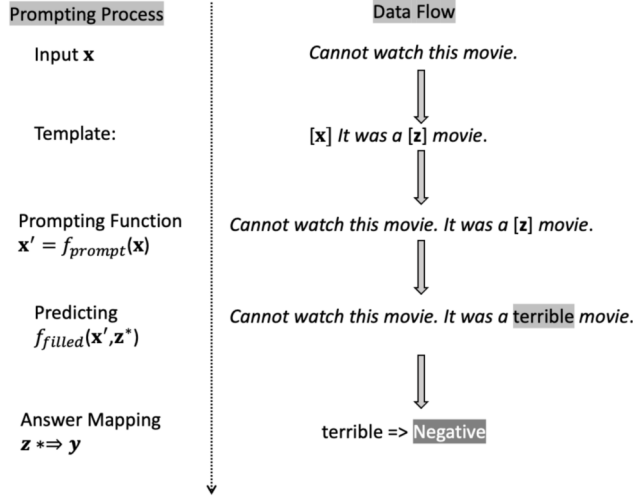


Fig. 3.8: Prompt-based learning is a process consisting of applying a prompting function to the input, filling the mask in the resulting text, and mapping the output to the appropriate answer

was a $[z]$ movie” to generate a prompt “Cannot watch this movie. It was a $[z]$ movie.”

3.2.2.2 Answer Search

The next step involves calculating the probability of all potential answers in the designated slot within the prompt. This process, often called *answer search*, is the model-based inference stage. Considering our prompt template again, we can adopt the potential answers “great” and “terrible” as our choices to represent the positive and negative classes. Whichever is calculated to be the most likely fill-in token is taken as the predicted answer.

Formally, the answer-searching process encompasses two primary steps.

1. Initially, the function $f_{\text{fill}}(x', z)$ populates the designated slot with a potential answer $[z]$. This answer may be selected from the entire vocabulary of the model or from a smaller subset of words $\mathcal{Z}$, depending on the task. The outcome of this process is referred to as the filled prompt.
2. Subsequently, a search function, such as *argmax* search, is employed to identify the highest-scoring output. This is accomplished by computing the probability of the corresponding filled prompts using a pre-trained language model, denoted as $P(\cdot; \theta)$. By selecting the output with the highest probability, the search function ensures that the most contextually relevant and semantically coherent answer is extracted to satisfy the prompt’s requirements.

This search function can be expressed as follows:

$$\hat{\mathbf{z}} = \underset{\mathbf{z} \in \mathcal{Z}}{\text{search}} P(f_{fill}(\mathbf{x}, \mathbf{z}); \theta) \quad (3.3)$$

Here, we search across all possible answers $\mathcal{Z}$ for the answer with the highest probability ($\mathbf{z}$) given the template function f_{fill} and the model parameters θ . We can then map the output $\hat{\mathbf{z}}$ to a more natural answer space that is easier to interpret, as described below.

3.2.2.3 Answer Mapping

Once we have obtained the intermediate text or tokens generated during the answer search process, we must map them into the ultimate desired output format. This process is referred to as *answer mapping*.

For instance, in a movie sentiment analysis task, the prompt-based model may produce words such as “terrible” or “great” as intermediate text to fill the slot during answer searching. Answer mapping subsequently associates these intermediate texts with the corresponding sentiment labels or numerical values (e.g., 1 for positive, 0 for negative) tailored to the specific task. Formally, this mapping sends the highest-scoring answer $\hat{\mathbf{z}}$ to the highest-scoring output $\hat{\mathbf{y}}$.

This step is necessary because the most natural words for the template may not exactly correspond to the labels. An example of this mismatch is sentiment classification on restaurant reviews, generally denoted as one to five stars. “One star” is a less natural answer than “terrible”, so in the prompt, we might use the latter as a candidate and then map it to the “one-star” category after the fact. It is essential to convert the tokens used for answer search into an appropriate format that aligns with the task objectives.

3.2.3 Prompt-based Knowledge Extraction

During pre-training, language models learn both generalized knowledge of semantics and innumerable specific factual claims, generally referred to as *parametric knowledge*. A prompting task such as the one given in Fig. 3.7 can be accomplished simply with knowledge of semantic relationships between words. However, consider the following question: “What city was Dante Alighieri born in?” This question cannot be answered only on semantics – it requires exposure to factual details. The massive size of LLM pre-training corpora makes this possible. For example, if the pre-training corpus contains the Wikipedia entry for Dante Alighieri, then the model weights will have been influenced by a sentence of this sort: “Dante was born in Florence, Republic of Florence, in what is now Italy.” A properly crafted prompt can induce the model to regurgitate this parametric knowledge, for example:

Dante was born in [MASK]

(3.4)

Because the model has been tuned on data that answers this question, “Florence” will be calculated as a highly probable fill for this mask token. This example demonstrates a basic and fundamental promise of prompt-based inference from LLMs: the possibility of using LLMs as knowledge bases.

This is in contrast to the use of standard knowledge bases, the development of which requires significant efforts in a) the extraction of relational knowledge from various data sources, and b) NLP pipeline solutions for entity extraction, co-reference resolution, entity linking, and relation extraction (Petroni et al., 2019). Each of these NLP pipeline requirements has challenges, and errors are inevitable. This can mean that the utility of the resulting knowledge base is particularly sensitive to errors propagating through and accumulating within the NLP pipeline (Petroni et al., 2019). A conceptual comparison of the two approaches is shown in Fig. 3.9.

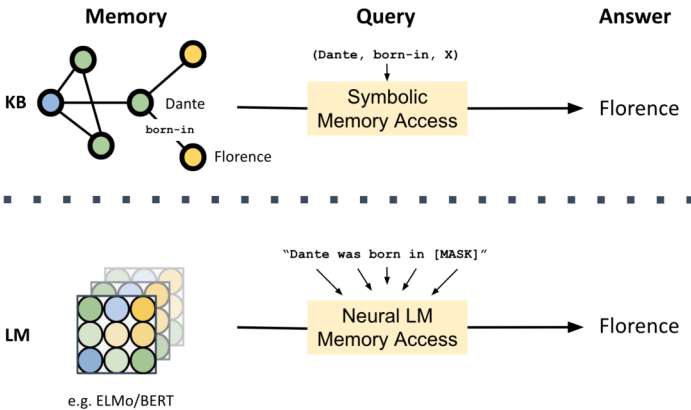


Fig. 3.9: Querying traditional knowledge bases and language models for factual information. In this example, the knowledge base (above the dashed line) has been purposefully designed to be queried for the entity relation, Dante, born in Florence. This is in contrast to the language model (below the dashed line) which was designed to predict masked words given associated context, and can therefore be induced to report facts that it was exposed to during pre-training.

LLM prompting holds a few key advantages over using standard knowledge bases. First, it is schema-free, as its relational knowledge is built within the language model as an emergent property of the pre-training process rather than as a specific task against which the model is developed. It is also highly generalizable given the vast scope of information contained in modern pre-training corpora. In theory, the same language model can support many di-

verse knowledge-based use cases and a much more comprehensive range of common NLP tasks.

! Practical Tips

Significant challenges and risks are associated with LLM-based knowledge extraction. Explainability is difficult because tracing the casual events leading to a specific response from prompt-based inference is often impossible. The accuracy of these responses must also be validated. The knowledge we are trying to elicit from a language model is an emergent property of the training process used during pre-training. As such, it has not been intentionally trained to learn these knowledge facts. Similarly, the datasets used in pre-training are impractically large from a knowledge validation/quality ranking perspective, and where these data have come from the internet, a similar lack of epistemological analysis can result in similarly untrue “facts”. Thus, users should maintain a healthy skepticism and safeguard against these errors with sound evaluation methodologies. Finally, the consistency of a prompt-based knowledge base strongly depends on the quality of the engineered prompt. We will discuss optimization approaches in detail in Sect. 3.3.

3.2.4 Prompt-based Learning Across NLP Tasks

Numerous NLP tasks are well suited to the prompt-based paradigm. In this section, we list many common tasks that can be accomplished with prompting, including a description of their inputs, templates, prompts, and answer mappings. By understanding these elements and their interactions, we aim to provide a comprehensive view of how NLP tasks can be effectively adapted and executed within the prompt-based learning framework.

We divide these tasks into three broad NLP categories:

1. **Text classification:** This category involves assigning an appropriate class label to a given input sentence. For these tasks, the prompt is designed to accept the input sentence and includes a dedicated slot for generating intermediate answers, which can later be mapped to classification labels.
2. **Tagging:** This category involves assigning labels or tags to individual elements within a given text, such as words or phrases. For these tasks, the prompt includes the string of text containing the element to be tagged and then queries specifically about that element, providing options for the model to decide between.
3. **Text generation:** This category involves generating a string of text, generally more than just one token, to accomplish a task given in the prompt. For these tasks, the prompt includes some relevant context, such as a paragraph to sum-

marize or a sentence to translate, and a specific directive to the model for what to do with the context.

Table 3.1 lists seventeen total tasks that fall within these three categories, gives a short description of the task, and a sample input, template, and answer space that can be used to accomplish the task. The wide variety of use cases exemplifies the flexibility of prompt-based learning. However, prompts must be carefully crafted to suit each individual task. In the next section, we will further break down the process into several areas that can be optimized to achieve the best results from prompt-based learning.

3.3 Prompt Engineering

In the previous section, we discussed how various NLP tasks can be solved with prompts, illustrated through several straightforward examples. The precise formulation of these prompts is critical for achieving good results. The development of suitable prompting functions to optimize performance on a target tasks downstream is referred to as *prompt engineering*. The process of designing prompts necessitates meticulous consideration and the integration of various elements. These elements include the selection of pre-trained models, the determination of the optimal prompt shape, the engineering of prompt templates, and answer engineering. Template engineering approaches fall broadly into two categories:

- manual templates
- automated templates

The former uses human expertise and trial-and-error to arrive at an optimized prompt, and the latter uses various automated processes to discern the best approach template for a given task. Fig. 3.10 shows an overview of the structure of the next two sections. In the following section, we will introduce basic terminology central to prompt categorization, overview the manual prompt engineering approach, and detail several automated approaches used in the literature.

3.3.1 Prompt Shape

Prompt templates can be broadly categorized into two main types: (a) prefix prompts and (b) cloze prompts. We refer to these as types of *prompt shape*.

Table 3.1: Summary of prompt-based NLP approaches. Each row contains an NLP task with a definition on the left, and an example on the right. The example includes an input sentence to perform the task on, a suggested template for prompt-based inference, and a potential answer space. These tasks are divided into three categories: text classification, tagging, and text generation.

Text Classification	
Task	Example
Sentiment analysis: Classifying the sentiment of a text as positive, negative, or neutral.	Input: I hate this movie. Template: [x] It was a [z] movie. Answers: great, terrible, . . .
Author attribution: Identifying the author of a given text from a predefined set of authors.	Input: It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness Template: The author of [x] is most likely [z]. Answers: Dickens, Carroll, Austin, . . .
Spam detection: Classifying an email or text message as spam or not spam.	Input: Congratulations! You have won! Click here to claim your free vacation. Template: This message: [x] is classified as [z]. Answers: Spam, Non-Spam
Emotion classification: Classifying the emotion expressed in a text from a predefined set of emotions.	Input: I just won the lottery! Template: This text: [x] expresses the emotion [z]. Answers: anger, surprise, sadness, happiness
Intent detection: Identifying the intent behind a user’s query or message, often used in chatbots and virtual assistants.	Input: What’s the weather like today? Template: [x] The user’s intent is [z]. Answers: get_weather, set_alarm
Language identification: Determining the language in which a given text is written.	Input: ¿Cómo estás? Template: [x] The language is [z]. Answers: Spanish, French, . . .
Hate speech detection: Identifying whether a given text contains hate speech.	Input: I can’t stand them. Template: [x] The text contains [z] speech. Answers: hate, non-hate
Tagging	
Task	Example
Part-of-speech (POS) tagging: Assigning grammatical categories to words, such as nouns, verbs, adjectives, and adverbs.	Input: She is running in the park. Template: In the sentence [x ₁ , . . . , x _n], the word [x _i] has POS-tag [z _j]. Answers: noun, verb, adjective, . . .
Named entity recognition (NER): Identifying and classifying entities mentioned in the text, such as people, dates, locations, organizations, etc.	Input: John met Mary in London. Template: In the sentence [x ₁ , . . . , x _n], the word [x _i] the named entity label is [z _j]. Answers: location, organization, . . .
Chunking or shallow parsing: Grouping adjacent words or tokens into larger units called “chunks” based on their grammatical structure, such as noun phrases or verb phrases.	Input: She is running in the park. Template: In the sentence [x ₁ , . . . , x _n], the word [x _i] the chunk label is [z _j]. Answers: ‘B-VP’ - beginning of a verb phrase, ‘I-VP’ - inside a verb phrase, . . .

Continued on next page

Table 3.1 – *Continued from previous page*

Task	Example
Dependency parsing: Identifying syntactic dependencies between words in a sentence, which includes labeling words as subjects, objects, modifiers, etc., and showing their relationships.	Input: She is running in the park. Template: In the sentence $[x_1, \dots, x_n]$, the word $[x_i]$ the dependency relation is $[z_j]$. Answers: 'nsubj' - nominal subject, 'root' - root of the sentence, 'dobj' - direct object, ...
Constituent parsing or phrase structure parsing: Identifying the constituent structure of sentences, where words are grouped into grammatical phrases, such as noun phrases (NPs) and verb phrases (VPs).	Input: She is running in the park. Template: In the sentence $[x_1, \dots, x_n]$, the word $[x_i]$ the constituent category is $[z_j]$. Answers: 'NP' - noun phrase, 'PP' - prepositional phrase, 'VP' - verb phrase, ...
Semantic role labeling (SRL): Assigning roles to words or phrases in a sentence, such as agent, instrument, etc., based on their semantic relationships with the predicate (usually a verb).	Input: John gave Mary a book. Template: In the sentence $[x_1, \dots, x_n]$, the word $[x_i]$ has semantic role $[z_j]$. Answers: Agent, Theme, Location, ...
Coreference resolution: Identifying words or phrases in a text that refer to the same entity and linking them together.	Input: Jane is a talented software engineer. She was recently promoted to team lead. Template: In the text with words: $[x_1, \dots, x_n]$, does the word $[x_i]$ refer to the word x_j? Answers: Yes, No

Text Generation

Task	Example
Summarization: Given a long piece of text, generate a shorter version that captures the original text's main points or key information.	Input: <Long text to be summarized.> Template: Please provide a summary for the following text: $[x]$. Summary: $[z]$. Answer: <summarized version of the long text>
Question-answering: Given a question and a context, generate an answer based on the information available in the context.	Input: <Context or passage>, <question> Template: Here is the context: $[x]$ What is the answer to the question: $[w]$? Answer: $[z]$. Answer: <answer to the question based on the context>
Machine translation: Translating a piece of text from one language to another while preserving the original meaning and context.	Input: ¿Cómo estás? Template: Translate the following text from the source language to the target language: $[x]$ Translation: $[z]$. Answer: <translated text in target language>

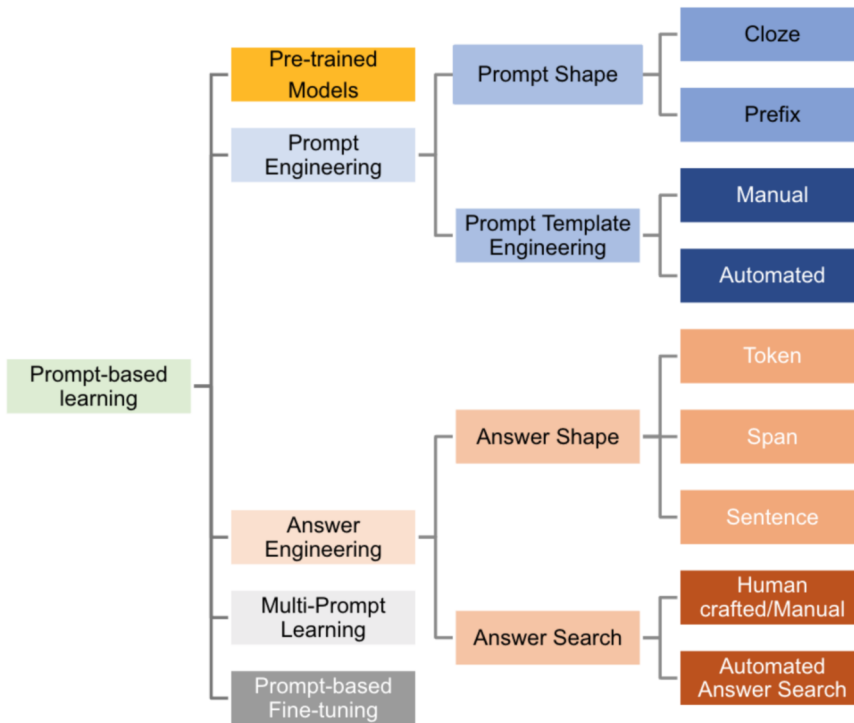


Fig. 3.10: Relationships between the various design options and design decisions within the prompt-based learning paradigm.

3.3.1.1 Prefix Prompts

In a *prefix prompt*, both the input and a string referring to the input are positioned as a prefix to the answer slot. For instance, in the context of movie sentiment analysis, a prefix prompt template can be formulated as

$$“\mathbf{x}' = [\mathbf{x}] \text{ This movie is } [\mathbf{z}]”, \quad (3.5)$$

When this template is applied to the input, it generates a filled prompt, such as, “Cannot watch this movie. This movie is [z]”.

Prefix prompts tend to perform well in tasks where the response is directly linked to the input with left-to-right mappings, and a simple, unambiguous connection can be established. For example, in machine translation tasks, prefix prompts can effectively generate translations by providing clear guidance on the target language. Consider this template:

$$“\mathbf{x}' = \text{Translate the following English sentence to French: } [\mathbf{x}][\mathbf{z}]”, \quad (3.6)$$

By explicitly stating the desired language, prefix prompts offer a straightforward directive to the model, resulting in accurate translations.

3.3.1.2 Cloze Prompts

Unlike prefix prompts, cloze prompts feature template tokens on either side of the answer slots, encompassing the answer in the interior of the template. For example, in the movie sentiment analysis task, a cloze prompt template can be expressed as

$$\mathbf{x}' = [\mathbf{x}] \text{ This is a } [\mathbf{z}] \text{ movie.}'' \quad (3.7)$$

where contextual words surround the answer slot. When applied to the input, this template results in the filled prompt, “Cannot watch this movie. This is a [z] movie.” Note that punctuation marks count as template text in this context, so a template ending in the answer slot [z] followed by a period is considered a cloze prompt.

Cloze prompts are characterized by their flexibility and ability to create more natural language structures, making them particularly well suited for tasks such as multiple-choice question answering. By embedding the answer slot within a contextual phrase, cloze prompts encourage the model to generate outputs that conform to the surrounding linguistic patterns. By providing context on either side of the answer slot, cloze prompts enable the model to consider the linguistic features and situational cues present in the input, ultimately resulting in more accurate and meaningful outputs.

To summarize the distinction, the two broad categories are prompt shape are:

- **Prefix prompts:** In these prompts, the input and template text come before the answer slot.
 - Ex.: “Cannot watch this movie. This movie is [z]”
- **Cloze prompts:** In these prompts, the input and template text surrounds the answer slot.
 - Ex.: “Cannot watch this movie. It was a [z] movie.”

3.3.2 Manual Template Design

We turn now to the creation and optimization of templates. The most straightforward approach is *manual template design*, which uses human expertise, intuition, and understanding of the task to design a suitable template. This often involves considering the specific characteristics of the task, as well as employing heuristics to determine the optimal structure and wording of the prompt. This process may require iterative adjustments to refine the template for improved performance (Brown et al., 2020;

Petroni et al., 2019; Schick and Schütze, 2020a,b). A final decision should be made based on performance against a labeled dataset.

There is no one-size-fits-all approach to generating manual prompts, but the most critical guideline to follow is experimentation with many candidates. To demonstrate the importance of trial-and-error in this process, consider a prompt designed to return capital cities of countries. Here are four candidate prompt templates:

1. "the capital city of [x] is [z] ."
2. "[z] is the capital city of [x] ."
3. "what is the capital city of [x]? It is [z] ."
4. "[z] is located in [x], and is its capital city ."

Listing 3.1: Country capital prompt templates

Each of these templates looks like a plausible choice, but are they equally effective? As a check, we use the AllenNLP Masked Language Modeling demo³ to test a input example. To use this demo, you enter a sentence including a mask token, and the model returns the top predicted tokens to fill the blank space. Taking Poland as our sample [x], we predict the top three tokens and report the results in Table 3.2.

Table 3.2: Prediction scores for the templates in Listing 3.1, using the masked language model demo from AllenNLP. For each prompt, probabilities of the top three predicted tokens to fill [z] given [x] = “Poland” are shown (in percent).

Input	Warsaw	Kraków	Poznan	Poland	here	It
Template 1	37.7	23.2	19.3	—	—	—
Template 2	55.7	14.8	15.0	—	—	—
Template 3	10.1	—	—	21.2	3.6	—
Template 4	29.3	—	12.3	—	—	33.0

Templates 1 and 2 return the correct answer, “Warsaw”, as the top predicted token, with template 2 predicting “Warsaw” by a wider margin. Notably, these are the most simple and direct templates of the four, without multiple sentences or inefficient clause ordering. Template 3 returns “Poland” as the top answer, and template 4 predicts the pronoun “It”. Both have Warsaw as their second guess, but it is clear that these templates did not activate the latent knowledge in the LLM as effectively.

In a manual prompt design project the engineer should test many different sample templates with many labeled examples similar to the above, allowing for statistical optimization. The optimal prompt should be determined relative to a metric, for example, the top-1 prompt selection approach:

$$A(t_r, i) = \frac{\sum_{(x,y) \in \mathcal{R}} \delta(y = \arg \max_{y'} P_{LM}(y'|x, t_r, i))}{|\mathcal{R}|} \quad (3.8)$$

Here, $\mathcal{R}$ is the labeled test set of *subject-object* pairs with relation r , and $\delta(\cdot)$ is Kronecker’s delta function, which returns 1 where y is equal to the top prediction

³ <https://demo.allennlp.org/masked-lm>

from the LM, and 0 where it is not. The final prompt is then with the highest accuracy on the set of *subject-object* pair training samples.

3.3.3 Automated Template Design: Discrete Search

Automated template design involves using some form of search or generation for the most effective prompt template in a predefined search space. While more complex to implement, automated prompt development will usually outperform manual prompt engineering, as it is generally more complete in its search of parameter space. Automated prompt engineering can be divided into two categories: (a) discrete search and (b) continuous search.

The primary distinction for these automated prompt template design methods is whether they use *discrete tokens/prompts* or *continuous tokens/prompts* to prompt the language model. This distinction relates to whether the prompt template itself is made up entirely of natural language tokens/phrases (discrete prompts) or continuous, tunable parameters (continuous prompts). Discrete prompts encompass the templates we have encountered in this section, where the tokens relating the input $\mathbf{x}$ to the masked output $\mathbf{z}$ are held fixed. Continuous prompts have non-fixed tokens, which can vary as a model training component. For example, the discrete template “the capital city of [x] is [z].” could be replaced by the continuous prompt “[a₁] [a₂] [a₃] [a₄] [x] [a₅] [z]”, where the tokens a_n are fine-tuned to optimize results during training. The following subsections will examine representative methods and their promise within these prompt template categories. A summary of the different approaches is shown in Table 3.3 at the end of the section.

3.3.3.1 Prompt Mining

Prompt mining, first proposed by Jiang et al. (2020), is a method where prompts are mined from a large corpus of text based on the logic that words in the vicinity of a subject x and the object y frequently describe the relation between them.

Take again our example of capital cities; in a large corpus, instances where *Poland* and *Warsaw* closely co-occur are likely, on average, to imply some relation between a country and its capital. If you assemble many samples of *subject-object* pairs with the same relationship (i.e., more countries and their capital city) and extracted sentences from the corpus where they co-occur, these sentences can provide the basis for useful prompt templates for this information retrieval task.

Prompts generated using this corpus mining approach can be defined using one of two prompt generation methods. The first generation approach, known as middle-

word prompt extraction, works by taking sentences from the search corpus that contain the *subject-object* pair and extracting the text token(s) between them, which then serve as the prompt template itself. To illustrate, imagine again that we are mining for prompts to maximize the activation of the knowledge that the capital city of *Poland* is *Warsaw*. By searching within a corpus for sentences containing these two entities, we find the following:

Warsaw is the capital city of Poland, and
has a population of 1.86 million people.

By extracting only the words between the *subject-object* pair, we get the following:

"is the capital city of"

Which is then formulated as the following prompt template:

"[z] is the capital city of [x]"

This process is iterated for the complete set of in-scope *subject-object* pairs derived from the small training set, and middle-word prompts are searched for and extracted for each pair.

The second approach for mining prompt templates from [Jiang et al. \(2020\)](#) leverages a more linguistically sophisticated extraction process. Namely, syntactic analysis extracts templates that represent the shortest dependency paths between the *subject-object* pair within the matched sentence.

To illustrate, a middle-word prompt template extracted from the sentence "*The capital of Poland is Warsaw.*" would be "[x] is [z]", which is clearly too simplistic to accomplish our task. However, dependency analysis on this same sentence would result in the following dependency path; "*Poland* $\xleftarrow{pobj}$ *of* $\xleftarrow{prep}$ *capital* $\xleftarrow{nsubj}$ *is* $\xleftarrow{attr}$ *Warsaw*", which gives the template "*capital of [x] is [z]*", which looks like a more plausible template for the capital city retrieval task. It is also possible that these dependency-parsed templates will be better for activating the types of knowledge being targeted since they are derived from stable linguistic rules, which the LLM is expected to have learned during pre-training ([Jawahar et al., 2019](#)).

3.3.3.2 Prompt Paraphrasing

Prompt paraphrasing aims to take a preexisting prompt and maximize lexical diversity by generating template variants. With our capital city example, we can create several slightly different versions:

- **Original Prompt:** "[z] is the capital city of [x]."
- **Paraphrased Prompt 1:** "[z], the capital city of [x]."
- **Paraphrased Prompt 2:** "[z] is the capital of [x]."
- **Paraphrased Prompt 3:** "[x]'s capital city is [z]."

- **Paraphrased Prompt n :** “[x]’s capital, [z].”

To automate this prompt paraphrasing process, [Mallinson et al. \(2017\)](#) developed a back-translation approach. This method follows a process wherein the original prompt to be paraphrased is translated into B candidate translations in a different language. Each of these is then translated into the same language as the original prompt to give B^2 candidate templates. These candidate prompt templates are then downselected by ranking their round-trip probabilities, which are calculated as,

$$P(t) = P_{forward}(\bar{t}|\hat{t}) \cdot P_{backward}(t|\bar{t}) \quad (3.9)$$

where $\hat{t}$ is the original prompt, $\bar{t}$ is the translation of the original prompt $\hat{t}$, and t is the final prompt candidate being ranked. Prompts are then retained by selecting the top T ranked candidates. These prompt candidates can then be subjected to additional downselection and ensembling to optimize their utility in solving the target NLP task. These prompt selection and ensembling techniques are discussed further in Sect. 3.5.

3.3.3.3 Gradient-directed Search

Another approach is to design prompt templates using a *gradient-directed search* method. This concept was initially proposed by [Wallace et al. \(2019\)](#), who were interested in adversarial attacks on generative models. These authors created an algorithm that iteratively updated “trigger tokens” appended to a prefix prompt just before the response slot to minimize the loss when an incorrect response is filled into the answer slot. For a concrete example, consider the following question answering (QA) prompt shape:

$$\text{“Question: } [\mathbf{x}] \text{ Context: } [\mathbf{y}] \text{ Answer: } [\mathbf{T}] [\mathbf{T}] [\mathbf{T}] [\mathbf{z}_{adv}] \text{”}, \quad (3.10)$$

where $[\mathbf{x}]$ and $[\mathbf{y}]$ are the question and context, $[\mathbf{z}_{adv}]$ is an adversarial output that we are trying to trick the model into producing, and $[\mathbf{T}]$ are a series of nonstatic “trigger” tokens that can be iteratively updated to minimize the loss of the sequence according to some language model. These updates are done by a gradient-guided search based on the HotFlip approach ([Ebrahimi et al., 2018](#)). This procedure induced the model to generate an adversarial response, and critically the authors found that in many instances the optimized sequence of trigger tokens were robust to changes in the input text, producing the same inappropriate output for many different inputs. An example from their work, using a question/answer pair from the SQuAD dataset:

Question: Why did he walk?

For exercise, Tesla walked between 8 to 10 miles per day. He squished his toes one hundred times for each foot every night, saying that it stimulated his brain cells. **why how because** to kill american people

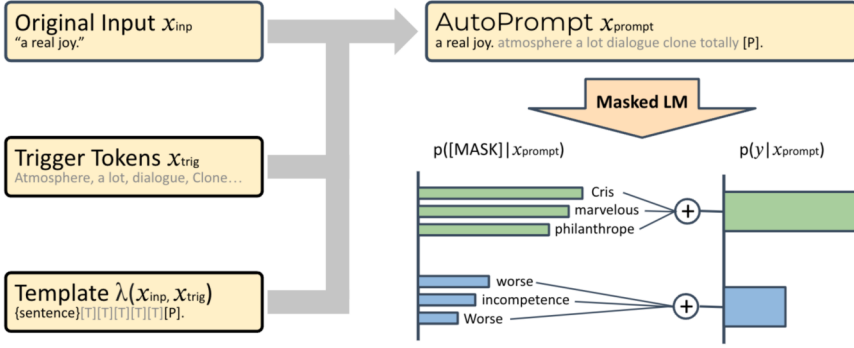


Fig. 3.11: An illustration of AutoPrompt applied to probe a masked language model's (MLM's) ability to perform sentiment analysis. Each input, x_{inp} , is placed into a natural language prompt, x_{prompt} , which contains a single [MASK] token to be predicted. The prompt is created using a template, λ , which combines the original input with a set of learned trigger tokens, x_{trig} . The trigger tokens are shared across all inputs and are learned using the described gradient-based search process. Probabilities for each class label, y , are obtained by marginalizing the MLM predictions, $p([MASK] | x_{prompt})$, over sets of automatically detected label tokens

The three tokens **why how because** are the product of their gradient optimization, and cause GPT-2 to generate the adversarial underlined response for many different inputs.

The promise of this approach for optimizing templates for the purpose of prompt engineering were quickly recognized. Building from this work [Shin et al. \(2020\)](#) proposed AutoPrompt as an approach to construct prompt templates automatically. These authors took a series of initial templates, including trigger tokens, similar to Equation 3.10, and optimized the tokens by a gradient-guided search, iterating over a sizable set of labeled input/output examples. Their method is depicted in Fig. 3.11, with an example of the sentiment analysis task. As seen in this figure, the input to the language model is constructed from three key components:

- **Original input (x_{inp})**: This maps to input x from Fig. 3.8.
- **Trigger Tokens (x_{trig})**: These are the natural language tokens learned through gradient search. The number of tokens learned depends on how many tokens the gradient search method is initialized with and can be considered a hyperparameter in this context.
- **Answer Slot**: This is represented by [P] or [MASK] in Fig. 3.11, and maps to the [z] slot in the example provided in Fig. 3.8

Each component is combined within the structural definition of a given prompt template to provide the optimized input to the language model (i.e., x_{prompt}). The label class is then determined by summing the probabilities of a number of automatically selected output tokens. In this example, *Cris*, *marvelous* and *philanthrope*

were derived for the positive class, and *worse*, *incompetence*, and *Worse* comprise the negative class. The cumulative probability of the positive labels exceeds that of the negative labels, denoting a positive sentiment classification.

Although the optimized tokens may not seem intuitive to a human, [Shin et al. \(2020\)](#) reported a complete 9% accuracy points gain over the Top-1 paraphrased prompts evaluated in [Jiang et al. \(2020\)](#) when tested on the same LAMA T-REx entity-relation subset benchmark relative to manual templates. They also show that using BERT and RoBERTa variants, AutoPrompt outperforms manual prompting by 10-20% on tasks such as answer mapping, natural language inference, fact retrieval, and entity relation extraction. Critically, they show that optimized prompting can even out-compete fine-tuned variants, particularly in low-data situations, where you may have only have a handful of labeled samples.

3.3.3.4 Prompt Generation

So far, all of the discrete prompt searching methods we have reviewed have leveraged masked language models, where singular tokens are predicted. Taking inspiration

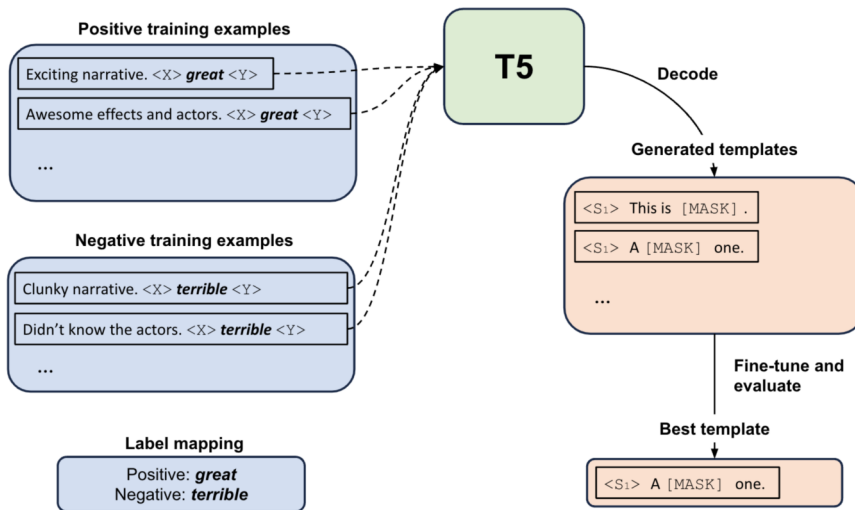


Fig. 3.12: Illustration of the prompt generation process, wherein input examples from D_{train} are partitioned by class, formulated as suitable inputs for T5, and used to decode to a single or small set of templates that maximizes the sum given in Equation 3.11.

from the “in-context” learning capabilities of LLMs demonstrated in Radford et al. (2019), Brown et al. (2020), and others, Gao et al. (2021) introduce the *Better Few-shot Fine-tuning of Language Models* (LM-BFF) approach. Instead of the MLMs used in the previous examples, these authors use T5, a large text-to-text or sequence-to-sequence model (Raffel et al., 2020). Their approach uses a few training examples in a “few-shot” configuration to prompt T5 to search for optimal prompts.

Fig. 3.12 illustrates how the T5 model is used in Gao et al. (2021) to generate prompt templates. Several samples are extracted from the small D_{train} dataset and passed to T5, which is prompted to construct a template $\mathcal{T}$ without the need to be explicit about the number of tokens the template should contain. The inputs to T5 are carefully designed to achieve the prompt generation outcome. Fig. 3.12 shows how D_{train} are grouped into class-specific samples, which are formulated as appropriate inputs for T5 and then used to generate a template or set of templates $\mathcal{T}$ that maximizes:

$$\sum_{(x_{in}, y) \in D_{train}} \log P_{T5}(\mathcal{T} | \mathcal{T}(x_{in}, y)), \quad (3.11)$$

where P_{T5} is the output probability distribution from the T5 language model.

Once a given set of templates is generated using these formulations as input to T5, the generated templates are decoded, and the *best template* is selected following fine-tuning of $\mathcal{L}$ using D_{train} , and evaluation on D_{dev} . Gao et al. (2021) demonstrated that their novel prompt template generation method, coupled with providing semantically similar demonstrations along with a given input, significantly improves performance over manually designed prompts. Additionally, leveraging manually or automatically generated prompts with T5 outperforms standard fine-tuning solutions, demonstrating the utility of prompt-based learning in NLP.

Building from work presented in Gao et al. (2021), Ben-David et al. (2022) introduced *Prompt learning algorithm for on-the-fly Any-Domain Adaptation* (PADA), which is a method that also leverages the text-to-text prompt generation capabilities of the T5 language model, but aims explicitly to generate human-readable prompts that represent multiple source domains (Ben-David et al., 2022). Thus, aiming to solve the common challenge of predicting out-of-distribution data, the PADA algorithm maps multiple specific domains into a shared semantic space, providing greater generalization potential. Ben-David et al. (2022) reported impressive performance relative to robust baseline solutions for both text classification and sequence tagging tasks using this approach.

3.3.4 Automated Template Design: Continuous Search

Considering that the primary goal of prompt construction is to develop a method that empowers an LLM to efficiently accomplish a task rather than solely generating prompts for human understanding, it is not essential to confine the prompt to human-interpretable natural language (Li et al., 2019). Consequently, alternative approaches

Table 3.3: Summary of discrete automated prompt design approaches

Method	Summary	Example	Pros	Cons
Prompt mining	Searching large corpora for sentences which contain given query/answer pairs, and using them to create a prompt template.	Test pair: Poland :: Warsaw Mined sentence: “Warsaw is the capital city of Poland, and has a population 1.86 million people” (found in Wikipedia) Derived template: “[z] is the capital city of [x] .”	• Programmatically simple. • Can be expanded with more sophisticated linguistic extraction processes.	• Domain of output templates constrained to sentences in corpora. • Optimization far from guaranteed.
Prompt paraphrasing	A seed prompt is iterated on with translation chains to produce many subtle variants, and the best performing one is selected.	Seed prompt: [z] is the capital city of [x] . Back-translation variants: • “[z], the capital city of [x] .” • “[x]’s capital city is [z] .” • “[x]’s capital, [z] .”	• Programmatically simple. • Variety of tested prompts helps to optimize.	• Domain of responses fairly narrow and limited. • Optimization far from guaranteed.
Gradient search	A series of variable trigger tokens (here, [T]), combined with input/prediction pairs, are combined into a template that is optimized during the training process to produce the best prediction results.	Review/Sentiment pair: “a real joy” :: positive Initial prompt: “a real joy [T][T][T] [T][T] positive” Gradient-optimized prompt: ‘ a real joy atmosphere alot dialogue Clone totally positive’ Optimized template: [x] atmosphere alot dialogue Clone totally [z]	• Can produce highly optimized input tokens. • Does not rely on existing sentence corpora for its domain.	• Computationally expensive and programmatically complex. • Unintuitive template results. • Output templates constrained to human language embeddings.
Prompt generation	An encoder-decoder model (e.g., T5) predicts tokens in a seed template created with training query/response pairs. The resulting predictions are converted templates and tested for quality.	Seed templates for review sentiment: • A pleasure to watch. <X> great <Y> • No reason to watch. <X> terrible <Y> T5-filled templates: • A pleasure to watch. This is great . • No reason to watch. A terrible one. Derived templates for testing: • [x] This is [z]. • [x] A [z] one.	• Variety of tested prompts helps to optimize. • LLM-derived templates may be constructed fairly well optimized for LLM usage.	• Computationally expensive and programmatically complex. • Human input required for seed templates. • Optimization not guaranteed.

have emerged that investigate continuous prompts, also called soft prompts, enabling prompting directly within the model’s embedding space.

Importantly, continuous prompts address two critical limitations of discrete prompts:

1. They reduce the necessity for template word embeddings to align with the embeddings of natural language words, such as those found in English.
2. They remove the constraint that pre-trained LM parameters parameterize the template. Instead, the templates have parameters that can be fine-tuned based on the training data obtained from the downstream task.

3.3.4.1 Prefix Tuning

Prefix tuning was initially presented in [Li and Liang \(2021\)](#). Inspired by the success of in-context learning with prompts (see Sect. 3.5.2), prefix tuning introduces task-specific “virtual tokens” that are added to the beginning of the input text (Fig. 3.13). These vectors do not represent actual tokens, but their dimensions are initialized such that the language model can attend to them in the same manner as hard tokens. They can then be treated as continuous vectors for training, whereas hard tokens have a fixed representation. This approach makes it possible for the language model to learn the nature of the task by tuning the prefix rather than relying solely on the explicit discrete features in the prompt’s text.

Indeed, [Li and Liang \(2021\)](#) reported that their prefix-tuning trials outperformed fine-tuning in low-data settings and were competitive with full data fine-tuning. By applying the prefix-tuning approach to BART [Lewis et al. \(2019\)](#) for summarization and to GPT-2 [Radford et al. \(2019\)](#) for table-to-text, the method achieved strong results on both tasks relative to the established adaptor and full data fine-tuning benchmarks. Importantly, these results indicate that the prefix-tuning approach generalizes well across language model types and was specifically shown to do so for encoder-decoder and autoregressive models.

As with AutoPrompt, where training datasets are used to optimize a set of discrete prompts through a gradient-directed search in discrete space (Sect. 3.3.3.3), prefix-tuning leverages training data to learn a set of continuous vectors (i.e., the prefix) that maximizes:

$$\max_{\phi} \log p_{\phi}(y|x) = \sum_{i \in Y_{idx}} \log p_{\phi}(z_i|h_{<i}) \quad (3.12)$$

where p_{ϕ} , which typically represents the trainable parameters of an LLM, are replaced with P_{θ} , representing the prefix parameters θ , since the LLM’s parameter are fixed. h_i is the concatenation of all activation layers, including the prefix at time step i . Since prefix-tuning leverages left-to-right or autoregressive language models,

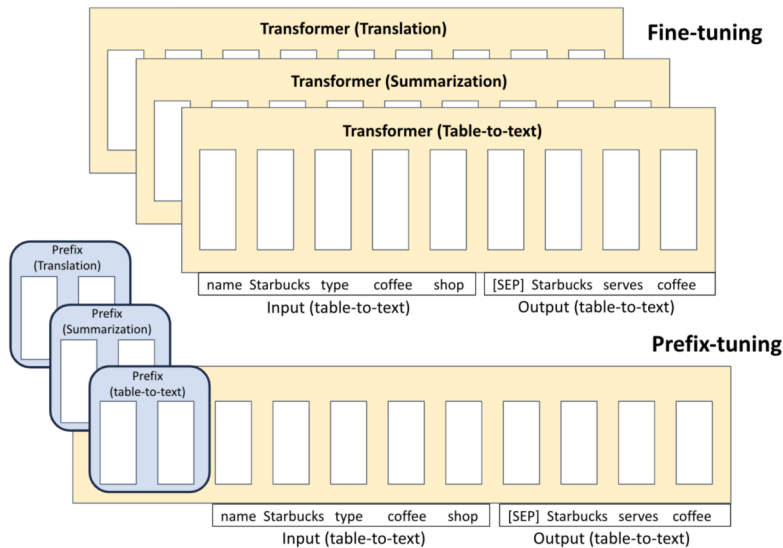


Fig. 3.13: Pre-training and fine-tuning (top) requires that the pre-trained model’s parameters be copied and tuned for each downstream task, which, given the scale of some language models, represents a significant cost and technical challenge. Prefix-tuning (bottom) aims to overcome this challenge by freezing the parameters of the model and tuning only a task-specific prefix. Then by swapping in different tuned prefixes, this allows a single LLM to be used across multiple downstream tasks enabling both modularity in task solutions and a more space-efficient solution overall.

and as the name suggests, the learned vectors are prefixed to the leftmost layers of the language model, the influence of these prefixes percolates through the language model from left to right through all of the LM’s fixed layers.

3.3.4.2 Hybrid and Discrete initialized Prompts

One key challenge identified within the work from [Li and Liang \(2021\)](#) was the prefix-tuning instability resulting from prefix parameter initialization and sensitivity to the learning rate. In that work, the solution was to parameterize the prefixes instead of using a smaller matrix generated using an extensive feed-forward neural network. However, another approach for initializing continuous tokens is to use informed discrete tokens. These tokens can be learned, as in previous automated discrete template search (e.g., [Zhong et al. \(2021\)](#)), or can be manually defined, and have shown promise in entity-relation knowledge probing tasks when used as the initialization point when learning continuous tokens ([Qin and Eisner, 2021](#); [Zhong et al., 2021](#)).

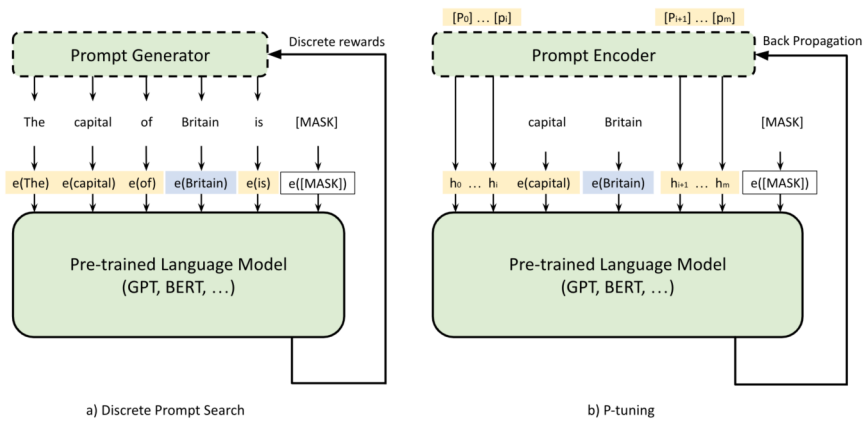


Fig. 3.14: An example of prompt search for “The capital of Britain is [MASK]”. Given the context (darkest gray, “Britain”) and target (white box, “[MASK]”), the lightest gray regions refer to the prompt tokens. In (a), the prompt generator only receives discrete rewards, while in (b), the pseudo-prompts and prompt encoder can be optimized differently. Sometimes, adding a few task-related anchor tokens, such as “capital” in (b), will further improve downstream task performance.

Another promising automated prompt template design innovation is the use of hard and soft tokens to compose the final template, as proposed in [Liu et al. \(2021\)](#). Fig. 3.14 illustrates how these hybrid prompts are generated (Fig. 3.14b), in contrast to how discrete prompt templates are generated (Fig. 3.14a). Importantly, [Liu et al. \(2021\)](#) demonstrated that their P-tuning method for prompt template generation outperformed all other tested discrete prompt templates in a knowledge probing task using BERT and achieved SOTA performance on the few-shot SuperGLUE benchmark using ALBERT ([Lan et al., 2020](#)).

3.3.5 Prompt-based Fine-tuning

The prompting approaches discussed thus far have all assumed that we vary a prompt or prefix to optimize against a static inference model. An alternative approach is to unfreeze the model parameters and fine-tune them using traditional backpropagation methodology on a dataset of input/output pairs arranged in a fixed template – this is called *prompt-based fine-tuning*. Consider again the example given in Fig. 3.7:

“Cannot watch this movie. This is [z].”

Instead of performing inference with this template, we can tune the model to accurately predict a value of [z] assigned by hand. Successful prompt-based fine-tuning

can require sizable samples of input/output pairs templated in a consistent manner, although frequently, the quantity of labeled samples required for good performance is lower than for standard PTFT (see the tutorial in Sect. 3.6). This approach is viable both for cloze prompting, which fine-tunes on a series of filled prompts of common shape and different input/output pairs to predict the token in the answer slot, and for prefix prompting, which fine-tunes on a prefix answer to iteratively predict the next token in a trailing answer of arbitrary length. Inference is then performed with the tuned model using the standard prompt-based learning procedure.

Prompt-based fine-tuning is the fundamental technique used for instruction tuning, which is a critical step in the development of SOTA chatbots such as ChatGPT. We will discuss instruction tuning in detail in Sect. 4.2, so we defer further discussion until then and simply note here that prompt-based training generally outperforms zero/few-shot prompt-based learning (at potentially significant computational cost), and in many cases outperforms PTFT, particularly in the data-poor regime.

3.4 Answer engineering

Similar to how prompt engineering facilitates optimal choice of template, *answer engineering* encompasses designing and optimizing answer formats to guide the language model in generating the most accurate and contextually relevant responses to specific tasks or questions. This involves carefully considering various factors, formulating answer shapes, and exploring the answer search space in distinct ways to map to labels.

3.4.1 Answer Shape

The first consideration is the *answer shape*. This property determines the granularity of the model's outputs, ranging from individual tokens to entire sentences or phrases. Different tasks require varying levels of granularity in the responses; hence, selecting an appropriate answer shape is crucial for the success of prompt-based learning techniques. There are three basic types of answer shape:

- **Tokens:** These represent one or more individual tokens from the pre-trained LM's vocabulary or a subset thereof. Token-based answer shapes are often used in classification tasks such as sentiment classification, relation extraction, or named entity recognition (Cui et al., 2021; Petroni et al., 2019; Yin et al., 2019). For instance, in sentiment classification, the model's answer could be a single

token, such as “positive”, “negative”, or “neutral”. For this answer shape, the answer space is usually restricted to a few choices of token, and thus falls into the constrained answer space category.

- **Chunks:** A chunk or a span includes a short multitoken sequence typically used in conjunction with cloze prompts. The distinction from the token answer shape is that they are not of fixed length and are generally in the unconstrained answer space category. This makes them useful for question-answering tasks, such as, for instance, the response to a prompt such as “Dante was born in [z].”
- **Sentences:** Sentence-based shapes are the answers that comprise one or more sentences or even an entire document based on the task. Sentence-based answers are commonly employed with prefix prompts and are frequently used in language generation tasks that require more detailed responses such as summarization or translation (Radford et al., 2019). They are unconstrained.

3.4.2 Defining the Answer Space

The *answer space*, which we denote as $\mathcal{Z}$, is defined as the set of potential answers that a model can provide in response to an input. In many instances, this answer space maps to a series of output classes, denoted as $\mathcal{Y}$. There are two general classes of answer space: constrained and unconstrained.

- In **constrained answer spaces**, the output space is limited to a predefined set of answers. This is useful in tasks with a finite number of output labels, such as text classification, entity recognition, or multiple-choice question answering.
 - For this configuration, every element in $\mathcal{Z}$ maps to an element in the label space $\mathcal{Y}$ for the final output (e.g., the answer “terrible” maps to the negative label class in the sentiment analysis task of Fig. 3.4).
 - Associated with the token answer shape.
- In **unconstrained answer spaces**, $\mathcal{Z}$ encompasses all tokens (Petroni et al., 2019), fixed-length spans (Jiang et al., 2020), or token sequences (Radford et al., 2019).
 - These outputs generally do not map to a distinct label space, but instead are themselves the final outputs.
 - Associated with the chunk and sentence answer shapes.

As noted, constrained answer spaces require specific tokens to be selected to comprise $\mathcal{Z}$. In the next three sections, we discuss approaches for choosing these elements.

3.4.3 Manual Answer Mapping

The simplest approach to defining $\mathcal{Z}$ is to do so by hand. This involves a trial-and-error process where different sets of answers are selected to correspond to each label and tested against data to determine efficacy. Considering again our sentiment analysis example, “ $\mathbf{x}' = [\mathbf{x}]$ This is a $[\mathbf{z}]$ movie.”, we have initially adopted $\mathcal{Z} = \{great \rightarrow positive, terrible \rightarrow negative\}$. We could alternatively test $\mathcal{Z} = \{good \rightarrow positive, bad \rightarrow negative\}$, and can only determine which performs better empirically.

Note that mappings need not be one-to-one. Multiple answers could correspond to single labels. For example, we could expand the answer space to $\mathcal{Z} = \{great, terrible, good, bad\}$, and select the label based on whether the highest probability is in the positive class ($great|good$) or the negative class ($terrible|bad$).

3.4.4 Automated Answer Mapping: Discrete Search

Some authors have developed techniques to automate the answer selection process. [Jiang et al. \(2020\)](#) employs an iterative process, initiating with an elementary answer space $\mathcal{Z}'$ and expanding its scope through paraphrasing techniques. For this approach, the authors collect answer-label pairs $(\mathbf{z}', \mathbf{y})$, and vary the answer $\mathbf{z}'$ using back-translation to find similar but distinct terms. These can then be tested for efficacy.

[Gao et al. \(2021\)](#) also introduced an approach to defining a label word mapping $\mathcal{Z} \rightarrow \mathcal{Y}$ that aims to maximize accuracy on a small validation dataset. They use an LLM to suggest possible answer mappings by passing them templates filled with input data and aggregating the highest likelihood predictions. This produces a ranked list of tokens predicted by your LLM to fill in the mask for each label in your dataset. You can then take the top n values to create your answer mapping.

3.4.5 Automated Answer Mapping: Continuous Search

Continuous answer searching operates directly within the model’s embedding space. Similar to the case of continuous prompts (Sect. 3.3.4), the requirement that the output map to a known token is removed and further optimization becomes possible. Consider, for instance, the labels “positive” and “negative”. In human language, these words are perhaps the closest representation of the desired outputs. However, this does not necessarily mean that the embeddings of those words are the best possible outputs for the model’s solution to the task. Continuous searching allows the model to give answers closer to its own information representation without forced translation into human language.

3.5 Multi-Prompt Inference

There is a tacit feature shared by all prompt shapes that we have discussed thus far in this chapter: they consist of a single query with a single (masked) response token(s). For example, “The capital city of Poland is [MASK]” provides one question and asks for one answer. With this approach, the model has only its pre-trained weights and this single prompt to benefit from at the time of inference. This can limit the predictive capabilities of LLMs when the context is sufficiently sparse – this counts doubly so in the zero-shot context where we are prompting a model that has not been fine-tuned for the requested task. Furthermore, while our template may have been chosen through an optimization process, the particular biases of its formulation may lead to inaccuracies or systematic errors in predictions that are difficult to combat.

Several so-called *multi-prompt inference* approaches have been considered in the literature to address these shortcomings. Characteristically, these approaches do not rely on the LLM’s response to a single prompt but provide additional context in the form of question/answer pairs passed to the model at inference time or aggregate the results from separate prompts or sub-prompts to improve results on average. We will briefly discuss a few multi-prompt approaches and demonstrate some advantages with a practical example.

3.5.1 Ensembling

The simplest approach to multi-prompt inference is *ensembling*. In ensembling, the user applies multiple prompt templates f_{prompt} to a given query/answer pair $(\mathbf{x}, \mathbf{z})$ and aggregates the various prediction scores to calculate a cumulative best guess for an inference problem. This approach takes advantage of the fact that different prompt templates will have different subtle biases, which can cause individuals to make incorrect guesses but which, in the aggregate, will be more accurate than any of them.

To demonstrate the efficacy of ensembling, we re-purpose the task and prompts from Sect. 3.3.2 to query for the capital city of Canada, again using AllenNLP. We assemble four separate masked-language prompts as follows:

1. [MASK] is located in Canada, and is its capital city.
2. I am in the capital of Canada, I am in [MASK].
3. [MASK] is the capital city of Canada.
4. The capital city of Canada is [MASK].

Listing 3.2: Capital of Canada prompts

In Table 3.4, we show the top five scores for each prompt, encompassing a variety of Canadian cities (and “Canada” itself). Prompt 1 correctly predicts Ottawa, but only marginally over the second-place Winnipeg. Prompt 3 is similar. Prompt 4 performed the best at identifying Ottawa as the capital; prompt 2 lists Ottawa fourth

Table 3.4: Prediction scores for a number of Canadian cities (and “Canada”) based on zero-shot inference from AllenNLP using the prompts given in Listing 3.2. The bottom row reports the simple average of each column, adopting a score of 0 when a city was not among the top-5 predictions (denoted by —). Note that because we only record the top 5 samples, these averages are not strictly correct – e.g., prompt 2 would likely produce a non-zero prediction for Winnipeg, which would slightly increase its score. However, this impact is limited and does not change the conclusions.

Input	Otta.	Winn.	Mont.	Toro.	Calg.	Lond.	Canada	Edmo.	Vanc.
Prompt 1	17.3	15.8	10	6.8	8.1	—	—	—	—
Prompt 2	7.3	—	5.5	11.3	—	13.4	12.2	—	—
Prompt 3	24.9	19.8	10	10.4	7.2	—	—	—	—
Prompt 4	29.8	10.6	14.4	—	—	—	—	7.6	6.2
Average	19.83	11.55	9.98	7.13	3.83	3.35	3.05	1.9	1.55

behind several other Canadian cities. In the bottom row, we average the prediction scores between the different prompts for each unique response token, adopting a prediction score of zero when a city is not in the top 5 (see the table caption). The ensemble has correctly reported the capital of Canada, outperforming prompts 1, 2, and 3 in the score difference between the first and second samples.

There are numerous ways to create an array of templates for ensembling, several of which we have discussed in Sect. 3.3.2-3.3.4. Aside from the choice of which prompts to aggregate, an additional design consideration in ensembling is how to translate the various probability calculations from multiple prompts into a single number. Several approaches to score aggregation have been considered in the literature.

- **Simple majority:** The simplest approach is to pick the unique response $[z]$ that appears as the top choice for the largest number of prompts (e.g. [Lester et al., 2021](#)). The simple majority has long been shown to perform as well as more advanced ensembling approaches in many contexts ([Lam and Suen, 1997](#)), and is a reasonable base case for comparing more sophisticated ensemble results against.
- **Simple average:** For each unique response $[z]$, we sum the probabilities $P(z|x')$ across every filled prompt template x' in the ensemble, and divide by the total number of prompts. The response with the highest average score is then selected.
- **Weighted average:** Optionally, weights can be applied to different prompts in the ensemble to increase or decrease the individual contributions to the average. This can be considered the generalized case of simple averaging. These weights may be picked *ad hoc* based on subjective performance, selected based on objective performance metrics such as a test-set prediction accuracy (e.g. [Schick and Schütze, 2020a](#)), or tuned in combination with prompt-based learning ([Jiang et al., 2020](#)).
- **Knowledge distillation:** This final method uses the ensembled predictions against an unlabeled dataset as the training input to an additional model, which becomes

the classifier used for the prediction task. The advantage of this approach is that inference can be made less expensive by downsampling from an ensemble to a single model while still retaining the benefits of ensembling through training on features of the ensemble results (Hinton et al., 2015). This approach has been successfully leveraged in the prompt-learning context by, e.g., Schick and Schütze (2021).

3.5.2 In-context Learning

A second approach to multi-prompting is *in-context learning* (ICL), also called prompt augmentation or demonstration learning (Liu et al., 2023). In ICL, a given prompt is modified by prepending (or postpending) one or more template instances filled with query/answer pairs. This multi-prompt string is then passed to the model for inference. For example, instead of ``The capital of Canada is [MASK] .'', you might pass ``The capital of France is Paris. The capital of Canada is [MASK] .''. The additional context of the ICL prompt can greatly enhance the accuracy of LLM classification and generation tasks (e.g. Brown et al., 2020) by leveraging the pattern recognition capabilities of language models.

As a demonstration, the following are scores from AllenNLP for the two queries in the paragraph above.

```
The capital of Canada is [MASK] .
  "Ottawa": 0.28438693284988403
  "Montreal": 0.1832585334777832
  ...

The capital of France is Paris. The capital of Canada is [MASK] .
  "Ottawa": 0.3411746919155121
  "Vancouver": 0.1890593022108078
  ...
```

Listing 3.3: In-context learning examples for the capitals of Canada and France

Using an in-context filled prompt, we find a modest increase in the prediction score of the (correct) top choice, demonstrating the value of ICL. Notably, the choice of in-context augmentation samples has a significant impact on the precision gain and can, to quote Lu et al. (2022), “make the difference between near state-of-the-art and random guess performance.” As an example, we show the results from prepending or postpending an additional in-context sample to the prompt:

```
The capital of France is Paris. The capital of
Thailand is Bangkok. The capital of Canada is [MASK] .
  "Vancouver": 0.31520649790763855
  "Ottawa": 0.277070015668869
  ...

The capital of France is Paris. The capital of Canada is
```



```
[MASK] . The capital of Thailand is Bangkok.
      "Ottawa": 0.4077857434749603
      "Vancouver": 0.2586808502674103
      ...
```

Listing 3.4: Expanded in-context learning examples for the capitals of France, Thailand, and Canada

When adding the additional in-context sample to the preamble, we find that the model now incorrectly names Vancouver as the capital of Canada. However, if we add the additional sample to the end of the prompt, we find further improvement. Prediction scores are highly sensitive to small details of the ICL samples and their placement. Consequently, great care (and perhaps automated optimization, e.g., [Lu et al. 2022](#)) must be taken in creating an ICL prompt template.

3.5.3 Prompt Decomposition

A final style of multi-prompt learning involves breaking a complex prompting task into a number of individual discrete tasks with their unique prompts. Problems suited to this approach are characteristically those where an input string of text elicits multiple responses instead of a single response. An example is named entity recognition, where specific labels are applied to more than one token within the input sentence. More generally, the approach is suited for any sequence labeling tasks, such as part-of-speech identification.

Taking named-entity recognition, we can see why prompt decomposition is valuable with an example. Consider the following sentence:

```
Serena won the tennis tournament at Flushing Meadows.
```

Listing 3.5: Serena's victory at Flushing Meadows

If we wanted to identify each of the named entities in this sentence, it would be very challenging to create a single prompt aimed at reporting Serena as a name, tennis as a sport, and Flushing Meadows as a location. It would certainly be beyond the capacity of *cloze*-style prompts. Instead, we can simplify by creating a series of prompts that ask about each token in the sentence.

```
Serena won the tennis tournament at Flushing Meadows. Choosing
between name, location, sport, or none, "Serena" is a [MASK] .
      "nickname": 0.17498865723609924
      "choice": 0.042668603360652924
Serena won the tennis tournament at Flushing Meadows. Choosing
between name, location, sport, or none, tennis is a [MASK] .
      "sport": 0.08051449060440063,
      "synonym": 0.03813215345144272
Serena won the tennis tournament at Flushing Meadows. Choosing
between name, location, sport, or none, "Flushing Meadows" is
a [MASK] .
```

```
"suburb": 0.1842394471168518,  
"neighborhood": 0.09695208817720413,  
Serena won the tennis tournament at Flushing Meadows. Choosing  
between name, location, sport, or none, "the" is a [MASK] .  
"suffix": 0.11233088374137878  
"noun": 0.08212842047214508
```

Listing 3.6: Prompt Decomposition Examples

Even with this crude approach, the model identifies the three entities with reasonable accuracy and provides a non-entity response for the non-entity token “the”. In practice, for a true named-entity recognition solution, you would create a prompt for every n-gram within the input sentence and generate responses for each — otherwise, you would be unable to capture multi-token entities (such as Flushing Meadows). This approach becomes quite powerful with an associated verbalizer to constrain the allowed options and model tuning on a series of input examples (also decomposed into component prompts) to sharpen the accuracy.

Each of the three approaches discussed in this section is summarized in Table 3.5, along with individual strengths and weaknesses. Now that we have surveyed several important methodological innovations in prompt engineering for prompt-based learning literature, the next thing to do is to get some hands-on experience in their practical application. The next section of this chapter will dive into how you can experiment with these solutions for your projects.

3.6 First Tutorial: Prompt vs. Pre-train and Fine-tune Methods in Text Classification and NER

3.6.1 Overview

This chapter has introduced the concept of prompt-based learning and detailed several potential configurations for prompt and answer shape, but we have not yet demonstrated one of the most significant benefits of prompt-based approaches over PTFT approaches: its zero- and few-shot performance. This tutorial will show how prompt-based learning can achieve better results with fewer training examples than traditional head-based fine-tuning. This property allows LLMs to be adapted to new tasks with fewer data and cheaper computation cycles.

Goals:

- Compare and contrast prompt-based learning with head-based fine-tuning.
- Demonstrate that prompts can be effectively structured to accomplish various tasks.
- Introduce the OpenPrompt library as an example of how the techniques discussed throughout the chapter have been implemented.

Table 3.5: Summary of multi-prompt inference approaches

Design Method	Summary	Example	Pros	Cons
Ensembling	An inference task is completed with several templates, and the predictions scores for each test sample ensembled in some way (simple majority, average, distillation) to a final answer.	Prompt 1: "I am in the capital of Canada, I am in [MASK]." Prompt 2: "[MASK] is located in Canada, and is its capital city." Prompt 3: "[MASK] is the capital of Canada". Top results: 1) Toronto; 2) Ottawa; 3) Ottawa Simple majority: Ottawa (correct)	• Averaging of multiple prompts smooths over individual fail-states. • Makes full use of prompt engineering approaches that generate multiple candidates.	• Requires multiple prediction runs per test sample at inference time. • Only as good as your input prompts – little room for optimization.
In-context learning	Examples of a filled template with query/answer pairs is pre- or post-pended to the un-filled template before inference to help guide the LLM to a desired prediction.	Original prompt: "The capital of Canada is [MASK]" In-context prompt: "The capital of France is Paris. The capital of Canada is [MASK]" Top results: 1) Ottawa (p=0.284); 2) Ottawa (p=0.341)	• Does not require multiple predictions per test sample at inference time. • Very simple and intuitive to implement.	• Accuracy highly dependent on choice of in-context filled prompts. • Proper application likely requires fine-tuning of in-context prompt.
Prompt decomposition	A number of sub-prompts are created to probe specific aspects of a complex query, for example in identifying named entities within a sentence.	Original prompt: [x] = "Serena won the tennis tournament" Decomposed prompt: • "[x]. 'Serena' is a [MASK]" (person) • "[x]. 'Tennis' is a [MASK]" (sport) • "[x]. 'the' is a [MASK]" (non-entity)	• Can probe more fine-grained details of a query.	• Requires multiple prediction runs per test sample at inference time. • Multiple sub-prompts may have to be individually optimized.

- Plot learning curves to illustrate the strong performance of prompts in few-shot settings.

Please note that this is a condensed version of the tutorial. The full version is available at <https://github.com/springer-llms-deep-dive/llms-deep-dive-tutorials>.

3.6.2 Experimental Design

Our experiment will directly compare the zero-shot and few-shot capabilities of the PTFT and prompt-based learning approaches in their application to text classification and named-entity recognition. We adopt BERT as the basis for our fine-tuning exercises for this test. Using PyTorch, supplemented with OpenPrompt for the prompt-based portion, we will iteratively refine our BERT models with increasingly larger subsets of the training data, predicting on the validation sets at regular intervals to show how the model responds to few-shot learning. Finally, we will compare learning curves for the two tuning approaches for each NLP task and discuss the implications.

3.6.3 Results and Analysis

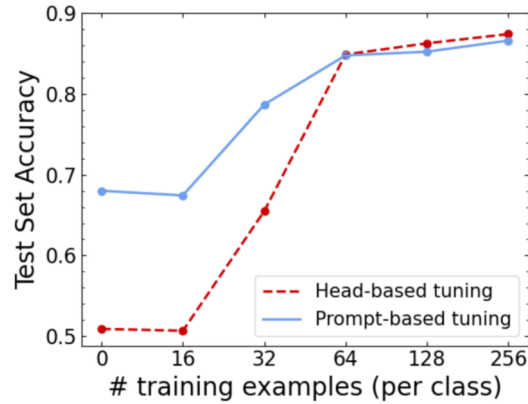
3.6.3.1 Text Classification

We begin with traditional head-based fine-tuning of a pre-trained language model for text classification. As described in Sect. 3.1.2, this process involves tuning a task-specific head with sentence/label pairs to enable transfer learning from the rich language representation of the LM (in this case, BERT) to the classification task. Our dataset is the sentiment analysis corpus *SST-2 GLUE* which consists of sentences extracted from movie reviews, hand-labeled for positive or negative sentiment about the film. We divide the train set into positives and negatives and train models with varying quantities of train samples. We then evaluate performance against the test set.

The results are shown in the middle column of Table 3.6. For zero-shot, accuracy is almost exactly 50%, no better than random guesses. There is little improvement with the first few tranches, but a marked improvement by 64 samples, eventually reaching 87% accuracy with 256 samples. It should be noted that the numbers in this table are sensitive to precisely which training samples are selected for the experiment and will vary somewhat in different runs. However, the basic story they tell does not change.

We turn now to a prompt-based training approach to see how the results compare. The resulting accuracies are shown in the right-hand column of Table 3.6. With only

Fig. 3.15 Graphical representation of the data from Table 3.6, showing the comparative accuracy of pre-train/fine tuning and prompt-based learning as a function of training examples for our text classification exercise.



32 examples from each class, the prompt model dramatically increases in accuracy. It then levels off quickly and gains relatively little ground with additional data.

Table 3.6: A comparison of the prediction Accuracy vs Num Train Samples of Pre-train/Fine-tune and Prompt-based text classification for the *SST-2 GLUE* dataset.

# Train Samples	PT/FT Accuracy	Prompting Accuracy
0	0.5092	0.6800
16	0.5069	0.6743
32	0.6548	0.7867
64	0.8486	0.8475
128	0.8624	0.8521
256	0.8739	0.8658

Fig. 3.15 plots the accuracy as a function of training examples for the two models, starting with zero-shot performance and progressively adding larger volumes of training data. In contrast to the head-based classifier, the prompt model achieves impressive results with very few training samples. The PTFT model eventually becomes competitive with the prompt-based model but requires 32 training samples (per class) to match the zero-shot performance of the prompt.

3.6.3.2 Named-entity recognition

We will now turn to a second classification task, named-entity recognition, expected to have poorer zero-shot performance and determine whether prompting still outperforms fine-tuning in the few-shot context. For this experiment our dataset is *CoNLL-2003*, a collection of news headlines and quotes with token-by-token entity tags as-

Table 3.7: A comparison of the F1-scores vs. number of train samples of pre-train/fine-tune and prompt-based named entity recognition for the *CoNLL-2003* dataset.

# Train Samples	PT/FT F1	Prompt F1
0	0.0687	0.0712
8	0.0492	0.5788
16	0.0023	0.6482
32	0.0034	0.7274
64	0.3323	0.7867
128	0.5578	0.8365
256	0.7157	0.8672
512	0.7894	0.8304
1024	0.8526	0.8551

signed to people, organizations, and locations, with a final category for miscellaneous entities.

The results of PTFT are shown in the left column of Table 3.7. Similar to the text classification exercise, BERT shows poor performance without any training. This is not surprising, as NER is a label identification exercise and BERT does not know yet what the labels in this dataset signify. It primarily predicts values > 0 , whereas most labels = 0, thus producing many false positives and a poor F1-score. The first few data points show that performance worsens as we introduce more training samples, likely due to catastrophic forgetting. After that, performance improves with each additional tranche of data, and by 1024 samples, we achieved $F1 = 85\%$.

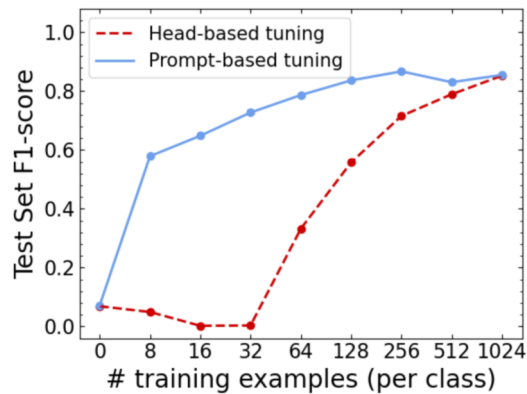
Next, using prompt-based tuning, we iteratively train the model with larger and larger quantities of training samples for five epochs and examine the learning curve. The results are in the right column of Table 3.7. Performance significantly improves with only a few sentences and gradually increases to 87% F1 at 256 samples. Tuning with larger amounts of data does not improve performance any further.

As a summary, we compare the PTFT and prompt-based tuning results in Fig. 3.16. The comparison is similar to the text classification situation – with sufficient data, pre-train/finetune becomes competitive with prompt-based learning, but in a data-starved regime, prompt-based tuning achieves much better results.

3.6.4 Conclusion

The defining conclusion from both experiments in this tutorial is that prompt-based tuning is superior to head-based fine-tuning when the available training set is limited in quantity. The few-shot learning results are especially impressive considering the long-running observation that acquiring an adequately large set of good-quality training data is the crux of most machine learning problems. In this prompting paradigm,

Fig. 3.16 Graphical representation of the data from Table 3.7, showing the comparative F1-score performance of pre-train/fine tuning and prompt-based learning as a function of training examples for our named entity recognition exercise.



the key to a high-quality model is instead the optimal design of prompt templates and answer formats. Properly selecting these critical ingredients produces high-quality NLP results with only a few dozen examples. The following chapter will explore this in greater depth.

3.7 Second Tutorial: Approaches to Prompt Engineering

3.7.1 Overview

Another central theme of this chapter is the use of template engineering to improve the analytic capabilities of prompt-tuned LLMs. In Listing 3.1 and Table 3.2, we demonstrated the sensitivity of LLM inference outputs to choices in template architecture and the fine details of prompt composition. That demo was accomplished with a web application, a useful proof-of-concept but inherently limited in its capabilities as it cannot be fine-tuned. Therefore in this tutorial, we will expand on these exercises by exploring few- and many-shot prompt-tuning, discussing results for variable prompt template designs, and aiming to grasp the critical importance of prompt template optimization.

Goals:

- Illustrate that task performance is highly sensitive to prompt template design, with even subtle variations making a notable difference.
- Explore some of the factors that lead to higher quality prompt templates.
- Conduct automatic tuning with soft prompts to demonstrate how they compare to manually constructed prompts.

Please note that this is a condensed version of the tutorial. The full version is available at <https://github.com/springer-llms-deep-dive/llms-deep-dive-tutorials>.

3.7.2 Experimental Design

This tutorial will consider several different approaches to template engineering and assess their performance in training a model against a benchmark dataset. We begin with the simplest approach: manual template engineering (see Sect. 3.3.2). In manual template engineering, it's up to the developer to create a template that best suits the task. One can reference the existing literature suggesting templates for all prompt-based learning tasks (see e.g., Sect. 3.2.4) or experiment with different configurations.

We will also explore automatic template design using training data. We have discussed some automated approaches in Sect. 3.3.3 and Sect. 3.3.4, and consider in this tutorial a style of gradient-based template optimization called *soft prompting*. In contrast to manual prompting, soft prompting uses a variable template that can be tuned to an optimal representation without the constraint of mapping to discrete tokens. The soft prompt is initialized with a template that combines the dataset features with “soft” tokens, which themselves may optionally be initialized to a given word or phrase, and refines the respective embeddings through backpropagation to achieve the training objective. We implement soft prompting using the OpenPrompt code base (Ding et al., 2021). Our experiments will show that soft prompts can outperform manually engineered prompts.

In this tutorial, we make use of the *SuperGLUE BoolQ* dataset, which provides triplets of an informational paragraph, a yes or no question related to the paragraph's content, and the correct response. The BoolQ dataset is very expansive in its topics, including history, science, geography, law, sports, pop culture, and more, making it a fascinating dataset for exploring LLMs' natural language inference capabilities.

3.7.3 Results and Analysis

For the manual prompt experiment, we will run prompt-based tuning on a t5-base model with three different prompt templates, and for several data sample quantities. The training samples are evenly split between the two label classes. We test three different templates, which we call 1) the “simplest” template, 2) the “simplest + punctuation” template, and 3) a more “suitable” template. The first simply concatenates the passage, question, and mask. The second adds some punctuation for guidance. The third adds guiding text to indicate the meaning of each portion of text, and uses a reasonable cloze-style formulation for the mask token.

```
temp1 = "{passage} {question} {mask}"
```


Table 3.8: A series of zero- and few-shot accuracy scores using *SuperGLUE BoolQ* for three different prompt templates.

Num Train Samples	Naive Template	Naive Punct	+ Improved Template
0	0.501	0.526	0.564
16	0.511	0.508	0.489
32	0.513	0.482	0.503
64	0.518	0.516	0.535
128	0.526	0.598	0.638
256	0.578	0.628	0.686
512	0.546	0.638	0.640

```
temp2 = "{passage} . {question} ? {mask}"
temp3 = "hypothesis: {passage} premise: {question} The answer was
{mask} ."
```

We run the fine-tuning experiment first with the simplest template, and show the results in the left column of Table 3.8. Overall, the model performance is poor:

- Zero-shot inference predicts the negative class for every sample, thus reproducing with its accuracy score the ratio of negative to total samples in the validation set (50/50).
- The few-shot examples do better, but only marginally better than random – not far from flipping a coin for each query.
- Model performance peaks with around 256 samples but never achieves impressive results.

We then test the simple change of adding a period to the passage if missing and a question mark to the end of the question. The result of this minor change, shown in the middle column of Table 3.8, is interesting. The zero-shot performance improves a bit – from ~50% to ~53% – simply from adding a “?” and a “.” in the appropriate places. Once fine-tuning begins, the punctuated template improves more rapidly than the simplest template, indicative of improved prompting.

Finally, we test the more suitable manual template. This template should produce better results, as it provides helpful context and precisely queries the model for an answer. Running the identical experiment with the improved template gives the right-hand column in Table 3.8. Fig. 3.17 depicts the three learning curves. The improvement is notable; its zero-shot performance is the best of the three templates. However, with a small number of tuning examples, accuracy declines due to catastrophic forgetting before beginning to increase again with further tuning. After 256 samples, the model correctly answered ~69% of prompts, a significant improvement over the other templates.

We then go on to test a thoughtfully designed set of 10 candidate templates encompassing both cloze and prefix styles, repeating the exercise described above (see

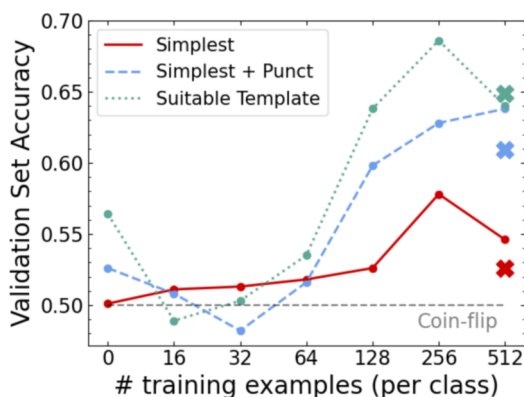
the full tutorial for more details). The results reveal a few interesting features about templates. First, there is some coherence of behavior within each of the categories.

- The prefix prompts have some success in zero-shot mode, degrade due to catastrophic forgetting with a small number of tuning samples, and then improve greatly in predictive power.
- The cloze prompts do somewhat worse in the zero-shot mode and degrade somewhat with a small number of training samples, but after that, perform better, eventually reaching parity with the prefix prompts.
- Prompts that provide less context are notably worse. They do a little better than random in zero-shot and only do a few percentage points better after the full suite of training examples. However, it is noteworthy that each prompt does better than random after the full train – the model does encode the answers to some of these questions.
- There is a significant scatter in overall performance within each category, which tends to increase with greater training data. This suggests that minute differences in template structure can have meaningful consequences.

For the final experiment, we instantiated two soft prompts, one with a simple template and one with a well-engineered template. In each case, we fine-tune with 128 SuperGLUE BoolQ samples for several epochs. For this test, the t5-base LLM is frozen, so only the prompt is tuned. We show these two models' changing validation set performances in the left panel of Fig. 3.18.

- The red dashed line shows the featureless prompt, which fails to improve despite 60 epochs of fine-tuning. Given the sparsity of this template, the features that could be fine-tuned are simply lacking, so no fine-tuning improves the performance.
- The black line shows the second prompt. Here, we do see significant improvement with additional fine-tuning, with the accuracy increasing by approximately 4.5% over 60 epochs. The template has arrived at a better state than our input template due to soft-prompt tuning.

Fig. 3.17 The change in *SuperGLUE BoolQ* validation set accuracy for a model trained with three different prompts. Template shape impacts both zero-shot and few-shot performance in solving the question/answer task.



Notably, the performance of the engineered soft-prompt model after 60 epochs out-performs the zero-shot performance of any model in the manual template experiments, and indeed out-performs any of the models before they have been fine-tuned with 128 training samples. This is achieved with a smaller investment of computing power, given the much smaller size of the prompt compared to the LLM itself. Thus, for a situation with a limited number of data points, prompt tuning may be preferable to LLM tuning when considering the computation expenses of training.

Next, we explore four different tuning approaches:

1. Soft prompt tuning only
2. LLM tuning only
3. Soft prompt tuning, followed by LLM tuning
4. Simultaneous soft prompt and LLM tuning

The results are shown in the right hand panel of Fig. 3.18. A few observations:

- First, for solving the BoolQ dataset, it is clear that LLM tuning is advantageous over pure soft prompt tuning. The prompt-only model shows significant improvement with additional tuning, but at 512 samples, it is well below the performance of all three models, which allowed the LLM variables to vary. This performance gap should narrow with longer training times, as shown by [Lester et al. \(2021\)](#), who achieved a performance score around 0.9 with well-optimized training and 30,000 training steps.
- Of the three other models, the one initialized with a tuned soft prompt shows the best zero-shot performance, which is unsurprising given that it has already been exposed to the training data. After that, each model allowing for a nonstatic LLM shows similar improvement rates with additional training samples. At each training step, there is a small preference for the model where both the LLM and prompt are simultaneously tuned over the model, tuning only the LLM. This is likely due to the larger number of parameters being tuned in the prompt+LLM model (248 million vs. 222 million), representing the highest-performing model in this tutorial. However, the gain over a well-engineered prompt+LLM tuning is fairly small.

3.7.4 Conclusion

We have shown the vital importance of prompt engineering in optimizing LLM performance. To be sure, many additional parameters must be fine-tuned to achieve peak performance that we have not focused on, including the size of the training set, the number of training epochs, learning rates, choice of LLM, and more. Nonetheless, from our weakest performing to best performing model, we have shown an improvement over 25% in prediction accuracy solely from template engineering. Thus, great attention must be paid to this component of any prompting model.

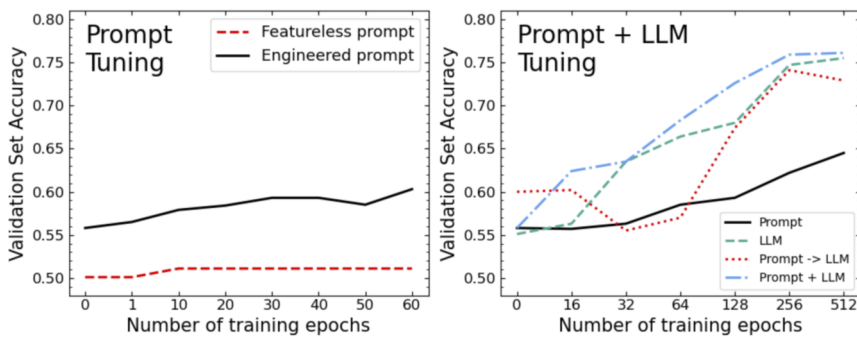


Fig. 3.18: *Left*: Results of soft prompt tuning starting with a naive prompt and an engineered prompt. *Right*: Learning curves for the four modes of learning given in the key. The model that allowed simultaneous prompt and LLM tuning performed the best at all stages of the training process.

References

- Eyal Ben-David, Nadav Oved, and Roi Reichart. Pada: Example-based prompt learning for on-the-fly adaptation to unseen domains, 2022.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Leyang Cui, Yu Wu, Jian Liu, Sen Yang, and Yue Zhang. Template-based named entity recognition using bart. *arXiv preprint arXiv:2106.01760*, 2021.
- Ning Ding, Shengding Hu, Weilin Zhao, Yulin Chen, Zhiyuan Liu, Hai-Tao Zheng, and Maosong Sun. Openprompt: An open-source framework for prompt-learning, 2021.
- Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. Hotflip: White-box adversarial examples for text classification, 2018.
- Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3816–3830, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.295. URL <https://aclanthology.org/2021.acl-long.295>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.
- Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3651–3657, Florence, Italy, July

2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1356. URL <https://aclanthology.org/P19-1356>.
- Zhengbao Jiang, Frank F Xu, Jun Araki, and Graham Neubig. How can we know what language models know? *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.
- Uday Kamath, John Liu, and James Whitaker. Transfer learning: Domain adaptation. *Deep learning for NLP and speech recognition*, pages 495–535, 2019.
- Sotiris B Kotsiantis, Ioannis Zaharakis, P Pintelas, et al. Supervised machine learning: A review of classification techniques. *Emerging artificial intelligence applications in computer engineering*, 160(1):3–24, 2007.
- Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1378–1387, New York, New York, USA, 20–22 Jun 2016. PMLR. URL <https://proceedings.mlr.press/v48/kumar16.html>.
- L. Lam and S. Y. Suen. Application of majority voting to pattern recognition: An analysis of its behavior and performance. *Trans. Sys. Man Cyber. Part A*, 27(5): 553–568, sep 1997. ISSN 1083-4427. doi: 10.1109/3468.618255. URL <https://doi.org/10.1109/3468.618255>.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2020.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.
- Xiaoya Li, Jingrong Feng, Yuxian Meng, Qinghong Han, Fei Wu, and Jiwei Li. A unified mrc framework for named entity recognition. *arXiv preprint arXiv:1910.11476*, 2019.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.
- Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. Gpt understands, too, 2021.
- Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity, 2022.
- Jonathan Mallinson, Rico Sennrich, and Mirella Lapata. Paraphrasing revisited with neural machine translation. In *Proceedings of the 15th Conference of the Euro-*

- pean Chapter of the Association for Computational Linguistics: Volume 1, Long Papers, pages 881–893, Valencia, Spain, April 2017. Association for Computational Linguistics. URL <https://aclanthology.org/E17-1083>.
- Franz Josef Och, Daniel Gildea, Sanjeev Khudanpur, Anoop Sarkar, Kenji Yamada, Alexander Fraser, Shankar Kumar, Libin Shen, David A Smith, Katherine Eng, et al. A smorgasbord of features for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*, pages 161–168, 2004.
- Matthew E Peters, Sebastian Ruder, and Noah A Smith. To tune or not to tune? adapting pretrained representations to diverse tasks. *arXiv preprint arXiv:1903.05987*, 2019.
- Fabio Petroni, Tim Rocktäschel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, Alexander H Miller, and Sebastian Riedel. Language models as knowledge bases? *arXiv preprint arXiv:1909.01066*, 2019.
- Guanghui Qin and Jason Eisner. Learning how to ask: Querying LMs with mixtures of soft prompts. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5203–5212, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.410. URL <https://aclanthology.org/2021.naacl-main.410>.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2020.
- Timo Schick and Hinrich Schütze. Exploiting cloze questions for few shot text classification and natural language inference. *arXiv preprint arXiv:2001.07676*, 2020a.
- Timo Schick and Hinrich Schütze. It’s not just size that matters: Small language models are also few-shot learners. *arXiv preprint arXiv:2009.07118*, 2020b.
- Timo Schick and Hinrich Schütze. Few-shot text generation with pattern-exploiting training, 2021.
- Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, and Sameer Singh. AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4222–4235, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.346. URL <https://aclanthology.org/2020.emnlp-main.346>.
- Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. Universal adversarial triggers for attacking and analyzing NLP. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Process-*

- ing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2153–2162, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1221. URL <https://aclanthology.org/D19-1221>.
- Caiming Xiong, Stephen Merity, and Richard Socher. Dynamic memory networks for visual and textual question answering. In *International conference on machine learning*, pages 2397–2406. PMLR, 2016.
- Wenpeng Yin, Jamaal Hay, and Dan Roth. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. *arXiv preprint arXiv:1909.00161*, 2019.
- Yue Zhang and Joakim Nivre. Transition-based dependency parsing with rich non-local features. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, pages 188–193, 2011.
- Zexuan Zhong, Dan Friedman, and Danqi Chen. Factual probing is [mask]: Learning vs. learning to recall, 2021.



Chapter 4

LLM Adaptation and Utilization

Abstract This chapter considers a number of approaches for effectively adapting and utilizing LLMs for the user's purpose. We begin by discussing the instruction tuning technique for LLMs, a critical step toward tailoring these powerful tools to specific tasks and domains. Central to our discussion is parameter-efficient fine-tuning, a technique that optimizes the model's performance with minimal adjustments to its parameters, thus conserving computational resources while maintaining high accuracy. We extend this conversation to cover various strategies that enhance compute-resource efficiency during fine-tuning. We offer readers insights into methods that reduce the computational burden without sacrificing the model's effectiveness through quantization. From this perspective of LLM adaptation, we pivot to a view of LLM utilization, wherein we explore some of the most important concepts around end-user prompting of LLMs, and we also provide practical guidance for prompt engineering. We conclude with a comprehensive tutorial on fine-tuning LLMs in resource-constrained settings.

4.1 Introduction

As we learned in Chapter 2, large language models can be pre-trained in many different ways considering the neural architecture used, the data used, and the learning objective or tasks they are pre-trained on. Regardless of this pre-training variation, these models generally learn beneficial patterns in the language they are exposed to. Owing to the unprecedented scale of pre-training, these learned patterns often enable LLMs in zero-shot language task competencies that often outcompete smaller models that have been explicitly fine-tuned for such tasks (Brown et al., 2020).

However, much research has shown that LLMs are capable of even greater domain/task competency when further adapted or fine-tuned in said domains/tasks. Such performance responses should not be surprising, considering the often generic or, at best, uniform representation of domains in the pre-training corpus. In light of

these observations, this chapter aims to provide readers with a solid understanding of the various techniques and concepts associated with efficient LLM adaptation and utilization.

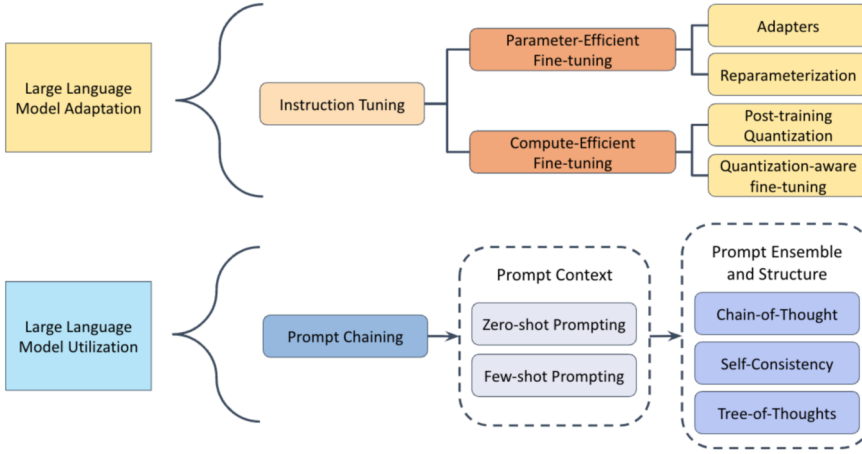


Fig. 4.1: Taxonomy of concepts introduced in this chapter, which focuses on the efficient adaptation and utilization of LLMs.

To do this, we survey the research literature for the most illuminating or practically promising tools, techniques, and procedures. Fig. 4.1 summarizes the scope of these within the chapter. In the context of LLM adaptation, we surmise that the majority of readers will be budget-constrained in regard to fine-tuning LLMs, and in light of this assumption, we prioritize the coverage of parameter-efficient and resource-efficient fine-tuning methods over more parameter- or resource-intensive tuning techniques such as adaptive pre-training. We have also dedicated a full chapter (Chapter 3) to prompt-based learning methods, so we do not address them in this chapter. In the context of LLM utilization, we highlight the most fundamental end-user prompt engineering concepts, including *prompt chaining* and *chain-of-thought* prompting. However, before diving into LLM adaptation through fine-tuning, we first introduce the reader to the core concepts within *instruction tuning*, the workhorse of fine-tuning LLMs.

4.2 Instruction Tuning

In Sect. 3.3.5, we introduced the concept of prompt-based fine-tuning of LLMs. The basic procedure is to use a labeled dataset of question/answer pairs representing a given task, develop a prompting template that converts the questions into natural language instructions, and fine-tune the model with these instructions to predict the

manually curated answers. This methodology is very good at accomplishing targeted tasks. However, SOTA chatbots such as ChatGPT and Claude 2 are not only performant against a single problem they were trained to solve. They are highly extensible, performing well on various subjects outside their fine-tuning objectives. How can models be tuned to perform accurate zero-shot tasks that differ from their training data? The answer is *instruction tuning* (IT).

4.2.1 Instruction Tuning Procedure

At the simplest level, IT is the fine-tuning of LMs with prompts formatted as natural language instructions for the model. These prompts usually contain an instruction portion describing a task to complete, the context needed to complete the task, and a prompt for an answer. Consider the example given in Sect. 3.3.5:

“Cannot watch this movie. This is [z].”

In prompt-based fine-tuning, we collect many examples for the film review sentence, and tune the model on prompts completed with “great” or “terrible”, corresponding to a positive and negative class. An alternative way to prompt the model for classification is to create a template with explicit natural language instructions for the model to follow, instead of the implicit directions of this cloze-style prompt:

```
‘## Instruction: Please read the following film review and
determine if it is a positive or negative review. Respond
‘positive’ or ‘negative’, according to your classification.
## Review: Cannot watch this movie.
## Response: ’’
```

The more explicit, natural-language-style instruction of this approach lends itself more naturally to the style of communication humans are used to. By fine-tuning with many examples of instruction-based templates, the LLM is trained not just to fill-in-the-blank for a movie review, but to learn how to follow natural language instructions, and generate responses, in a way that is desirable to humans.

A remarkable property of this approach is how well instruction-following extends to new tasks that were not explicitly trained during fine-tuning. In IT, an LLM is fine-tuned with questions and target answers rendered in human language and spanning a wide array of natural language tasks. These may include reading comprehension, sentiment analysis, translation, commonsense reasoning, and more. They also cover both discrete tasks with unique answers (i.e. “What is the capital of Canada?”), and open-ended tasks with no single correct answer (i.e. “Write a poem about pirates:”), in which case multiple demonstrations can be provided for the same prompt. These training sets are transformed into natural language instructions using instruction templates, and the model is tuned to predict the answer paired with each training

input. Tuning occurs on a token-by-token basis – starting with the full prompt, the model is tuned through backpropagation to predict the first token of the response. Then, with the prompt and the first token, it is tuned to predict the second token, continuing on in this way. Numerous studies (Wei et al. 2021; Ouyang et al. 2022; see Zhang et al. 2023c for a comprehensive overview) have showed that instruction-tuned LLMs show improved performance on NLP tasks not exposed to the model during fine-tuning, allowing for impressive levels of generalization.

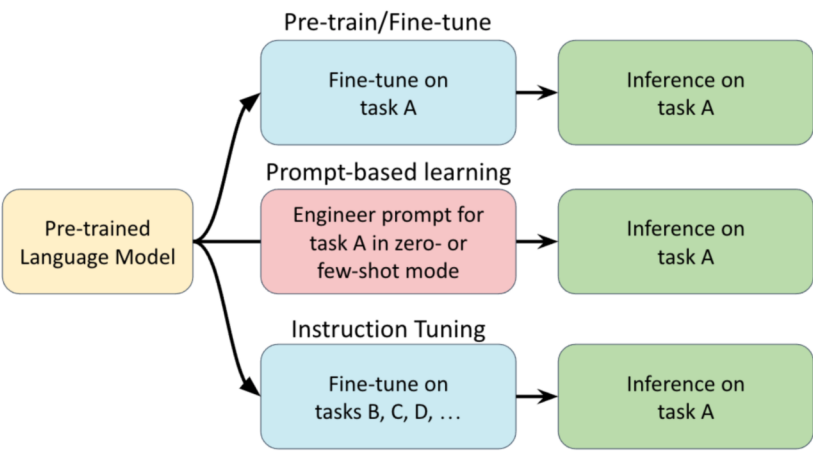


Fig. 4.2: A comparison of the high-level workflows for the pre-train/fine-tune approach, prompt-based inference, and instruction tuning. In PTFT, a user fine-tunes on a single task and then performs inference on that task. In prompt-based learning, a prompt is engineered, potentially with in-context examples, and the model performs inference with the prompt. In instruction tuning, a model is fine-tuned on many different tasks with many different datasets, generalizing its capabilities to new tasks unseen by training.

IT is closely related to the PTFT procedures discussed in Sect. 3.1.2 and prompt-based learning and inference detailed in Chapter 3.2.2. The relationships are illustrated in Fig. 4.2.

PTFT uses supervised fine-tuning with an LLM to accomplish a specific machine learning task; this approach encompasses both head-based fine-tuning and prompt-based fine-tuning. In contrast, instruction tuning uses supervised fine-tuning to train an LLM by *templatizing input/output pairs from a wide variety of natural language inference tasks using a consistent template*. Through this standardization of inputs into natural language prompts, the model learns to perform the tasks and correctly interpret the meaning behind instructions written by a human. This procedure is critical for adjusting an autoregressive

LLM from its objective purpose – predicting the most likely next token in a string based on the data it was trained on – to the purpose desired by humans – giving useful and accurate responses to instructions.

In this section, we discuss the approaches researchers have taken to collect these instruction tuning datasets, and demonstrate examples of instruction tuning for domain adaptation from the literature.

4.2.2 Instruction Tuning Data

IT training sets consist of question/answer pairs, expressed as natural language instructions and desired responses, across various NLP tasks. Researchers have taken several approaches to assembling these datasets. The primary variants are:

1. Transforming existing NLP training datasets into IT data with prompt templates.
2. Collecting human-generated prompts and generating answers by hand.
3. Collecting human-generated prompts and generating answers using existing LLMs.

Frequently, some mixture of these three approaches is employed to create variety and expand the size of the training set. In this section, we briefly describe each of these approaches and demonstrate how they work. A visual summary is shown in Fig. 4.3.

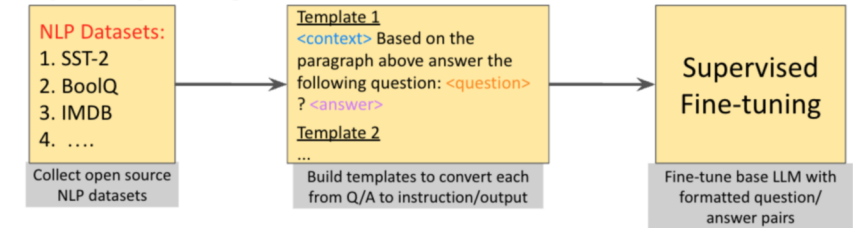
4.2.2.1 IT with Templatized Datasets

An enormous quantity of NLP datasets that can be leveraged for IT are available on sources such as Hugging Face Hub and Tensorflow Datasets. These datasets are very diverse in structure and purpose but generally have one or more inputs (e.g., question, context, instructions) and one or more potential target outputs (possibly ranked by preference). Some are definitive yes or no questions (positive/negative sentiment), some involve extracting information from a contextualizing paragraph (open-book QA), and some are more open-ended without a single correct answer (summarization, translation). The wide variety of tasks and topics provides expansive coverage of relevant NLP tasks and related domain knowledge.

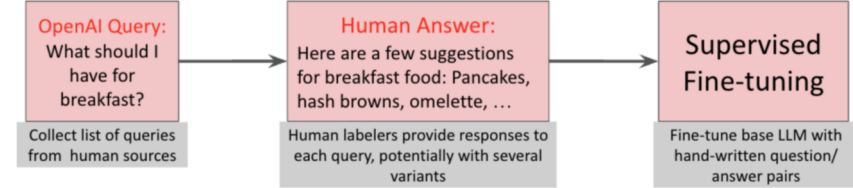
To leverage these data for IT, templates are created for each dataset to transform them from their native structure into natural language instructions and a target answer. For example, consider the context/question/answer triple in Listing 4.1, from the GLUE BoolQ dataset:

```
passage: "Look What You Made Me Do" is a song recorded by
        American singer-songwriter Taylor Swift, released on August
        24, 2017 by Big Machine Records as the lead single from her
        sixth studio album Reputation (2017). Swift wrote the
```

Templatizing existing datasets



Human-generated samples



LLM-generated samples

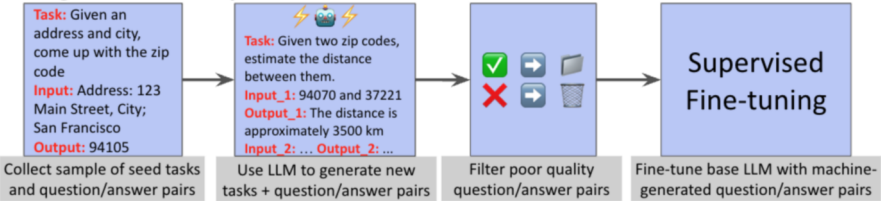


Fig. 4.3: Three different approaches to creating IT datasets: 1) Collect various open-source datasets from different inference tasks, format them into a consistent template framework, and fine-tune them; 2) Collect a large number of instructions, perhaps from queries sent to the OpenAI API, and have humans write responses; 3) Create a network of LLMs that can generate and respond to queries, building up a large IT dataset.

```
song with her producer Jack Antonoff....

question: "did taylor swift write look what you made me do"

label: 1 (yes)
```

Listing 4.1: *GLUE BoolQ* example

Each entry of this dataset contains a context paragraph, a question about the paragraph, and a yes or no answer. We can template this according to Template 1 in the left-hand column of Fig. 4.3:

```
"Look What You Made Me Do" is a song recorded by American singer-
songwriter Taylor Swift, released on August 24, 2017 by Big
Machine Records as the lead single from her sixth studio
album Reputation (2017). Swift wrote the song with her
producer Jack Antonoff... .
```

```
Based on the paragraph above answer the following question: did  
taylor swift write "look what you made me do ?"  
yes
```

Listing 4.2: Question and Answer Example

During training, the question is passed to the model as initial conditions, and the model is fine-tuned to respond “yes” correctly. Note that there is value in using several different prompt templates for each dataset. This prevents possible overfitting on the specific wording of a single template and helps to even out potential weaknesses of any individual choice.

A prominent example of an IT model relying primarily on formatting existing data is FLAN (Wei et al., 2021), an IT adaptation of the LaMDA LLM (Thoppilan et al., 2022). These authors collected 62 labeled NLP training datasets from open-source databases and grouped them into 12 categories related to the task. Most were natural language understanding tasks such as reading comprehension and sentiment analysis, but a few were generation tasks such as machine translations and summarization. They designed several templates for each dataset. They then tested the generalization capabilities of IT models by holding out specific task clusters and tuning on the remaining 11 clusters, scoring the model based on the performance of the holdout task. They demonstrated substantial performance improvement compared with the non-fine-tuned LaMDA model, especially in translation and closed-book QA tasks. FLAN also outperformed significantly larger non-IT LLMs such as GPT-3, showing that IT is an essential procedure for maximizing performance regardless of parameter count.

4.2.2.2 IT with Human-Generated Samples

Another approach is to craft or assemble a series of instruction tasks and have humans write answers for the model to train on. We refer to these as human-generated samples. This approach is beneficial for collecting data for natural language generation tasks for which there is no specific correct answer but where certain outputs are preferred over others, such as poetry writing. Training on handwritten texts helps attune the model to more human-like speech patterns when answering questions. The significant upside of human-generated samples is that the model architect has finer control over the question topics and details of the answers. The obvious downside is that human labeling is slow and costly, so the size of boutique human-generated datasets tends to be smaller than the aggregation of existing sets.

A significant model that leverages human-generated samples is InstructGPT (Ouyang et al., 2022). These authors collected queries passed to the OpenAI API and added some hand-written questions to construct a set of inputs. They then hired a team of labelers to write answers to these queries, completing their dataset. The details of their training methods align closely with those of reinforcement learning with human feedback, which is the subject of Chapter 5. Interestingly, after training, the labelers tended to prefer the outputs of InstructGPT over those of FLAN, produc-

ing an approximately 73% win-rate over the baseline for InstructGPT compared to $\sim 30\%$ for FLAN. Although this is partly a consequence of the training technique, it also reflects that humans created the fine-tuning outputs—the model built with hand-crafted answers was more closely aligned to human preferences than a model created by templating a heterogeneous collection of datasets.

4.2.2.3 IT with LLM-Generated Samples

The final approach to IT that we will discuss is to construct a dataset by generating answers to queries with an LLM. This can be done using the same model you are training or with another model with properties you wish to emulate with your fine-tuning. Typically, a user generates several answers from the model and applies a quality filter (programmatic or manual) to select the best responses. Compared to human-generated answers, this approach is much faster and less expensive, but the capabilities of the LLM limit the domain of answers.

There is also an obvious data contamination issue when answers are fed into the very model that produced them in the first place. Can this impart new information? It is reasonable to wonder if a model can learn anything by being fine-tuned on its generated responses. A plausible framing is that the model can be fine-tuned to output answers more reliably by generating many answers to a single question and picking the one or two that best reflect human judgment as to what constitutes a good answer to the prompt. Thus, even if you are not exposing it to new information, as in the case of human-generated tuning, it will learn desirable tendencies.

To empirically test a similar idea [Wang et al. \(2023b\)](#) developed Self-Instruct, an iterative bootstrapping framework that takes in a sample of seed tasks along with sample inputs/outputs for each and uses GPT-3 to generate additional queries inspired by the seed inputs. These queries are then passed into GPT-3 and paired with the output to build an IT dataset. The authors apply several quality filters to ensure the accuracy of the diversity of responses. Ultimately, they show that their models significantly outperform vanilla GPT-3 in natural language generation tasks and outperform an instruction-tuned variant of the smaller T5 LLM.

4.2.3 Instruction Tuning for Domain Adaptation

While this section has mostly concerned inference on hidden tasks, instruction tuning is also a popular approach for adapting LLMs to specific domains. Domain-adapted IT models have been shown to outperform generalized chatbots for highly-specific tasks requiring knowledge of technical jargon or information outside of the model's pre-training data. In this section, we overview examples in the education, medicine, and financial domains, with the understanding that this is just the tip of the iceberg for applications of IT.

In the education domain, [Zhang et al. \(2023d\)](#) released *Writing-Alpaca* for writing tasks. This model was tuned to make suggestions for improvement to writing, including correcting grammar, improving clarity, simplifying a confusing sentence, or paraphrasing text. Tuning involved simple, one sentence instructions for each of these tasks, an input sentence to correct, and the corrected output sentence. For example:

```
###Instruction:
Fix grammatical errors in the text
###Input:
She went to the markt
###Response:
She went to the market
```

The model is thus taught to catch and correct spelling mistakes among other grammatical errors. [Zhang et al.](#) tuned the Llama-7B model with roughly 60,000 text-improvement examples representing seven different copy-editing tasks, and an additional 52,000 general instruction prompts taken from the *Stanford-Alpaca* project ([Taori et al., 2023](#)). This work significantly improved over the foundation model baseline and models tuned with less task-specific datasets, though did not quite rival the performance of PTFT models trained on vastly larger (millions) text editing datasets. This shows that while IT can be brought up to high quality with a relatively modest dataset, peak performance will sometimes require larger datasets.

In the domain of medicine, [Li et al. \(2023b\)](#) introduced *ChatDoctor*, another instruction-tuned version of the Llama-7B LLM. These authors used real conversations between doctors and patients over text chat channels as the basis for a model that can recommend patient actions based on their requests. For their project, they tuned a model using instruction prompts for three sequential tasks:

1. They used an instruction prompt to teach the model to extract keywords from a patient's request.
2. They used an instruction prompt to consider context sourced from internal databases and Wikipedia related to those keywords, and pull out information relevant to the patient's question.
3. They used an instruction prompt to consider the extracted context, and suggest a course of action to the patient.

The training data were extracted programmatically from the back-and-forth conversations of the patients and doctors, and used to construct approximately 100,000 sequential examples. This instruction-tuned model significantly improved performance in understanding patients' symptoms and providing relevant advice on those symptoms, and demonstrates the power of chaining together instructions to accomplish more sophisticated tasks than can be accomplished with a single prompt. Other IT LLMs in medicine have targeted even more specific disciplines, such as radiology and oncology.

In finance, instruction-tuned versions of LLMs have been proposed for various tasks. As one example, *Instruct-FinGPT* was developed by [Zhang et al. \(2023a\)](#) to automatically classify financial headlines by sentiment: positive, neutral, or negative.

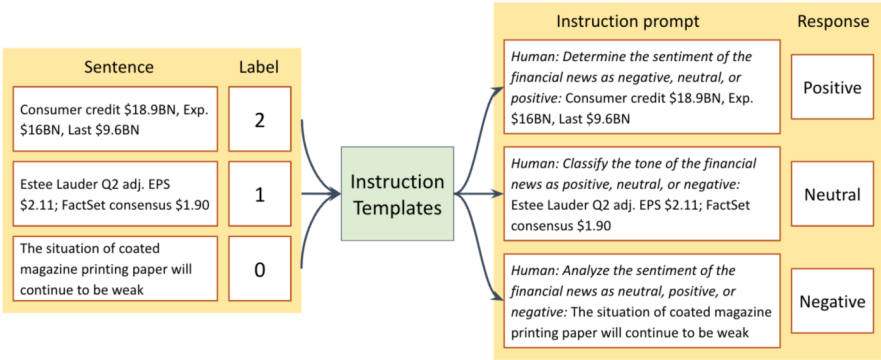


Fig. 4.4: Examples of templatized financial data used to tune Instruct FinGPT (Zhang et al., 2023a).

They took literature sample of sentiment-tagged financial headlines, formulated ten distinct sentiment classification instructions, and generated prompts filled with the annotated answers. Examples of this templatzation can be seen in Fig. 4.4. It is clear from these examples why domain-specific adaptation is beneficial to this task: many financial headlines use specialized language and syntax which may require additional emphasis on a model for superior performance. Using these data, they tuned Llama-7B and demonstrated a greater than 20% increase in F1 score compared to both the baseline Llama model and SOTA chatbots like ChatGPT, underlining the value of domain-adaptation with IT.

Now that we have seen a few examples of how data can be structured and fed to LLMs to boost their performance and abilities in the fine-tuning setting, the next sections will explore various efficiency-based methods for achieving LLM fine-tuning.

4.3 Parameter-Efficient Fine-Tuning

While the scale of LLMs is critical for the emergence of some of their most valuable competencies, it also introduces several practical constraints. Challenges associated with the efficient transport of large models between environments and their storage are the most apparent (Ding et al., 2023). Indeed, these particular challenges are compounded when multiple copies of the model are required for each task, use case, or application. Such challenges apply even for smaller language models such as BERT, the first Transformer model for which adapters, which we will discuss at length below, were proposed by the Google Research team in Houlsby et al. (2019).

In addition to deployment challenges, LLM scale often introduces prohibitive time and cost if fine-tuning strategies targeting the LLM’s full parameter set are pursued. Such practical constraints necessarily limit researchers’ and developers’ ability

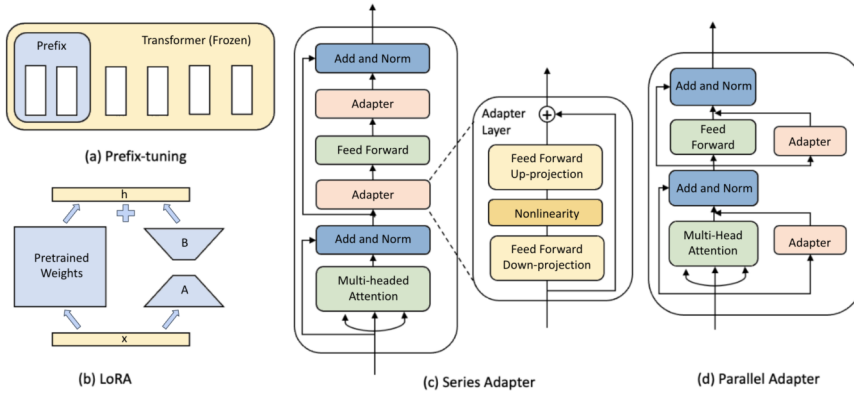


Fig. 4.5: Model architecture details for four parameter-efficient fine-tuning methods: (a) depicts prefix-tuning from the prompt-based fine-tuning category; (b) depicts LoRA from the reparameterization fine-tuning category; (c) depicts how an adapter is integrated into a Transformer in series with pre-existing network layers; (d) depicts how an adapter is integrated into a Transformer in parallel with pre-existing network layers.

to experiment and iteratively improve tuning strategies. As such, significant upfront effort is required to ensure that the small number of iterations one might have a budget available to conduct are as effective and error-free as possible since mistakes can be very costly.

Similarly, with the extremely impressive generalized capabilities that LLMs gain through pre-training, fine-tuning, by definition, aims to specialize their behaviors along one or more relatively narrow dimensions of competency. Fine-tuning that potentially modifies all of the LLM's parameters to achieve the target task(s) will necessarily modify some of those generalized capabilities, thereby rendering the fine-tuned LLM less generally applicable than the original pre-trained only version; thus, many versions and copies of LLMs may be required depending on the number of in-scope tasks.

Parameter-efficient fine-tuning, or *PEFT*, is a set of fine-tuning techniques that aim to overcome these challenges. In general, PEFT techniques aim to maximize task outcomes while minimizing the number of total tunable parameters required to do so. Often, the goal of retaining the beneficial generalized competencies the LLM gained through pre-training is part of the design considerations of PEFT systems, typically achieved by tuning adapters or augmentations of the LLM on a per-task basis, while the LLM's pre-trained parameters remain entirely intact (Houlsby et al., 2019).

Following Hu et al. (2023), in general, there are four categories of PEFT methods, as illustrated in Fig. 4.5. These are:

- **Prompt methods** relate to prompt-based learning approaches for fine-tuning LLMs. In general, the goal of these methods is to construct learnable prompt vectors, which can then be appended to either the input embeddings (*prompt tuning*; Lester et al. (2021)) or the hidden states of the PLM layers (*prefix tuning*; Li and Liang (2021)). Fig. 4.5 illustrates these methods.
- **Series adapter methods** involve integrating additional neural modules in series with the existing layers of the PLM. Examples within this category include that of Houlsby et al. (2019), where the adapter networks are integrated immediately after the feed-forward and attention layers of the Transformer. Fig. 4.5c illustrates these methods.
- **Parallel adapter methods**, similar to series adapters, leverage small additional neural modules, but rather than incorporating them in series with existing PLM sub-layer components, these modules are incorporated in parallel with the existing network's sub-layers. In this configuration, intermediate activations from the PLM are passed to these adapter modules through shortcut connections (e.g., Sung et al. (2022)). Fig. 4.5d illustrates these methods.
- **Reparameterization methods** are a set of approaches that exploit the concept of intrinsic dimensions to re-parameterize the pre-trained network according to a given fine-tuning task (Aghajanyan et al., 2020). Methods in this category include LoRA (Hu et al., 2021) and VeRA (Kopieczko et al., 2023). Fig. 4.5b illustrates these methods.

! Practical Tips

The primary tradeoff to consider with PEFT methods is computational requirements vs. analytic quality. Generally speaking, less computationally intensive approaches incur larger analytic quality hits. We can construct an approximate trend in this trade-off considering the PEFT categories listed above. In order of decreasing computational load, and thus decreasing analytic quality:

1. Full fine-tuning
2. Prompt-based tuning
3. Series adapters
4. Parallel adapters
5. Standard reparameterization
6. Optimized reparameterization

Note that standard reparameterization represents techniques like LoRA that use standard float precision and standard optimization algorithms (e.g. Adam; Kingma and Ba 2017), while optimized reparameterization represents techniques like QLoRA, within which float precision is compute optimized and parameter updates occur selectively thanks to optimization algorithms.

In the following sections, we will discuss the architectural and analytic benefits and costs associated with some of the more prominent PEFT approaches, includ-

ing reparameterization, series adapters, and parallel adapters. We will not discuss prompt-based learning PEFT approaches, as their fundamentals have already been covered in detail in Chapter 3. Readers are encouraged to explore Hugging Face’s curated view of PEFT methods¹ from a practical perspective, as well as the coded tutorial of this chapter, where we will demonstrate the comparative benefits and costs associated with a few of these methods.

4.3.1 Adapters

Fundamentally, adapters are small (relative to the number of parameters present in the LM) neural network modules introduced into the layers of the pre-trained model being adapted (Houlsby et al., 2019). The factors that make adapters attractive from a practical perspective are as follows:

- **Analytic performance:** Adapters can attain close to full fine-tuning performance on many tasks despite tuning many times fewer parameters.
- **Modular task tuning:** Since adapters are task-specific modules that are incorporated into the layers of a Transformer model, they can be developed sequentially (i.e., adapting a language model to multiple tasks can be done on a per-task basis, rather than requiring training on data representing all tasks in parallel. This pattern allows developers to focus on optimizing the specific target outcomes per task rather than relying on a joint measure across all tasks. This property effectively ameliorates the catastrophic forgetting challenge associated with full or partial fine-tuning of the original PLM on multiple tasks (Pfeiffer et al., 2021).
- **Scalable Deployment:** Adapters typically have a fraction of the parameters that the target language model has. As such, task-specific adapter modules can be readily deployed on standard computing infrastructure.

Contrasting adapters with traditional fine-tuning and feature-based transfer tuning techniques will help us understand key innovations. Consider a neural network with parameters $\mathbf{w} : \phi_w(\mathbf{x})$. For traditional fine-tuning, the original parameters $\mathbf{w}$ are adjusted for each task, which limits compactness since new copies of $\mathbf{w}$ are necessary for each task. Conversely, for feature-based transfer, the model function is reformulated using a new function X_v to give $x_v(\phi_w(\mathbf{x}))$, wherein only the new task-specific parameters v are tuned. This approach provides good compactness properties, since the same original model parameters $\mathbf{w}$ remain unchanged.

Despite being much less computationally intensive than full-parameter fine-tuning, adapters have been shown deliver performance that is on par or better than fine-tuning, owing to innovative approaches in how task-specific parameters $\mathbf{x}_v$ are composed with the original model parameters $\mathbf{w}$. This is done by initially setting the new task-specific parameters $\mathbf{v}_0$ so that the new model function is as close to the original as possible, $\psi_{\mathbf{w}, \mathbf{v}_0}(\mathbf{x}) \approx \phi_{\mathbf{w}}(\mathbf{x})$, and only tuning $\mathbf{v}$ at training time (Houlsby et al., 2019).

¹ <https://github.com/huggingface/peft>

It is usually the case when fine-tuning LLMs that $|\mathbf{v}| \ll |\mathbf{w}|$; in other words, the number of tuned parameters in the adapters is a tiny fraction of the number of parameters in the original LLM. For the adapter architecture proposed in [Houlsby et al. \(2019\)](#), the number of trainable parameters can be calculated as $2md + d + m$, where d is the original dimensionality of features from the Transformer layer feed-forward projection, while m is the bottleneck dimensionality chosen for the adapter layer. By selecting a small m , the additional parameters required for task fine-tuning can be kept low. Indeed, in practice, [Houlsby et al. \(2019\)](#) reported successful fine-tuning outcomes even when using 0.5% of the parameters of the original pre-trained model.

4.3.1.1 Series Adapters

Series adapters are the style of adapters that are integrated in series with the pre-existing layers of the pre-trained network. This type of PEFT method results in the following reformulation:

$$H_o \leftarrow H_o + f(H_o W_{down}) W_{up} \quad (4.1)$$

Here, H_o is the output of a given network layer. When series adapters are installed, this output is down-projected to a lower dimension with $W_{down} \in \mathbb{R}^{d \times r}$, where r is the bottleneck size defined for the adapter, and is usually small. A nonlinear function f is applied to the down-projection, and then the output is up-projected back to the original dimensionality of H_o with $W_{up} \in \mathbb{R}^{d \times r}$. These three features, W_{down} , f , and W_{up} , constitute the series adapter and are fine-tuned during adapter tuning.

Fig. 4.6 depicts the placement of the adapters immediately after the feed-forward layer that is itself preceded by the multi-head attention layer, and the two feed-forward layers preceding the Transformer output normalization. [Hu et al. \(2023\)](#) demonstrate that this may not always be the best placement for certain tasks. Indeed, an evaluation of the analytic impacts of adapter placement ([Hu et al., 2023](#)) reveals that placing the adapter modules only after the feed-forward layers results in improved performance on mathematical reasoning when compared to placement after the multi-head attention layer and placement after both the multi-head attention and feed-forward layers. This aligns with the more efficient adapter variant proposed in [Pfeiffer et al. \(2021\)](#).

In addition to adapter placement, the bottleneck size (r) used in the initial down-projection is also an extremely important hyperparameter in adapter design. In general, setting r too small is likely to limit the retention of valuable information between the input layers to the adapter and the bottleneck layer within the adapter. On the other hand, setting r too high, while potentially improving task performance, will diminish the parameter-efficiency of the fine-tuning itself, although [Hu et al. \(2023\)](#) find that setting r too high can also negatively impact analytic outcomes.

Serial adapters generally perform well in reducing computational consumption during fine-tuning. However, because they are essentially extra serial layers through

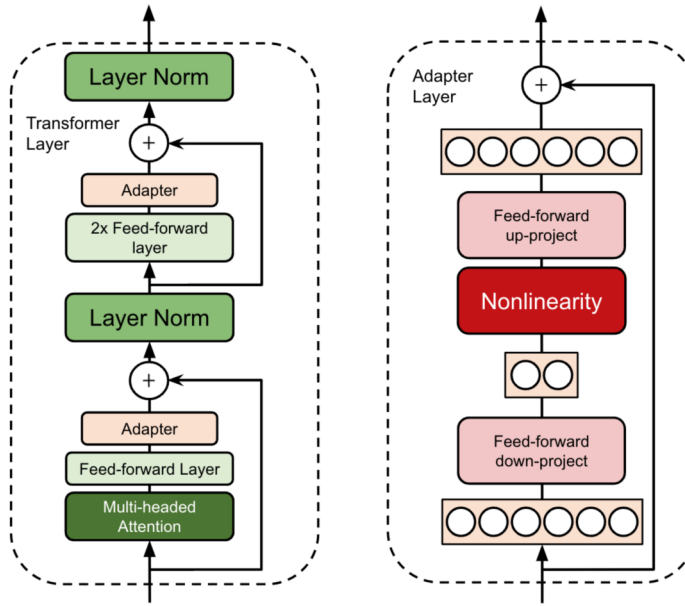


Fig. 4.6: Architectural view of the location of adapters within a Transformer layer. The adapters are integrated into two locations within the Transformer. The first is after the feed-forward projection immediately after the multi-head attention layer, while the second is after the two feed-forward layers. The key features of the adapter include the bottleneck architecture, which projects the input to the adapter layer into a smaller feature space on the way in, after which nonlinearity is applied before projection back into the original input dimensionality.

which inputs must be propagated to make predictions, they have been reported to incur nontrivial inference-time costs.

4.3.1.2 Parallel Adapters

The first parallel connection method for adapters was introduced to improve the performance degradation problem associated with multilingual machine translation (Zhu et al., 2021). Effectively, the goal in Zhu et al. (2021) was to leverage parallel adapters to close the performance gap between the then superior multiple bilingual machine translation models and a single multilingual machine translation model, which was successfully demonstrated for two out of the three multilingual machine translation benchmark datasets tested. The architecture and placement of parallel adapters from Zhu et al. (2021) are illustrated in Fig. 4.7.

Parallel adapters result in the following reformulation:

$$H_o \leftarrow H_o + f(H_i W_{down}) W_{up} \quad (4.2)$$

where H_i/H_o are the input/output of the specific layer and adapter.

Integrating adapters in parallel with the backbone network has one key advantage over serially integrated adapters in that training can be much less computationally intensive, not only because of the already significantly reduced tunable parameter-space but also because parameter updates typically occur without having to back-propagate through the PLM network to calculate gradients (Sung et al., 2022).

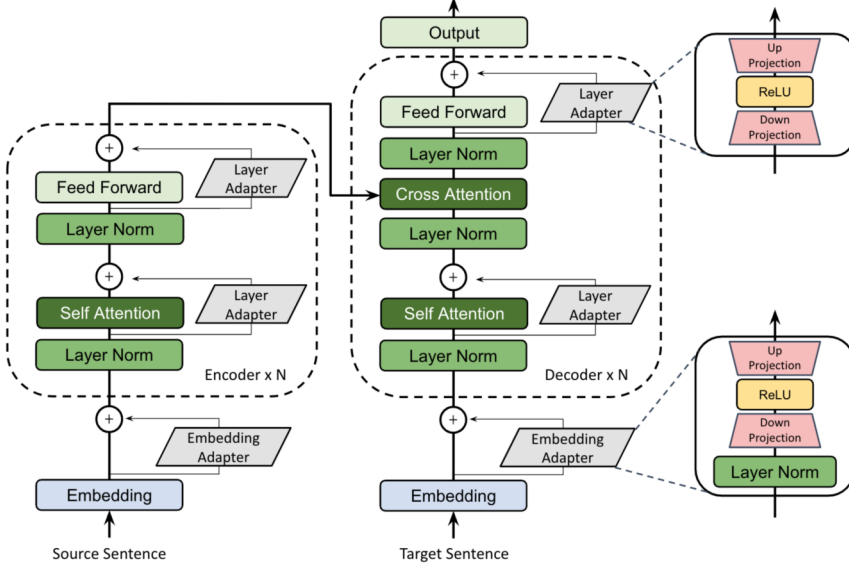


Fig. 4.7: Location and architecture of parallel adapters used to fine-tune multilingual machine translation performance. In this architecture, the non-adapter Transformers are pre-trained as a multilingual model. At the same time, layer adapters are fine-tuned on bilingual corpora to enhance machine translation performance for those language pairs.

4.3.2 Reparameterization

Reparameterization methods, unlike adapters, typically do not involve additional neural network modules, which improves upon the inference latency impacts from adapters (Hu et al., 2021). At their core, these methods take advantage of the fact that many NLP tasks have orders of magnitude lower intrinsic dimensions relative to the pre-trained model and therefore can be effectively fine-tuned for many tasks in a relatively parameter-efficient manner.

Fundamentally, reparameterization methods apply some rank decomposition followed by a learning phase, wherein low-rank representations of higher-dimensional representations from the pre-trained model are optimized. In the following sections, we will explore three representative reparameterization methods for parameter-efficient fine-tuning of LLMs, namely, *Low-Rank Adapters* (LoRA) (Hu et al., 2021), *Kronecker Adapters* (KronA) (Edalati et al., 2022) and *Vector-based Random Matrix Adaptation* (VeRA) (Kopiczko et al., 2023).

4.3.2.1 Low-Rank Adapters

Low-Rank Adapter fine-tuning involves learning low-rank matrices that approximate parameter updates according to whatever task the fine-tuning is happening on (He et al., 2022). He et al. (2022) report the following four key advantages of the LoRA method:

- A single pre-trained model can be shared across many NLP tasks for which task-specific LoRA modules have been learned. Switching tasks is achieved by swapping the learned low-rank matrices, which significantly reduces the storage and task-switching overhead.
- Since optimization occurs only on the injected low-rank matrices and not on the full parameter set of the PLM, the training computation and hardware requirements are reduced by up to 3x.
- The linear design of LoRA fine-tuning allows the learned low-rank matrices to be merged with the fixed weights of the PLM, thereby introducing no additional inference latency.
- Since LoRA aims to find lower-dimensional representations of fine-tuned NLP tasks, it is, by definition, orthogonal to other tuning methods that do not optimize rank. As such, LoRA can be combined with many of these other fine-tuning techniques.

How are these advantages achieved? LoRA aims to optimize a much smaller set of parameters Θ for each fine-tuned NLP task. Consider the following modeling objective that is optimized in full-parameter fine-tuning:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})) \quad (4.3)$$

where $\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$ is a set of N context-target pairs for a given NLP task. In the case of a summarization task, x_i is the full text to be summarized, while y_i is its summary. As such, during fine-tuning, Φ_0 is initialized with the pre-trained model's weights, which are updated to $\Phi_0 + \Delta\Phi$ by iteratively following the gradient to maximize Equation 4.3.

However, because the pre-trained model's weights are updated directly during full fine-tuning, as mentioned, scalable deployment can be prohibitive in practice. As such, Hu et al. (2021) proposed estimating the task-specific parameter updates

$\Delta\Phi$ with $\Delta\Phi = \Delta\Phi(\Theta)$, where $|\Theta| \ll |\Phi_0|$ thanks to the low intrinsic dimension of the NLP task relative to the pre-trained model. This means that $\Delta\Phi$ can now be estimated by maximizing Θ as follows:

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t})) \quad (4.4)$$

From an algorithmic perspective, [Hu et al. \(2021\)](#) targeted the dense layers of the Transformer architecture, wherein they hypothesized that the pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ updates could be constrained to a lower rank decomposition $W_0 + \Delta W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and r are much less than either k or d , which are the dimensions of the dense layer weight matrices that have full rank.

During training, only A and B have learnable parameters (i.e., W_0 is frozen); as such, for a given input x , the forward pass output is given as:

$$h = W_0 x + \Delta W x = W_0 x + BA x \quad (4.5)$$

A and B are initialized randomly (the original LoRA paper uses random Gaussian initialization) but constrained to fulfill $\Delta W = BA = 0$. After initialization, ΔW is approximated, as noted previously, by optimizing over Equation 4.4.

While LoRA can technically be applied to any dense layer weight matrix, [Hu et al. \(2021\)](#) limit their original application to the self-attention weights (the key and value weight matrices W_k and W_v). Despite this limited application of the technique, when applied to GPT-3 ([Brown et al., 2020](#)), LoRA either matched or exceeded full parameter fine-tuning performance on three standard benchmarks (GLUE, WikiSQL, and SAMSUM).

[Hu et al. \(2021\)](#) were also able to empirically demonstrate that adapting matrices from variable layer types using a lower rank (r) delivers a more efficient parameter/quality trade-off than adapting only a few different types of layer matrix types and a larger rank, showing that the fundamental assumptions of the intrinsic dimension framing of fine-tuning hold in an empirical setting ([Hu et al., 2021](#)).

The success of LoRA has led to the rapid emergence of several significant research and applied outcomes. Such notable works include that of LoRAHub ([Huang et al. \(2023\)](#)), which aims to optimize the interoperability of LoRA adapter modules in an applied setting. Similarly, as we will see in Sect. 4.4.1 of this chapter, the addition of quantization methods to the LoRA method is beginning to emerge as another interesting innovation in the ongoing effort to make the fine-tuning and deployment of LLMs increasingly realistic ([Dettmers et al., 2023](#)).

Generally, LoRA remains a popular approach for fine-tuning LLMs, because of its generalizability to many NLP tasks and the computational and data efficiency with which those tasks can be accomplished, even when using the largest LLMs. Therefore, the core idea behind LoRA, low-rank decomposition, has been further modified in various research efforts to improve both its parameter-efficiency and analytical quality. Such works include *AdaLoRA* ([Zhang et al., 2023b](#)), which aims to selectively update fine-tuning parameters based on an adaptive allocation of the

overall parameter budget for a given task based on a differential importance metric. Additionally, *QLoRA* (Dettmers et al., 2023) introduces floating point precision-based quantization on the PLM, for further computational efficiency during gradient backpropagation. More details are provided in Sect. 4.4.1 below).

As promising as these low-rank methods are, as we will see in the next section, LoRA’s use of rank decomposition can indeed be improved upon in specific settings where such low rank is insufficient to capture essential patterns necessary for some tasks. Specifically, we will look at a method with similar parameter efficiency to LoRA but without the low-rank assumptions of LoRA, namely, KronA (Edalati et al., 2022).

4.3.2.2 Kronecker Adapters

Kronecker adapters, which were originally proposed in Edalati et al. (2022), use Kronecker product decomposition to achieve parameter-efficient fine-tuning while avoiding the strong assumptions implied by the intrinsic dimension framing of NLP tasks. Other methods that use Kronecker products have been proposed previously Edalati et al. (2022), such as Compactor (Mahabadi et al., 2021), which leverages a mixture of rank-one matrices and Kronecker products to improve the parameter efficiency of fine-tuning. However, while achieving good analytic performance, such methods have lower training and inference-time computation efficiencies than KronA (Edalati et al., 2022). KronA improves on this noted deficiency of other re-factorization methods by optimizing the calculations involved (see Fig. 4.8). Typically, the Kronecker product of two matrices, $\mathbf{A}$ and $\mathbf{B}$, is given as:

$$\mathbf{W} = \mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \dots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \dots & a_{mn}\mathbf{B} \end{bmatrix} \quad (4.6)$$

where $\mathbf{W}$ is the resulting block matrix from the Kronecker product of $\mathbf{A}$ and $\mathbf{B}$, and (m, n) are the row and column dimensions of $\mathbf{A}$. However, rather than recovering $\mathbf{W}$ directly, Edalati et al. (2022) leverages a more efficient calculation:

$$(\mathbf{A} \otimes \mathbf{B})\mathbf{x} = \gamma(\mathbf{B}_{\eta b_2 \times a_2}(\mathbf{x})\mathbf{A}^T) \quad (4.7)$$

where $(\mathbf{A} \otimes \mathbf{B})\mathbf{x}$ is the Kronecker product of matrix $\mathbf{A}$ and $\mathbf{B}$ multiplied by input vector $\mathbf{x} \in \mathbb{R}^{d_h}$, where d_h is the input embedding dimension, $\mathbf{A}^T$ is transposition of matrix $\mathbf{A}$. $\eta m \times n(\mathbf{x})$ is an operation that converts a vector $\mathbf{x}$ and converts it to a matrix of dimension mn , while $\gamma(\mathbf{x})$ is an operation that converts a matrix into a vector by stacking its columns.

In the context of Fig. 4.8a, the y output for a given input $\mathbf{X}$ is given as:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} + s\mathbf{X}[\mathbf{A}_k \otimes \mathbf{B}_k] \quad (4.8)$$

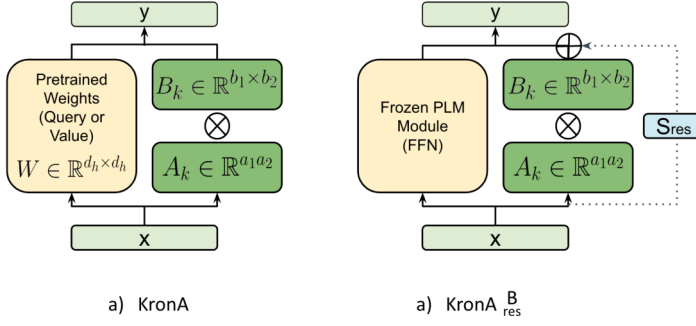


Fig. 4.8: Architectural illustration of the (a) KronA and (b) KronA^B_{res}. $\otimes$ represents the Kronecker product of matrix A and B. KronA^B_{res} contains the residual connection, S_{res} , which when removed reverts the fine-tuning adapter back to KronA^B.

where s is a scaling factor, and $\mathbf{W}$ are the frozen weights of the PLM. Therefore, the tuned weights for a given NLP task fine-tuned using KronA are given as:

$$W_{tuned} = W + s[\mathbf{A}_k \otimes \mathbf{B}_k] \quad (4.9)$$

Essentially, A_k and B_k replace the down- and up-projections of the LoRA architecture (see Fig. 4.5b), and similar to LoRA, they are merged with the weights of the LLM. This final weight merging operation and the freezing of the LLM weights, as is the case with LoRA, enable efficient fine-tuning without introducing additional inference latency.

Edalati et al. (2022) also proposed a parallel-adaptor blueprint for implementing KronA (referred to as *KronA^B*) in parallel to feed-forward network modules of a PLM, as well as the same architecture, but with the addition of a residual scale factor to further improve analytic performance. However, both of these architectures are less efficient from a computational perspective in terms of both fine-tuning time and inference time and will not be covered in any additional detail. Interested readers are encouraged to read Edalati et al. (2022) to understand these methods.

How does KronA perform analytically and computationally relative to other PEFT approaches? Edalati et al. (2022) report that when applied to T5 (Raffel et al., 2020), KronA on average outperforms full fine-tuning, Compactor (Mahabadi et al., 2021), BitFit (Zaken et al., 2022), LoRA (Hu et al., 2021), and the parallel adapter method presented in He et al. (2022), when evaluated on the GLUE benchmark. These results are impressive when considering that this analytic performance is achieved through fine-tuning, which reduces training time by 25% (vs. 28% for LoRA) and incurs no additional inference latency compared to full fine-tuning. Both KronA^B and KronA^B_{res} outperform KronA on this same benchmark.

4.3.2.3 Vector-based Random Matrix Adaptation

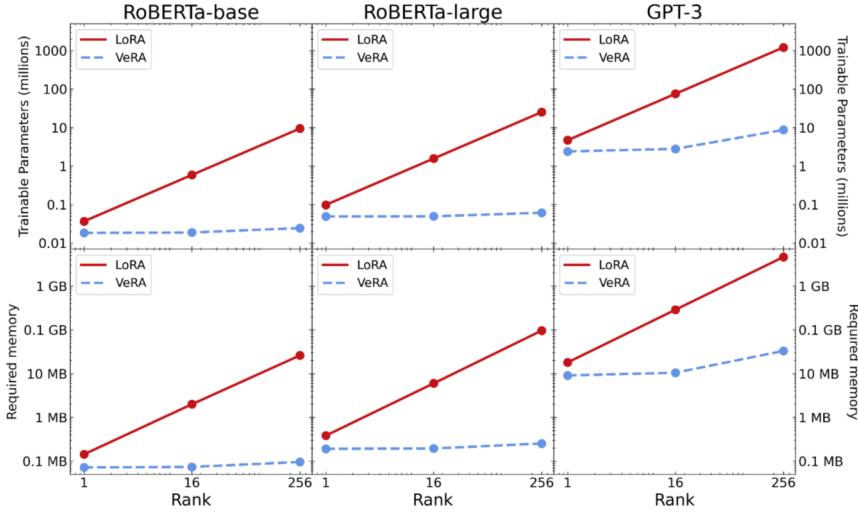


Fig. 4.9: Comparative theoretical memory required (in bytes) and number of trainable parameters for $Rank \in \{1, 16, 256\}$ for LoRA vs VeRA, calculated for three different LLMs (RoBERTa-base, RoBERTa-large, and GPT-3, assuming both LoRA and VeRA methods are applied only to the query and key layers of the Transformer self-attention head. VeRA has consistently lower memory and trainable parameter count than LoRA. Memory requirements in bytes and the number of trainable parameters are scaled to log base 10 for visualization purposes. Parameter calculations for VeRA follow: $|\Theta| = L_{tuned} \times (d_{model} + r)$. LoRA follows: $|\Theta| = 2 \times L_{tuned} \times d_{model} \times r$. In each of these equations, L_{tuned} , d_{model} , and r represent the number of layers being fine-tuned, the dimensions of those layers, and the rank of the adapter matrices, respectively.

Reparameterization methods like LoRA can reduce the number of trainable parameters by up to 10,000 times and the GPU memory requirements by up to 3x. However, there exist some use cases where not only task-specific adaptation of LLMs are required, but potentially user-specific adaptation across such tasks as well (e.g., personalized assistants, personalized recommendations, edge devices). [Kopiczko et al. \(2023\)](#) recognized that even the parameter-efficiency achieved by LoRA would still result in prohibitive storage and network overheads in a production runtime setting. This recognition, in combination with further inspiration from the work of [Aghajanyan et al. \(2020\)](#) on intrinsic dimensionality in NLP task fine-tuning, led to *Vector-based Random Matrix Adaptation* (VeRA) (Fig. 4.10). This method enables the further reduction of tunable parameters during fine-tuning by an additional

10x compared to LoRA (Fig. 4.9), thus further alleviating the significant operational challenges associated with applied use cases for increasingly large LMs.

Fundamentally, this efficiency gain is achieved by using a pair of randomly initialized (see below for initialization details) matrices, A and B as in LoRA (Fig. 4.5b), which are frozen and shared across all Transformer layers during fine-tuning. However, to learn weight updates from fine-tuning (ΔW), VeRA leverages a pair of scaling vectors (i.e., d and b from Fig. 4.10), which are tunable and effectively adapt the frozen weight matrices according to a given NLP task. The efficiency gain of this design is in the storage of lighter-weight, task-adapted vector modules rather than the reparameterized matrices of LoRA, which allows many more versions of the adapted LLM to exist on a given compute node.

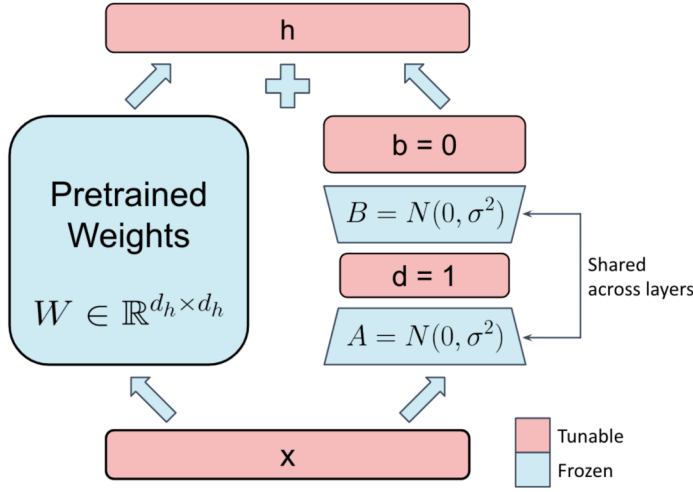


Fig. 4.10: Architectural overview of VeRA adapter components. In contrast with Fig. 4.5b, VeRA freezes matrices A and B , which are shared across all network layers. During fine-tuning, only vectors d and b are trainable, greatly reducing the number of tunable parameters.

Referring back to Equation 4.5, wherein ΔW is recovered by tuning the product of the two low-rank decomposition matrices, A and B , VeRA formulates the computation of model weights for a given input x as:

$$h = W_0 x + \Delta W x = W_0 x + \Lambda_b B \Lambda_d A x \quad (4.10)$$

where, in contrast to Equation 4.5, A and B are frozen, randomly initialized, and shared across Transformer layers. Interestingly, within the VeRA method, A and B do not necessarily have to be reduced in rank relative to the LLM; however, the rank of these matrices results in a linear increase in the number of trainable parameters. As we will see below, this factor of VeRA, coupled with its impressive analytic quality

relative to LoRA, despite using $>10\times$ fewer parameters, represents a powerful option. Scaling vectors b and d (denoted as diagonal matrices Λb and Λd), which are initialized as a vector of zeros and a single nonzero value for all elements, respectively, are trainable during fine-tuning. They serve to scale up and scale down rows and columns of matrices A and B depending on the NLP task of interest, through layer-wise adaptation.

As mentioned, matrices A and B in VeRA are randomly initialized. This random initialization means that only the seed for the random number generator required to reproduce the matrices need be tracked. As such, the storage and memory requirements for VeRA are limited to that random seed and the trained vectors b and d , which, as seen in Fig. 4.9, are significantly reduced as compared to LoRA. Matrix initialization for VeRA leverages Kaiming initialization (He et al., 2015), which maintains a uniform matrix variance independent of rank. This relaxes the need to fine-tune the learning rate per rank, which is another training time efficiency.

VeRA stacks up surprisingly well against other PEFT methods in terms of analytic performance, considering it has an order of magnitude fewer parameters than LoRA. VeRA performs only slightly worse when evaluated against the GLUE benchmark using RoBERTa-base and on par using RoBERTa-large. Additionally, when evaluating VeRA against LoRA on the E2E benchmark, GPT-2 VeRA out-competes it in four of the five E2E tasks.

Next, we will explore alternative methods for improving the efficiency of adapting and fine-tuning LLMs that, rather than attempting to reparameterize or side-car additional task-specific neural networks, aim to reduce the training time memory requirements by optimizing how data are represented or through more efficient optimization functions. Helpfully, many of the techniques we will discuss can be adopted in addition to PEFT methods, thus compounding the efficiencies gained.

4.4 Compute-Efficient Fine-Tuning

While PEFT eases the cost of LLM fine-tuning by only training a fraction of the total parameters in the model, *compute-efficient fine-tuning* focuses on quantization methods that reduce the memory requirements for fine-tuning or doing inference with a given number of parameters. These methods generally enable better trade-off points between training and inference cost versus analytic performance. Some do so with some degradation of analytical performance relative to popular methods such as LoRA, but others improve outcomes along both the computational resource efficiency and analytical performance dimensions, delivering state-of-the-art or near-state-of-the-art results.

Table 4.1: Commonly used data types in LLMs, indicating whether they are standard data types borrowed from other areas of computation versus machine learning optimized representations, other common names for them, and the number of memory bits required for their storage.

Data Type	Standard Data Type?	Other Names	#Bits
float32	Yes	FP32, single-precision floating-point format	32
float16	Yes	FP16, half-precision floating-point format	16
bfloat16	ML optimized	BF16, brain floating point format	16
INT8	Yes	-	8
INT4	Yes	-	4
NF4	ML optimized	-	4

4.4.1 LLM Quantization

Quantization is fundamentally a model compression technique, which reduces the total size of the model by representing its parameters in lower information bit forms (Zhao et al., 2023). This has the effect of reducing the computational resource requirements in the inference setting. Typically, quantization is applied to the parameter weights of the Transformer attention layers and feed-forward layers, as the matrix multiplication operations at these layers represent more than 95% of the memory consumption during LLM inference; thus, targeting the data types involved can result in significant reductions in memory consumption (Dettmers et al., 2022).

Naturally, data precision has a fundamental trade-off with compute efficiency. Table 4.1 shows the bitwidths for different commonly used data types for neural networks in general and for LLMs specifically. As can be surmised, by quantizing parameter weights from, say float32 $\rightarrow$ int8, one can effectively achieve a near 4x reduction in memory required (give or take for layers/parameters that are not quantization targets). Such memory requirements are significant, considering that some models require much more working memory during inference than is available in even most cutting-edge GPU hardware. For example, the 175 billion-parameter GPT3 model requires 325GB of storage at float16 precision, effectively meaning that it can only be run across complex, multi-GPU clusters, precluding its use on more commoditized hardware (e.g., NVIDIA A100 @ 80GB) (Frantar et al., 2023).

Broadly, there are two types of quantization regimes when in regard to LLMs: *Post-Training Quantization* (PTQ), and *Quantization-Aware Training* (QAT). We will first explore the influential applications of PTQ on LLMs, prioritizing coverage of work that a) achieves inference resource consumption that is within the limits of commodity hardware such as NVIDIA A100 or NVIDIA A600 and b) does so while recovering similar analytic performance to unquantized versions of the same models.

After exploring interesting applications of PTQ, we will cover QAT methods in the fine-tuning setting, where the pre-trained LLM is not exposed to QAT but rather to the fine-tuned adapters. Such applications again represent improvements to the

computational-resource efficiency of inference for LLMs, making them viable options for practitioners with limited budgets or other resource constraints (e.g., micro-controllers or edge-computing use cases).

4.4.1.1 Post-Training Quantization

As the name suggests, PTQ is applied to LLMs after the pre-training stage. Typically, the goal is to reduce the memory requirement for inference while maintaining parity in analytic performance with the original LLM. While naive quantization, where weights are more or less indiscriminately quantized to lower-precision data types, has been shown to be effective for smaller language models, drastic drops in analytic performance have been observed for LLMs exceeding 6.7B parameters (see Fig. 4.11; [Dettmers et al. \(2022\)](#)). This phenomenon is linked to the emergence of outlier features, which present as large values in hidden activations of the network, first described in the context of LLM quantization in [Dettmers et al. \(2022\)](#).

Considering the challenge of preserving the precision with which these influential outlier features could be represented while also meeting inference budgets, [Dettmers et al. \(2022\)](#) introduced `LLM.int8()`, which applies INT8 quantization in a vector-wise fashion to 99.9% of target features, but aims to preserve outlier features by isolating them and preserving them in 16-bit precision during matrix multiplications. While this introduces complexity in applying quantization, this targeted mixed-precision regime, which reduces the memory requirements of inference by 2x in the BLOOM-176B model, proved to be impressively effective in preserving the analytic performance of the original LLM, as illustrated across several benchmark tasks (Fig. 4.11).

Another method, *SqueezeLLM*, aims to preserve outlier features and other features sensitive to precision changes by searching for optimal bit precision based on second-order information about the features. Applying this regime in a layer-wise fashion, with precision as low as 3 bit, *SqueezeLLM* can gain up to 2.3x speedup during inference over the original LLM, again with minimal loss ([Kim et al., 2023](#)).

With even more fine-grained quantization, *ZeroQuant* introduced a method that applies different quantization schemes to weights and activations and a novel knowledge distillation mechanism to offset analytic performance degradation. This approach again results in impressive efficiencies (up to 5x inference efficiency), with minimal accuracy loss ([Yao et al., 2022](#)).

In addition to the methods described above, one of the more popular post-training quantization regimes is *GPTQ*. Building on the same ideas as previous methods, GPTQ also leverages second-order information on features to search for the optimal bitwidth for quantization. By targeting weights in such a selective manner and allowing for extreme quantization in the 4-, 3-, and 2-bit widths, GPTQ enabled the use of the BLOOM-176B parameter model on a single NVIDIA A100, with up to 4.5x inference efficiency gains. [Liu et al. \(2023\)](#) provides another example of work aiming to improve the effectiveness of quantization in the extreme range of 3-bit precision through knowledge distillation techniques.

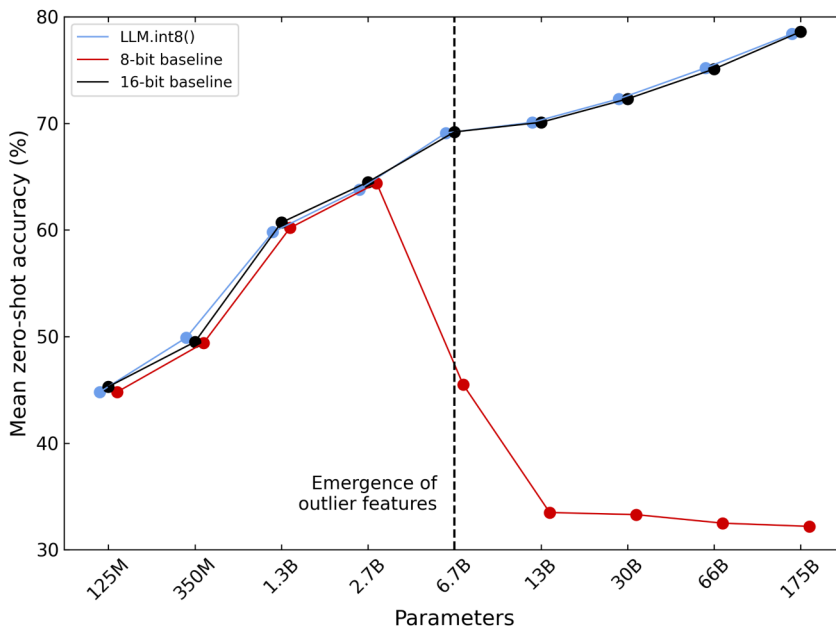


Fig. 4.11: Analytic performance of three different numeric storage precision/quantization regimes for language models with increasing parameters, on a variety of natural language inference tasks. The significant drop in analytic performance between 2.7B and 6.7B parameters is attributed to the emergence of outlier features. LLM.int8() applies 8bit quantization to LLM weights in a way that aims to preserve these features; thus, this method delivers similar analytic performance relative to the full 16-bit version of the LM, despite using half the precision to store parameter weights.

4.4.1.2 Quantization-Aware Training

As described in Sect. 4.4.1.1, PTQ methods do not explicitly attempt to minimize the loss introduced by the act of quantization during the learning process. However, it is important to note that the fine-grained way in which PTQ is applied through methods such as LLM.int8(), GPTQ, or SqueezeLLM does leverage knowledge from the learning process to some extent. One of the key motivators for PTQ approaches in LLM pre-training is to avoid the significant increase in computational overhead due to the scale of the parameters to be quantized as the training loop iterates. As a result, much research has been aimed at combining the inference efficiencies gained through quantization approaches with the training efficiencies gained through PEFT methods, thus reducing the computational overhead introduced by quantization at training time relative to full fine-tuning.

In much the same way that PTQ methods enable LLM inference on more accessible hardware, QAT reduces the fine-tuning overhead to levels where more accessible hardware can be leveraged (Dettmers et al., 2023). In the following sections, we will highlight three of the most promising PEFT-based QAT methods based on a) the extent to which they reduce the fine-tuning overhead and b) the extent to which they preserve analytic performance relative to unquantized PEFT.

QLoRA

Building off the insights and recommendations by Wortsman et al. (2023) regarding techniques to bring some of the efficiency benefits of quantization at inference time into training, QLoRA (Dettmers et al., 2023) has emerged as one of the most widely adopted QAT methods for LLMs. At a high level, QLoRA applies a novel 4-bit quantization to a given LLM, the parameters of which are subsequently frozen during fine-tuning. This work introduced a novel data type named *NF4* or *4-bit NormalFloat*, which is considered to have better quantization precision for normally distributed tensor data than is achieved using either 4-bit integers or 4-bit floats. Following quantization, gradients for LoRA weight updates are backpropagated through the frozen 4-bit quantized LLM, thus ensuring that the error resulting from quantization is part of the fine-tuning process.

By applying not only quantization using the novel NF4 data type mentioned above but also a novel double quantization regime, designed to further reduce the memory overhead introduced by quantization constants, as well as the use of paged-optimizers, QLoRA achieves remarkable computational efficiency during fine-tuning. To put this into quantitative terms, by applying all three of these novel innovations to carry out instruction fine-tuning of the 65B parameter Llama LLM using the LoRA fine-tuning approach and the Alpaca and FLAN v2 datasets Dettmers et al. (2023) demonstrate 99.3% of the analytic performance of ChatGPT, despite fine-tuning requiring only 24 hours on a single GPU. Effectively, the memory requirement for fine-tuning using QLoRA was reduced from more than 780GB of GPU memory in the full-parameter fine-tuning setting with 16-bit precision to less than 48GB of GPU memory, all while preserving near-SOTA analytic performance.

LoftQ

Li et al. (2023a) noted that the fine-tuning outcomes of LoRA-tuned models are adversely affected by quantization of the PLM, especially in the extreme-low bit regime. Explicitly aiming to alleviate the precision discrepancy introduced through low-bitwidth quantization, these authors introduced *LoftQ*, a novel QAT technique that attempts to minimize the disparity between the original weight matrices of the LLM and the weights derived from the joint application of quantization and low-rank weight approximation.

This optimization is formulated as a Frobenius norm minimization as follows:

$$\min_{Q,A,B} ||W - Q - AB^T||_{F'} \quad (4.11)$$

where $||\cdot||_{F'}$ denotes the Frobenius norm, W denotes the original parameter weights, Q denotes the quantized weights, and AB^T denotes LoRA.

Formulating the fine-tuning problem in this way not only allows for the approximation of a more effective quantized initialization of the LoRA matrices A and B but also provides a good approximation of the original LLM parameter weights W . This is achieved by jointly optimizing both the quantization objective, which primarily aims to minimize the memory requirements for weight matrix operations, and the fine-tuning objective through LoRA, which primarily aims to maximize analytic performance with the low-rank constraint on A and B .

LoftQ achieves this joint loss minimization by iteratively alternating between finding Q given the estimation of A and B that minimizes the Frobenius norm in the current step and subsequently, given this new estimate for Q , finding the singular value decomposition low-rank approximation for A and B that minimizes the residual of the quantized weight, Q , and the original weight W (i.e., $Q - W$). By alternating between the quantization estimates and the quantization-aware singular value decomposition (SVD) step, LoftQ effectively finds a better balance between the two, such that they both contribute to the maximization of fine-tuning outcomes. Following this alternating joint-optimization phase, the optimal value for Q is frozen, and standard LoRA fine-tuning can proceed.

This balance between the quantization error and the error introduced by the low-rank representations in LoRA contrasts with QLoRA, where quantization error is not explicitly minimized for fine-tuning. Since quantization introduces a precision discrepancy relative to the original LLM, QLoRA results in less effective generalization than does LoftQ. Supporting this, LoftQ has been shown to outperform QLoRA in all benchmarks tested in [Li et al. \(2023a\)](#).

4.5 End-User Prompting

Thus far, in this chapter, we have discussed learning strategies that involve tuning either all of the LLM parameters, a subset of them, or additional adapters that are appended to the LLM parameters. The commonality of each of these approaches is that they fall into the category of *LLM adaptation*, which we introduced in Sect. 1.5.2. In contrast, in this section we explore *end-user prompting*, which leverages an LLM’s autoregressive text generation and in-context learning abilities to achieve the desired outcomes ([Minaee et al., 2024](#); [Zhao et al., 2023](#)).

Generally, these approaches aim to navigate the various limitations and abilities of an LLM by constructing prompt structures that maximize output quality within the application context. These prompts are engineered using a combination of language comprehension/usage skills, especially in the context of the domain of application, an understanding of the LLM’s strengths and weaknesses, and a traditional engineering mindset that aims to structure and sequence information within the prompt, or chain of prompts, to elicit the most valuable outputs from the model. As with traditional data science and machine learning engineering, prompt engineering is both *science and art*, requiring the interweaving of both creativity and rigid adherence to the details that matter to be successful.

! Practical Tips

Conceptually, it is helpful to imagine any given output of an LLM as the single outcome in an enormous landscape of other possible outcomes, prompting as the user's way of biasing the generation process toward the most useful. In the most capable LLMs, these biases can be induced at every level of language structure, from single subword tokens up to higher-level structures such as grammatical relations, since language modeling has been shown to enable effective learning of this (e.g. [Jawahar et al., 2019](#)). The most effective prompts are usually designed by methodically experimenting with content and structure, such as assessing the influence of domain-specific tokens/words on the alignment of LLM responses or the influence of formal vs. colloquial grammar as in [Chen et al. \(2023\)](#).

In the final sections of this chapter, we explore some of the most popular end-user prompting strategies and their application. While we do not aim to survey every end-user prompting technique comprehensively, we will introduce the most popular of them, as well as the most important concepts. We point the reader to the excellent survey paper; [Chen et al. \(2023\)](#) and the impressive *Prompt Engineering Guide*² to review others. These techniques all leverage various structural patterns better to control the suitability of the LLM outputs, and having an appreciation for their effectiveness in different settings will aid the reader in more effective LLM utilization and application.

4.5.1 Zero-Shot Prompting

A prompt that contains only the task instructions is considered a *zero-shot prompt*. No additional examples or demonstrations of the task solution are included in the prompt. As such, these prompts must be carefully designed to appropriately elicit the useful information or ability required for the target task. Such tasks include sentiment classification, where the example shown in Listing 4.3 might be applied.

```
Please classify the following sentence as either 'Positive',  
'Neutral' or 'Negative' with respect to its sentiment.  
Sentence: I hated the color of the front door!  
Sentiment:
```

Listing 4.3: Zero-shot sentiment classification prompt

As mentioned, zero-shot prompts simply elicit existing knowledge or abilities within the LLM. In the sentiment classification shown in Listing 4.3, it is assumed that the LLM already has knowledge of the concept of sentiment and how it is encoded in text.

² <https://www.promptingguide.ai/>

4.5.2 Few-Shot Prompting

When Zero-shot prompting is ineffective for eliciting knowledge or abilities from LLMs, another option is the use of *few-shot prompts*. In contrast to zero-shot prompts, few-shot prompts contain both the task description and one or more examples or demonstrations of the task solution. The addition of demonstrations of the task the LLM is being asked to complete activates the LLM's in-context learning ability, thus improving task performance over zero-shot solutions (Touvron et al., 2023).

With respect to the sentiment classification task used in Sect. 4.5.1, Listing 4.4 shows a few-shot prompt example.

```
Sentence: I just love it when I wake up to the sun shining
through my window.
Sentiment: Positive
Sentence: I was walking through the town yesterday.
Sentiment: Neutral
Sentence: I can't see a way to solve this problem without it
costing a lot.
Sentiment: Negative
Sentence: That sounds like such an exciting opportunity.
Sentiment:
```

Listing 4.4: few-shot sentiment classification prompt

Interestingly, for few-shot prompting, Min et al. (2022) reported that several prompt attributes are important, while others appear less so. As an example, the prompt in Listing 4.4 follows a structured format, repeating the Sentence the Sentiment sequence to demonstrate the task. This structure is more important to task performance than the demonstrations' correctness (i.e., even using incorrect labels can elicit better task performance than not providing any labels at all). As effective as few-shot prompting can be for tasks such as classification or entity extraction, it has significant limitations for tasks involving complex reasoning. Next, we will look at chain-of-thought and tree-of-thoughts prompting for these tasks.

4.5.3 Prompt Chaining

Prompt chaining aims to simplify and modularize interactions with an LLM in the context of solving a given problem. Generally, prompt chaining is a useful LLM interaction pattern when the use of a single prompt is ineffective, usually due to the complexity of the problem and the inability of the LLM to solve it based on a single prompt. By breaking a larger problem into multiple prompts and chaining them together in a modular, sequentially aware way, better control and quality can often be achieved.

```
Please provide a short summary of the financial dealings
between each business entity pair within the following
document:
```

```
{{document}}  
Summaries:
```

Listing 4.5: Zero-shot sentiment classification prompt

Hypothetically, consider a task where one would like to write a short summary of the various financial dealings between business entities within a document. One approach might be constructing a simple prompt such as the one in Listing 4.5, which tasks the LLM to solve the entire problem in a single inference run. At a low level, this single prompt approach requires the LLM to understand the instructions, reason between the instructions and the document, reason over the identified entities and the document, and finally generate the summary for each entity pair. Even the most capable LLM might struggle with this task.

```
Please list all business entity pairs within the following  
document. Only entity pairs recorded in the document as  
having had business dealings should be listed.  
Document: {{document}}  
Entity Pairs with business dealings:
```

Listing 4.6: Zero-shot sentiment classification prompt

Given the complexity of this task, prompt chaining, where an initial prompt such as that in Listing 4.6 is used first to identify and list all business entity pairs with financial dealings in the document, the results of which are then passed to additional downstream prompt(s) (e.g., Listing 4.7 shows a prompt template for obtaining individual financial dealings summaries) could help improve task performance, as well as control over task performance. By modularizing larger problems into smaller tasks, developers can evaluate LLM performance on intermediate solution steps and modify only those steps to improve the overall task performance.

```
Please summarize the financial dealings between two entities  
listed below, as recorded in the following document.  
Entities: {{entity-pair}}  
Document: {{document}}  
Summary:
```

Listing 4.7: Zero-shot sentiment classification prompt

Multiple frameworks have been developed around the concept of prompt chaining, and are discussed in more detail in Chapter 8. Two of the most popular are LangChain and DSPy, the former being much higher-level than the latter. These frameworks are designed to streamline the development of complex prompting chains and better align their development lifecycle to traditional software development practices.

4.5.4 Chain-of-Thought

First highlighted in [Wei et al. \(2023\)](#), *Chain-of-Thought* (CoT) prompting structures the prompt's context and examples in such a way as to replicate the sequential thinking/reasoning process that humans would typically leverage when solving problems. Generally, problems that can be naturally broken down into a chain of intermediate problems align well with the chain-of-thought prompting paradigm. The most effective prompts within this technique leverage few-shot examples of the type of reasoning steps necessary to solve the problem posed. Chain-of-thought prompting has three core variants worth highlighting:

- **Zero-shot chain-of-thought** was presented in [Kojima et al. \(2023\)](#) and is the most simple and straight-forward of the three variants. It is as simple as adding the text “*Let’s think step by step*” or some text with similar meaning at the end of the prompt. Surprisingly, [Kojima et al. \(2023\)](#) found that this simple addition was sufficient to improve the accuracy of the LLM from 17.7% to 78.7% on the MultiArith dataset ([Roy and Roth, 2015](#)) and from 10.4% to 40.7% on the GSM8K dataset ([Cobbe et al., 2021](#)).
- **Manual chain-of-thought** refers to prompts manually constructed by prompt engineers to contain one or more demonstrations of the reasoning steps the LLM is expected to follow to solve the examples. Including these demonstrations has been shown to enable performance in line with the state of the art on challenging math problems.
- **Automatic chain-of-thought** is a technique proposed in [Zhang et al. \(2022\)](#) that reduces the manual effort required to develop effective CoT prompts. CoT works most effectively when diverse demonstrations and manual construction of such prompts can be laborious. As such, automatic CoT uses question clustering and sampling across clusters to maximize demonstration question diversity while leveraging a zero-shot CoT prompting approach to generate the chain of reasoning through an LLM for these demonstrations. Auto-generated demonstrations are then included in a prompt template and used for inference. This approach was shown in [Zhang et al. \(2022\)](#) to match or exceed manual CoT prompting performance on relevant benchmarks.

! Practical Tips

As all CoT prompting strategies capitalize on LLM’s emergent reasoning abilities, it has been shown to be effective only when the LLM exceeds a certain scale (number of parameters). Smaller LLMs do not exhibit the levels of task performance improvements seen for larger models. For example, the largest performance improvement from using CoT rather than standard prompting on the GSM8K benchmark was seen in the 175B parameter GPT-3 model, with standard prompting achieving 15.6% and CoT prompting achieving 46.9%. In contrast, the 7B parameter GPT-3 model with standard and CoT prompting achieved 4% and 2.4%, respectively ([Wei et al., 2023](#)).

Given such results, developers must verify that CoT prompting is effective in their chosen LLM.

4.5.5 Self-Consistency

As we have discussed, LLMs are prone to confabulation/hallucination in their outputs. In applications with high consistency or factuality requirements, *self-consistency prompting* is an effective approach. The general principle is that the more consistently an LLM responds to the same query, the more likely these responses are to be correct (Wang et al., 2023a).

Leveraging a few-shot CoT prompting approach, self-consistency aims to query the LLM with this same prompt multiple times to elicit multiple responses. The correct answer to the prompt is then derived from this pool of responses based on several options. Simple majority answer selection can be effective in arithmetic tasks, while semantic similarity or n-gram overlap methods can help in language tasks such as question answering.

4.5.6 Tree-of-Thoughts

Tree-of-Thoughts (ToT) prompting builds on the core logic of chain-of-thought prompting in that it focuses the LLM on demonstrations or descriptions of the reasoning steps necessary to solve the task. However, ToT aims to more closely replicate the multi-path exploration that the human mind appears to follow when searching for the correct answer to a problem (Long, 2023). Rather than prompting the LLM with a linear chain of reasoning, ToT aims to enable the LLM to traverse multiple reasoning paths through the problem. This design minimizes the risk of incorrect solutions due to incorrect derivative reasoning steps while increasing the probability of correct answers by exploring more solution pathways.

ToT aligns to the way humans solve problems, leveraging insights from research into human problem solving, where it has been observed that people find solutions based on a cognitive search across a combinatorial problem-space (Simon and Newell, 1971). This process in humans occurs across an ever-narrowing set of pathways, each being filtered as a result of some step in the reasoning process that occurs for that particular branch. Unlike earlier prompting designs, ToT effectively enables both the construction of multiple pathways through a problem, as well as planning, look-ahead and backtracking across them to determine the most effective path to solving the problem.

Tree-of-thoughts as an idea appears to have been independently introduced by both Yao et al. (2023) and Long (2023), differing mainly in the way search across “thoughts” is performed, with the former work leveraging either a *breadth-first search* or *depth-first search* and the latter leveraging a specialized controller module trained through reinforcement learning. In general, ToT can be considered a further enhancement over self-consistency by not only selecting the majority vote answer but also allowing for the sampling of additional intermediate reasoning steps that eventually lead to correct answers.

4.6 Tutorial: Fine-Tuning LLMs in a Resource-Constrained Setting

4.6.1 Overview

We have covered several parameter-efficient fine-tuning techniques and outlined two major approaches to fine-tuning LLMs: instruction and alignment tuning. This tutorial leverages LoRA and QLoRA to train LLMs to accomplish a specific instruction-based task. While this is not strictly instruction tuning, as we focus on a single task instead of a wide range of tasks, our templating approach follows the methodology of instruction tuning.

Goals:

- Demonstrate the advantages of parameter-efficient fine-tuning in terms of both memory requirements and resulting output quality.
- Examine the relative capabilities of a larger LLM and a scaled-down LLM.
- Implement an evaluation rubric for generated text outputs, using a more sophisticated LLM as the grader.

Please note that this is a condensed version of the tutorial. The full version is available at <https://github.com/springer-llms-deep-dive/llms-deep-dive-tutorials>.

4.6.2 Experimental Design

In this tutorial, we create an LLM that can take in a conversation between a customer and a service agent and return a summary of the salient points. The results captured here are based on the performance of a Google Colab session with a 16GB V100 GPU. We use the TWEETSUMM dataset (Feigenblat et al., 2021), which consists

of back-and-forth conversations between customers and service agents from various companies on x.com (formerly Twitter). Paired with each conversation are hand-written two-sentence summaries of the conversation, noting the customer's request and the agent's response. In most cases, there are multiple summaries written by different annotators.

To assess the quality of LLM-generated summaries, we establish three criteria that define a summary score.

1. Is the description of the customer's question/complaint reasonably accurate?
2. Is the description of the agent's response reasonably accurate?
3. Is the summary two sentences in length?

The summary receives one point for meeting each of these criteria. Following [Dettmers et al. \(2023\)](#), we will use GPT-4 to grade the summaries and assign scores. We pass GPT-4 a rubric with these scoring criteria, along with the input conversation and generated summary and ask it to return a score out of 3.

We first test DistilGPT-2, an 85 million parameter autoregressive LLM trained with supervision from GPT-2, selected because its relatively low memory requirements allow us to easily fine-tune it in our Colab environment.

We then try to improve the results by moving to a larger LLM, whose better knowledge of the language could help improve its ability to parse what is happening in these messages. To do this, we adopt Llama-2-7B, a 7 billion parameter autoregressive text-generation LLM released by Meta in 2023. While this model is much more capable, it runs out of memory when we attempt to fine-tune it in the same manner as DistilGPT-2. This motivates the need for parameter-efficient fine-tuning techniques, so we then apply LoRA and QLoRA to compare both model performance and training times across the various training methods.

4.6.3 Results and Analysis

4.6.3.1 DistilGPT-2

As a baseline, we first ask DistilGPT-2 to generate summaries for each test set conversation without fine-tuning. We define a `transformers` pipeline for text generation and then pass in prompts from the templated TWEETSUMM test set. Unsurprisingly, the output is poor. DistilGPT-2 is too small of an LLM for any type of impressive emergent capabilities without additional fine-tuning. Next we fine-tune the model on the training data using the python package `trl`, which implements a convenient wrapper around the `transformers` functionality. The fine-tuned DistilGPT-2 works better than the base model, especially in the summary length criteria, but the descriptions of the customer and agent conversation are still low quality.

To test the overall performance, we generate summaries for 50 conversations in the test dataset using both the base and the tuned models and grade them using GPT-4. The cumulative score for the base model summaries is 2 out of a possible 150,

Table 4.2: Final score out of 150 for each model approach to tuning on the TWEET-SUMM train set and doing casual inference with the test set. Also listed are tuning times for each model.

Model Configuration	Summary score (/150)	Tuning time (m)
Base DistilGPT2	2	0
Fine-tuned DistilGPT2	67	9.7
LoRA-tuned DistilGPT2	58	6.9
QLoRA-tuned DistilGPT2	52	14.3
Base Llama-2-7B	25.5	0
Fine-tuned Llama-2-7B	Failed	
LoRA-tuned Llama-2-7B	131	75.1
QLoRA-tuned Llama-2-7B	125	21.3

which is an extremely poor performance and unsuitable for the task. The tuned model performs considerably better, with a score of 67/150. However, this is still far from ideal.

As discussed in Sect. 4.3.2.1, using low-rank adapters is a popular and efficient method for reducing the memory requirements of training. Instead of fine-tuning the entire weight matrix, we only tune two low-rank matrices, which are then added to the full weights at inference time, thus significantly reducing the number of parameters whose gradients are stored in memory during training. We also test an even more efficient version, QLoRA, which involves quantizing the model weights to 4-bits before applying a LoRA approach to tuning.

The relative performances of LoRA-tuning and QLoRA-tuning for the TWEET-SUM dataset are shown in Table 4.2. They do not reach the level of full-parameter fine-tuning, but are still much better than the baseline. Despite the lower performance for DistilGPT-2, we observe a smaller total GPU workload during training. Compared to full-parameter fine-tuning, the maximum GPU RAM occupancy is 228 MB lower for LoRA tuning and 336 MB lower for QLoRA tuning. This is a significant amount as that DistilGPT-2’s weight matrix is approximately 356 MB.

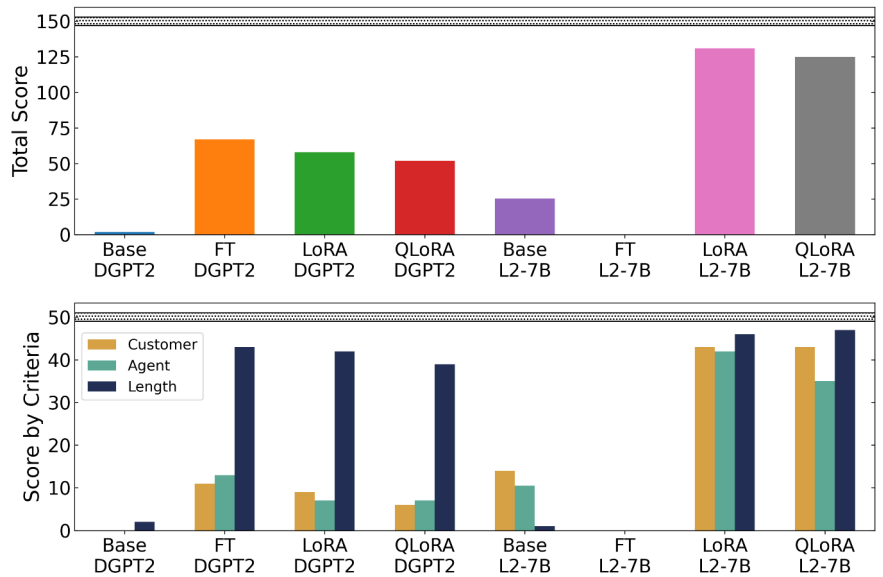
4.6.3.2 Llama-2-7B

We next attempt to improve our results by by moving to a larger LLM, whose better knowledge of the language could help improve its ability to parse what is happening in these messages. Llama-2-7B fits the bill. Repeating the base-line zero-shot summarization experimert, we find that Llama-2 scores 25.5/150. Still a poor performance, but a significant upgrade over baseline DistilGPT2. Next we test full-parameter fine-tuning of Llama-2, and unfortunately run out of memory on our GPU. At seven billion parameters, the model weights alone on Llama-2-7B consume around 12GB of memory, and when fine-tuning gradients are added the total balloons to around 64 GB of memory, well above the 16 GB on our V100 GPU.

Motivated by this failure, we test our PEFT methods on Llama-2-7B, which allow us to enter the training loops without CUDA errors. We tune for a single epoch, which takes 75 minutes for the LoRA loop and just 21 minutes for the QLoRA loop. With this approach, we find a remarkable improvement in performance, with the LoRA-tuned test set evaluation scoring 131/150 and the QLoRA evaluation scoring 125/131.

Fig. 4.12 summarizes the test set evaluation results of every configuration considered in this tutorial. The two adapter-tuned Llama-2-7B models dominate the overall score and are the best for each grading criterion. We see on the bottom how the fine-tuned DistilGPT-2 models effectively learned to limit their summaries to two sentences but were not able to make them accurate enough for the liking of GPT-4. Base Llama-2-7B produced an equal number of summaries deemed accurate as the full-parameter fine-tuned DistilGPT-2 but could not follow the formatting rules without reinforcement.

Fig. 4.12: Final scores on the TWEETSUMM summarization task for each inference framework. On the top, we show raw score out of 150, and on the bottom, we break down the score into the three criteria: successful customer summary, successful agent summary, and length (is the response 2 sentences long?). Note that full-parameter fine-tuning for Llama-2-7B did not produce a model due to memory constraints.



4.6.4 Conclusion

This experiment shows how smaller LLMs can be tuned to follow specific instructions but ultimately cannot compete with the semantic capabilities of large LLMs due to their low information capacity. Among the Llama-2 tuned models, QLoRA slightly underperforms LoRA but finishes tuning in less than a third of the time. This trade-off is critical for situations with large training datasets. Overall, low-rank adapter tuning took advantage of the large number of parameters in the Llama-2-7B model, producing a high-quality and reliable summarization bot.

References

- Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning, 2020.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Banghao Chen, Zhaofeng Zhang, Nicolas Langren  , and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review, 2023.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.
- Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235, 2023.
- Ali Edalati, Marzieh Tahaei, Ivan Kobzyev, Vahid Partovi Nia, James J. Clark, and Mehdi Rezagholizadeh. Krona: Parameter efficient tuning with kronecker adapter, 2022.
- Guy Feigenblat, Chulaka Gunasekara, Benjamin Sznajder, Sachindra Joshi, David Konopnicki, and Ranit Aharonov. TWEETSUMM - a dialog summarization dataset for customer service. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 245–260, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. URL <https://aclanthology.org/2021.findings-emnlp.24>.

- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023.
- Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning, 2022.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- Zhiqiang Hu, Lei Wang, Yihuai Lan, Wanyu Xu, Ee-Peng Lim, Lidong Bing, Xing Xu, Soujanya Poria, and Roy Ka-Wei Lee. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models, 2023.
- Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. Lorahub: Efficient cross-task generalization via dynamic lora composition, 2023.
- Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3651–3657, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1356. URL <https://aclanthology.org/P19-1356>.
- Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization, 2023.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners, 2023.
- Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki Markus Asano. Vera: Vector-based random matrix adaptation, 2023.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.
- Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, and Tuo Zhao. Loftq: Lora-fine-tuning-aware quantization for large language models, 2023a.
- Yunxiang Li, Zihan Li, Kai Zhang, Ruilong Dan, Steve Jiang, and You Zhang. Chatdoctor: A medical chat model fine-tuned on a large language model meta-ai (llama) using medical domain knowledge, 2023b.
- Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. Llm-qat: Data-free quantization aware training for large language models, 2023.
- Jieyi Long. Large language model guided tree-of-thought, 2023.

- Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers, 2021.
- Sewon Min, Xinxu Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work?, 2022.
- Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey, 2024.
- Long Ouyang et al. Training language models to follow instructions with human feedback, 2022.
- Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning, 2021.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2020.
- Subho Roy and Dan Roth. Solving general arithmetic word problems. In Lluís Màrquez, Chris Callison-Burch, and Jian Su, editors, *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1743–1752, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1202. URL <https://aclanthology.org/D15-1202>.
- Herbert A Simon and Allen Newell. Human problem solving: The state of the theory in 1970. *American psychologist*, 26(2):145, 1971.
- Yi-Lin Sung, Jaemin Cho, and Mohit Bansal. Lst: Ladder side-tuning for parameter and memory efficient transfer learning, 2022.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>, 3(6):7, 2023.
- Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.
- Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023a.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2023b.
- Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- Mitchell Wortsman, Tim Dettmers, Luke Zettlemoyer, Ari Morcos, Ali Farhadi, and Ludwig Schmidt. Stable and low-precision training for large-scale vision-language models, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.
- Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers, 2022.
- Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, 2022.
- Boyu Zhang, Hongyang Yang, and Xiao-Yang Liu. Instruct-fingpt: Financial sentiment analysis by instruction tuning of general-purpose large language models, 2023a.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning, 2023b.
- Shengyu Zhang et al. Instruction tuning for large language models: A survey, 2023c.
- Yue Zhang, Leyang Cui, Deng Cai, Xinting Huang, Tao Fang, and Wei Bi. Multi-task instruction tuning of llama for specific scenarios: A preliminary study on writing assistance, 2023d.
- Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models, 2022.
- Wayne Xin Zhao et al. A survey of large language models, 2023.
- Yaoming Zhu, Jiangtao Feng, Chengqi Zhao, Mingxuan Wang, and Lei Li. Counter-interference adapter for multilingual machine translation, 2021.



Chapter 5

Tuning for LLM Alignment

Abstract LLM training traditionally involves self-supervised learning using pre-training and supervised learning with fine-tuning, which relies on large datasets with predefined input-output pairs. These models learn to predict the next word in a sequence, attempting to mimic the training data as closely as possible. However, the optimal behavior of LLMs often involves more than replicating seen examples; it requires an understanding and integration of nuanced human preferences and societal norms that are not explicitly present in the raw data. This chapter starts by defining what alignment to human preferences means and introducing the three Hs – Helpful, Harmless, and Honest. Human preferences are abstract, multifaceted, and often challenging to encode directly into training datasets comprehensively. This is where Reinforcement Learning (RL) comes to the rescue. After establishing a foundational understanding of reinforcement learning, this chapter explores the seminal work, process, research, and architectures that have paved the way for human feedback to assist LLMs in aligning with human values. By tracing the contributions of key studies and methodologies, this chapter delves into the progressive evolution of reinforcement learning techniques and their role in enabling LLMs to better align with and embody human values, leading to enhanced ethical and responsible language generation. Although RLHF is a useful technique for alignment, it faces primarily two challenges: (1) it requires a large number of human evaluators to rank AI-generated responses, a process that is resource and cost-intensive, and (2) its scalability is limited due to the need to maintain multiple LLMs (LLM acts as a reward model to learn human preferences). We spotlight two pivotal research breakthroughs addressing RLHF challenges: “Constitutional AI” and “Direct Preference Optimization”, which offer solutions to enhance training efficiency, model reliability, and scalability.